

action

The action namespace is produced by one regex pass over one combined text string: tweet body + OCR text + transcript text + media-description text, joined with separators. A tag is added when its regex appears anywhere in that combined string.

action:detention (2,278 (22.8%))

Precise plain-English rule

Matches arrest/arrested/arresting, detain/detained/detaining, apprehend/apprehended/apprehending, in custody, or nab/nabbed.

Code text for this tag

```
518: PATTERN_ACTION_DETENTION = _compile(  
519:     r"\b(arrest(?:ed|ing)?|detain(?:ed|ing)?|apprehend(?:ed|ing)?|in custody|nab(?:bed)?)\b"  
520: )  
  
1539:     (PATTERN_FRAME_CRIMINAL, "theme:criminal"),  
1540:     (PATTERN_ACTION_DETENTION, "action:detention"),  
1541:     (PATTERN_ACTION_SELF_DEPORTATION, "action:self-deportation"),
```

action:deportation (1,304 (13.0%))

Precise plain-English rule

Matches deport/deported/deporting/deportation, remove/removed/removal, or repatriated/repatriation, but the deport match is guarded so self-deport does not count here.

Code text for this tag

```
528: PATTERN_ACTION_DEPORTATION = _compile(  
529:     r"(?!self-)(?!self-)\b(deport(?:ed|ing|ation)?|remov(?:ed|al)|repatriat(?:ed|ion))\b"  
530: )  
  
1541:     (PATTERN_ACTION_SELF_DEPORTATION, "action:self-deportation"),  
1542:     (PATTERN_ACTION_DEPORTATION, "action:deportation"),  
1543:     (PATTERN_ACTION_REPORT_TO_ICE, "action:report-immigrants"),
```

action:self-deportation (191 (1.9%))

Precise plain-English rule

Matches self-deport/self deport variants, voluntary/voluntarily departure/departed/departing, leave voluntarily, report <your/their/his/her/an/the> departure, or take control of <your/their/his/her> departure.

Code text for this tag

```
521: PATTERN_ACTION_SELF_DEPORTATION = _compile(  
522:     r"\bself[- ]deport(?:ed|ing|ation)?\b"  
523:     r"|\bvolutar(?:y|ily)\s+depart(?:ure|ed|ing)?\b"  
524:     r"|\bleave voluntarily\b"  
525:     r"|\breport\s+(?:your|their|his|her|an|the)\s*\s*departure\b"  
526:     r"|\btake\s+control\s+of\s+(?:your|their|his|her)\s*\s*departure\b"  
527: )  
  
1540:     (PATTERN_ACTION_DETENTION, "action:detention"),  
1541:     (PATTERN_ACTION_SELF_DEPORTATION, "action:self-deportation"),  
1542:     (PATTERN_ACTION_DEPORTATION, "action:deportation"),
```

action:report-immigrants (7 (0.1%))

Precise plain-English rule

Matches report/call/tip language only when ICE/DHS/contact information and immigrant/alien/migrant/immigration-violation subject language occur in the same bounded expression. The bounds in the regex are 80, 120, 180, or 240 characters depending on phrase order.

Code text for this tag

```
546: PATTERN_ACTION_REPORT_TO_ICE = _compile(  
547:     rf"\b(?:"  
548:     rf"(?:call|contact)\s+(?:{ICE_DIRECT_TARGET}|{ICE_REPORT_PHONE})\b"  
549:     rf"|-.{0,180}\b{ICE_REPORT_SUBJECT}\b)"  
550:     rf"|(?:report|tip|submit|send|notify)\b.{0,80}\b{ICE_REPORT_SUBJECT}\b"  
551:     rf"|.{{0,80}}\b(?:to|at|with)\s+(?:{ICE_OR_DHS_TARGET}|{ICE_REPORT_PHONE})\b)"  
552:     rf"|(?:submit|send)\s+(?:a\s+)?tip\b.{0,80}\b(?:to|with)\s+"  
553:     rf"|(?:{ICE_OR_DHS_TARGET}|{ICE_REPORT_PHONE})\b)"  
554:     rf"|-.{0,120}\b{ICE_REPORT_SUBJECT}\b)"  
555:     rf"|{ICE_REPORT_SUBJECT}\b.{0,240}\b(?:call|contact)\s+"  
556:     rf"|(?:our|the)\s+(?:{ICE_DIRECT_TARGET}|ICE\s+)"  
557:     rf"|(?:tip\s*line|hotline|{ICE_REPORT_PHONE})\b)"  
558:     rf")"  
559: )  
  
1542:     (PATTERN_ACTION_DEPORTATION, "action:deportation"),  
1543:     (PATTERN_ACTION_REPORT_TO_ICE, "action:report-immigrants"),  
1544:     (PATTERN_SUBJECT_CBP_HOME_APP, "subject:cbp-home-app"),
```

artist

The artist namespace is produced only from written or transcribed text. The code does not identify music by listening to audio. It matches artist names or listed signature song phrases in the combined text string.

artist:lee-greenwood (4 (0.0%))

Precise plain-English rule

Matches Lee Greenwood, God Bless the USA, or Proud to be an American in written/transcribed text.

Code text for this tag

```
941: ARTIST_GAZETTEER: tuple[tuple[re.Pattern[str], str], ...] = (  
942:     (_compile(r"\bsydney\s+sweeney\b"), "sydney-sweeney"),  
943:     (  
944:         _compile(  
945:             r"\blee\s+greenwood\b|\bgod\s+bless\s+the\s+u\?.?s\?.?a\?.?b"  
946:             r"|\bproud\s+to\s+be\s+an\s+american\b"  
947:         ),  
948:         "lee-greenwood",  
949:     ),  
950:     (_compile(r"\bkid\s+rock\b"), "kid-rock"),  
951:     (_compile(r"\bvillage\s+people\b|\bmacho\s+man\b"), "village-people"),  
952:     (_compile(r"\bjason\s+aldean\b|\btry\s+that\s+in\s+a\s+small\s+town\b"), "jason-aldean"),  
953:     (_compile(r"\boliver\s+anthony\b|\brich\s+men\s+north\s+of\s+richmond\b"), "oliver-anthony"),  
954:     (_compile(r"\btaylor\s+swift\b"), "taylor-swift"),  
955:     (_compile(r"\bbeyonc[eé]\b"), "beyonce"),  
956: )  
  
1672: # artist:<name> - named cultural figures / musicians (and signature songs).  
1673: for artist_pat, artist_slug in ARTIST_GAZETTEER:  
1674:     if m := artist_pat.search(text):  
1675:         add(f"artist:{artist_slug}", span=m.span())
```

artist:kid-rock (2 (0.0%))

Precise plain-English rule

Matches Kid Rock in written/transcribed text.

Code text for this tag

```
941: ARTIST_GAZETTEER: tuple[tuple[re.Pattern[str], str], ...] = (  
942:     (_compile(r"\bsydney\s+sweeney\b"), "sydney-sweeney"),  
943:     (  
944:         _compile(  
945:             r"\blee\s+greenwood\b|\bgod\s+bless\s+the\s+u\?.?s\?.?a\?.?b"  
946:             r"|\bproud\s+to\s+be\s+an\s+american\b"  
947:         ),  
948:         "lee-greenwood",  
949:     ),  
950:     (_compile(r"\bkid\s+rock\b"), "kid-rock"),  
951:     (_compile(r"\bvillage\s+people\b|\bmacho\s+man\b"), "village-people"),  
952:     (_compile(r"\bjason\s+aldean\b|\btry\s+that\s+in\s+a\s+small\s+town\b"), "jason-aldean"),  
953:     (_compile(r"\boliver\s+anthony\b|\brich\s+men\s+north\s+of\s+richmond\b"), "oliver-anthony"),  
954:     (_compile(r"\btaylor\s+swift\b"), "taylor-swift"),  
955:     (_compile(r"\bbeyonc[eé]\b"), "beyonce"),  
956: )  
  
1672: # artist:<name> - named cultural figures / musicians (and signature songs).  
1673: for artist_pat, artist_slug in ARTIST_GAZETTEER:  
1674:     if m := artist_pat.search(text):  
1675:         add(f"artist:{artist_slug}", span=m.span())
```

country

The country namespace is produced by validated extraction. The code first captures a one- or two-word place after one of these prepositions: from, in, to, of, with, by. It then resolves that capture against the country vocabulary. It also adds country tags for curated demonyms and curated foreign city names.

country:Mexico (771 (7.7%))

Precise plain-English rule

Emitted as country:Mexico when a validated candidate resolves to Mexico: a country name after from/in/to/of/with/by, a curated demonym for Mexico, or a curated foreign city mapped to Mexico.

Code text for this tag

```
153:     "Mexico",  
  
1217: # second word excludes those same prepositions via lookahead, so a greedy  
1218: # capture can't swallow the preposition that introduces the *next* place  
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). `_resolve_vocab`  
1220: # then validates against the vocab and falls back to the leading word for any  
1221: # other trailing token. Kept conservative because raw country names  
1222: # false-positive easily ("Chad", "Georgia").  
1223: _PREPOSITIONS = "from|in|to|of|with|by"  
1224: _PLACE = rf"[A-Za-z][a-zA-Z](?:\s+(?!({_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]+)?"  
1225: # "from <Country>" - anchors the ORIGIN validator.  
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s+", "  
1227: # Any sovereign-state mention after a preposition.  
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{_PREPOSITIONS})\s+({_PLACE})\b")  
1229: # "<Place>, <State>" - anchors the STATE validator.  
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE})\s+({_PLACE})\b")  
  
1683: # country:<NAME> - broader: any contextual country mention.  
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):  
1685:     resolved = _resolve_vocab(m.group(1)) or "", COUNTRY_BY_LOWER  
1686:     if resolved:  
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))  
1688:  
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
```

```

1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:     country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:     if country and country.lower() in COUNTRY_LOWER:
1701:         add(f"country:{_normalize_country(country)}", span=m.span(1))

2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
2052: if low == "burma":
2053:     return "Myanmar"
2054: return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:El-Salvador (266 (2.7%))

Precise plain-English rule

Emitted as country:El-Salvador when a validated candidate resolves to El Salvador: a country name after from/in/to/of/with/by, a curated demonym for El Salvador, or a curated foreign city mapped to El Salvador.

Code text for this tag

```

98:     "El Salvador",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]+)?"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*",)
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:({_PREPOSITIONS})\s+({_PLACE})\b)"
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE})\s+({_PLACE})\b")

1683: # country:<NAME> - broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:     country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:     if country and country.lower() in COUNTRY_LOWER:
1701:         add(f"country:{_normalize_country(country)}", span=m.span(1))

2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
2052: if low == "burma":
2053:     return "Myanmar"
2054: return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Guatemala (189 (1.9%))

Precise plain-English rule

Emitted as country:Guatemala when a validated candidate resolves to Guatemala: a country name after from/in/to/of/with/by, a curated demonym for Guatemala, or a curated foreign city mapped to Guatemala.

Code text for this tag

```
113:     "Guatemala",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). `._resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE})\s+({_PLACE})\b")

1683:     # country:<NAME> - broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:
1687:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:     # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:     # country, which the bare-name validators above can't see. A demonym
1691:     # followed by a person noun also earns origin:.
1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:         country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:         if country and country.lower() in COUNTRY_LOWER:
1701:             add(f"country:{_normalize_country(country)}", span=m.span(1))

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token
```

country:Honduras (180 (1.8%))

Precise plain-English rule

Emitted as country:Honduras when a validated candidate resolves to Honduras: a country name after from/in/to/of/with/by, a curated demonym for Honduras, or a curated foreign city mapped to Honduras.

Code text for this tag

```
117:     "Honduras",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). `._resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE})\s+({_PLACE})\b")

1683:     # country:<NAME> - broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:
1687:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:     # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:     # country, which the bare-name validators above can't see. A demonym
1691:     # followed by a person noun also earns origin:.
1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:         country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
```

```

1700:         if country and country.lower() in COUNTRY_LOWER:
1701:             add(f"country:{_normalize_country(country)}", span=m.span(1))

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Iran (133 (1.3%))

Precise plain-English rule

Emitted as country:Iran when a validated candidate resolves to Iran: a country name after from/in/to/of/with/by, a curated demonym for Iran, or a curated foreign city mapped to Iran.

Code text for this tag

```

122:     "Iran",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). `_resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{[_PREPOSITIONS]})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE})\s+({_PLACE})\b")

1683:     # country:<NAME> - broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:
1687:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:     # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:     # country, which the bare-name validators above can't see. A demonym
1691:     # followed by a person noun also earns origin:.
1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:         country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:         if country and country.lower() in COUNTRY_LOWER:
1701:             add(f"country:{_normalize_country(country)}", span=m.span(1))

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Venezuela (133 (1.3%))

Precise plain-English rule

Emitted as country:Venezuela when a validated candidate resolves to Venezuela: a country name after from/in/to/of/with/by, a curated demonym for Venezuela, or a curated foreign city mapped to Venezuela.

Code text for this tag

```

225:     "Venezuela",

1217: # second word excludes those same prepositions via lookahead, so a greedy

```

```

1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). '_resolve_vocab'
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z](?:\s+(?!({_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]+)?"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{[_PREPOSITIONS]})\s+({_PLACE})\b")
1229: # "<Place>, <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{[_PLACE]}\s+({_PLACE})\b")

1683: # country:<NAME> - broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:     country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:     if country and country.lower() in COUNTRY_LOWER:
1701:         add(f"country:{_normalize_country(country)}", span=m.span(1))

2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
2052: if low == "burma":
2053:     return "Myanmar"
2054: return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Cuba (100 (1.0%))

Precise plain-English rule

Emitted as country:Cuba when a validated candidate resolves to Cuba: a country name after from/in/to/of/with/by, a curated demonym for Cuba, or a curated foreign city mapped to Cuba.

Code text for this tag

```

88:     "Cuba",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). '_resolve_vocab'
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z](?:\s+(?!({_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]+)?"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{[_PREPOSITIONS]})\s+({_PLACE})\b")
1229: # "<Place>, <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{[_PLACE]}\s+({_PLACE})\b")

1683: # country:<NAME> - broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:     country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:     if country and country.lower() in COUNTRY_LOWER:
1701:         add(f"country:{_normalize_country(country)}", span=m.span(1))

2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"

```

```

2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:China (98 (1.0%))

Precise plain-English rule

Emitted as country:China when a validated candidate resolves to China: a country name after from/in/to/of/with/by, a curated demonym for China, or a curated foreign city mapped to China.

Code text for this tag

```

82:     "China",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!(?:{_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE})\s+({_PLACE})\b")

1683: # country:<NAME> - broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:     country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:     if country and country.lower() in COUNTRY_LOWER:
1701:         add(f"country:{_normalize_country(country)}", span=m.span(1))

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Dominican-Republic (70 (0.7%))

Precise plain-English rule

Emitted as country:Dominican-Republic when a validated candidate resolves to Dominican Republic: a country name after from/in/to/of/with/by, a curated demonym for Dominican Republic, or a curated foreign city mapped to Dominican Republic.

Code text for this tag

```

95:     "Dominican Republic",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!(?:{_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")

```

```

1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{:_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>, <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1683: # country:<NAME> – broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:     country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:     if country and country.lower() in COUNTRY_LOWER:
1701:         add(f"country:{_normalize_country(country)}", span=m.span(1))

2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
2052: if low == "burma":
2053:     return "Myanmar"
2054: return s.replace(" ", ".")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", ".")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Colombia (54 (0.5%))

Precise plain-English rule

Emitted as country:Colombia when a validated candidate resolves to Colombia: a country name after from/in/to/of/with/by, a curated demonym for Colombia, or a curated foreign city mapped to Colombia.

Code text for this tag

```

83:     "Colombia",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?:{:_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]*(?)"
1225: # "from <country>," – anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:{:_PREPOSITIONS})\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{:_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>, <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1683: # country:<NAME> – broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:     country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:     if country and country.lower() in COUNTRY_LOWER:
1701:         add(f"country:{_normalize_country(country)}", span=m.span(1))

2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
2052: if low == "burma":
2053:     return "Myanmar"
2054: return s.replace(" ", ".")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", ".")
2059:
2060:

```

```

2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Laos (51 (0.5%))

Precise plain-English rule

Emitted as country:Laos when a validated candidate resolves to Laos: a country name after from/in/to/of/with/by, a curated demonym for Laos, or a curated foreign city mapped to Laos.

Code text for this tag

```

136:     "Laos",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE}),\s+({_PLACE})\b")

1683:     # country:<NAME> - broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:
1687:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:     # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:     # country, which the bare-name validators above can't see. A demonym
1691:     # followed by a person noun also earns origin:.
1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:         country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:         if country and country.lower() in COUNTRY_LOWER:
1701:             add(f"country:{_normalize_country(country)}", span=m.span(1))

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Ecuador (49 (0.5%))

Precise plain-English rule

Emitted as country:Ecuador when a validated candidate resolves to Ecuador: a country name after from/in/to/of/with/by, a curated demonym for Ecuador, or a curated foreign city mapped to Ecuador.

Code text for this tag

```

96:     "Ecuador",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE}),\s+({_PLACE})\b")

1683:     # country:<NAME> - broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:

```

```

1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:         # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:         # country, which the bare-name validators above can't see. A demonym
1691:         # followed by a person noun also earns origin:.
1692:         for m in PATTERN_DEMONYM.finditer(text):
1693:             country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:             if country and country.lower() in COUNTRY_LOWER:
1695:                 add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:                 if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                     add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:         for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:             country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:             if country and country.lower() in COUNTRY_LOWER:
1701:                 add(f"country:{_normalize_country(country)}", span=m.span(1))
1702:
2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Israel (43 (0.4%))

Precise plain-English rule

Emitted as country:Israel when a validated candidate resolves to Israel: a country name after from/in/to/of/with/by, a curated demonym for Israel, or a curated foreign city mapped to Israel.

Code text for this tag

```

125:     "Israel",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z](?:\s+(?!{:_PREPOSITIONS})\b)[A-Za-z][a-zA-Z])?"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{:_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE}\s+({_PLACE})\b")

1683:     # country:<NAME> - broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:
1687:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:         # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:         # country, which the bare-name validators above can't see. A demonym
1691:         # followed by a person noun also earns origin:.
1692:         for m in PATTERN_DEMONYM.finditer(text):
1693:             country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:             if country and country.lower() in COUNTRY_LOWER:
1695:                 add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:                 if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                     add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:         for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:             country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:             if country and country.lower() in COUNTRY_LOWER:
1701:                 add(f"country:{_normalize_country(country)}", span=m.span(1))
1702:
2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Vietnam (39 (0.4%))

Precise plain-English rule

Emitted as country:Vietnam when a validated candidate resolves to Vietnam: a country name after from/in/to/of/with/by, a curated demonym for Vietnam, or a curated foreign city mapped to Vietnam.

Code text for this tag

```
226:     "Vietnam",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the *next* place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). `_resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE}),\s+({_PLACE})\b")

1683: # country:<NAME> - broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:     country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:     if country and country.lower() in COUNTRY_LOWER:
1701:         add(f"country:{_normalize_country(country)}", span=m.span(1))

2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
2052: if low == "burma":
2053:     return "Myanmar"
2054: return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token
```

country:India (36 (0.4%))

Precise plain-English rule

Emitted as country:India when a validated candidate resolves to India: a country name after from/in/to/of/with/by, a curated demonym for India, or a curated foreign city mapped to India.

Code text for this tag

```
120:     "India",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the *next* place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). `_resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE}),\s+({_PLACE})\b")

1683: # country:<NAME> - broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
```

```

1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:         country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:         if country and country.lower() in COUNTRY_LOWER:
1701:             add(f"country:{_normalize_country(country)}", span=m.span(1))

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", ".")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", ".")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Somalia (35 (0.3%))

Precise plain-English rule

Emitted as country:Somalia when a validated candidate resolves to Somalia: a country name after from/in/to/of/with/by, a curated demonym for Somalia, or a curated foreign city mapped to Somalia.

Code text for this tag

```

192:     "Somalia",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the *next* place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]{?:(?!({_PREPOSITIONS}))\b}[A-Za-z][a-zA-Z]{+}"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"b(?:{:_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"b({_PLACE})\s+({_PLACE})\b")

1683:     # country:<NAME> - broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:
1687:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:     # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:     # country, which the bare-name validators above can't see. A demonym
1691:     # followed by a person noun also earns origin:.
1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:         country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:         if country and country.lower() in COUNTRY_LOWER:
1701:             add(f"country:{_normalize_country(country)}", span=m.span(1))

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", ".")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", ".")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Haiti (33 (0.3%))

Precise plain-English rule

Emitted as country:Haiti when a validated candidate resolves to Haiti: a country name after from/in/to/of/with/by, a curated demonym for Haiti, or a curated foreign city mapped to Haiti.

Code text for this tag

```
116:     "Haiti",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the *next* place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). `_resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!{:_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{:_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>,<State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1683: # country:<NAME> - broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:     country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:     if country and country.lower() in COUNTRY_LOWER:
1701:         add(f"country:{_normalize_country(country)}", span=m.span(1))

2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
2052: if low == "burma":
2053:     return "Myanmar"
2054: return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token
```

country:Brazil (31 (0.3%))

Precise plain-English rule

Emitted as country:Brazil when a validated candidate resolves to Brazil: a country name after from/in/to/of/with/by, a curated demonym for Brazil, or a curated foreign city mapped to Brazil.

Code text for this tag

```
72:     "Brazil",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the *next* place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). `_resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!{:_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{:_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>,<State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1683: # country:<NAME> - broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:     country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:     if country and country.lower() in COUNTRY_LOWER:
1701:         add(f"country:{_normalize_country(country)}", span=m.span(1))

2046: if low in ("usa", "united states"):
```

```

2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Canada (31 (0.3%))

Precise plain-English rule

Emitted as country:Canada when a validated candidate resolves to Canada: a country name after from/in/to/of/with/by, a curated demonym for Canada, or a curated foreign city mapped to Canada.

Code text for this tag

```

79:     "Canada",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*(?)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"b(?:{:_PREPOSITIONS})\s+({_PLACE})\s*",")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"b(?:{:_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"b{:_PLACE}\s+({_PLACE})\b")

1683:     # country:<NAME> - broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:
1687:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:     # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:     # country, which the bare-name validators above can't see. A demonym
1691:     # followed by a person noun also earns origin:.
1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:         country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:         if country and country.lower() in COUNTRY_LOWER:
1701:             add(f"country:{_normalize_country(country)}", span=m.span(1))

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Afghanistan (27 (0.3%))

Precise plain-English rule

Emitted as country:Afghanistan when a validated candidate resolves to Afghanistan: a country name after from/in/to/of/with/by, a curated demonym for Afghanistan, or a curated foreign city mapped to Afghanistan.

Code text for this tag

```

50:     "Afghanistan",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names

```

```

1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z](?:\s+(?!{:_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]?"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{:_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE}\s+({_PLACE})\b")

1683: # country:<NAME> - broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:     country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:     if country and country.lower() in COUNTRY_LOWER:
1701:         add(f"country:{_normalize_country(country)}", span=m.span(1))

2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
2052: if low == "burma":
2053:     return "Myanmar"
2054: return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Jamaica (25 (0.2%))

Precise plain-English rule

Emitted as country:Jamaica when a validated candidate resolves to Jamaica: a country name after from/in/to/of/with/by, a curated demonym for Jamaica, or a curated foreign city mapped to Jamaica.

Code text for this tag

```

127:     "Jamaica",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the *next* place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). ` _resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z](?:\s+(?!{:_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]?"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{:_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE}\s+({_PLACE})\b")

1683: # country:<NAME> - broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:     country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:     if country and country.lower() in COUNTRY_LOWER:
1701:         add(f"country:{_normalize_country(country)}", span=m.span(1))

2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
2052: if low == "burma":
2053:     return "Myanmar"
2054: return s.replace(" ", "-")
2055:

```

```

2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Russia (25 (0.2%))

Precise plain-English rule

Emitted as country:Russia when a validated candidate resolves to Russia: a country name after from/in/to/of/with/by, a curated demonym for Russia, or a curated foreign city mapped to Russia.

Code text for this tag

```

182:     "Russia",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the *next* place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*", "")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{[_PREPOSITIONS]})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE}),\s+({_PLACE})\b")

1683:     # country:<NAME> - broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:
1687:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:     # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:     # country, which the bare-name validators above can't see. A demonym
1691:     # followed by a person noun also earns origin:.
1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:         country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:         if country and country.lower() in COUNTRY_LOWER:
1701:             add(f"country:{_normalize_country(country)}", span=m.span(1))

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Nicaragua (22 (0.2%))

Precise plain-English rule

Emitted as country:Nicaragua when a validated candidate resolves to Nicaragua: a country name after from/in/to/of/with/by, a curated demonym for Nicaragua, or a curated foreign city mapped to Nicaragua.

Code text for this tag

```

167:     "Nicaragua",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the *next* place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*", "")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{[_PREPOSITIONS]})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE}),\s+({_PLACE})\b")

```

```

1683: # country:<NAME> – broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:     country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:     if country and country.lower() in COUNTRY_LOWER:
1701:         add(f"country:{_normalize_country(country)}", span=m.span(1))

2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
2052: if low == "burma":
2053:     return "Myanmar"
2054: return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Pakistan (15 (0.1%))

Precise plain-English rule

Emitted as country:Pakistan when a validated candidate resolves to Pakistan: a country name after from/in/to/of/with/by, a curated demonym for Pakistan, or a curated foreign city mapped to Pakistan.

Code text for this tag

```

172:     "Pakistan",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the *next* place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). `_resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," – anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE}\s+({_PLACE})\b")

1683: # country:<NAME> – broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:     country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:     if country and country.lower() in COUNTRY_LOWER:
1701:         add(f"country:{_normalize_country(country)}", span=m.span(1))

2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
2052: if low == "burma":
2053:     return "Myanmar"
2054: return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy

```

2066: case-insensitive capture that swallowed a trailing lowercase token

country:Ukraine (15 (0.1%))

Precise plain-English rule

Emitted as country:Ukraine when a validated candidate resolves to Ukraine: a country name after from/in/to/of/with/by, a curated demonym for Ukraine, or a curated foreign city mapped to Ukraine.

Code text for this tag

```
215: "Ukraine",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the *next* place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). `resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!{:_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*",)
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{:_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE}\s+({_PLACE})\b")

1683: # country:<NAME> - broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:     country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:     if country and country.lower() in COUNTRY_LOWER:
1701:         add(f"country:{_normalize_country(country)}", span=m.span(1))

2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
2052: if low == "burma":
2053:     return "Myanmar"
2054: return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token
```

country:Georgia (14 (0.1%))

Precise plain-English rule

Emitted as country:Georgia when a validated candidate resolves to Georgia: a country name after from/in/to/of/with/by, a curated demonym for Georgia, or a curated foreign city mapped to Georgia.

Code text for this tag

```
108: "Georgia",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the *next* place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). `resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!{:_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*",)
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{:_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE}\s+({_PLACE})\b")

1683: # country:<NAME> - broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
```

```

1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:         country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:         if country and country.lower() in COUNTRY_LOWER:
1701:             add(f"country:{_normalize_country(country)}", span=m.span(1))

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Nigeria (13 (0.1%))

Precise plain-English rule

Emitted as country:Nigeria when a validated candidate resolves to Nigeria: a country name after from/in/to/of/with/by, a curated demonym for Nigeria, or a curated foreign city mapped to Nigeria.

Code text for this tag

```

169:     "Nigeria",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't't swallow the preposition that introduces the 'next' place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]+)?"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE})\s+({_PLACE})\b")

1683: # country:<NAME> - broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:         country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:         if country and country.lower() in COUNTRY_LOWER:
1701:             add(f"country:{_normalize_country(country)}", span=m.span(1))

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Argentina (11 (0.1%))

Precise plain-English rule

Emitted as country:Argentina when a validated candidate resolves to Argentina: a country name after from/in/to/of/with/by, a curated demonym for Argentina, or a curated foreign city mapped to Argentina.

Code text for this tag

```
55:     "Argentina",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). '_resolve_vocab'
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>" - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*", "")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>, <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1683:     # country:<NAME> - broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:
1687:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:     # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:     # country, which the bare-name validators above can't see. A demonym
1691:     # followed by a person noun also earns origin:.
1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:         country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:         if country and country.lower() in COUNTRY_LOWER:
1701:             add(f"country:{_normalize_country(country)}", span=m.span(1))

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token
```

country:Peru (11 (0.1%))

Precise plain-English rule

Emitted as country:Peru when a validated candidate resolves to Peru: a country name after from/in/to/of/with/by, a curated demonym for Peru, or a curated foreign city mapped to Peru.

Code text for this tag

```
176:     "Peru",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). '_resolve_vocab'
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>" - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*", "")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>, <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1683:     # country:<NAME> - broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:
1687:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:     # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:     # country, which the bare-name validators above can't see. A demonym
1691:     # followed by a person noun also earns origin:.
1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:         country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
```

```

1700:         if country and country.lower() in COUNTRY_LOWER:
1701:             add(f"country:{_normalize_country(country)}", span=m.span(1))

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Lebanon (10 (0.1%))

Precise plain-English rule

Emitted as country:Lebanon when a validated candidate resolves to Lebanon: a country name after from/in/to/of/with/by, a curated demonym for Lebanon, or a curated foreign city mapped to Lebanon.

Code text for this tag

```

138:     "Lebanon",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). '_resolve_vocab'
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:({_PREPOSITIONS})\s+({_PLACE}))\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE})\s+({_PLACE})\b")

1683:     # country:<NAME> - broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:
1687:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:     # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:     # country, which the bare-name validators above can't see. A demonym
1691:     # followed by a person noun also earns origin:.
1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:         country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:         if country and country.lower() in COUNTRY_LOWER:
1701:             add(f"country:{_normalize_country(country)}", span=m.span(1))

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:United-States (10 (0.1%))

Precise plain-English rule

Emitted as country:United-States when a validated candidate resolves to United States: a country name after from/in/to/of/with/by, a curated demonym for United States, or a curated foreign city mapped to United States.

Code text for this tag

```

220:     "United States",

1217: # second word excludes those same prepositions via lookahead, so a greedy

```

```

1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). '_resolve_vocab'
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z](?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]?"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*", "")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>, <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE})\s+({_PLACE})\b")

1683: # country:<NAME> - broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:         country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:         if country and country.lower() in COUNTRY_LOWER:
1701:             add(f"country:{_normalize_country(country)}", span=m.span(1))

2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
2052: if low == "burma":
2053:     return "Myanmar"
2054: return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Egypt (8 (0.1%))

Precise plain-English rule

Emitted as country:Egypt when a validated candidate resolves to Egypt: a country name after from/in/to/of/with/by, a curated demonym for Egypt, or a curated foreign city mapped to Egypt.

Code text for this tag

```

97:     "Egypt",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). '_resolve_vocab'
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z](?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]?"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*", "")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>, <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE})\s+({_PLACE})\b")

1683: # country:<NAME> - broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:         country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:         if country and country.lower() in COUNTRY_LOWER:
1701:             add(f"country:{_normalize_country(country)}", span=m.span(1))

2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"

```

```

2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Iraq (8 (0.1%))

Precise plain-English rule

Emitted as country:Iraq when a validated candidate resolves to Iraq: a country name after from/in/to/of/with/by, a curated demonym for Iraq, or a curated foreign city mapped to Iraq.

Code text for this tag

```

123:     "Iraq",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!(?:{_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*", "")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE})\s+({_PLACE})\b")

1683:     # country:<NAME> - broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:
1687:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:     # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:     # country, which the bare-name validators above can't see. A demonym
1691:     # followed by a person noun also earns origin:.
1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:         country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:         if country and country.lower() in COUNTRY_LOWER:
1701:             add(f"country:{_normalize_country(country)}", span=m.span(1))

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Panama (8 (0.1%))

Precise plain-English rule

Emitted as country:Panama when a validated candidate resolves to Panama: a country name after from/in/to/of/with/by, a curated demonym for Panama, or a curated foreign city mapped to Panama.

Code text for this tag

```

174:     "Panama",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!(?:{_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*", "")

```

```

1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{:_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>, <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1683: # country:<NAME> – broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:     country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:     if country and country.lower() in COUNTRY_LOWER:
1701:         add(f"country:{_normalize_country(country)}", span=m.span(1))

2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
2052: if low == "burma":
2053:     return "Myanmar"
2054: return s.replace(" ", ".")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", ".")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Cambodia (7 (0.1%))

Precise plain-English rule

Emitted as country:Cambodia when a validated candidate resolves to Cambodia: a country name after from/in/to/of/with/by, a curated demonym for Cambodia, or a curated foreign city mapped to Cambodia.

Code text for this tag

```

77:     "Cambodia",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?:{:_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]*(?)"
1225: # "from <country>," – anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:{:_PREPOSITIONS})\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{:_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>, <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1683: # country:<NAME> – broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:     country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:     if country and country.lower() in COUNTRY_LOWER:
1701:         add(f"country:{_normalize_country(country)}", span=m.span(1))

2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
2052: if low == "burma":
2053:     return "Myanmar"
2054: return s.replace(" ", ".")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", ".")
2059:
2060:

```

```

2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Sierra-Leone (7 (0.1%))

Precise plain-English rule

Emitted as country:Sierra-Leone when a validated candidate resolves to Sierra Leone: a country name after from/in/to/of/with/by, a curated demonym for Sierra Leone, or a curated foreign city mapped to Sierra Leone.

Code text for this tag

```

188:     "Sierra Leone",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*",")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1683:     # country:<NAME> - broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:
1687:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:     # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:     # country, which the bare-name validators above can't see. A demonym
1691:     # followed by a person noun also earns origin:.
1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:         country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:         if country and country.lower() in COUNTRY_LOWER:
1701:             add(f"country:{_normalize_country(country)}", span=m.span(1))

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:South-Korea (7 (0.1%))

Precise plain-English rule

Emitted as country:South-Korea when a validated candidate resolves to South Korea: a country name after from/in/to/of/with/by, a curated demonym for South Korea, or a curated foreign city mapped to South Korea.

Code text for this tag

```

194:     "South Korea",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*",")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1683:     # country:<NAME> - broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:

```

```

1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:     # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:     # country, which the bare-name validators above can't see. A demonym
1691:     # followed by a person noun also earns origin:.
1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:         country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:         if country and country.lower() in COUNTRY_LOWER:
1701:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1702:
2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:France (6 (0.1%))

Precise plain-English rule

Emitted as country:France when a validated candidate resolves to France: a country name after from/in/to/of/with/by, a curated demonym for France, or a curated foreign city mapped to France.

Code text for this tag

```

105:     "France",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!{:_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]*(?)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{:_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE}\s+({_PLACE})\b")

1683:     # country:<NAME> - broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:
1687:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:     # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:     # country, which the bare-name validators above can't see. A demonym
1691:     # followed by a person noun also earns origin:.
1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:         country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:         if country and country.lower() in COUNTRY_LOWER:
1701:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1702:
2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Guyana (6 (0.1%))

Precise plain-English rule

Emitted as country:Guyana when a validated candidate resolves to Guyana: a country name after from/in/to/of/with/by, a curated demonym for Guyana, or a curated foreign city mapped to Guyana.

Code text for this tag

```
115:     "Guyana",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the *next* place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). `_resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE}),\s+({_PLACE})\b")

1683:     # country:<NAME> - broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:
1687:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:     # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:     # country, which the bare-name validators above can't see. A demonym
1691:     # followed by a person noun also earns origin:.
1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:         country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:         if country and country.lower() in COUNTRY_LOWER:
1701:             add(f"country:{_normalize_country(country)}", span=m.span(1))

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token
```

country:Jordan (6 (0.1%))

Precise plain-English rule

Emitted as country:Jordan when a validated candidate resolves to Jordan: a country name after from/in/to/of/with/by, a curated demonym for Jordan, or a curated foreign city mapped to Jordan.

Code text for this tag

```
129:     "Jordan",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the *next* place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). `_resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE}),\s+({_PLACE})\b")

1683:     # country:<NAME> - broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:
1687:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:     # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:     # country, which the bare-name validators above can't see. A demonym
1691:     # followed by a person noun also earns origin:.
1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
```

```

1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:         country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:         if country and country.lower() in COUNTRY_LOWER:
1701:             add(f"country:{_normalize_country(country)}", span=m.span(1))

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", ".")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", ".")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Sudan (6 (0.1%))

Precise plain-English rule

Emitted as country:Sudan when a validated candidate resolves to Sudan: a country name after from/in/to/of/with/by, a curated demonym for Sudan, or a curated foreign city mapped to Sudan.

Code text for this tag

```

198:     "Sudan",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the *next* place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]{?:(?!(?:{_PREPOSITIONS}))\b}[A-Za-z][a-zA-Z]{+}"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"b(?:{_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"b({_PLACE})\s+({_PLACE})\b")

1683:     # country:<NAME> - broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:
1687:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:     # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:     # country, which the bare-name validators above can't see. A demonym
1691:     # followed by a person noun also earns origin:.
1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:         country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:         if country and country.lower() in COUNTRY_LOWER:
1701:             add(f"country:{_normalize_country(country)}", span=m.span(1))

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", ".")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", ".")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Syria (6 (0.1%))

Precise plain-English rule

Emitted as country:Syria when a validated candidate resolves to Syria: a country name after from/in/to/of/with/by, a curated demonym for Syria, or a curated foreign city mapped to Syria.

Code text for this tag

```
202: "Syria",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the 'next' place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). `_resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!{:_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{:_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>,<State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1683: # country:<NAME> - broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:     country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:     if country and country.lower() in COUNTRY_LOWER:
1701:         add(f"country:{_normalize_country(country)}", span=m.span(1))

2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
2052: if low == "burma":
2053:     return "Myanmar"
2054: return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token
```

country:Ghana (5 (0.0%))

Precise plain-English rule

Emitted as country:Ghana when a validated candidate resolves to Ghana: a country name after from/in/to/of/with/by, a curated demonym for Ghana, or a curated foreign city mapped to Ghana.

Code text for this tag

```
110: "Ghana",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the 'next' place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). `_resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!{:_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{:_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>,<State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1683: # country:<NAME> - broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:     country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:     if country and country.lower() in COUNTRY_LOWER:
1701:         add(f"country:{_normalize_country(country)}", span=m.span(1))

2046: if low in ("usa", "united states"):
```

```

2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Japan (5 (0.0%))

Precise plain-English rule

Emitted as country:Japan when a validated candidate resolves to Japan: a country name after from/in/to/of/with/by, a curated demonym for Japan, or a curated foreign city mapped to Japan.

Code text for this tag

```

128:     "Japan",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z](?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"b(?:{_PREPOSITIONS})\s+({_PLACE})\s*",")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"b(?:{_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"b{_PLACE}\s+({_PLACE})\b")

1683:     # country:<NAME> - broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:
1687:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:     # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:     # country, which the bare-name validators above can't see. A demonym
1691:     # followed by a person noun also earns origin:.
1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:         country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:         if country and country.lower() in COUNTRY_LOWER:
1701:             add(f"country:{_normalize_country(country)}", span=m.span(1))

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Myanmar (5 (0.0%))

Precise plain-English rule

Emitted as country:Myanmar when a validated candidate resolves to Myanmar: a country name after from/in/to/of/with/by, a curated demonym for Myanmar, or a curated foreign city mapped to Myanmar.

Code text for this tag

```

160:     "Myanmar",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names

```

```

1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!{:_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]*(?)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{:_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1683: # country:<NAME> - broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:     country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:     if country and country.lower() in COUNTRY_LOWER:
1701:         add(f"country:{_normalize_country(country)}", span=m.span(1))

2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
2052: if low == "burma":
2053:     return "Myanmar"
2054: return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Turkey (5 (0.0%))

Precise plain-English rule

Emitted as country:Turkey when a validated candidate resolves to Turkey: a country name after from/in/to/of/with/by, a curated demonym for Turkey, or a curated foreign city mapped to Turkey.

Code text for this tag

```

212:     "Turkey",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the *next* place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). ` _resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!{:_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]*(?)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{:_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1683: # country:<NAME> - broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:     country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:     if country and country.lower() in COUNTRY_LOWER:
1701:         add(f"country:{_normalize_country(country)}", span=m.span(1))

2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
2052: if low == "burma":
2053:     return "Myanmar"
2054: return s.replace(" ", "-")
2055:

```

```

2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Barbados (4 (0.0%))

Precise plain-English rule

Emitted as country:Barbados when a validated candidate resolves to Barbados: a country name after from/in/to/of/with/by, a curated demonym for Barbados, or a curated foreign city mapped to Barbados.

Code text for this tag

```

63:     "Barbados",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the *next* place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*", "")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:({_PREPOSITIONS}))\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE})\s+({_PLACE})\b")

1683:     # country:<NAME> - broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:
1687:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:     # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:     # country, which the bare-name validators above can't see. A demonym
1691:     # followed by a person noun also earns origin:.
1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:         country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:         if country and country.lower() in COUNTRY_LOWER:
1701:             add(f"country:{_normalize_country(country)}", span=m.span(1))

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Cameroon (4 (0.0%))

Precise plain-English rule

Emitted as country:Cameroon when a validated candidate resolves to Cameroon: a country name after from/in/to/of/with/by, a curated demonym for Cameroon, or a curated foreign city mapped to Cameroon.

Code text for this tag

```

78:     "Cameroon",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the *next* place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*", "")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:({_PREPOSITIONS}))\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE})\s+({_PLACE})\b")

```

```

1683: # country:<NAME> – broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:     country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:     if country and country.lower() in COUNTRY_LOWER:
1701:         add(f"country:{_normalize_country(country)}", span=m.span(1))

2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
2052: if low == "burma":
2053:     return "Myanmar"
2054: return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Chile (4 (0.0%))

Precise plain-English rule

Emitted as country:Chile when a validated candidate resolves to Chile: a country name after from/in/to/of/with/by, a curated demonym for Chile, or a curated foreign city mapped to Chile.

Code text for this tag

```

81:     "Chile",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the *next* place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). ` _resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," – anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{ _PREPOSITIONS })\s+({_PLACE})\b")
1229: # "<Place>," <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE}\s+({_PLACE})\b")

1683: # country:<NAME> – broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:     country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:     if country and country.lower() in COUNTRY_LOWER:
1701:         add(f"country:{_normalize_country(country)}", span=m.span(1))

2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
2052: if low == "burma":
2053:     return "Myanmar"
2054: return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy

```

2066: case-insensitive capture that swallowed a trailing lowercase token

country:Ethiopia (4 (0.0%))

Precise plain-English rule

Emitted as country:Ethiopia when a validated candidate resolves to Ethiopia: a country name after from/in/to/of/with/by, a curated demonym for Ethiopia, or a curated foreign city mapped to Ethiopia.

Code text for this tag

```
102: "Ethiopia",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the *next* place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). `resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{:_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1683: # country:<NAME> - broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:     country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:     if country and country.lower() in COUNTRY_LOWER:
1701:         add(f"country:{_normalize_country(country)}", span=m.span(1))

2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
2052: if low == "burma":
2053:     return "Myanmar"
2054: return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token
```

country:Germany (4 (0.0%))

Precise plain-English rule

Emitted as country:Germany when a validated candidate resolves to Germany: a country name after from/in/to/of/with/by, a curated demonym for Germany, or a curated foreign city mapped to Germany.

Code text for this tag

```
109: "Germany",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the *next* place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). `resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{:_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1683: # country:<NAME> - broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
```

```

1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:         country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:         if country and country.lower() in COUNTRY_LOWER:
1701:             add(f"country:{_normalize_country(country)}", span=m.span(1))

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Greece (4 (0.0%))

Precise plain-English rule

Emitted as country:Greece when a validated candidate resolves to Greece: a country name after from/in/to/of/with/by, a curated demonym for Greece, or a curated foreign city mapped to Greece.

Code text for this tag

```

111:     "Greece",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]+)?"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE})\s+({_PLACE})\b")

1683:     # country:<NAME> - broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:
1687:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:     # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:     # country, which the bare-name validators above can't see. A demonym
1691:     # followed by a person noun also earns origin:.
1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:         country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:         if country and country.lower() in COUNTRY_LOWER:
1701:             add(f"country:{_normalize_country(country)}", span=m.span(1))

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Ireland (4 (0.0%))

Precise plain-English rule

Emitted as country:Ireland when a validated candidate resolves to Ireland: a country name after from/in/to/of/with/by, a curated demonym for Ireland, or a curated foreign city mapped to Ireland.

Code text for this tag

```
124:     "Ireland",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). '_resolve_vocab'
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1683:     # country:<NAME> - broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:
1687:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:     # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:     # country, which the bare-name validators above can't see. A demonym
1691:     # followed by a person noun also earns origin:.
1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:         country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:         if country and country.lower() in COUNTRY_LOWER:
1701:             add(f"country:{_normalize_country(country)}", span=m.span(1))

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token
```

country:Kenya (4 (0.0%))

Precise plain-English rule

Emitted as country:Kenya when a validated candidate resolves to Kenya: a country name after from/in/to/of/with/by, a curated demonym for Kenya, or a curated foreign city mapped to Kenya.

Code text for this tag

```
131:     "Kenya",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). '_resolve_vocab'
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1683:     # country:<NAME> - broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:
1687:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:     # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:     # country, which the bare-name validators above can't see. A demonym
1691:     # followed by a person noun also earns origin:.
1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:         country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
```

```

1700:         if country and country.lower() in COUNTRY_LOWER:
1701:             add(f"country:{_normalize_country(country)}", span=m.span(1))

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Liberia (4 (0.0%))

Precise plain-English rule

Emitted as country:Liberia when a validated candidate resolves to Liberia: a country name after from/in/to/of/with/by, a curated demonym for Liberia, or a curated foreign city mapped to Liberia.

Code text for this tag

```

140:     "Liberia",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). `_resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{[_PREPOSITIONS]})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE})\s+({_PLACE})\b")

1683: # country:<NAME> - broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:     country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:     if country and country.lower() in COUNTRY_LOWER:
1701:         add(f"country:{_normalize_country(country)}", span=m.span(1))

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Philippines (4 (0.0%))

Precise plain-English rule

Emitted as country:Philippines when a validated candidate resolves to Philippines: a country name after from/in/to/of/with/by, a curated demonym for Philippines, or a curated foreign city mapped to Philippines.

Code text for this tag

```

177:     "Philippines",

1217: # second word excludes those same prepositions via lookahead, so a greedy

```

```

1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). '_resolve_vocab'
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z](?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]?"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*",")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>, <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE})\s+({_PLACE})\b")

1683: # country:<NAME> - broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:     country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:     if country and country.lower() in COUNTRY_LOWER:
1701:         add(f"country:{_normalize_country(country)}", span=m.span(1))

2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
2052: if low == "burma":
2053:     return "Myanmar"
2054: return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Qatar (4 (0.0%))

Precise plain-English rule

Emitted as country:Qatar when a validated candidate resolves to Qatar: a country name after from/in/to/of/with/by, a curated demonym for Qatar, or a curated foreign city mapped to Qatar.

Code text for this tag

```

180:     "Qatar",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). '_resolve_vocab'
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z](?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]?"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*",")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>, <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE})\s+({_PLACE})\b")

1683: # country:<NAME> - broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:     country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:     if country and country.lower() in COUNTRY_LOWER:
1701:         add(f"country:{_normalize_country(country)}", span=m.span(1))

2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"

```

```

2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Rwanda (4 (0.0%))

Precise plain-English rule

Emitted as country:Rwanda when a validated candidate resolves to Rwanda: a country name after from/in/to/of/with/by, a curated demonym for Rwanda, or a curated foreign city mapped to Rwanda.

Code text for this tag

```

183:     "Rwanda",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!(?:{_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*", "")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE})\s+({_PLACE})\b")

1683: # country:<NAME> - broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:     country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:     if country and country.lower() in COUNTRY_LOWER:
1701:         add(f"country:{_normalize_country(country)}", span=m.span(1))

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:South-Africa (4 (0.0%))

Precise plain-English rule

Emitted as country:South-Africa when a validated candidate resolves to South Africa: a country name after from/in/to/of/with/by, a curated demonym for South Africa, or a curated foreign city mapped to South Africa.

Code text for this tag

```

193:     "South Africa",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!(?:{_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*", "")

```

```

1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>, <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1683: # country:<NAME> – broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:     country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:     if country and country.lower() in COUNTRY_LOWER:
1701:         add(f"country:{_normalize_country(country)}", span=m.span(1))

2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
2052: if low == "burma":
2053:     return "Myanmar"
2054: return s.replace(" ", ".")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", ".")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Trinidad (4 (0.0%))

Precise plain-English rule

Emitted as country:Trinidad when a validated candidate resolves to Trinidad: a country name after from/in/to/of/with/by, a curated demonym for Trinidad, or a curated foreign city mapped to Trinidad.

Code text for this tag

```

209:     "Trinidad",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?:{PREPOSITIONS})\b)[A-Za-z][a-zA-Z]*(?)"
1225: # "from <country>," – anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>, <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1683: # country:<NAME> – broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:     country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:     if country and country.lower() in COUNTRY_LOWER:
1701:         add(f"country:{_normalize_country(country)}", span=m.span(1))

2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
2052: if low == "burma":
2053:     return "Myanmar"
2054: return s.replace(" ", ".")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", ".")
2059:
2060:

```

```

2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Australia (3 (0.0%))

Precise plain-English rule

Emitted as country:Australia when a validated candidate resolves to Australia: a country name after from/in/to/of/with/by, a curated demonym for Australia, or a curated foreign city mapped to Australia.

Code text for this tag

```

57:     "Australia",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*",")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:({_PREPOSITIONS}))\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE}),\s+({_PLACE})\b")

1683: # country:<NAME> - broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:     country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:     if country and country.lower() in COUNTRY_LOWER:
1701:         add(f"country:{_normalize_country(country)}", span=m.span(1))

2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
2052: if low == "burma":
2053:     return "Myanmar"
2054: return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Bahrain (3 (0.0%))

Precise plain-English rule

Emitted as country:Bahrain when a validated candidate resolves to Bahrain: a country name after from/in/to/of/with/by, a curated demonym for Bahrain, or a curated foreign city mapped to Bahrain.

Code text for this tag

```

61:     "Bahrain",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*",")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:({_PREPOSITIONS}))\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE}),\s+({_PLACE})\b")

1683: # country:<NAME> - broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:

```

```

1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:         # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:         # country, which the bare-name validators above can't see. A demonym
1691:         # followed by a person noun also earns origin:.
1692:         for m in PATTERN_DEMONYM.finditer(text):
1693:             country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:             if country and country.lower() in COUNTRY_LOWER:
1695:                 add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:                 if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                     add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:         for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:             country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:             if country and country.lower() in COUNTRY_LOWER:
1701:                 add(f"country:{_normalize_country(country)}", span=m.span(1))
1702:
2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Bangladesh (3 (0.0%))

Precise plain-English rule

Emitted as country:Bangladesh when a validated candidate resolves to Bangladesh: a country name after from/in/to/of/with/by, a curated demonym for Bangladesh, or a curated foreign city mapped to Bangladesh.

Code text for this tag

```

62:     "Bangladesh",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z](?:\s+(?!{:_PREPOSITIONS})\b)[A-Za-z][a-zA-Z])?"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{:_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE}\s+({_PLACE})\b")

1683:     # country:<NAME> - broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:
1687:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:     # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:     # country, which the bare-name validators above can't see. A demonym
1691:     # followed by a person noun also earns origin:.
1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:         country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:         if country and country.lower() in COUNTRY_LOWER:
1701:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1702:
2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Costa-Rica (3 (0.0%))

Precise plain-English rule

Emitted as country:Costa-Rica when a validated candidate resolves to Costa Rica: a country name after from/in/to/of/with/by, a curated demonym for Costa Rica, or a curated foreign city mapped to Costa Rica.

Code text for this tag

```
86:     "Costa Rica",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the *next* place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). `_resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*",)
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{[_PREPOSITIONS]})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1683: # country:<NAME> - broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:     country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:     if country and country.lower() in COUNTRY_LOWER:
1701:         add(f"country:{_normalize_country(country)}", span=m.span(1))

2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
2052: if low == "burma":
2053:     return "Myanmar"
2054: return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token
```

country:Italy (3 (0.0%))

Precise plain-English rule

Emitted as country:Italy when a validated candidate resolves to Italy: a country name after from/in/to/of/with/by, a curated demonym for Italy, or a curated foreign city mapped to Italy.

Code text for this tag

```
126:     "Italy",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the *next* place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). `_resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*",)
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{[_PREPOSITIONS]})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1683: # country:<NAME> - broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
```

```

1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:         country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:         if country and country.lower() in COUNTRY_LOWER:
1701:             add(f"country:{_normalize_country(country)}", span=m.span(1))

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", ".")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", ".")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Senegal (3 (0.0%))

Precise plain-English rule

Emitted as country:Senegal when a validated candidate resolves to Senegal: a country name after from/in/to/of/with/by, a curated demonym for Senegal, or a curated foreign city mapped to Senegal.

Code text for this tag

```

185:     "Senegal",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the *next* place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). `_resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!(?:{_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"b(?:{_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"b({_PLACE})\s+({_PLACE})\b")

1683:     # country:<NAME> - broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:
1687:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:     # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:     # country, which the bare-name validators above can't see. A demonym
1691:     # followed by a person noun also earns origin:.
1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:         country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:         if country and country.lower() in COUNTRY_LOWER:
1701:             add(f"country:{_normalize_country(country)}", span=m.span(1))

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", ".")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", ".")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Spain (3 (0.0%))

Precise plain-English rule

Emitted as country:Spain when a validated candidate resolves to Spain: a country name after from/in/to/of/with/by, a curated demonym for Spain, or a curated foreign city mapped to Spain.

Code text for this tag

```
196:     "Spain",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the 'next' place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). `_resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!{:_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{:_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>,<State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1683: # country:<NAME> - broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:     country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:     if country and country.lower() in COUNTRY_LOWER:
1701:         add(f"country:{_normalize_country(country)}", span=m.span(1))

2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
2052: if low == "burma":
2053:     return "Myanmar"
2054: return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token
```

country:Thailand (3 (0.0%))

Precise plain-English rule

Emitted as country:Thailand when a validated candidate resolves to Thailand: a country name after from/in/to/of/with/by, a curated demonym for Thailand, or a curated foreign city mapped to Thailand.

Code text for this tag

```
206:     "Thailand",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the 'next' place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). `_resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!{:_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{:_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>,<State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1683: # country:<NAME> - broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:     country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:     if country and country.lower() in COUNTRY_LOWER:
1701:         add(f"country:{_normalize_country(country)}", span=m.span(1))

2046: if low in ("usa", "united states"):
```

```

2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Uganda (3 (0.0%))

Precise plain-English rule

Emitted as country:Uganda when a validated candidate resolves to Uganda: a country name after from/in/to/of/with/by, a curated demonym for Uganda, or a curated foreign city mapped to Uganda.

Code text for this tag

```

214:     "Uganda",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z](?:\s+(?!({_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]+)?"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:{_PREPOSITIONS})\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE}\s+({_PLACE})\b")

1683:     # country:<NAME> - broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:
1687:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:     # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:     # country, which the bare-name validators above can't see. A demonym
1691:     # followed by a person noun also earns origin:.
1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:         country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:         if country and country.lower() in COUNTRY_LOWER:
1701:             add(f"country:{_normalize_country(country)}", span=m.span(1))

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Uzbekistan (3 (0.0%))

Precise plain-English rule

Emitted as country:Uzbekistan when a validated candidate resolves to Uzbekistan: a country name after from/in/to/of/with/by, a curated demonym for Uzbekistan, or a curated foreign city mapped to Uzbekistan.

Code text for this tag

```

223:     "Uzbekistan",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names

```

```

1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z](?:\s+(?!{:_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]?"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{:_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1683: # country:<NAME> - broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:     country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:     if country and country.lower() in COUNTRY_LOWER:
1701:         add(f"country:{_normalize_country(country)}", span=m.span(1))

2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
2052: if low == "burma":
2053:     return "Myanmar"
2054: return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Angola (2 (0.0%))

Precise plain-English rule

Emitted as country:Angola when a validated candidate resolves to Angola: a country name after from/in/to/of/with/by, a curated demonym for Angola, or a curated foreign city mapped to Angola.

Code text for this tag

```

54:     "Angola",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the *next* place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). ` _resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z](?:\s+(?!{:_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]?"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{:_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1683: # country:<NAME> - broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:     country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:     if country and country.lower() in COUNTRY_LOWER:
1701:         add(f"country:{_normalize_country(country)}", span=m.span(1))

2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
2052: if low == "burma":
2053:     return "Myanmar"
2054: return s.replace(" ", "-")
2055:

```

```

2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Armenia (2 (0.0%))

Precise plain-English rule

Emitted as country:Armenia when a validated candidate resolves to Armenia: a country name after from/in/to/of/with/by, a curated demonym for Armenia, or a curated foreign city mapped to Armenia.

Code text for this tag

```

56:     "Armenia",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the *next* place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*", "")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:({_PREPOSITIONS}))\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE})\s+({_PLACE})\b")

1683:     # country:<NAME> - broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:
1687:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:     # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:     # country, which the bare-name validators above can't see. A demonym
1691:     # followed by a person noun also earns origin:.
1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:         country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:         if country and country.lower() in COUNTRY_LOWER:
1701:             add(f"country:{_normalize_country(country)}", span=m.span(1))

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Azerbaijan (2 (0.0%))

Precise plain-English rule

Emitted as country:Azerbaijan when a validated candidate resolves to Azerbaijan: a country name after from/in/to/of/with/by, a curated demonym for Azerbaijan, or a curated foreign city mapped to Azerbaijan.

Code text for this tag

```

59:     "Azerbaijan",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the *next* place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*", "")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:({_PREPOSITIONS}))\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE})\s+({_PLACE})\b")

```

```

1683: # country:<NAME> – broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:     country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:     if country and country.lower() in COUNTRY_LOWER:
1701:         add(f"country:{_normalize_country(country)}", span=m.span(1))

2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
2052: if low == "burma":
2053:     return "Myanmar"
2054: return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Burkina-Faso (2 (0.0%))

Precise plain-English rule

Emitted as country:Burkina-Faso when a validated candidate resolves to Burkina Faso: a country name after from/in/to/of/with/by, a curated demonym for Burkina Faso, or a curated foreign city mapped to Burkina Faso.

Code text for this tag

```

75:     "Burkina Faso",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the *next* place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). `_resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]+(?:\s+(?!({_PREPOSITIONS}))b)[A-Za-z][a-zA-Z]?"
1225: # "from <Country>," – anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"b(?:{[_PREPOSITIONS]})\s+({_PLACE})\b")
1229: # "<Place>," <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"b({_PLACE})\s+({_PLACE})\b")

1683: # country:<NAME> – broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:     country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:     if country and country.lower() in COUNTRY_LOWER:
1701:         add(f"country:{_normalize_country(country)}", span=m.span(1))

2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
2052: if low == "burma":
2053:     return "Myanmar"
2054: return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy

```

2066: case-insensitive capture that swallowed a trailing lowercase token

country:Finland (2 (0.0%))

Precise plain-English rule

Emitted as country:Finland when a validated candidate resolves to Finland: a country name after from/in/to/of/with/by, a curated demonym for Finland, or a curated foreign city mapped to Finland.

Code text for this tag

```
104: "Finland",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the *next* place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). `resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*(?)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{[_PREPOSITIONS]})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{[_PLACE]}\s+({_PLACE})\b")

1683: # country:<NAME> - broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:     country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:     if country and country.lower() in COUNTRY_LOWER:
1701:         add(f"country:{_normalize_country(country)}", span=m.span(1))

2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
2052: if low == "burma":
2053:     return "Myanmar"
2054: return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token
```

country:Gabon (2 (0.0%))

Precise plain-English rule

Emitted as country:Gabon when a validated candidate resolves to Gabon: a country name after from/in/to/of/with/by, a curated demonym for Gabon, or a curated foreign city mapped to Gabon.

Code text for this tag

```
106: "Gabon",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the *next* place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). `resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*(?)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{[_PREPOSITIONS]})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{[_PLACE]}\s+({_PLACE})\b")

1683: # country:<NAME> - broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
```

```

1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:         country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:         if country and country.lower() in COUNTRY_LOWER:
1701:             add(f"country:{_normalize_country(country)}", span=m.span(1))

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Kazakhstan (2 (0.0%))

Precise plain-English rule

Emitted as country:Kazakhstan when a validated candidate resolves to Kazakhstan: a country name after from/in/to/of/with/by, a curated demonym for Kazakhstan, or a curated foreign city mapped to Kazakhstan.

Code text for this tag

```

130:     "Kazakhstan",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]+)?"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*",)
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE})\s+({_PLACE})\b")

1683:     # country:<NAME> - broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:
1687:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:     # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:     # country, which the bare-name validators above can't see. A demonym
1691:     # followed by a person noun also earns origin:.
1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:         country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:         if country and country.lower() in COUNTRY_LOWER:
1701:             add(f"country:{_normalize_country(country)}", span=m.span(1))

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Kyrgyzstan (2 (0.0%))

Precise plain-English rule

Emitted as country:Kyrgyzstan when a validated candidate resolves to Kyrgyzstan: a country name after from/in/to/of/with/by, a curated demonym for Kyrgyzstan, or a curated foreign city mapped to Kyrgyzstan.

Code text for this tag

```
135:     "Kyrgyzstan",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). `._resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1683:     # country:<NAME> - broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:
1687:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:     # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:     # country, which the bare-name validators above can't see. A demonym
1691:     # followed by a person noun also earns origin:.
1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:         country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:         if country and country.lower() in COUNTRY_LOWER:
1701:             add(f"country:{_normalize_country(country)}", span=m.span(1))

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token
```

country:Portugal (2 (0.0%))

Precise plain-English rule

Emitted as country:Portugal when a validated candidate resolves to Portugal: a country name after from/in/to/of/with/by, a curated demonym for Portugal, or a curated foreign city mapped to Portugal.

Code text for this tag

```
179:     "Portugal",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). `._resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1683:     # country:<NAME> - broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:
1687:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:     # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:     # country, which the bare-name validators above can't see. A demonym
1691:     # followed by a person noun also earns origin:.
1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:         country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
```

```

1700:         if country and country.lower() in COUNTRY_LOWER:
1701:             add(f"country:{_normalize_country(country)}", span=m.span(1))

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Romania (2 (0.0%))

Precise plain-English rule

Emitted as country:Romania when a validated candidate resolves to Romania: a country name after from/in/to/of/with/by, a curated demonym for Romania, or a curated foreign city mapped to Romania.

Code text for this tag

```

181:     "Romania",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). '_resolve_vocab'
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{[_PREPOSITIONS]})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE})\s+({_PLACE})\b")

1683:     # country:<NAME> - broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:
1687:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:     # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:     # country, which the bare-name validators above can't see. A demonym
1691:     # followed by a person noun also earns origin:.
1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:         country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:         if country and country.lower() in COUNTRY_LOWER:
1701:             add(f"country:{_normalize_country(country)}", span=m.span(1))

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:South-Sudan (2 (0.0%))

Precise plain-English rule

Emitted as country:South-Sudan when a validated candidate resolves to South Sudan: a country name after from/in/to/of/with/by, a curated demonym for South Sudan, or a curated foreign city mapped to South Sudan.

Code text for this tag

```

195:     "South Sudan",

1217: # second word excludes those same prepositions via lookahead, so a greedy

```

```

1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). '_resolve_vocab'
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*", "")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{[_PREPOSITIONS]})\s+({_PLACE})\b")
1229: # "<Place>, <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{[_PLACE]}\s+({_PLACE})\b")

1683: # country:<NAME> - broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:     country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:     if country and country.lower() in COUNTRY_LOWER:
1701:         add(f"country:{_normalize_country(country)}", span=m.span(1))

2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
2052: if low == "burma":
2053:     return "Myanmar"
2054: return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:United-Kingdom (2 (0.0%))

Precise plain-English rule

Emitted as country:United-Kingdom when a validated candidate resolves to United Kingdom: a country name after from/in/to/of/with/by, a curated demonym for United Kingdom, or a curated foreign city mapped to United Kingdom.

Code text for this tag

```

218:     "United Kingdom",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). '_resolve_vocab'
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*", "")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{[_PREPOSITIONS]})\s+({_PLACE})\b")
1229: # "<Place>, <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{[_PLACE]}\s+({_PLACE})\b")

1683: # country:<NAME> - broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:     country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:     if country and country.lower() in COUNTRY_LOWER:
1701:         add(f"country:{_normalize_country(country)}", span=m.span(1))

2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"

```

```

2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Uruguay (2 (0.0%))

Precise plain-English rule

Emitted as country:Uruguay when a validated candidate resolves to Uruguay: a country name after from/in/to/of/with/by, a curated demonym for Uruguay, or a curated foreign city mapped to Uruguay.

Code text for this tag

```

222:     "Uruguay",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!(?:{_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]+)?"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE})\s+({_PLACE})\b")

1683: # country:<NAME> - broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:     country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:     if country and country.lower() in COUNTRY_LOWER:
1701:         add(f"country:{_normalize_country(country)}", span=m.span(1))

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Yemen (2 (0.0%))

Precise plain-English rule

Emitted as country:Yemen when a validated candidate resolves to Yemen: a country name after from/in/to/of/with/by, a curated demonym for Yemen, or a curated foreign city mapped to Yemen.

Code text for this tag

```

227:     "Yemen",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!(?:{_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]+)?"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")

```

```

1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{:_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>, <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1683: # country:<NAME> – broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:     country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:     if country and country.lower() in COUNTRY_LOWER:
1701:         add(f"country:{_normalize_country(country)}", span=m.span(1))

2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
2052: if low == "burma":
2053:     return "Myanmar"
2054: return s.replace(" ", ".")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", ".")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Algeria (1 (0.0%))

Precise plain-English rule

Emitted as country:Algeria when a validated candidate resolves to Algeria: a country name after from/in/to/of/with/by, a curated demonym for Algeria, or a curated foreign city mapped to Algeria.

Code text for this tag

```

52:     "Algeria",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?:{:_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]*(?)"
1225: # "from <country>," – anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:{:_PREPOSITIONS})\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{:_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>, <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1683: # country:<NAME> – broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:     country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:     if country and country.lower() in COUNTRY_LOWER:
1701:         add(f"country:{_normalize_country(country)}", span=m.span(1))

2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
2052: if low == "burma":
2053:     return "Myanmar"
2054: return s.replace(" ", ".")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", ".")
2059:
2060:

```

```

2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Belarus (1 (0.0%))

Precise plain-English rule

Emitted as country:Belarus when a validated candidate resolves to Belarus: a country name after from/in/to/of/with/by, a curated demonym for Belarus, or a curated foreign city mapped to Belarus.

Code text for this tag

```

64:     "Belarus",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*",")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE}),\s+({_PLACE})\b")

1683:     # country:<NAME> - broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:
1687:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:     # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:     # country, which the bare-name validators above can't see. A demonym
1691:     # followed by a person noun also earns origin:.
1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:         country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:         if country and country.lower() in COUNTRY_LOWER:
1701:             add(f"country:{_normalize_country(country)}", span=m.span(1))

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Belgium (1 (0.0%))

Precise plain-English rule

Emitted as country:Belgium when a validated candidate resolves to Belgium: a country name after from/in/to/of/with/by, a curated demonym for Belgium, or a curated foreign city mapped to Belgium.

Code text for this tag

```

65:     "Belgium",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*",")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE}),\s+({_PLACE})\b")

1683:     # country:<NAME> - broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:

```

```

1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:         # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:         # country, which the bare-name validators above can't see. A demonym
1691:         # followed by a person noun also earns origin:.
1692:         for m in PATTERN_DEMONYM.finditer(text):
1693:             country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:             if country and country.lower() in COUNTRY_LOWER:
1695:                 add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:                 if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                     add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:         for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:             country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:             if country and country.lower() in COUNTRY_LOWER:
1701:                 add(f"country:{_normalize_country(country)}", span=m.span(1))
1702:
2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Belize (1 (0.0%))

Precise plain-English rule

Emitted as country:Belize when a validated candidate resolves to Belize: a country name after from/in/to/of/with/by, a curated demonym for Belize, or a curated foreign city mapped to Belize.

Code text for this tag

```

66:     "Belize",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!{:_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]*(?)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{:_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE}\s+({_PLACE})\b")

1683:     # country:<NAME> - broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:
1687:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:         # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:         # country, which the bare-name validators above can't see. A demonym
1691:         # followed by a person noun also earns origin:.
1692:         for m in PATTERN_DEMONYM.finditer(text):
1693:             country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:             if country and country.lower() in COUNTRY_LOWER:
1695:                 add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:                 if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                     add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:         for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:             country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:             if country and country.lower() in COUNTRY_LOWER:
1701:                 add(f"country:{_normalize_country(country)}", span=m.span(1))
1702:
2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Botswana (1 (0.0%))

Precise plain-English rule

Emitted as country:Botswana when a validated candidate resolves to Botswana: a country name after from/in/to/of/with/by, a curated demonym for Botswana, or a curated foreign city mapped to Botswana.

Code text for this tag

```
71:     "Botswana",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). `_resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!{:_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*",)
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{:_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1683: # country:<NAME> - broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:     country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:     if country and country.lower() in COUNTRY_LOWER:
1701:         add(f"country:{_normalize_country(country)}", span=m.span(1))

2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
2052: if low == "burma":
2053:     return "Myanmar"
2054: return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token
```

country:Chad (1 (0.0%))

Precise plain-English rule

Emitted as country:Chad when a validated candidate resolves to Chad: a country name after from/in/to/of/with/by, a curated demonym for Chad, or a curated foreign city mapped to Chad.

Code text for this tag

```
80:     "Chad",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). `_resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!{:_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*",)
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{:_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1683: # country:<NAME> - broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
```

```

1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:         country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:         if country and country.lower() in COUNTRY_LOWER:
1701:             add(f"country:{_normalize_country(country)}", span=m.span(1))

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", ".")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", ".")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Congo (1 (0.0%))

Precise plain-English rule

Emitted as country:Congo when a validated candidate resolves to Congo: a country name after from/in/to/of/with/by, a curated demonym for Congo, or a curated foreign city mapped to Congo.

Code text for this tag

```

85:     "Congo",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the *next* place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]{?:(?!(?:{_PREPOSITIONS}))\b}[A-Za-z][a-zA-Z]{+}"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"b(?:{_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"b({_PLACE})\s+({_PLACE})\b")

1683:     # country:<NAME> - broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:
1687:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:     # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:     # country, which the bare-name validators above can't see. A demonym
1691:     # followed by a person noun also earns origin:.
1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:         country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:         if country and country.lower() in COUNTRY_LOWER:
1701:             add(f"country:{_normalize_country(country)}", span=m.span(1))

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", ".")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", ".")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Djibouti (1 (0.0%))

Precise plain-English rule

Emitted as country:Djibouti when a validated candidate resolves to Djibouti: a country name after from/in/to/of/with/by, a curated demonym for Djibouti, or a curated foreign city mapped to Djibouti.

Code text for this tag

```
93:     "Djibouti",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the *next* place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). `_resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{[_PREPOSITIONS]})\s+({_PLACE})\b")
1229: # "<Place>,<State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1683: # country:<NAME> - broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:     country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:     if country and country.lower() in COUNTRY_LOWER:
1701:         add(f"country:{_normalize_country(country)}", span=m.span(1))

2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
2052: if low == "burma":
2053:     return "Myanmar"
2054: return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token
```

country:Dominica (1 (0.0%))

Precise plain-English rule

Emitted as country:Dominica when a validated candidate resolves to Dominica: a country name after from/in/to/of/with/by, a curated demonym for Dominica, or a curated foreign city mapped to Dominica.

Code text for this tag

```
94:     "Dominica",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the *next* place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). `_resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{[_PREPOSITIONS]})\s+({_PLACE})\b")
1229: # "<Place>,<State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1683: # country:<NAME> - broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:     country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:     if country and country.lower() in COUNTRY_LOWER:
1701:         add(f"country:{_normalize_country(country)}", span=m.span(1))

2046: if low in ("usa", "united states"):
```

```

2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Iceland (1 (0.0%))

Precise plain-English rule

Emitted as country:Iceland when a validated candidate resolves to Iceland: a country name after from/in/to/of/with/by, a curated demonym for Iceland, or a curated foreign city mapped to Iceland.

Code text for this tag

```

119:     "Iceland",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*(?)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:{_PREPOSITIONS})\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE}\s+({_PLACE})\b")

1683:     # country:<NAME> - broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:
1687:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:     # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:     # country, which the bare-name validators above can't see. A demonym
1691:     # followed by a person noun also earns origin:.
1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:         country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:         if country and country.lower() in COUNTRY_LOWER:
1701:             add(f"country:{_normalize_country(country)}", span=m.span(1))

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Latvia (1 (0.0%))

Precise plain-English rule

Emitted as country:Latvia when a validated candidate resolves to Latvia: a country name after from/in/to/of/with/by, a curated demonym for Latvia, or a curated foreign city mapped to Latvia.

Code text for this tag

```

137:     "Latvia",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names

```

```

1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!{:_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{:_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1683: # country:<NAME> - broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:     country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:     if country and country.lower() in COUNTRY_LOWER:
1701:         add(f"country:{_normalize_country(country)}", span=m.span(1))

2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
2052: if low == "burma":
2053:     return "Myanmar"
2054: return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Mauritania (1 (0.0%))

Precise plain-English rule

Emitted as country:Mauritania when a validated candidate resolves to Mauritania: a country name after from/in/to/of/with/by, a curated demonym for Mauritania, or a curated foreign city mapped to Mauritania.

Code text for this tag

```

151:     "Mauritania",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the *next* place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!{:_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{:_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1683: # country:<NAME> - broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:     country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:     if country and country.lower() in COUNTRY_LOWER:
1701:         add(f"country:{_normalize_country(country)}", span=m.span(1))

2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
2052: if low == "burma":
2053:     return "Myanmar"
2054: return s.replace(" ", "-")
2055:

```

```

2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Mauritius (1 (0.0%))

Precise plain-English rule

Emitted as country:Mauritius when a validated candidate resolves to Mauritius: a country name after from/in/to/of/with/by, a curated demonym for Mauritius, or a curated foreign city mapped to Mauritius.

Code text for this tag

```

152:     "Mauritius",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the *next* place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*", "")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:({_PREPOSITIONS}))\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE}),\s+({_PLACE})\b")

1683:     # country:<NAME> - broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:
1687:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:     # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:     # country, which the bare-name validators above can't see. A demonym
1691:     # followed by a person noun also earns origin:.
1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:         country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:         if country and country.lower() in COUNTRY_LOWER:
1701:             add(f"country:{_normalize_country(country)}", span=m.span(1))

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Montenegro (1 (0.0%))

Precise plain-English rule

Emitted as country:Montenegro when a validated candidate resolves to Montenegro: a country name after from/in/to/of/with/by, a curated demonym for Montenegro, or a curated foreign city mapped to Montenegro.

Code text for this tag

```

157:     "Montenegro",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the *next* place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*", "")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:({_PREPOSITIONS}))\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE}),\s+({_PLACE})\b")

```

```

1683: # country:<NAME> – broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:     country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:     if country and country.lower() in COUNTRY_LOWER:
1701:         add(f"country:{_normalize_country(country)}", span=m.span(1))

2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
2052: if low == "burma":
2053:     return "Myanmar"
2054: return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Namibia (1 (0.0%))

Precise plain-English rule

Emitted as country:Namibia when a validated candidate resolves to Namibia: a country name after from/in/to/of/with/by, a curated demonym for Namibia, or a curated foreign city mapped to Namibia.

Code text for this tag

```

162:     "Namibia",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the *next* place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). ` _resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," – anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{ _PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE}\s+({_PLACE})\b")

1683: # country:<NAME> – broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:     country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:     if country and country.lower() in COUNTRY_LOWER:
1701:         add(f"country:{_normalize_country(country)}", span=m.span(1))

2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
2052: if low == "burma":
2053:     return "Myanmar"
2054: return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy

```

2066: case-insensitive capture that swallowed a trailing lowercase token

country:Poland (1 (0.0%))

Precise plain-English rule

Emitted as country:Poland when a validated candidate resolves to Poland: a country name after from/in/to/of/with/by, a curated demonym for Poland, or a curated foreign city mapped to Poland.

Code text for this tag

```
178: "Poland",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the *next* place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). `resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!{:_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{:_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1683: # country:<NAME> - broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:     country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:     if country and country.lower() in COUNTRY_LOWER:
1701:         add(f"country:{_normalize_country(country)}", span=m.span(1))

2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
2052: if low == "burma":
2053:     return "Myanmar"
2054: return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token
```

country:Togo (1 (0.0%))

Precise plain-English rule

Emitted as country:Togo when a validated candidate resolves to Togo: a country name after from/in/to/of/with/by, a curated demonym for Togo, or a curated foreign city mapped to Togo.

Code text for this tag

```
207: "Togo",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the *next* place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). `resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!{:_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{:_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1683: # country:<NAME> - broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
```

```

1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:         country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:         if country and country.lower() in COUNTRY_LOWER:
1701:             add(f"country:{_normalize_country(country)}", span=m.span(1))

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:Tonga (1 (0.0%))

Precise plain-English rule

Emitted as country:Tonga when a validated candidate resolves to Tonga: a country name after from/in/to/of/with/by, a curated demonym for Tonga, or a curated foreign city mapped to Tonga.

Code text for this tag

```

208:     "Tonga",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't't swallow the preposition that introduces the 'next' place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]+)?"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*",)
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE})\s+({_PLACE})\b")

1683: # country:<NAME> - broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:         country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:         if country and country.lower() in COUNTRY_LOWER:
1701:             add(f"country:{_normalize_country(country)}", span=m.span(1))

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

country:United-Arab-Emirates (1 (0.0%))

Precise plain-English rule

Emitted as country:United-Arab-Emirates when a validated candidate resolves to United Arab Emirates: a country name after from/in/to/of/with/by, a curated demonym for United Arab Emirates, or a curated foreign city mapped to United Arab Emirates.

Code text for this tag

```
216:     "United Arab Emirates",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). `_resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]+)?"
1225: # "from <Country>" - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*", "")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>, <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE}),\s+({_PLACE})\b")

1683: # country:<NAME> - broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:     country = FOREIGN_CITY_TO_COUNTRY.get(m.group(1).lower())
1700:     if country and country.lower() in COUNTRY_LOWER:
1701:         add(f"country:{_normalize_country(country)}", span=m.span(1))

2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
2052: if low == "burma":
2053:     return "Myanmar"
2054: return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token
```

crime

The crime namespace is produced by iterating CRIME_VOCAB and CRIME_SUBTYPE_VOCAB. Each regex match emits one crime tag. Some matches also emit parent tags through CRIME_PARENT_TAGS, such as rape -> sexual, murder -> homicide, fentanyl -> narcotics.

crime:assault (911 (9.1%))

Precise plain-English rule

Matches the assault entry in CRIME_VOCAB/CRIME_SUBTYPE_VOCAB. If this tag has a configured parent in CRIME_PARENT_TAGS, the parent is emitted from the same text span.

Code text for this tag

```
297:     ("assault", rf"\bassault(?:ed|ing|s)?\b"),

1638: # crime:<TYPE> - every distinct match emits one tag entry.
1639: for slug, pat_str in CRIME_VOCAB:
1640:     for m in re.finditer(pat_str, text, re.I):
1641:         add(f"crime:{slug}", span=m.span())
1642:         for parent in CRIME_PARENT_TAGS.get(f"crime:{slug}", ()):
1643:             add(parent, span=m.span())
1644: for slug, pat_str in CRIME_SUBTYPE_VOCAB:
1645:     for m in re.finditer(pat_str, text, re.I):
1646:         tag = f"crime:{slug}"
1647:         add(tag, span=m.span())
1648:         for parent in CRIME_PARENT_TAGS.get(tag, ()):
1649:             add(parent, span=m.span())
1650:
1651: if m := _martyrdom_match(text, account_category, entries):
1652:     add("theme:martyrdom", span=m.span())
1653:
```

crime:homicide (737 (7.4%))

Precise plain-English rule

Matches the homicide entry in CRIME_VOCAB/CRIME_SUBTYPE_VOCAB. If this tag has a configured parent in CRIME_PARENT_TAGS, the parent is emitted from the same text span.

Code text for this tag

```
293:      ("homicide", r"\bhomicides?\b"),

1638:  # crime:<TYPE> - every distinct match emits one tag entry.
1639:  for slug, pat_str in CRIME_VOCAB:
1640:      for m in re.finditer(pat_str, text, re.I):
1641:          add(f"crime:{slug}", span=m.span())
1642:          for parent in CRIME_PARENT_TAGS.get(f"crime:{slug}", ()):
1643:              add(parent, span=m.span())
1644:  for slug, pat_str in CRIME_SUBTYPE_VOCAB:
1645:      for m in re.finditer(pat_str, text, re.I):
1646:          tag = f"crime:{slug}"
1647:          add(tag, span=m.span())
1648:          for parent in CRIME_PARENT_TAGS.get(tag, ()):
1649:              add(parent, span=m.span())
1650:
1651:  if m := _martyrdom_match(text, account_category, entries):
1652:      add("theme:martyrdom", span=m.span())
1653:
```

crime:sexual (619 (6.2%))

Precise plain-English rule

Matches the sexual entry in CRIME_VOCAB/CRIME_SUBTYPE_VOCAB. If this tag has a configured parent in CRIME_PARENT_TAGS, the parent is emitted from the same text span.

Code text for this tag

```
305:      ("sexual", r"\b(?:sexual\s+assault|sex\s+crime|sex\s+offen[sc]e|sex\s+abuse)\b"),

333:  "crime:child-sexual": ("crime:sexual",),
334:  "crime:perjury": ("crime:disobedience",),

1638:  # crime:<TYPE> - every distinct match emits one tag entry.
1639:  for slug, pat_str in CRIME_VOCAB:
1640:      for m in re.finditer(pat_str, text, re.I):
1641:          add(f"crime:{slug}", span=m.span())
1642:          for parent in CRIME_PARENT_TAGS.get(f"crime:{slug}", ()):
1643:              add(parent, span=m.span())
1644:  for slug, pat_str in CRIME_SUBTYPE_VOCAB:
1645:      for m in re.finditer(pat_str, text, re.I):
1646:          tag = f"crime:{slug}"
1647:          add(tag, span=m.span())
1648:          for parent in CRIME_PARENT_TAGS.get(tag, ()):
1649:              add(parent, span=m.span())
1650:
1651:  if m := _martyrdom_match(text, account_category, entries):
1652:      add("theme:martyrdom", span=m.span())
1653:
```

crime:murder (614 (6.1%))

Precise plain-English rule

Matches the murder entry in CRIME_VOCAB/CRIME_SUBTYPE_VOCAB. If this tag has a configured parent in CRIME_PARENT_TAGS, the parent is emitted from the same text span.

Code text for this tag

```
327: CRIME_SUBTYPE_VOCAB: tuple[tuple[str, str], ...] = (("murder", r"\bmurder(?:s|ed|ers?|ing)?\b"),)

1638:  # crime:<TYPE> - every distinct match emits one tag entry.
1639:  for slug, pat_str in CRIME_VOCAB:
1640:      for m in re.finditer(pat_str, text, re.I):
1641:          add(f"crime:{slug}", span=m.span())
1642:          for parent in CRIME_PARENT_TAGS.get(f"crime:{slug}", ()):
1643:              add(parent, span=m.span())
1644:  for slug, pat_str in CRIME_SUBTYPE_VOCAB:
1645:      for m in re.finditer(pat_str, text, re.I):
1646:          tag = f"crime:{slug}"
1647:          add(tag, span=m.span())
1648:          for parent in CRIME_PARENT_TAGS.get(tag, ()):
1649:              add(parent, span=m.span())
1650:
1651:  if m := _martyrdom_match(text, account_category, entries):
1652:      add("theme:martyrdom", span=m.span())
1653:
```

crime:narcotics (516 (5.2%))

Precise plain-English rule

Matches the narcotics entry in CRIME_VOCAB/CRIME_SUBTYPE_VOCAB. If this tag has a configured parent in CRIME_PARENT_TAGS, the parent is emitted from the same text span.

Code text for this tag

```
315:      ("narcotics", r"\bnarcotic[s]?\b"),

335:  "crime:fentanyl": ("crime:narcotics",),
336:  "crime:cocaine": ("crime:narcotics",),

1638:  # crime:<TYPE> - every distinct match emits one tag entry.
1639:  for slug, pat_str in CRIME_VOCAB:
1640:      for m in re.finditer(pat_str, text, re.I):
1641:          add(f"crime:{slug}", span=m.span())
1642:          for parent in CRIME_PARENT_TAGS.get(f"crime:{slug}", ()):
1643:              add(parent, span=m.span())
```

```

1643:         add(parent, span=m.span())
1644:     for slug, pat_str in CRIME_SUBTYPE_VOCAB:
1645:         for m in re.finditer(pat_str, text, re.I):
1646:             tag = f"crime:{slug}"
1647:             add(tag, span=m.span())
1648:             for parent in CRIME_PARENT_TAGS.get(tag, ()):
1649:                 add(parent, span=m.span())
1650:
1651:     if m := _martyrdom_match(text, account_category, entries):
1652:         add("theme:martyrdom", span=m.span())
1653:

```

crime:trafficking (405 (4.0%))

Precise plain-English rule

Matches the trafficking entry in CRIME_VOCAB/CRIME_SUBTYPE_VOCAB. If this tag has a configured parent in CRIME_PARENT_TAGS, the parent is emitted from the same text span.

Code text for this tag

```

302:     ("trafficking", r"\btraffick(?:ing|er|ers)\b"),

1638:     # crime:<TYPE> — every distinct match emits one tag entry.
1639:     for slug, pat_str in CRIME_VOCAB:
1640:         for m in re.finditer(pat_str, text, re.I):
1641:             add(f"crime:{slug}", span=m.span())
1642:             for parent in CRIME_PARENT_TAGS.get(f"crime:{slug}", ()):
1643:                 add(parent, span=m.span())
1644:     for slug, pat_str in CRIME_SUBTYPE_VOCAB:
1645:         for m in re.finditer(pat_str, text, re.I):
1646:             tag = f"crime:{slug}"
1647:             add(tag, span=m.span())
1648:             for parent in CRIME_PARENT_TAGS.get(tag, ()):
1649:                 add(parent, span=m.span())
1650:
1651:     if m := _martyrdom_match(text, account_category, entries):
1652:         add("theme:martyrdom", span=m.span())
1653:

```

crime:weapon (377 (3.8%))

Precise plain-English rule

Matches the weapon entry in CRIME_VOCAB/CRIME_SUBTYPE_VOCAB. If this tag has a configured parent in CRIME_PARENT_TAGS, the parent is emitted from the same text span.

Code text for this tag

```

323:     ("weapon", r"\bweapon[s]?\\b|\\billegal firearm[s]?\\b"),

1638:     # crime:<TYPE> — every distinct match emits one tag entry.
1639:     for slug, pat_str in CRIME_VOCAB:
1640:         for m in re.finditer(pat_str, text, re.I):
1641:             add(f"crime:{slug}", span=m.span())
1642:             for parent in CRIME_PARENT_TAGS.get(f"crime:{slug}", ()):
1643:                 add(parent, span=m.span())
1644:     for slug, pat_str in CRIME_SUBTYPE_VOCAB:
1645:         for m in re.finditer(pat_str, text, re.I):
1646:             tag = f"crime:{slug}"
1647:             add(tag, span=m.span())
1648:             for parent in CRIME_PARENT_TAGS.get(tag, ()):
1649:                 add(parent, span=m.span())
1650:
1651:     if m := _martyrdom_match(text, account_category, entries):
1652:         add("theme:martyrdom", span=m.span())
1653:

```

crime:gang (337 (3.4%))

Precise plain-English rule

Matches the gang entry in CRIME_VOCAB/CRIME_SUBTYPE_VOCAB. If this tag has a configured parent in CRIME_PARENT_TAGS, the parent is emitted from the same text span.

Code text for this tag

```

312:     ("gang", r"\bgang(?:member|members|affiliation)\\b|\\bgang-related\\b"),

1638:     # crime:<TYPE> — every distinct match emits one tag entry.
1639:     for slug, pat_str in CRIME_VOCAB:
1640:         for m in re.finditer(pat_str, text, re.I):
1641:             add(f"crime:{slug}", span=m.span())
1642:             for parent in CRIME_PARENT_TAGS.get(f"crime:{slug}", ()):
1643:                 add(parent, span=m.span())
1644:     for slug, pat_str in CRIME_SUBTYPE_VOCAB:
1645:         for m in re.finditer(pat_str, text, re.I):
1646:             tag = f"crime:{slug}"
1647:             add(tag, span=m.span())
1648:             for parent in CRIME_PARENT_TAGS.get(tag, ()):
1649:                 add(parent, span=m.span())
1650:
1651:     if m := _martyrdom_match(text, account_category, entries):
1652:         add("theme:martyrdom", span=m.span())
1653:

```

crime:rape (276 (2.8%))

Precise plain-English rule

Matches the rape entry in CRIME_VOCAB/CRIME_SUBTYPE_VOCAB. If this tag has a configured parent in CRIME_PARENT_TAGS, the parent is emitted from the same text span.

Code text for this tag

```
291:      ("rape", r"\brap(?:e|ed|ing|ist)\b"),

1638:  # crime:<TYPE> – every distinct match emits one tag entry.
1639:  for slug, pat_str in CRIME_VOCAB:
1640:      for m in re.finditer(pat_str, text, re.I):
1641:          add(f"crime:{slug}", span=m.span())
1642:          for parent in CRIME_PARENT_TAGS.get(f"crime:{slug}", ()):
1643:              add(parent, span=m.span())
1644:  for slug, pat_str in CRIME_SUBTYPE_VOCAB:
1645:      for m in re.finditer(pat_str, text, re.I):
1646:          tag = f"crime:{slug}"
1647:          add(tag, span=m.span())
1648:          for parent in CRIME_PARENT_TAGS.get(tag, ()):
1649:              add(parent, span=m.span())
1650:
1651:  if m := _martyrdom_match(text, account_category, entries):
1652:      add("theme:martyrdom", span=m.span())
1653:
```

crime:child (256 (2.6%))

Precise plain-English rule

Matches the child entry in CRIME_VOCAB/CRIME_SUBTYPE_VOCAB. If this tag has a configured parent in CRIME_PARENT_TAGS, the parent is emitted from the same text span.

Code text for this tag

```
311:      ("child", r"\bchild(?:abuse|sex|pornography|molest)|child sexual\b"),

1638:  # crime:<TYPE> – every distinct match emits one tag entry.
1639:  for slug, pat_str in CRIME_VOCAB:
1640:      for m in re.finditer(pat_str, text, re.I):
1641:          add(f"crime:{slug}", span=m.span())
1642:          for parent in CRIME_PARENT_TAGS.get(f"crime:{slug}", ()):
1643:              add(parent, span=m.span())
1644:  for slug, pat_str in CRIME_SUBTYPE_VOCAB:
1645:      for m in re.finditer(pat_str, text, re.I):
1646:          tag = f"crime:{slug}"
1647:          add(tag, span=m.span())
1648:          for parent in CRIME_PARENT_TAGS.get(tag, ()):
1649:              add(parent, span=m.span())
1650:
1651:  if m := _martyrdom_match(text, account_category, entries):
1652:      add("theme:martyrdom", span=m.span())
1653:
```

crime:fraud (226 (2.3%))

Precise plain-English rule

Matches the fraud entry in CRIME_VOCAB/CRIME_SUBTYPE_VOCAB. If this tag has a configured parent in CRIME_PARENT_TAGS, the parent is emitted from the same text span.

Code text for this tag

```
316:      ("fraud", r"\bfraud(?:ulent|ster)?\b"),

1638:  # crime:<TYPE> – every distinct match emits one tag entry.
1639:  for slug, pat_str in CRIME_VOCAB:
1640:      for m in re.finditer(pat_str, text, re.I):
1641:          add(f"crime:{slug}", span=m.span())
1642:          for parent in CRIME_PARENT_TAGS.get(f"crime:{slug}", ()):
1643:              add(parent, span=m.span())
1644:  for slug, pat_str in CRIME_SUBTYPE_VOCAB:
1645:      for m in re.finditer(pat_str, text, re.I):
1646:          tag = f"crime:{slug}"
1647:          add(tag, span=m.span())
1648:          for parent in CRIME_PARENT_TAGS.get(tag, ()):
1649:              add(parent, span=m.span())
1650:
1651:  if m := _martyrdom_match(text, account_category, entries):
1652:      add("theme:martyrdom", span=m.span())
1653:
```

crime:cocaine (222 (2.2%))

Precise plain-English rule

Matches the cocaine entry in CRIME_VOCAB/CRIME_SUBTYPE_VOCAB. If this tag has a configured parent in CRIME_PARENT_TAGS, the parent is emitted from the same text span.

Code text for this tag

```
300:      ("cocaine", r"\bcocaine\b"),

336:  "crime:cocaine": ("crime:narcotics",),
337:  "crime:meth": ("crime:narcotics",),

1638:  # crime:<TYPE> – every distinct match emits one tag entry.
1639:  for slug, pat_str in CRIME_VOCAB:
1640:      for m in re.finditer(pat_str, text, re.I):
1641:          add(f"crime:{slug}", span=m.span())
1642:          for parent in CRIME_PARENT_TAGS.get(f"crime:{slug}", ()):
1643:              add(parent, span=m.span())
1644:  for slug, pat_str in CRIME_SUBTYPE_VOCAB:
1645:      for m in re.finditer(pat_str, text, re.I):
```

```

1646:         tag = f"crime:{slug}"
1647:         add(tag, span=m.span())
1648:         for parent in CRIME_PARENT_TAGS.get(tag, ()):
1649:             add(parent, span=m.span())
1650:
1651:     if m := _martyrdom_match(text, account_category, entries):
1652:         add("theme:martyrdom", span=m.span())
1653:

```

crime:child-sexual (215 (2.1%))

Precise plain-English rule

Matches the child-sexual entry in CRIME_VOCAB/CRIME_SUBTYPE_VOCAB. If this tag has a configured parent in CRIME_PARENT_TAGS, the parent is emitted from the same text span.

Code text for this tag

```

333:     "crime:child-sexual": ("crime:sexual",),
334:     "crime:perjury": ("crime:disobedience",),
335:
1638: # crime:<TYPE> – every distinct match emits one tag entry.
1639: for slug, pat_str in CRIME_VOCAB:
1640:     for m in re.finditer(pat_str, text, re.I):
1641:         add(f"crime:{slug}", span=m.span())
1642:         for parent in CRIME_PARENT_TAGS.get(f"crime:{slug}", ()):
1643:             add(parent, span=m.span())
1644: for slug, pat_str in CRIME_SUBTYPE_VOCAB:
1645:     for m in re.finditer(pat_str, text, re.I):
1646:         tag = f"crime:{slug}"
1647:         add(tag, span=m.span())
1648:         for parent in CRIME_PARENT_TAGS.get(tag, ()):
1649:             add(parent, span=m.span())
1650:
1651: if m := _martyrdom_match(text, account_category, entries):
1652:     add("theme:martyrdom", span=m.span())
1653:

```

crime:battery (187 (1.9%))

Precise plain-English rule

Matches the battery entry in CRIME_VOCAB/CRIME_SUBTYPE_VOCAB. If this tag has a configured parent in CRIME_PARENT_TAGS, the parent is emitted from the same text span.

Code text for this tag

```

298:     ("battery", r"\bbatter(?:y|ies|ed)\b"),
299:
1638: # crime:<TYPE> – every distinct match emits one tag entry.
1639: for slug, pat_str in CRIME_VOCAB:
1640:     for m in re.finditer(pat_str, text, re.I):
1641:         add(f"crime:{slug}", span=m.span())
1642:         for parent in CRIME_PARENT_TAGS.get(f"crime:{slug}", ()):
1643:             add(parent, span=m.span())
1644: for slug, pat_str in CRIME_SUBTYPE_VOCAB:
1645:     for m in re.finditer(pat_str, text, re.I):
1646:         tag = f"crime:{slug}"
1647:         add(tag, span=m.span())
1648:         for parent in CRIME_PARENT_TAGS.get(tag, ()):
1649:             add(parent, span=m.span())
1650:
1651: if m := _martyrdom_match(text, account_category, entries):
1652:     add("theme:martyrdom", span=m.span())
1653:

```

crime:firearm (177 (1.8%))

Precise plain-English rule

Matches the firearm entry in CRIME_VOCAB/CRIME_SUBTYPE_VOCAB. If this tag has a configured parent in CRIME_PARENT_TAGS, the parent is emitted from the same text span.

Code text for this tag

```

324:     ("firearm", r"\bfirearm[s]?b"),
325:
1638: # crime:<TYPE> – every distinct match emits one tag entry.
1639: for slug, pat_str in CRIME_VOCAB:
1640:     for m in re.finditer(pat_str, text, re.I):
1641:         add(f"crime:{slug}", span=m.span())
1642:         for parent in CRIME_PARENT_TAGS.get(f"crime:{slug}", ()):
1643:             add(parent, span=m.span())
1644: for slug, pat_str in CRIME_SUBTYPE_VOCAB:
1645:     for m in re.finditer(pat_str, text, re.I):
1646:         tag = f"crime:{slug}"
1647:         add(tag, span=m.span())
1648:         for parent in CRIME_PARENT_TAGS.get(tag, ()):
1649:             add(parent, span=m.span())
1650:
1651: if m := _martyrdom_match(text, account_category, entries):
1652:     add("theme:martyrdom", span=m.span())
1653:

```

crime:theft (166 (1.7%))

Precise plain-English rule

Matches the theft entry in CRIME_VOCAB/CRIME_SUBTYPE_VOCAB. If this tag has a configured parent in CRIME_PARENT_TAGS, the parent is emitted from the same text span.

Code text for this tag

```
295:      ("theft", r"\btheft\b|\bstole(?:ing)?\b|\bstole\b"),

1638:  # crime:<TYPE> – every distinct match emits one tag entry.
1639:  for slug, pat_str in CRIME_VOCAB:
1640:      for m in re.finditer(pat_str, text, re.I):
1641:          add(f"crime:{slug}", span=m.span())
1642:          for parent in CRIME_PARENT_TAGS.get(f"crime:{slug}", ()):
1643:              add(parent, span=m.span())
1644:  for slug, pat_str in CRIME_SUBTYPE_VOCAB:
1645:      for m in re.finditer(pat_str, text, re.I):
1646:          tag = f"crime:{slug}"
1647:          add(tag, span=m.span())
1648:          for parent in CRIME_PARENT_TAGS.get(tag, ()):
1649:              add(parent, span=m.span())
1650:
1651:  if m := _martyrdom_match(text, account_category, entries):
1652:      add("theme:martyrdom", span=m.span())
1653:
```

crime:robbery (164 (1.6%))

Precise plain-English rule

Matches the robbery entry in CRIME_VOCAB/CRIME_SUBTYPE_VOCAB. If this tag has a configured parent in CRIME_PARENT_TAGS, the parent is emitted from the same text span.

Code text for this tag

```
296:      ("robbery", r"\brobber(?:y|ies)\b|\brobbled\b"),

1638:  # crime:<TYPE> – every distinct match emits one tag entry.
1639:  for slug, pat_str in CRIME_VOCAB:
1640:      for m in re.finditer(pat_str, text, re.I):
1641:          add(f"crime:{slug}", span=m.span())
1642:          for parent in CRIME_PARENT_TAGS.get(f"crime:{slug}", ()):
1643:              add(parent, span=m.span())
1644:  for slug, pat_str in CRIME_SUBTYPE_VOCAB:
1645:      for m in re.finditer(pat_str, text, re.I):
1646:          tag = f"crime:{slug}"
1647:          add(tag, span=m.span())
1648:          for parent in CRIME_PARENT_TAGS.get(tag, ()):
1649:              add(parent, span=m.span())
1650:
1651:  if m := _martyrdom_match(text, account_category, entries):
1652:      add("theme:martyrdom", span=m.span())
1653:
```

crime:fentanyl (156 (1.6%))

Precise plain-English rule

Matches the fentanyl entry in CRIME_VOCAB/CRIME_SUBTYPE_VOCAB. If this tag has a configured parent in CRIME_PARENT_TAGS, the parent is emitted from the same text span.

Code text for this tag

```
299:      ("fentanyl", r"\bfentanyl\b"),

335:  "crime:fentanyl": ("crime:narcotics",),
336:  "crime:cocaine": ("crime:narcotics",),

1638:  # crime:<TYPE> – every distinct match emits one tag entry.
1639:  for slug, pat_str in CRIME_VOCAB:
1640:      for m in re.finditer(pat_str, text, re.I):
1641:          add(f"crime:{slug}", span=m.span())
1642:          for parent in CRIME_PARENT_TAGS.get(f"crime:{slug}", ()):
1643:              add(parent, span=m.span())
1644:  for slug, pat_str in CRIME_SUBTYPE_VOCAB:
1645:      for m in re.finditer(pat_str, text, re.I):
1646:          tag = f"crime:{slug}"
1647:          add(tag, span=m.span())
1648:          for parent in CRIME_PARENT_TAGS.get(tag, ()):
1649:              add(parent, span=m.span())
1650:
1651:  if m := _martyrdom_match(text, account_category, entries):
1652:      add("theme:martyrdom", span=m.span())
1653:
```

crime:ms13 (156 (1.6%))

Precise plain-English rule

Matches the ms13 entry in CRIME_VOCAB/CRIME_SUBTYPE_VOCAB. If this tag has a configured parent in CRIME_PARENT_TAGS, the parent is emitted from the same text span.

Code text for this tag

```
313:      ("ms13", r"\bMS-?13\b"),

1638:  # crime:<TYPE> – every distinct match emits one tag entry.
1639:  for slug, pat_str in CRIME_VOCAB:
1640:      for m in re.finditer(pat_str, text, re.I):
1641:          add(f"crime:{slug}", span=m.span())
1642:          for parent in CRIME_PARENT_TAGS.get(f"crime:{slug}", ()):
1643:              add(parent, span=m.span())
1644:  for slug, pat_str in CRIME_SUBTYPE_VOCAB:
1645:      for m in re.finditer(pat_str, text, re.I):
```

```

1646:         tag = f"crime:{slug}"
1647:         add(tag, span=m.span())
1648:         for parent in CRIME_PARENT_TAGS.get(tag, ()):
1649:             add(parent, span=m.span())
1650:
1651:     if m := _martyrdom_match(text, account_category, entries):
1652:         add("theme:martyrdom", span=m.span())
1653:

```

crime:dui (148 (1.5%))

Precise plain-English rule

Matches the dui entry in CRIME_VOCAB/CRIME_SUBTYPE_VOCAB. If this tag has a configured parent in CRIME_PARENT_TAGS, the parent is emitted from the same text span.

Code text for this tag

```

303:     ("dui", r"\bDUI\b|\bDWI\b"),

1638: # crime:<TYPE> – every distinct match emits one tag entry.
1639: for slug, pat_str in CRIME_VOCAB:
1640:     for m in re.finditer(pat_str, text, re.I):
1641:         add(f"crime:{slug}", span=m.span())
1642:         for parent in CRIME_PARENT_TAGS.get(f"crime:{slug}", ()):
1643:             add(parent, span=m.span())
1644: for slug, pat_str in CRIME_SUBTYPE_VOCAB:
1645:     for m in re.finditer(pat_str, text, re.I):
1646:         tag = f"crime:{slug}"
1647:         add(tag, span=m.span())
1648:         for parent in CRIME_PARENT_TAGS.get(tag, ()):
1649:             add(parent, span=m.span())
1650:
1651: if m := _martyrdom_match(text, account_category, entries):
1652:     add("theme:martyrdom", span=m.span())
1653:

```

crime:burglary (132 (1.3%))

Precise plain-English rule

Matches the burglary entry in CRIME_VOCAB/CRIME_SUBTYPE_VOCAB. If this tag has a configured parent in CRIME_PARENT_TAGS, the parent is emitted from the same text span.

Code text for this tag

```

294:     ("burglary", r"\bburglar(?:y|ize|ized|ies)\b"),

1638: # crime:<TYPE> – every distinct match emits one tag entry.
1639: for slug, pat_str in CRIME_VOCAB:
1640:     for m in re.finditer(pat_str, text, re.I):
1641:         add(f"crime:{slug}", span=m.span())
1642:         for parent in CRIME_PARENT_TAGS.get(f"crime:{slug}", ()):
1643:             add(parent, span=m.span())
1644: for slug, pat_str in CRIME_SUBTYPE_VOCAB:
1645:     for m in re.finditer(pat_str, text, re.I):
1646:         tag = f"crime:{slug}"
1647:         add(tag, span=m.span())
1648:         for parent in CRIME_PARENT_TAGS.get(tag, ()):
1649:             add(parent, span=m.span())
1650:
1651: if m := _martyrdom_match(text, account_category, entries):
1652:     add("theme:martyrdom", span=m.span())
1653:

```

crime:meth (132 (1.3%))

Precise plain-English rule

Matches the meth entry in CRIME_VOCAB/CRIME_SUBTYPE_VOCAB. If this tag has a configured parent in CRIME_PARENT_TAGS, the parent is emitted from the same text span.

Code text for this tag

```

301:     ("meth", r"\bmethamphetamine\b|\bmeth\b"),

337:     "crime:meth": ("crime:narcotics",),
338: }

1638: # crime:<TYPE> – every distinct match emits one tag entry.
1639: for slug, pat_str in CRIME_VOCAB:
1640:     for m in re.finditer(pat_str, text, re.I):
1641:         add(f"crime:{slug}", span=m.span())
1642:         for parent in CRIME_PARENT_TAGS.get(f"crime:{slug}", ()):
1643:             add(parent, span=m.span())
1644: for slug, pat_str in CRIME_SUBTYPE_VOCAB:
1645:     for m in re.finditer(pat_str, text, re.I):
1646:         tag = f"crime:{slug}"
1647:         add(tag, span=m.span())
1648:         for parent in CRIME_PARENT_TAGS.get(tag, ()):
1649:             add(parent, span=m.span())
1650:
1651: if m := _martyrdom_match(text, account_category, entries):
1652:     add("theme:martyrdom", span=m.span())
1653:

```

crime:kidnap (123 (1.2%))

Precise plain-English rule

Matches the kidnap entry in CRIME_VOCAB/CRIME_SUBTYPE_VOCAB. If this tag has a configured parent in CRIME_PARENT_TAGS, the parent is emitted from the same text span.

Code text for this tag

```
304:      ("kidnap", r"\bkidnap(?:ped|ping|per)?\b"),

1638:  # crime:<TYPE> – every distinct match emits one tag entry.
1639:  for slug, pat_str in CRIME_VOCAB:
1640:      for m in re.finditer(pat_str, text, re.I):
1641:          add(f"crime:{slug}", span=m.span())
1642:          for parent in CRIME_PARENT_TAGS.get(f"crime:{slug}", ()):
1643:              add(parent, span=m.span())
1644:  for slug, pat_str in CRIME_SUBTYPE_VOCAB:
1645:      for m in re.finditer(pat_str, text, re.I):
1646:          tag = f"crime:{slug}"
1647:          add(tag, span=m.span())
1648:          for parent in CRIME_PARENT_TAGS.get(tag, ()):
1649:              add(parent, span=m.span())
1650:
1651:  if m := _martyrdom_match(text, account_category, entries):
1652:      add("theme:martyrdom", span=m.span())
1653:
```

crime:tren-de-aragua (117 (1.2%))

Precise plain-English rule

Matches the tren-de-aragua entry in CRIME_VOCAB/CRIME_SUBTYPE_VOCAB. If this tag has a configured parent in CRIME_PARENT_TAGS, the parent is emitted from the same text span.

Code text for this tag

```
314:      ("tren-de-aragua", r"\bTren de Aragua\b\bTdA\b"),

1638:  # crime:<TYPE> – every distinct match emits one tag entry.
1639:  for slug, pat_str in CRIME_VOCAB:
1640:      for m in re.finditer(pat_str, text, re.I):
1641:          add(f"crime:{slug}", span=m.span())
1642:          for parent in CRIME_PARENT_TAGS.get(f"crime:{slug}", ()):
1643:              add(parent, span=m.span())
1644:  for slug, pat_str in CRIME_SUBTYPE_VOCAB:
1645:      for m in re.finditer(pat_str, text, re.I):
1646:          tag = f"crime:{slug}"
1647:          add(tag, span=m.span())
1648:          for parent in CRIME_PARENT_TAGS.get(tag, ()):
1649:              add(parent, span=m.span())
1650:
1651:  if m := _martyrdom_match(text, account_category, entries):
1652:      add("theme:martyrdom", span=m.span())
1653:
```

crime:arson (29 (0.3%))

Precise plain-English rule

Matches the arson entry in CRIME_VOCAB/CRIME_SUBTYPE_VOCAB. If this tag has a configured parent in CRIME_PARENT_TAGS, the parent is emitted from the same text span.

Code text for this tag

```
322:      ("arson", r"\barson(?:ist)?\b"),

1638:  # crime:<TYPE> – every distinct match emits one tag entry.
1639:  for slug, pat_str in CRIME_VOCAB:
1640:      for m in re.finditer(pat_str, text, re.I):
1641:          add(f"crime:{slug}", span=m.span())
1642:          for parent in CRIME_PARENT_TAGS.get(f"crime:{slug}", ()):
1643:              add(parent, span=m.span())
1644:  for slug, pat_str in CRIME_SUBTYPE_VOCAB:
1645:      for m in re.finditer(pat_str, text, re.I):
1646:          tag = f"crime:{slug}"
1647:          add(tag, span=m.span())
1648:          for parent in CRIME_PARENT_TAGS.get(tag, ()):
1649:              add(parent, span=m.span())
1650:
1651:  if m := _martyrdom_match(text, account_category, entries):
1652:      add("theme:martyrdom", span=m.span())
1653:
```

crime:sodomy (23 (0.2%))

Precise plain-English rule

Matches the sodomy entry in CRIME_VOCAB/CRIME_SUBTYPE_VOCAB. If this tag has a configured parent in CRIME_PARENT_TAGS, the parent is emitted from the same text span.

Code text for this tag

```
292:      ("sodomy", r"\bsodom(?:y|ize|ized)\b"),

1638:  # crime:<TYPE> – every distinct match emits one tag entry.
1639:  for slug, pat_str in CRIME_VOCAB:
1640:      for m in re.finditer(pat_str, text, re.I):
1641:          add(f"crime:{slug}", span=m.span())
1642:          for parent in CRIME_PARENT_TAGS.get(f"crime:{slug}", ()):
1643:              add(parent, span=m.span())
1644:  for slug, pat_str in CRIME_SUBTYPE_VOCAB:
1645:      for m in re.finditer(pat_str, text, re.I):
1646:          tag = f"crime:{slug}"
1647:          add(tag, span=m.span())
1648:          for parent in CRIME_PARENT_TAGS.get(tag, ()):
1649:              add(parent, span=m.span())
```

```

1649:         add(parent, span=m.span())
1650:
1651:     if m := _martyrdom_match(text, account_category, entries):
1652:         add("theme:martyrdom", span=m.span())
1653:

```

crime:disobedience (8 (0.1%))

Precise plain-English rule

Matches the disobedience entry in CRIME_VOCAB/CRIME_SUBTYPE_VOCAB. If this tag has a configured parent in CRIME_PARENT_TAGS, the parent is emitted from the same text span.

Code text for this tag

```

334:     "crime:perjury": ("crime:disobedience",),
335:     "crime:fentanyl": ("crime:narcotics",),

1638: # crime:<TYPE> – every distinct match emits one tag entry.
1639: for slug, pat_str in CRIME_VOCAB:
1640:     for m in re.finditer(pat_str, text, re.I):
1641:         add(f"crime:{slug}", span=m.span())
1642:         for parent in CRIME_PARENT_TAGS.get(f"crime:{slug}", ()):
1643:             add(parent, span=m.span())
1644: for slug, pat_str in CRIME_SUBTYPE_VOCAB:
1645:     for m in re.finditer(pat_str, text, re.I):
1646:         tag = f"crime:{slug}"
1647:         add(tag, span=m.span())
1648:         for parent in CRIME_PARENT_TAGS.get(tag, ()):
1649:             add(parent, span=m.span())
1650:
1651: if m := _martyrdom_match(text, account_category, entries):
1652:     add("theme:martyrdom", span=m.span())
1653:

```

crime:perjury (1 (0.0%))

Precise plain-English rule

Matches the perjury entry in CRIME_VOCAB/CRIME_SUBTYPE_VOCAB. If this tag has a configured parent in CRIME_PARENT_TAGS, the parent is emitted from the same text span.

Code text for this tag

```

321:     ("perjury", r"\bperjur(?:y|ed)\b|blying\s+under\s+oath\b"),

334:     "crime:perjury": ("crime:disobedience",),
335:     "crime:fentanyl": ("crime:narcotics",),

1638: # crime:<TYPE> – every distinct match emits one tag entry.
1639: for slug, pat_str in CRIME_VOCAB:
1640:     for m in re.finditer(pat_str, text, re.I):
1641:         add(f"crime:{slug}", span=m.span())
1642:         for parent in CRIME_PARENT_TAGS.get(f"crime:{slug}", ()):
1643:             add(parent, span=m.span())
1644: for slug, pat_str in CRIME_SUBTYPE_VOCAB:
1645:     for m in re.finditer(pat_str, text, re.I):
1646:         tag = f"crime:{slug}"
1647:         add(tag, span=m.span())
1648:         for parent in CRIME_PARENT_TAGS.get(tag, ()):
1649:             add(parent, span=m.span())
1650:
1651: if m := _martyrdom_match(text, account_category, entries):
1652:     add("theme:martyrdom", span=m.span())
1653:

```

event

The event namespace is produced by direct event regexes and disturbance-cluster regexes. Disturbance clusters require the city name and unrest/federal-response language within 180 characters of each other.

event:palestine (36 (0.4%))

Precise plain-English rule

Matches Palestine/Palestinian(s), Gaza/Gazan(s), Hamas, Israel-Hamas, Israel-Palestine/Palestinian, or West Bank.

Code text for this tag

```

639: PATTERN_EVENT_PALESTINE = _compile(
640:     r"\b(?:palestin(?:e|ian)s?|gaza(?:n)?s?|hamas|israel[- ]hamas|"
641:     r"israel[- ]palestin(?:e|ian)|west\s+bank)\b"
642: )

1550: (PATTERN_TOPIC_LAUDATORY, "topic:laudatory"),
1551: (PATTERN_EVENT_PALESTINE, "event:palestine"),
1552: (PATTERN_THEME_BORDER, "policy:border"),

```

event:los-angeles-disturbance (15 (0.1%))

Precise plain-English rule

Matches the los angeles disturbance disturbance cluster: the city name must occur within 180 characters of riot/unrest/protest/federal-response language, in either order.

Code text for this tag

```
731: (
732:     "los-angeles-disturbance",
733:     _compile(
734:         r"\b(?:Los\s+Angeles|L\.A\.|LA)\b.{0,180}\b(?:riot(?:s|ers|ing)?|"
735:         r"civil\s+disturbance|civil\s+unrest|violent\s+protests?|anti[- ]ICE\s+"
736:         r"(\?:riot(?:s|ers|ing)?|protests?)|federal\s+building|concrete\s+at\s+DHS\s+agents)\b"
737:         r"|b(?:riot(?:s|ers|ing)?|civil\s+disturbance|civil\s+unrest|violent\s+protests?)"
738:         r"anti[- ]ICE\s+(\?:riot(?:s|ers|ing)?|protests?)|federal\s+building|"
739:         r"concrete\s+at\s+DHS\s+agents)\b.{0,180}\b(?:Los\s+Angeles|L\.A\.|LA)\b"
740:     ),
1609:     if m := _coded_nativism_match(text, entries):
1610:         add("theme:nativism", span=m.span())
1611:
1612:     if m := _theme_religion_match(text):
```

event:portland-disturbance (6 (0.1%))

Precise plain-English rule

Matches the portland disturbance disturbance cluster: the city name must occur within 180 characters of riot/unrest/protest/federal-response language, in either order.

Code text for this tag

```
753: (
754:     "portland-disturbance",
755:     _compile(
756:         r"\bPortland\b.{0,180}\b(?:riot(?:s|ers|ing)?|civil\s+disturbance|civil\s+unrest|"
757:         r"violent\s+protests?|anti[- ]ICE\s+(\?:riot(?:s|ers|ing)?|protests?)|"
758:         r"ICE\s+(\?:detention|facility)|Antifa|domestic\s+terrorism)\b"
759:         r"|b(?:riot(?:s|ers|ing)?|civil\s+disturbance|civil\s+unrest|violent\s+protests?)"
760:         r"anti[- ]ICE\s+(\?:riot(?:s|ers|ing)?|protests?)|ICE\s+(\?:detention|facility)|"
761:         r"Antifa|domestic\s+terrorism)\b.{0,180}\bPortland\b"
762:     ),
1609:     if m := _coded_nativism_match(text, entries):
1610:         add("theme:nativism", span=m.span())
1611:
1612:     if m := _theme_religion_match(text):
```

event:minneapolis-disturbance (5 (0.0%))

Precise plain-English rule

Matches the minneapolis disturbance disturbance cluster: the city name must occur within 180 characters of riot/unrest/protest/federal-response language, in either order.

Code text for this tag

```
742: (
743:     "minneapolis-disturbance",
744:     _compile(
745:         r"\bMinneapolis\b.{0,180}\b(?:Bovino|riot(?:s|ers|ing)?|civil\s+disturbance|"
746:         r"civil\s+unrest|violent\s+protests?|protests?|tear\s+gas|pellet\s+guns?|"
747:         r"heavy[- ]handed|lawsuits?|killing|killed|disaster)\b"
748:         r"|b(?:Bovino|riot(?:s|ers|ing)?|civil\s+disturbance|civil\s+unrest|"
749:         r"violent\s+protests?|protests?|tear\s+gas|pellet\s+guns?|heavy[- ]handed|"
750:         r"lawsuits?|killing|killed|disaster)\b.{0,180}\bMinneapolis\b"
751:     ),
1609:     if m := _coded_nativism_match(text, entries):
1610:         add("theme:nativism", span=m.span())
1611:
1612:     if m := _theme_religion_match(text):
```

genre

The genre namespace is produced from produced-video regexes, parody franchise matches, media-description sidecar tags, and the lineup composite. The lineup composite is exact: reply + theme:criminal + exactly one attached media item.

genre:lineup (670 (6.7%))

Precise plain-English rule

Composite rule: tweet_type is reply, theme:criminal already fired, and media_count equals exactly 1.

Code text for this tag

```
1709: # genre:lineup: composite replies that hit theme:criminal with one photo.
1710: if (
1711:     tweet_type == "reply"
1712:     and any(e["tag"] == "theme:criminal" for e in entries)
1713:     and media_count == 1
1714: ):
1715:     add("genre:lineup")
```

genre:advertisement (439 (4.4%))

Precise plain-English rule

On video posts or sidecar text, matches new ad/advert/spot/commercial, ad/advertisement/commercial/promo/promotional video/campaign spot, campaign/recruitment ad/spot/video/campaign, join ICE/CBP/HSI language, join.ice.gov, apply today/now, Promises Made/Kept, or most

secure border in American history.

Code text for this tag

```
1045: PATTERN_VIDEO_AD = _compile(
1046:     r"\b(?:new\s+(?:ad|advert|spot|commercial))\b"
1047:     r"|\b(?:ad|advertisement|commercial|promo|promotional\s+video|campaign\s+spot)\b"
1048:     r"|\b(?:campaign|recruitment)\s+(?:ad|spot|video|campaign)\b"
1049:     r"|\bjoin\s+(?:ice|cbp|hsi|the\s+(?:ice|cbp)\s+(?:team|family))\b"
1050:     r"|\bjoin\.ice\.gov\b"
1051:     r"|\bapply\s+(?:today|now)\b"
1052:     r"|\bpromises?\s+made[,.\s]+promises?\s+kept\b"
1053:     r"|\bmost\s+secure\s+border\s+in\s+american\s+history\b"
1054: )
```

genre:psa (128 (1.3%))

Precise plain-English rule

On video posts or sidecar text, matches PSA/public service announcement, learn/find out/more info/visit website at, did you know, know your rights/facts/signs, protect yourself, stay safe, report a/suspicious, or call 1-800/888/877/866 numbers.

Code text for this tag

```
1019: PATTERN_VIDEO_PSA = _compile(
1020:     r"\b(?:psa|public\s+service\s+announcement)\b"
1021:     r"|\b(?:learn\s+more|find\s+out\s+more|more\s+info|visit\s+(?:our\s+)?website)\s+at\b"
1022:     r"|\b(?:did\s+you\s+know(?:\s+that)?\b"
1023:     r"|\b(?:know\s+(?:your\s+rights|the\s+facts|the\s+signs)|protect\s+yourself|stay\s+safe)"
1024:     r"report\s+(?:a|suspicious))\b"
1025:     r"|\bcall\s+(?:1-7800|1-7888|1-7877|1-7866)[- ]?\d"
1026: )
```

genre:recruitment (126 (1.3%))

Precise plain-English rule

On video posts or sidecar text, matches join.ice.gov, DHS/ICE/CBP/HSI career/join/job/recruiting URLs, hiring/recruiting, recruitment/hiring ad/spot/video/campaign, join ICE/CBP/HSI/DHS/team/family, apply today/now, answer the call, or serve your/our/their/his/her country/nation/community.

Code text for this tag

```
1033: PATTERN_VIDEO_RECRUITMENT = _compile(
1034:     r"\bjoin\.ice\.gov\b"
1035:     r"|\b(?:ice|cbp|hsi|dhs)\.gov/(?:careers?|join|jobs?|recruit(?:ment|ing)?|homeland-security-careers)\b"
1036:     r"|\b(?:careers?|jobs?)\.(?:ice|cbp|hsi|dhs)\.gov\b"
1037:     r"|\b(?:dhs|ice|cbp|hsi)\s+(?:is\s+)?(?:hiring|recruiting)\b"
1038:     r"|\b(?:recruitment|hiring)\s+(?:ad|spot|video|campaign)\b"
1039:     r"|\bjoin\s+(?:ice|cbp|hsi|dhs|the\s+(?:ice|cbp|hsi|dhs)\s+(?:team|family))\b"
1040:     r"|\bapply\s+(?:today|now)\b"
1041:     r"|\bcareer(?:s)?\s+(?:with|at)\s+(?:ice|cbp|hsi|dhs)\b"
1042:     r"|\banswer\s+the\s+call\b"
1043:     r"|\bserve\s+(?:your|our|their|his|her)\s+(?:country|nation|countrymen|community)\b"
1044: )
```

genre:utopian (44 (0.4%))

Precise plain-English rule

Matches utopian, idealized, aspirational, golden age, bright future, morning in America, or sunlit/heroic/triumphal within 80 characters of montage/vision/future/aesthetic.

Code text for this tag

```
1067: ),
1068: (
1069:     "genre:utopian",
1070:     _compile(
1071:         r"\butopian\b|bidealized\b|baspirational\b|b(?:golden\s+age|bright\s+future|morning\s+in\s+america)\b"
1072:         r"|\b(?:sunlit|heroic|triumphal)\b.{0,80}\b(?:montage|vision|future|aesthetic)\b"
1073:     ),
1074: ),
1075: (
1076:     "genre:dystopian",
1077:     _compile(
1078:         r"\bdystopian\b|bsci[- ]?fi\b|bscience[- ]fiction\b|bcyberpunk\b"
1079:         r"|\b(?:dark|bleak|apocalyptic|hellscape|surveillance[- ]state|futuristic)\b.{0,80}\b(?:future|city|vision|scene|aesthetic)\b"
1080:     )
1081: )
1082: )
1083:
1084: # long : > 120s
1085: # We tag the bucket regardless of whether a kind matched so the viewer
1086: # can filter "all videos under 30s" without depending on text cues.
1087: if video_count > 0:
1088:     for pat, tag in (
1089:         (PATTERN_VIDEO_BODYCAM, "video:bodycam"),
1090:         (PATTERN_VIDEO_INTERVIEW, "video:interview"),
1091:         (PATTERN_VIDEO_NEWS_CLIP, "video:news-clip"),
1092:         (PATTERN_VIDEO_SPEECH, "video:speech"),
1093:     ):
1094:         m = pat.search(text)
1095:         if m:
1096:             add(tag, span=m.span())
1097:             if PATTERN_AUDIO_MUSIC_CONTEXT.search(text):
1098:                 add("audio:music-likely")
1099:             if PATTERN_AUDIO_MUSIC_CONTEXT.search(reply_context_text):
1100:                 add("audio:music-likely", source="reply-context")
1101:             if any(e["tag"] in {"genre:music-video"} for e in entries):
1102:                 add("audio:music-likely")
1103:             if m := PATTERN_PRODUCED_VIDEO_STYLE.search(text):
1104:                 add("video:produced", span=m.span())
1105:             if video_max_duration_sec is not None and video_max_duration_sec > 0:
1106:                 if video_max_duration_sec <= 30:
```

```

1753:         add("video:short")
1754:     elif video_max_duration_sec <= 120:
1755:         add("video:medium")
1756:     else:
1757:         add("video:long")
1758:     # Derive genre:music-video ONLY from an explicit genre:music-video
1759:     # signal (set by the conservative describe_media / manual-review rules) –
1760:     # NEVER from audio:music-likely, which is an incidental acoustic heuristic.
1761:     # Also suppress it on speech / press-conference clips.
1762:     if not speech_indicator_present and any(e["tag"] == "genre:music-video" for e in entries):
1763:         add("genre:music-video")
1764:     if any(
1765:         e["tag"] in PRODUCED_VIDEO_STRUCTURE_TAGS or str(e["tag"]).startswith("genre:")
1766:         for e in entries
1767:     ):
1768:         add("video:produced")

```

genre:music-video (36 (0.4%))

Precise plain-English rule

On video posts, matches explicit music-video wording: official music video, set to music/the song/the track, official video/audio for, or lyric/lyrics video. Speech/press-conference indicators suppress this tag.

Code text for this tag

```

987: PATTERN_VIDEO_MUSIC_VIDEO = _compile(
988:     r"\b(?:official\s+)?music\s+video\b"
989:     r"|\bset\s+to\s+(?:music|the\s+song|the\s+track)\b"
990:     r"|\b(?:official\s+(?:video|audio)\s+for\b"
991:     r"|\b(?:lyric|lyrics)\s+video\b"
992: )

```

genre:dystopian (18 (0.2%))

Precise plain-English rule

Matches dystopian, sci-fi/science fiction, cyberpunk, or dark/bleak/apocalyptic/hellscape/surveillance-state/futuristic within 80 characters of future/city/vision/scene/aesthetic.

Code text for this tag

```

1074:     ),
1075:     (
1076:         "genre:dystopian",
1077:         _compile(
1078:             r"\bdystopian\b|\bsci[- ]?fi\b|\bscience[- ]fiction\b|\bcyberpunk\b"
1079:             r"|\b(?:dark|bleak|apocalyptic|hellscape|surveillance[- ]state|futuristic)\b.{0,80}\b(?:future|city|vision|scene|aesthetic)\b"
1080:         ),
1081:     ),
1082: )
1083: PRODUCED_VIDEO_STRUCTURE_TAGS = {
1084:     "video:produced",
1085:     "genre:music-video",
1086:     "video:montage",
1087: }
1730: # long : > 120s
1731: # We tag the bucket regardless of whether a kind matched so the viewer
1732: # can filter "all videos under 30s" without depending on text cues.
1733: if video_count > 0:
1734:     for pat, tag in (
1735:         (PATTERN_VIDEO_BODYCAM, "video:bodycam"),
1736:         (PATTERN_VIDEO_INTERVIEW, "video:interview"),
1737:         (PATTERN_VIDEO_NEWS_CLIP, "video:news-clip"),
1738:         (PATTERN_VIDEO_SPEECH, "video:speech"),
1739:     ):
1740:         m = pat.search(text)
1741:         if m:
1742:             add(tag, span=m.span())
1743:     if PATTERN_AUDIO_MUSIC_CONTEXT.search(text):
1744:         add("audio:music-likely")
1745:     if PATTERN_AUDIO_MUSIC_CONTEXT.search(reply_context_text):
1746:         add("audio:music-likely", source="reply-context")
1747:     if any(e["tag"] in {"genre:music-video"} for e in entries):
1748:         add("audio:music-likely")
1749:     if m := PATTERN_PRODUCED_VIDEO_STYLE.search(text):
1750:         add("video:produced", span=m.span())
1751:     if video_max_duration_sec is not None and video_max_duration_sec > 0:
1752:         if video_max_duration_sec <= 30:
1753:             add("video:short")
1754:         elif video_max_duration_sec <= 120:
1755:             add("video:medium")
1756:         else:
1757:             add("video:long")
1758:     # Derive genre:music-video ONLY from an explicit genre:music-video
1759:     # signal (set by the conservative describe_media / manual-review rules) –
1760:     # NEVER from audio:music-likely, which is an incidental acoustic heuristic.
1761:     # Also suppress it on speech / press-conference clips.
1762:     if not speech_indicator_present and any(e["tag"] == "genre:music-video" for e in entries):
1763:         add("genre:music-video")
1764:     if any(
1765:         e["tag"] in PRODUCED_VIDEO_STRUCTURE_TAGS or str(e["tag"]).startswith("genre:")
1766:         for e in entries
1767:     ):
1768:         add("video:produced")

```

genre:parody (5 (0.0%))

Precise plain-English rule

Emitted when any franchise parody pattern fires; it is not emitted by itself.

```

1656:         if m := franchise_pat.search(text):
1657:             add("genre:parody", span=m.span())
1658:             add(f"parody:{franchise_slug}", span=m.span())
1659:             add("theme:pop-culture-reference")
1660: # Also catch "this is the way" + one more Star Wars cue (e.g. "a galaxy").
1661: if (
1662:     PATTERN_PARODY_STAR_WARS_MULTI.search(text)
1663:     and PATTERN_PARODY_STAR_WARS_EXTRA_CUE.search(text)
1664:     and not any(e["tag"] == "parody:star-wars" for e in entries)
1665: ):
1666:     m_way = PATTERN_PARODY_STAR_WARS_MULTI.search(text)
1667:     if m_way:
1668:         add("genre:parody", span=m_way.span())
1669:         add("parody:star-wars", span=m_way.span())
1670:         add("theme:pop-culture-reference")

```

Precise plain-English rule

```

1059: ("genre:advertisement", PATTERN_VIDEO_AD),
1060: (
1061:     "genre:war-movie",
1062:     _compile(
1063:         r"\bwar[- ]movie\b\bwar[- ]film\b\baction[- ]movie\b"
1064:         r"|\b(?:cinematic|movie[- ]trailer|trailer[- ]style|dramatic)\b.{0,80}\b(?:war|battle|combat|military|raid|operation)\b"
1065:         r"|\b(?:war|battle|combat|military|raid|operation)\b.{0,80}\b(?:cinematic|movie[- ]trailer|trailer[- ]style|dramatic)\b"
1066:     ),
1067: ),
1068: (
1069:     "genre:utopian",
1070:     _compile(
1071:         r"\butopian\b\bidealized\b\baspirational\b\b(?:golden\s+age|bright\s+future|morning\s+in\s+america)\b"
1072:     )
1073: )
1074: )
1075:
1076: # long : > 120s
1077: # We tag the bucket regardless of whether a kind matched so the viewer
1078: # can filter "all videos under 30s" without depending on text cues.
1079: if video_count > 0:
1080:     for pat, tag in (
1081:         (PATTERN_VIDEO_BODYCAM, "video:bodycam"),
1082:         (PATTERN_VIDEO_INTERVIEW, "video:interview"),
1083:         (PATTERN_VIDEO_NEWS_CLIP, "video:news-clip"),
1084:         (PATTERN_VIDEO_SPEECH, "video:speech"),
1085:     ):
1086:         m = pat.search(text)
1087:         if m:
1088:             add(tag, span=m.span())
1089:     if PATTERN_AUDIO_MUSIC_CONTEXT.search(text):
1090:         add("audio:music-likely")
1091:     if PATTERN_AUDIO_MUSIC_CONTEXT.search(reply_context_text):
1092:         add("audio:music-likely", source="reply-context")
1093:     if any(e["tag"] in {"genre:music-video"} for e in entries):
1094:         add("audio:music-likely")
1095:     if m := PATTERN_PRODUCED_VIDEO_STYLE.search(text):
1096:         add("video:produced", span=m.span())
1097:     if video_max_duration_sec is not None and video_max_duration_sec > 0:
1098:         if video_max_duration_sec <= 30:
1099:             add("video:short")
1100:         elif video_max_duration_sec <= 120:
1101:             add("video:medium")
1102:         else:
1103:             add("video:long")
1104:     # Derive genre:music-video ONLY from an explicit genre:music-video
1105:     # signal (set by the conservative describe_media / manual-review rules) –
1106:     # NEVER from audio:music-likely, which is an incidental acoustic heuristic.
1107:     # Also suppress it on speech / press-conference clips.
1108:     if not speech_indicator_present and any(e["tag"] == "genre:music-video" for e in entries):
1109:         add("genre:music-video")
1110:     if any(
1111:         e["tag"] in PRODUCED_VIDEO_STRUCTURE_TAGS or str(e["tag"]).startswith("genre:")
1112:         for e in entries
1113:     ):
1114:         add("video:produced")

```

```

1186:      r"|\bsentenc(?:ed|ing)\b"
1187:      r"|\bfelony\s+charges?\b"
1188:      r"|\bmisdemeanor\s+charges?\b"
1189: )

1545:      (PATTERN_SUBJECT_CELIBRITY, "subject:celebrity"),
1546:      (PATTERN_LEGAL_CRIMINAL_PROSECUTION, "legal:criminal-prosecution"),
1547:      (PATTERN_LEGAL_CIVIL_LAWSUIT, "legal:civil-lawsuit"),

```

legal:civil-lawsuit (30 (0.3%))

Precise plain-English rule

Matches civil lawsuit/suit/action/case/complaint/litigation, lawsuit, filed a lawsuit/suit/civil action/complaint, sue/sued/sues/suing, injunction, temporary restraining order, TRO, settlement agreement, or consent decree.

Code text for this tag

```

1190: PATTERN_LEGAL_CIVIL_LAWSUIT = _compile(
1191:     r"|\bcivil\s+(?:lawsuit|suit|action|case|complaint|litigation)\b"
1192:     r"|\blawsuits?\b"
1193:     r"|\bfiled\s+(?:a\s+)?(?:lawsuit|suit|civil\s+action|complaint)\b"
1194:     r"|\bsu(?:e|ed|es|ing)\b"
1195:     r"|\binjunction\b"
1196:     r"|\btemporary\s+restraining\s+order\b"
1197:     r"|\bTRO\b"
1198:     r"|\bsettlement\s+agreement\b"
1199:     r"|\bconsent\s+decree\b"
1200: )

1546:      (PATTERN_LEGAL_CRIMINAL_PROSECUTION, "legal:criminal-prosecution"),
1547:      (PATTERN_LEGAL_CIVIL_LAWSUIT, "legal:civil-lawsuit"),
1548:      (PATTERN_TOPIC_ECONOMY, "topic:economy"),

```

legal:birthright-citizenship (3 (0.0%))

Precise plain-English rule

Matches the literal phrase birthright citizenship.

Code text for this tag

```

814: PATTERN_LEGAL_BIRTHRIGHT_CITIZENSHIP = _compile(r"|\bbirthright\s+citizenship\b")

1561:      (PATTERN_THEME_POP_CULTURE_REFERENCE, "theme:pop-culture-reference"),
1562:      (PATTERN_LEGAL_BIRTHRIGHT_CITIZENSHIP, "legal:birthright-citizenship"),
1563:      (PATTERN_THEME_NATIVISM_BIRTHRIGHT, "theme:nativism"),

```

military

The military namespace is produced from text regexes for service branches and from account-mention aliases. Any military branch tag also adds topic:military.

military:navy (171 (1.7%))

Precise plain-English rule

Matches navy branch terms in MILITARY_VOCAB or a mention alias mapped to military:navy; then implies topic:military.

Code text for this tag

```

579:      (
580:          "navy",
581:          _compile(
582:              r"\b(?:u\.s\.s+)?navy|sailors?|naval\s+academy|USNA|midshipmen|"
583:              r"carrier\s+strike\s+group|aircraft\s+carrier|carrier\s+air\s+wing|"
584:              r"CVN\s+d+|USS\s+[A-Z][A-Za-z0-9-]+|USNS\s+[A-Z][A-Za-z0-9-]+\b"
585:          ),
586:      ),
587:      (
588:          "air-force",

1631:      for slug, pat in MILITARY_VOCAB:
1632:          for m in pat.finditer(text):
1633:              add(f"military:{slug}", span=m.span())
1634:
1635:      if _general_topic_score(text) >= 3:

1913:
1914:
1915: def _ensure_intrinsic_parent_topics(
1916:     entries: list[dict[str, Any]],
1917:     text: str,
1918:     add: Callable[..., None],
1919: ) -> None:
1920:     """Add broad topic parents implied by narrower deterministic tags."""
1921:     existing_tags = {str(e["tag"]) for e in entries}
1922:     if (

```

military:coast-guard (142 (1.4%))

Precise plain-English rule

Matches coast guard branch terms in MILITARY_VOCAB or a mention alias mapped to military:coast-guard; then implies topic:military.

Code text for this tag

```

593:     (
594:         "coast-guard",
595:         _compile(
596:             r"\b(?:coast\s+guard|USCG|USCGA|coast\s+guard\s+academy|coasties?|coast\s+guards?m[ae]n)\b"
597:         ),
598:     ),
599:     ("national-guard", _compile(r"\b(?:national\s+guard|national\s+guards?m[ae]n)\b")),
600: )
601: MILITARY_MENTION_ALIASES: dict[str, str] = {
602:     "usarmynorth": "army",

1631:     for slug, pat in MILITARY_VOCAB:
1632:         for m in pat.finditer(text):
1633:             add(f"military:{slug}", span=m.span())
1634:
1635:     if _general_topic_score(text) >= 3:

1913:
1914:
1915: def _ensure_intrinsic_parent_topics(
1916:     entries: list[dict[str, Any]],
1917:     text: str,
1918:     add: Callable[..., None],
1919: ) -> None:
1920:     """Add broad topic parents implied by narrower deterministic tags."""
1921:     existing_tags = {str(e["tag"]) for e in entries}
1922:     if (

```

military:army (43 (0.4%))

Precise plain-English rule

Matches army branch terms in MILITARY_VOCAB or a mention alias mapped to military:army; then implies topic:military.

Code text for this tag

```

577: MILITARY_VOCAB: tuple[tuple[str, re.Pattern[str]], ...] = (
578:     ("army", _compile(r"\b(?:?u\.s\.s+)?army[soldiers?|west\s+point|USMA]\b")),
579:     (
580:         "navy",
581:         _compile(
582:             r"\b(?:?u\.s\.s+)?navy[sailors?|naval\s+academy|USNA|midshipmen|"
583:             r"carrier\s+strike\s+group|aircraft\s+carrier|carrier\s+air\s+wing|"
584:             r"CVN\s+d+|USS\s+[A-Z][A-Za-z0-9-]+|USNS\s+[A-Z][A-Za-z0-9-]+)\b"
585:         ),
586:     ),

1631:     for slug, pat in MILITARY_VOCAB:
1632:         for m in pat.finditer(text):
1633:             add(f"military:{slug}", span=m.span())
1634:
1635:     if _general_topic_score(text) >= 3:

1913:
1914:
1915: def _ensure_intrinsic_parent_topics(
1916:     entries: list[dict[str, Any]],
1917:     text: str,
1918:     add: Callable[..., None],
1919: ) -> None:
1920:     """Add broad topic parents implied by narrower deterministic tags."""
1921:     existing_tags = {str(e["tag"]) for e in entries}
1922:     if (

```

military:national-guard (38 (0.4%))

Precise plain-English rule

Matches national guard branch terms in MILITARY_VOCAB or a mention alias mapped to military:national-guard; then implies topic:military.

Code text for this tag

```

598:     ),
599:     ("national-guard", _compile(r"\b(?:national\s+guard|national\s+guards?m[ae]n)\b")),
600: )
601: MILITARY_MENTION_ALIASES: dict[str, str] = {
602:     "usarmynorth": "army",
603:     "usnationalguard": "national-guard",
604:     "usguard": "national-guard",
605:     "usnavy": "navy",
606:     "usairforce": "air-force",
607:     "usspaceforce": "space-force",

1631:     for slug, pat in MILITARY_VOCAB:
1632:         for m in pat.finditer(text):
1633:             add(f"military:{slug}", span=m.span())
1634:
1635:     if _general_topic_score(text) >= 3:

1913:
1914:
1915: def _ensure_intrinsic_parent_topics(
1916:     entries: list[dict[str, Any]],
1917:     text: str,
1918:     add: Callable[..., None],
1919: ) -> None:
1920:     """Add broad topic parents implied by narrower deterministic tags."""
1921:     existing_tags = {str(e["tag"]) for e in entries}
1922:     if (

```

military:air-force (37 (0.4%))

Precise plain-English rule

Matches air force branch terms in MILITARY_VOCAB or a mention alias mapped to military:air-force; then implies topic:military.

Code text for this tag

```
587: (
588:     "air-force",
589:     _compile(r"\b(?:?:u\.s\.s+)?air\s+force|usaf|airm[ae]n|air\s+force\s+academy\b"),
590: ),
591: ("space-force", _compile(r"\b(?:?:u\.s\.s+)?space\s+force|ussf\b")),
592: ("marines", _compile(r"\b(?:marine\s+corps|u\.s\.s+marines?|usmc|marines)\b")),
593: (
594:     "coast-guard",
595:     _compile(
596:         r"\b(?:coast\s+guard|USCG|USCGA|coast\s+guard\s+academy|coasties?|coast\s+guards?m[ae]n)\b"
597:     ),
598: ),
599: ("national-guard", _compile(r"\b(?:national\s+guard|national\s+guards?m[ae]n)\b")),
600: )

1631: for slug, pat in MILITARY_VOCAB:
1632:     for m in pat.finditer(text):
1633:         add(f"military:{slug}", span=m.span())
1634:
1635: if _general_topic_score(text) >= 3:

1913:
1914:
1915: def _ensure_intrinsic_parent_topics(
1916:     entries: list[dict[str, Any]],
1917:     text: str,
1918:     add: Callable[..., None],
1919: ) -> None:
1920:     """Add broad topic parents implied by narrower deterministic tags."""
1921:     existing_tags = {str(e["tag"]) for e in entries}
1922:     if (
```

military:marines (15 (0.1%))

Precise plain-English rule

Matches marines branch terms in MILITARY_VOCAB or a mention alias mapped to military:marines; then implies topic:military.

Code text for this tag

```
591: ("space-force", _compile(r"\b(?:?:u\.s\.s+)?space\s+force|ussf\b")),
592: ("marines", _compile(r"\b(?:marine\s+corps|u\.s\.s+marines?|usmc|marines)\b")),
593: (
594:     "coast-guard",
595:     _compile(
596:         r"\b(?:coast\s+guard|USCG|USCGA|coast\s+guard\s+academy|coasties?|coast\s+guards?m[ae]n)\b"
597:     ),
598: ),
599: ("national-guard", _compile(r"\b(?:national\s+guard|national\s+guards?m[ae]n)\b")),
600: )

1631: for slug, pat in MILITARY_VOCAB:
1632:     for m in pat.finditer(text):
1633:         add(f"military:{slug}", span=m.span())
1634:
1635: if _general_topic_score(text) >= 3:

1913:
1914:
1915: def _ensure_intrinsic_parent_topics(
1916:     entries: list[dict[str, Any]],
1917:     text: str,
1918:     add: Callable[..., None],
1919: ) -> None:
1920:     """Add broad topic parents implied by narrower deterministic tags."""
1921:     existing_tags = {str(e["tag"]) for e in entries}
1922:     if (
```

military:space-force (5 (0.0%))

Precise plain-English rule

Matches space force branch terms in MILITARY_VOCAB or a mention alias mapped to military:space-force; then implies topic:military.

Code text for this tag

```
590: ),
591: ("space-force", _compile(r"\b(?:?:u\.s\.s+)?space\s+force|ussf\b")),
592: ("marines", _compile(r"\b(?:marine\s+corps|u\.s\.s+marines?|usmc|marines)\b")),
593: (
594:     "coast-guard",
595:     _compile(
596:         r"\b(?:coast\s+guard|USCG|USCGA|coast\s+guard\s+academy|coasties?|coast\s+guards?m[ae]n)\b"
597:     ),
598: ),
599: ("national-guard", _compile(r"\b(?:national\s+guard|national\s+guards?m[ae]n)\b")),
600: )

1631: for slug, pat in MILITARY_VOCAB:
1632:     for m in pat.finditer(text):
1633:         add(f"military:{slug}", span=m.span())
1634:
1635: if _general_topic_score(text) >= 3:

1913:
1914:
1915: def _ensure_intrinsic_parent_topics(
1916:     entries: list[dict[str, Any]],
1917:     text: str,
1918:     add: Callable[..., None],
1919: ) -> None:
1920:     """Add broad topic parents implied by narrower deterministic tags."""
1921:     existing_tags = {str(e["tag"]) for e in entries}
1922:     if (
```

origin

The origin namespace is stricter than country. It is produced when the text says from <country>, with a comma after the country, or when a curated demonym is immediately followed by a person-origin noun such as national, citizen, immigrant, migrant, man, woman, descent, or origin.

origin:Mexico (378 (3.8%))

Precise plain-English rule

Emitted as origin:Mexico when the text has from Mexico, with a comma after the country, or a demonym for Mexico followed immediately by a person-origin noun.

Code text for this tag

```
153:     "Mexico",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]+(?:\s+(?!{i:_PREPOSITIONS}}\b)[A-Za-z][a-zA-Z]+)?"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:{i:_PREPOSITIONS}}\s+({_PLACE}))\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{i:_PREPOSITIONS}}\s+({_PLACE}))\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE}),\s+({_PLACE}))\b")

1238: DEMONYM_TO_COUNTRY: dict[str, str] = {
1239:     "iranian": "Iran",
1240:     "mexican": "Mexico",
1241:     "venezuelan": "Venezuela",
1242:     "chinese": "China",
1243:     "russian": "Russia",
1244:     "cuban": "Cuba",
1245:     "colombian": "Colombia",
1246:     "salvadoran": "El Salvador",
1247:     "salvadorian": "El Salvador",
1248:     "honduran": "Honduras",
1249:     "guatemalan": "Guatemala",
1250:     "haitian": "Haiti",
1251:     "somali": "Somalia",
1252:     "somalian": "Somalia",
1253:     "afghan": "Afghanistan",
1254:     "syrian": "Syria",
1255:     "nigerian": "Nigeria",
1256:     "pakistani": "Pakistan",
1257:     "ukrainian": "Ukraine",
1258:     "israeli": "Israel",
1259:     "iraqi": "Iraq",
1260:     "egyptian": "Egypt",
1261:     "yemeni": "Yemen",
1262:     "brazilian": "Brazil",
1263:     "nicaraguan": "Nicaragua",
1264:     "ecuadorian": "Ecuador",
1265:     "peruvian": "Peru",
1266:     "jamaican": "Jamaica",
1267:     "filipino": "Philippines",
1268:     "vietnamese": "Vietnam",
1269:     "lebanese": "Lebanon",
1270:     "libyan": "Libya",
1271:     "sudanese": "Sudan",
1272:     "ethiopian": "Ethiopia",
1273:     "kenyan": "Kenya",
1274:     "moroccan": "Morocco",
1275:     "algerian": "Algeria",
1276:     "bangladeshi": "Bangladesh",
1277:     "cambodian": "Cambodia",
1278:     "indonesian": "Indonesia",
1279:     "dominican": "Dominican Republic",
1280:     "panamanian": "Panama",
1281:     "bolivian": "Bolivia",
1282:     "chilean": "Chile",
1283:     "argentine": "Argentina",
1284:     "argentinian": "Argentina",
1285:     "saudi": "Saudi Arabia",
1286:     "turkish": "Turkey",
1287: }
1288: FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
1289:     "tehran": "Iran",
1290:     "caracas": "Venezuela",
1291:     "beijing": "China",
1292:     "moscow": "Russia",
1293:     "havana": "Cuba",
1294:     "kabul": "Afghanistan",
1295:     "damascus": "Syria",
1296:     "baghdad": "Iraq",
1297:     "tripoli": "Libya",
1298:     "mogadishu": "Somalia",
1299:     "bogota": "Colombia",
1300:     "managua": "Nicaragua",
1301:     "tegucigalpa": "Honduras",
1302:     "kyiv": "Ukraine",
1303: }
1304:
1305:
1306: def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:
1307:     alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
1308:     return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)
1309:
```

```

1310:
1311: PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))
1312: PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
1313: # A demonym that immediately precedes one of these nouns describes a person's
1314: # nationality, so it also earns origin:<country>, not just a contextual country:.
1315: PATTERN_DEMONYM_PERSON = _compile(
1316:     r"^\W*(?:national|nationals|citizen|citizens|immigrant|immigrants|migrant|migrants|"
1677:     # origin:<COUNTRY> – validated against the sovereign-state vocab.
1678:     for m in PATTERN_ORIGIN_CANDIDATE.finditer(text):
1679:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1680:         if resolved:
1681:             add(f"origin:{_normalize_country(resolved)}", span=m.span(1))
1682:
1683:     # country:<NAME> – broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:
1687:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:     # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:     # country, which the bare-name validators above can't see. A demonym
1691:     # followed by a person noun also earns origin:.
1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

origin:El-Salvador (94 (0.9%))

Precise plain-English rule

Emitted as origin:El-Salvador when the text has from El Salvador, with a comma after the country, or a demonym for El Salvador followed immediately by a person-origin noun.

Code text for this tag

```

98:     "EL Salvador",
1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). `_resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*(?)"
1225: # "from <Country>," – anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,"")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")
1238: DEMONYM_TO_COUNTRY: dict[str, str] = {
1239:     "iranian": "Iran",
1240:     "mexican": "Mexico",
1241:     "venezuelan": "Venezuela",
1242:     "chinese": "China",
1243:     "russian": "Russia",
1244:     "cuban": "Cuba",
1245:     "colombian": "Colombia",
1246:     "salvadoran": "El Salvador",
1247:     "salvadorian": "El Salvador",
1248:     "honduran": "Honduras",
1249:     "guatemalan": "Guatemala",
1250:     "haitian": "Haiti",
1251:     "somali": "Somalia",
1252:     "somalian": "Somalia",
1253:     "afghan": "Afghanistan",
1254:     "syrian": "Syria",
1255:     "nigerian": "Nigeria",
1256:     "pakistani": "Pakistan",
1257:     "ukrainian": "Ukraine",
1258:     "israeli": "Israel",
1259:     "iraqi": "Iraq",
1260:     "egyptian": "Egypt",
1261:     "yemeni": "Yemen",
1262:     "brazilian": "Brazil",
1263:     "nicaraguan": "Nicaragua",
1264:     "ecuadorian": "Ecuador",
1265:     "peruvian": "Peru",
1266:     "jamaican": "Jamaica",

```

```

1267: "filipino": "Philippines",
1268: "vietnamese": "Vietnam",
1269: "lebanese": "Lebanon",
1270: "libyan": "Libya",
1271: "sudanese": "Sudan",
1272: "ethiopian": "Ethiopia",
1273: "kenyan": "Kenya",
1274: "moroccan": "Morocco",
1275: "algerian": "Algeria",
1276: "bangladeshi": "Bangladesh",
1277: "cambodian": "Cambodia",
1278: "indonesian": "Indonesia",
1279: "dominican": "Dominican Republic",
1280: "panamanian": "Panama",
1281: "bolivian": "Bolivia",
1282: "chilean": "Chile",
1283: "argentine": "Argentina",
1284: "argentinian": "Argentina",
1285: "saudi": "Saudi Arabia",
1286: "turkish": "Turkey",
1287: }
1288: FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
1289:     "tehran": "Iran",
1290:     "caracas": "Venezuela",
1291:     "beijing": "China",
1292:     "moscow": "Russia",
1293:     "havana": "Cuba",
1294:     "kabul": "Afghanistan",
1295:     "damascus": "Syria",
1296:     "baghdad": "Iraq",
1297:     "tripoli": "Libya",
1298:     "mogadishu": "Somalia",
1299:     "bogota": "Colombia",
1300:     "managua": "Nicaragua",
1301:     "tegucigalpa": "Honduras",
1302:     "kyiv": "Ukraine",
1303: }
1304:
1305:
1306: def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:
1307:     alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
1308:     return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)
1309:
1310:
1311: PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))
1312: PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
1313: # A demonym that immediately precedes one of these nouns describes a person's
1314: # nationality, so it also earns origin:<country>, not just a contextual country:.
1315: PATTERN_DEMONYM_PERSON = _compile(
1316:     r"^\w*(?:national|nationals|citizen|citizens|immigrant|immigrants|migrant|migrants|"
1677: # origin:<COUNTRY> – validated against the sovereign-state vocab.
1678: for m in PATTERN_ORIGIN_CANDIDATE.finditer(text):
1679:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1680:     if resolved:
1681:         add(f"origin:{_normalize_country(resolved)}", span=m.span(1))
1682:
1683: # country:<NAME> – broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
2052: if low == "burma":
2053:     return "Myanmar"
2054: return s.replace(" ", ".")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", ".")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

origin:Guatemala (91 (0.9%))

Precise plain-English rule

Emitted as origin:Guatemala when the text has from Guatemala, with a comma after the country, or a demonym for Guatemala followed immediately by a person-origin noun.

Code text for this tag

```

113:     "Guatemala",
1217: # second word excludes those same prepositions via lookahead, so a greedy

```

```

1218: # capture can't swallow the preposition that introduces the *next* place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). `_resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!{:_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:{:_PREPOSITIONS})\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{:_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>,<State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE}),\s+({_PLACE})\b")

1238: DEMONYM_TO_COUNTRY: dict[str, str] = {
1239:     "iranian": "Iran",
1240:     "mexican": "Mexico",
1241:     "venezuelan": "Venezuela",
1242:     "chinese": "China",
1243:     "russian": "Russia",
1244:     "cuban": "Cuba",
1245:     "colombian": "Colombia",
1246:     "salvadoran": "El Salvador",
1247:     "salvadorian": "El Salvador",
1248:     "honduran": "Honduras",
1249:     "guatemalan": "Guatemala",
1250:     "haitian": "Haiti",
1251:     "somali": "Somalia",
1252:     "somalian": "Somalia",
1253:     "afghan": "Afghanistan",
1254:     "syrian": "Syria",
1255:     "nigerian": "Nigeria",
1256:     "pakistani": "Pakistan",
1257:     "ukrainian": "Ukraine",
1258:     "israeli": "Israel",
1259:     "iraqi": "Iraq",
1260:     "egyptian": "Egypt",
1261:     "yemeni": "Yemen",
1262:     "brazilian": "Brazil",
1263:     "nicaraguan": "Nicaragua",
1264:     "ecuadorian": "Ecuador",
1265:     "peruvian": "Peru",
1266:     "jamaican": "Jamaica",
1267:     "filipino": "Philippines",
1268:     "vietnamese": "Vietnam",
1269:     "lebanese": "Lebanon",
1270:     "libyan": "Libya",
1271:     "sudanese": "Sudan",
1272:     "ethiopian": "Ethiopia",
1273:     "kenyan": "Kenya",
1274:     "moroccan": "Morocco",
1275:     "algerian": "Algeria",
1276:     "bangladeshi": "Bangladesh",
1277:     "cambodian": "Cambodia",
1278:     "indonesian": "Indonesia",
1279:     "dominican": "Dominican Republic",
1280:     "panamanian": "Panama",
1281:     "bolivian": "Bolivia",
1282:     "chilean": "Chile",
1283:     "argentine": "Argentina",
1284:     "argentinian": "Argentina",
1285:     "saudi": "Saudi Arabia",
1286:     "turkish": "Turkey",
1287: }
1288: FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
1289:     "tehran": "Iran",
1290:     "caracas": "Venezuela",
1291:     "beijing": "China",
1292:     "moscow": "Russia",
1293:     "havana": "Cuba",
1294:     "kabul": "Afghanistan",
1295:     "damascus": "Syria",
1296:     "baghdad": "Iraq",
1297:     "tripoli": "Libya",
1298:     "mogadishu": "Somalia",
1299:     "bogota": "Colombia",
1300:     "managua": "Nicaragua",
1301:     "tegucigalpa": "Honduras",
1302:     "kyiv": "Ukraine",
1303: }
1304:
1305:
1306: def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:
1307:     alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
1308:     return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)
1309:
1310:
1311: PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))
1312: PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
1313: # A demonym that immediately precedes one of these nouns describes a person's
1314: # nationality, so it also earns origin:<country>, not just a contextual country:.
1315: PATTERN_DEMONYM_PERSON = re.compile(
1316:     r"^\w*(?:national|nationals|citizen|citizens|immigrant|immigrants|migrant|migrants|"
1317:
1318:
1319:
1320:
1321:
1322:
1323:
1324:
1325:
1326:
1327:
1328:
1329:
1330:
1331:
1332:
1333:
1334:
1335:
1336:
1337:
1338:
1339:
1340:
1341:
1342:
1343:
1344:
1345:
1346:
1347:
1348:
1349:
1350:
1351:
1352:
1353:
1354:
1355:
1356:
1357:
1358:
1359:
1360:
1361:
1362:
1363:
1364:
1365:
1366:
1367:
1368:
1369:
1370:
1371:
1372:
1373:
1374:
1375:
1376:
1377:
1378:
1379:
1380:
1381:
1382:
1383:
1384:
1385:
1386:
1387:
1388:
1389:
1390:
1391:
1392:
1393:
1394:
1395:
1396:
1397:
1398:
1399:
1400:
1401:
1402:
1403:
1404:
1405:
1406:
1407:
1408:
1409:
1410:
1411:
1412:
1413:
1414:
1415:
1416:
1417:
1418:
1419:
1420:
1421:
1422:
1423:
1424:
1425:
1426:
1427:
1428:
1429:
1430:
1431:
1432:
1433:
1434:
1435:
1436:
1437:
1438:
1439:
1440:
1441:
1442:
1443:
1444:
1445:
1446:
1447:
1448:
1449:
1450:
1451:
1452:
1453:
1454:
1455:
1456:
1457:
1458:
1459:
1460:
1461:
1462:
1463:
1464:
1465:
1466:
1467:
1468:
1469:
1470:
1471:
1472:
1473:
1474:
1475:
1476:
1477:
1478:
1479:
1480:
1481:
1482:
1483:
1484:
1485:
1486:
1487:
1488:
1489:
1490:
1491:
1492:
1493:
1494:
1495:
1496:
1497:
1498:
1499:
1500:
1501:
1502:
1503:
1504:
1505:
1506:
1507:
1508:
1509:
1510:
1511:
1512:
1513:
1514:
1515:
1516:
1517:
1518:
1519:
1520:
1521:
1522:
1523:
1524:
1525:
1526:
1527:
1528:
1529:
1530:
1531:
1532:
1533:
1534:
1535:
1536:
1537:
1538:
1539:
1540:
1541:
1542:
1543:
1544:
1545:
1546:
1547:
1548:
1549:
1550:
1551:
1552:
1553:
1554:
1555:
1556:
1557:
1558:
1559:
1560:
1561:
1562:
1563:
1564:
1565:
1566:
1567:
1568:
1569:
1570:
1571:
1572:
1573:
1574:
1575:
1576:
1577:
1578:
1579:
1580:
1581:
1582:
1583:
1584:
1585:
1586:
1587:
1588:
1589:
1590:
1591:
1592:
1593:
1594:
1595:
1596:
1597:
1598:
1599:
1600:
1601:
1602:
1603:
1604:
1605:
1606:
1607:
1608:
1609:
1610:
1611:
1612:
1613:
1614:
1615:
1616:
1617:
1618:
1619:
1620:
1621:
1622:
1623:
1624:
1625:
1626:
1627:
1628:
1629:
1630:
1631:
1632:
1633:
1634:
1635:
1636:
1637:
1638:
1639:
1640:
1641:
1642:
1643:
1644:
1645:
1646:
1647:
1648:
1649:
1650:
1651:
1652:
1653:
1654:
1655:
1656:
1657:
1658:
1659:
1660:
1661:
1662:
1663:
1664:
1665:
1666:
1667:
1668:
1669:
1670:
1671:
1672:
1673:
1674:
1675:
1676:
1677:
1678:
1679:
1680:
1681:
1682:
1683:
1684:
1685:
1686:
1687:
1688:
1689:
1690:
1691:
1692:
1693:
1694:
1695:
1696:
1697:
1698:
1699:
1700:
1701:
1702:
1703:
1704:
1705:
1706:
1707:
1708:
1709:
1710:
1711:
1712:
1713:
1714:
1715:
1716:
1717:
1718:
1719:
1720:
1721:
1722:
1723:
1724:
1725:
1726:
1727:
1728:
1729:
1730:
1731:
1732:
1733:
1734:
1735:
1736:
1737:
1738:
1739:
1740:
1741:
1742:
1743:
1744:
1745:
1746:
1747:
1748:
1749:
1750:
1751:
1752:
1753:
1754:
1755:
1756:
1757:
1758:
1759:
1760:
1761:
1762:
1763:
1764:
1765:
1766:
1767:
1768:
1769:
1770:
1771:
1772:
1773:
1774:
1775:
1776:
1777:
1778:
1779:
1780:
1781:
1782:
1783:
1784:
1785:
1786:
1787:
1788:
1789:
1790:
1791:
1792:
1793:
1794:
1795:
1796:
1797:
1798:
1799:
1800:
1801:
1802:
1803:
1804:
1805:
1806:
1807:
1808:
1809:
1810:
1811:
1812:
1813:
1814:
1815:
1816:
1817:
1818:
1819:
1820:
1821:
1822:
1823:
1824:
1825:
1826:
1827:
1828:
1829:
1830:
1831:
1832:
1833:
1834:
1835:
1836:
1837:
1838:
1839:
1840:
1841:
1842:
1843:
1844:
1845:
1846:
1847:
1848:
1849:
1850:
1851:
1852:
1853:
1854:
1855:
1856:
1857:
1858:
1859:
1860:
1861:
1862:
1863:
1864:
1865:
1866:
1867:
1868:
1869:
1870:
1871:
1872:
1873:
1874:
1875:
1876:
1877:
1878:
1879:
1880:
1881:
1882:
1883:
1884:
1885:
1886:
1887:
1888:
1889:
1890:
1891:
1892:
1893:
1894:
1895:
1896:
1897:
1898:
1899:
1900:
1901:
1902:
1903:
1904:
1905:
1906:
1907:
1908:
1909:
1910:
1911:
1912:
1913:
1914:
1915:
1916:
1917:
1918:
1919:
1920:
1921:
1922:
1923:
1924:
1925:
1926:
1927:
1928:
1929:
1930:
1931:
1932:
1933:
1934:
1935:
1936:
1937:
1938:
1939:
1940:
1941:
1942:
1943:
1944:
1945:
1946:
1947:
1948:
1949:
1950:
1951:
1952:
1953:
1954:
1955:
1956:
1957:
1958:
1959:
1960:
1961:
1962:
1963:
1964:
1965:
1966:
1967:
1968:
1969:
1970:
1971:
1972:
1973:
1974:
1975:
1976:
1977:
1978:
1979:
1980:
1981:
1982:
1983:
1984:
1985:
1986:
1987:
1988:
1989:
1990:
1991:
1992:
1993:
1994:
1995:
1996:
1997:
1998:
1999:
2000:
2001:
2002:
2003:
2004:
2005:
2006:
2007:
2008:
2009:
2010:
2011:
2012:
2013:
2014:
2015:
2016:
2017:
2018:
2019:
2020:
2021:
2022:
2023:
2024:
2025:
2026:
2027:
2028:
2029:
2030:
2031:
2032:
2033:
2034:
2035:
2036:
2037:
2038:
2039:
2040:
2041:
2042:
2043:
2044:
2045:
2046:
2047:
2048:
2049:
2050:
2051:
2052:
2053:
2054:
2055:
2056:
2057:
2058:
2059:
2060:
2061:
2062:
2063:
2064:
2065:
2066:
2067:
2068:
2069:
2070:
2071:
2072:
2073:
2074:
2075:
2076:
2077:
2078:
2079:
2080:
2081:
2082:
2083:
2084:
2085:
2086:
2087:
2088:
2089:
2090:
2091:
2092:
2093:
2094:
2095:
2096:
2097:
2098:
2099:
2100:
2101:
2102:
2103:
2104:
2105:
2106:
2107:
2108:
2109:
2110:
2111:
2112:
2113:
2114:
2115:
2116:
2117:
2118:
2119:
2120:
2121:
2122:
2123:
2124:
2125:
2126:
2127:
2128:
2129:
2130:
2131:
2132:
2133:
2134:
2135:
2136:
2137:
2138:
2139:
2140:
2141:
2142:
2143:
2144:
2145:
2146:
2147:
2148:
2149:
2150:
2151:
2152:
2153:
2154:
2155:
2156:
2157:
2158:
2159:
2160:
2161:
2162:
2163:
2164:
2165:
2166:
2167:
2168:
2169:
2170:
2171:
2172:
2173:
2174:
2175:
2176:
2177:
2178:
2179:
2180:
2181:
2182:
2183:
2184:
2185:
2186:
2187:
2188:
2189:
2190:
2191:
2192:
2193:
2194:
2195:
2196:
2197:
2198:
2199:
2200:
2201:
2202:
2203:
2204:
2205:
2206:
2207:
2208:
2209:
2210:
2211:
2212:
2213:
2214:
2215:
2216:
2217:
2218:
2219:
2220:
2221:
2222:
2223:
2224:
2225:
2226:
2227:
2228:
2229:
2230:
2231:
2232:
2233:
2234:
2235:
2236:
2237:
2238:
2239:
2240:
2241:
2242:
2243:
2244:
2245:
2246:
2247:
2248:
2249:
2250:
2251:
2252:
2253:
2254:
2255:
2256:
2257:
2258:
2259:
2260:
2261:
2262:
2263:
2264:
2265:
2266:
2267:
2268:
2269:
2270:
2271:
2272:
2273:
2274:
2275:
2276:
2277:
2278:
2279:
2280:
2281:
2282:
2283:
2284:
2285:
2286:
2287:
2288:
2289:
2290:
2291:
2292:
2293:
2294:
2295:
2296:
2297:
2298:
2299:
2300:
2301:
2302:
2303:
2304:
2305:
2306:
2307:
2308:
2309:
2310:
2311:
2312:
2313:
2314:
2315:
2316:
2317:
2318:
2319:
2320:
2321:
2322:
2323:
2324:
2325:
2326:
2327:
2328:
2329:
2330:
2331:
2332:
2333:
2334:
2335:
2336:
2337:
2338:
2339:
2340:
2341:
2342:
2343:
2344:
2345:
2346:
2347:
2348:
2349:
2350:
2351:
2352:
2353:
2354:
2355:
2356:
2357:
2358:
2359:
2360:
2361:
2362:
2363:
2364:
2365:
2366:
2367:
2368:
2369:
2370:
2371:
2372:
2373:
2374:
2375:
2376:
2377:
2378:
2379:
2380:
2381:
2382:
2383:
2384:
2385:
2386:
2387:
2388:
2389:
2390:
2391:
2392:
2393:
2394:
2395:
2396:
2397:
2398:
2399:
2400:
2401:
2402:
2403:
2404:
2405:
2406:
2407:
2408:
2409:
2410:
2411:
2412:
2413:
2414:
2415:
2416:
2417:
2418:
2419:
2420:
2421:
2422:
2423:
2424:
2425:
2426:
2427:
2428:
2429:
2430:
2431:
2432:
2433:
2434:
2435:
2436:
2437:
2438:
2439:
2440:
2441:
2442:
2443:
2444:
2445:
2446:
2447:
2448:
2449:
2450:
2451:
2452:
2453:
2454:
2455:
2456:
2457:
2458:
2459:
2460:
2461:
2462:
2463:
2464:
2465:
2466:
2467:
2468:
2469:
2470:
2471:
2472:
2473:
2474:
2475:
2476:
2477:
2478:
2479:
2480:
2481:
2482:
2483:
2484:
2485:
2486:
2487:
2488:
2489:
2490:
2491:
2492:
2493:
2494:
2495:
2496:
2497:
2498:
2499:
2500:
2501:
2502:
2503:
2504:
2505:
2506:
2507:
2508:
2509:
2510:
2511:
2512:
2513:
2514:
2515:
2516:
2517:
2518:
2519:
2520:
2521:
2522:
2523:
2524:
2525:
2526:
2527:
2528:
2529:
2530:
2531:
2532:
2533:
2534:
2535:
2536:
2537:
2538:
2539:
2540:
2541:
2542:
2543:
2544:
2545:
2546:
2547:
2548:
2549:
2550:
2551:
2552:
2553:
2554:
2555:
2556:
2557:
2558:
2559:
2560:
2561:
2562:
2563:
2564:
2565:
2566:
2567:
2568:
2569:
2570:
2571:
2572:
2573:
2574:
2575:
2576:
2577:
2578:
2579:
2580:
2581:
2582:
2583:
2584:
2585:
2586:
2587:
2588:
2589:
2590:
2591:
2592:
2593:
2594:
2595:
2596:
2597:
2598:
2599:
2600:
2601:
2602:
2603:
2604:
2605:
2606:
2607:
2608:
2609:
2610:
2611:
2612:
2613:
2614:
2615:
2616:
2617:
2618:
2619:
2620:
2621:
2622:
2623:
2624:
2625:
2626:
2627:
2628:
2629:
2630:
2631:
2632:
2633:
2634:
2635:
2636:
2637:
2638:
2639:
2640:
2641:
2642:
2643:
2644:
2645:
2646:
2647:
2648:
2649:
2650:
2651:
2652:
2653:
2654:
2655:
2656:
2657:
2658:
2659:
2660:
2661:
2662:
2663:
2664:
2665:
2666:
2667:
2668:
2669:
2670:
2671:
2672:
2673:
2674:
2675:
2676:
2677:
2678:
2679:
2680:
2681:
2682:
2683:
2684:
2685:
2686:
2687:
2688:
2689:
2690:
2691:
2692:
2693:
2694:
2695:
2696:
2697:
2698:
2699:
2700:
2701:
2702:
2703:
2704:
2705:
2706:
2707:
2708:
2709:
2710:
2711:
2712:
2713:
2714:
2715:
2716:
2717:
2718:
2719:
2720:
2721:
2722:
2723:
2724:
2725:
2726:
2727:
2728:
2729:
2730:
2731:
2732:
2733:
2734:
2735:
2736:
2737:
2738:
2739:
2740:
2741:
2742:
2743:
2744:
2745:
2746:
2747:
2748:
2749:
2750:
2751:
2752:
2753:
2754:
2755:
2756:
2757:
2758:
2759:
2760:
2761:
2762:
2763:
2764:
2765:
2766:
2767:
2768:
2769:
2770:
2771:
2772:
2773:
2774:
2775:
2776:
2777:
2778:
2779:
2780:
2781:
2782:
2783:
2784:
2785:
2786:
2787:
2788:
2789:
2790:
2791:
2792:
2793:
2794:
2795:
2796:
2797:
2798:
2799:
2800:
2801:
2802:
2803:
2804:
2805:
2806:
2807:
2808:
2809:
2810:
2811:
2812:
2813:
2814:
2815:
2816:
2817:
2818:
2819:
2820:
2821:
2822:
2823:
2824:
2825:
2826:
2827:
2828:
2829:
2830:
2831:
2832:
2833:
2834:
2835:
2836:
2837:
2838:
2839:
2840:
2841:
2842:
2843:
2844:
2845:
2846:
2847:
2848:
2849:
2850:
2851:
2852:
2853:
2854:
2855:
2856:
2857:
2858:
2859:
2860:
2861:
2862:
2863:
2864:
2865:
2866:
2867:
2868:
2869:
2870:
2871:
2872:
2873:
2874:
2875:
2876:
2877:
2878:
2879:
2880:
2881:
2882:
2883:
2884:
2885:
2886:
2887:
2888:
2889:
2890:
2891:
2892:
2893:
2894:
2895:
2896:
2897:
2898:
2899:
2900:
2901:
2902:
2903:
2904:
2905:
2906:
2907:
2908:
2909:
2910:
2911:
2912:
2913:
2914:
2915:
2916:
2917:
2918:
2919:
2920:
2921:
2922:
2923:
2924:
2925:
2926:
2927:
2928:
2929:
2930:
2931:
2932:
2933:
2934:
2935:
2936:
2937:
2938:
2939:
2940:
2941:
2942:
2943:
2944:
2945:
2946:
2947:
2948:
2949:
2950:
2951:
2952:
2953:
2954:
2955:
2956:
2957:
2958:
2959:
2960:
2961:
2962:
2963:
2964:
2965:
2966:
2967:
2968:
2969:
2970:
2971:
2972:
2973:
2974:
2975:
2976:
2977:
2978:
2979:
2980:
2981:
2982:
2983:
2984:
2985:
2986:
2987:
2988:
2989:
2990:
2991:
2992:
2993:
2994:
2995:
2996:
2997:
2998:
2999:
3000:

```

```

1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", ".")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", ".")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

origin:Honduras (86 (0.9%))

Precise plain-English rule

Emitted as origin:Honduras when the text has from Honduras, with a comma after the country, or a demonym for Honduras followed immediately by a person-origin noun.

Code text for this tag

```

117:     "Honduras",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the *next* place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). `_resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]+(?:\s+?!(?i:{_PREPOSITIONS}))\b[A-Za-z][a-zA-Z]+)?"
1225: # "from <Country>." - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*", "")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:!:{_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>, <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE}),\s+({_PLACE})\b")

1238: DEMONYM_TO_COUNTRY: dict[str, str] = {
1239:     "iranian": "Iran",
1240:     "mexican": "Mexico",
1241:     "venezuelan": "Venezuela",
1242:     "chinese": "China",
1243:     "russian": "Russia",
1244:     "cuban": "Cuba",
1245:     "colombian": "Colombia",
1246:     "salvadoran": "El Salvador",
1247:     "salvadorian": "El Salvador",
1248:     "honduran": "Honduras",
1249:     "guatemalan": "Guatemala",
1250:     "haitian": "Haiti",
1251:     "somali": "Somalia",
1252:     "somalian": "Somalia",
1253:     "afghan": "Afghanistan",
1254:     "syrian": "Syria",
1255:     "nigerian": "Nigeria",
1256:     "pakistani": "Pakistan",
1257:     "ukrainian": "Ukraine",
1258:     "israeli": "Israel",
1259:     "iraqi": "Iraq",
1260:     "egyptian": "Egypt",
1261:     "yemeni": "Yemen",
1262:     "brazilian": "Brazil",
1263:     "nicaraguan": "Nicaragua",
1264:     "ecuadorian": "Ecuador",
1265:     "peruvian": "Peru",
1266:     "jamaican": "Jamaica",
1267:     "filipino": "Philippines",
1268:     "vietnamese": "Vietnam",
1269:     "lebanese": "Lebanon",
1270:     "libyan": "Libya",
1271:     "sudanese": "Sudan",
1272:     "ethiopian": "Ethiopia",
1273:     "kenyan": "Kenya",
1274:     "moroccan": "Morocco",
1275:     "algerian": "Algeria",
1276:     "bangladeshi": "Bangladesh",
1277:     "cambodian": "Cambodia",
1278:     "indonesian": "Indonesia",
1279:     "dominican": "Dominican Republic",
1280:     "panamanian": "Panama",
1281:     "bolivian": "Bolivia",
1282:     "chilean": "Chile",
1283:     "argentine": "Argentina",
1284:     "argentinian": "Argentina",
1285:     "saudi": "Saudi Arabia",
1286:     "turkish": "Turkey",
1287: }
1288: FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
1289:     "tehran": "Iran",
1290:     "caracas": "Venezuela",

```

```

1291:     "beijing": "China",
1292:     "moscow": "Russia",
1293:     "havana": "Cuba",
1294:     "kabul": "Afghanistan",
1295:     "damascus": "Syria",
1296:     "baghdad": "Iraq",
1297:     "tripoli": "Libya",
1298:     "mogadishu": "Somalia",
1299:     "bogota": "Colombia",
1300:     "managua": "Nicaragua",
1301:     "tegucigalpa": "Honduras",
1302:     "kyiv": "Ukraine",
1303: }
1304:
1305:
1306: def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:
1307:     alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
1308:     return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)
1309:
1310:
1311: PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))
1312: PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
1313: # A demonym that immediately precedes one of these nouns describes a person's
1314: # nationality, so it also earns origin:<country>, not just a contextual country:.
1315: PATTERN_DEMONYM_PERSON = _compile(
1316:     r"^\W*(?:national|nationals|citizen|citizens|immigrant|immigrants|migrant|migrants|"
1677:     # origin:<COUNTRY> – validated against the sovereign-state vocab.
1678:     for m in PATTERN_ORIGIN_CANDIDATE.finditer(text):
1679:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1680:         if resolved:
1681:             add(f"origin:{_normalize_country(resolved)}", span=m.span(1))
1682:
1683:     # country:<NAME> – broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:
1687:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:     # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:     # country, which the bare-name validators above can't see. A demonym
1691:     # followed by a person noun also earns origin:.
1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end(): m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
2052: if low == "burma":
2053:     return "Myanmar"
2054: return s.replace(" ", ".")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", ".")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

origin:Venezuela (54 (0.5%))

Precise plain-English rule

Emitted as origin:Venezuela when the text has from Venezuela, with a comma after the country, or a demonym for Venezuela followed immediately by a person-origin noun.

Code text for this tag

```

225:     "Venezuela",
1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]+(?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]+)?"
1225: # "from <Country>," – anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:({_PREPOSITIONS}))\s+({_PLACE})\b")
1229: # "<Place>," <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE})\s+({_PLACE})\b")
1238: DEMONYM_TO_COUNTRY: dict[str, str] = {
1239:     "iranian": "Iran",
1240:     "mexican": "Mexico",
1241:     "venezuelan": "Venezuela",
1242:     "chinese": "China",
1243:     "russian": "Russia",
1244:     "cuban": "Cuba",
1245:     "colombian": "Colombia",
1246:     "salvadoran": "El Salvador",
1247:     "salvadorian": "El Salvador",

```

```

1248: "honduran": "Honduras",
1249: "guatemalan": "Guatemala",
1250: "haitian": "Haiti",
1251: "somali": "Somalia",
1252: "somalian": "Somalia",
1253: "afghan": "Afghanistan",
1254: "syrian": "Syria",
1255: "nigerian": "Nigeria",
1256: "pakistanian": "Pakistan",
1257: "ukrainian": "Ukraine",
1258: "israeli": "Israel",
1259: "iraqi": "Iraq",
1260: "egyptian": "Egypt",
1261: "yemeni": "Yemen",
1262: "brazilian": "Brazil",
1263: "nicaraguan": "Nicaragua",
1264: "ecuadorian": "Ecuador",
1265: "peruvian": "Peru",
1266: "jamaican": "Jamaica",
1267: "filipino": "Philippines",
1268: "vietnamese": "Vietnam",
1269: "lebanese": "Lebanon",
1270: "libyan": "Libya",
1271: "sudanese": "Sudan",
1272: "ethiopian": "Ethiopia",
1273: "kenyan": "Kenya",
1274: "moroccan": "Morocco",
1275: "algerian": "Algeria",
1276: "bangladeshi": "Bangladesh",
1277: "cambodian": "Cambodia",
1278: "indonesian": "Indonesia",
1279: "dominican": "Dominican Republic",
1280: "panamanian": "Panama",
1281: "bolivian": "Bolivia",
1282: "chilean": "Chile",
1283: "argentine": "Argentina",
1284: "argentinian": "Argentina",
1285: "saudi": "Saudi Arabia",
1286: "turkish": "Turkey",
1287: }
1288: FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
1289:     "tehran": "Iran",
1290:     "caracas": "Venezuela",
1291:     "beijing": "China",
1292:     "moscow": "Russia",
1293:     "havana": "Cuba",
1294:     "kabul": "Afghanistan",
1295:     "damascus": "Syria",
1296:     "baghdad": "Iraq",
1297:     "tripoli": "Libya",
1298:     "mogadishu": "Somalia",
1299:     "bogota": "Colombia",
1300:     "managua": "Nicaragua",
1301:     "tegucigalpa": "Honduras",
1302:     "kyiv": "Ukraine",
1303: }
1304:
1305:
1306: def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:
1307:     alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
1308:     return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)
1309:
1310:
1311: PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))
1312: PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
1313: # A demonym that immediately precedes one of these nouns describes a person's
1314: # nationality, so it also earns origin:<country>, not just a contextual country:.
1315: PATTERN_DEMONYM_PERSON = _compile(
1316:     r"^\W(?:national|nationals|citizen|citizens|immigrant|immigrants|migrant|migrants|"
1677:     # origin:<COUNTRY> – validated against the sovereign-state vocab.
1678:     for m in PATTERN_ORIGIN_CANDIDATE.finditer(text):
1679:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1680:         if resolved:
1681:             add(f"origin:{_normalize_country(resolved)}", span=m.span(1))
1682:
1683:     # country:<NAME> – broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:
1687:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:     # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:     # country, which the bare-name validators above can't see. A demonym
1691:     # followed by a person noun also earns origin:.
1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical

```

```

2063: vocab spelling.
2064:
2065: Tries the full phrase first, then its leading word, so a greedy
2066: case-insensitive capture that swallowed a trailing lowercase token

```

origin:Cuba (52 (0.5%))

Precise plain-English rule

Emitted as origin:Cuba when the text has from Cuba, with a comma after the country, or a demonym for Cuba followed immediately by a person-origin noun.

Code text for this tag

```

88: "Cuba",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). `_resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z](?:\s+(?!({_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]+)?"
1225: # "from <Country>," - anchors the ORIGIN validator
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE})\s+({_PLACE})\b")

1238: DEMONYM_TO_COUNTRY: dict[str, str] = {
1239:     "iranian": "Iran",
1240:     "mexican": "Mexico",
1241:     "venezuelan": "Venezuela",
1242:     "chinese": "China",
1243:     "russian": "Russia",
1244:     "cuban": "Cuba",
1245:     "colombian": "Colombia",
1246:     "salvadoran": "El Salvador",
1247:     "salvadorian": "El Salvador",
1248:     "honduran": "Honduras",
1249:     "guatemalan": "Guatemala",
1250:     "haitian": "Haiti",
1251:     "somali": "Somalia",
1252:     "somalian": "Somalia",
1253:     "afghan": "Afghanistan",
1254:     "syrian": "Syria",
1255:     "nigerian": "Nigeria",
1256:     "pakistani": "Pakistan",
1257:     "ukrainian": "Ukraine",
1258:     "israeli": "Israel",
1259:     "iraqi": "Iraq",
1260:     "egyptian": "Egypt",
1261:     "yemeni": "Yemen",
1262:     "brazilian": "Brazil",
1263:     "nicaraguan": "Nicaragua",
1264:     "ecuadorian": "Ecuador",
1265:     "peruvian": "Peru",
1266:     "jamaican": "Jamaica",
1267:     "filipino": "Philippines",
1268:     "vietnamese": "Vietnam",
1269:     "lebanese": "Lebanon",
1270:     "libyan": "Libya",
1271:     "sudanese": "Sudan",
1272:     "ethiopian": "Ethiopia",
1273:     "kenyan": "Kenya",
1274:     "moroccan": "Morocco",
1275:     "algerian": "Algeria",
1276:     "bangladeshi": "Bangladesh",
1277:     "cambodian": "Cambodia",
1278:     "indonesian": "Indonesia",
1279:     "dominican": "Dominican Republic",
1280:     "panamanian": "Panama",
1281:     "bolivian": "Bolivia",
1282:     "chilean": "Chile",
1283:     "argentine": "Argentina",
1284:     "argentinian": "Argentina",
1285:     "saudi": "Saudi Arabia",
1286:     "turkish": "Turkey",
1287: }
1288: FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
1289:     "tehran": "Iran",
1290:     "caracas": "Venezuela",
1291:     "beijing": "China",
1292:     "moscow": "Russia",
1293:     "havana": "Cuba",
1294:     "kabul": "Afghanistan",
1295:     "damascus": "Syria",
1296:     "baghdad": "Iraq",
1297:     "tripoli": "Libya",
1298:     "mogadishu": "Somalia",
1299:     "bogota": "Colombia",
1300:     "managua": "Nicaragua",
1301:     "tegucigalpa": "Honduras",
1302:     "kyiv": "Ukraine",
1303: }
1304:
1305:
1306: def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:
1307:     alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
1308:     return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)
1309:
1310:
1311: PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))
1312: PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
1313: # A demonym that immediately precedes one of these nouns describes a person's
1314: # nationality, so it also earns origin:<country>, not just a contextual country:.

```

```

1315: PATTERN_DEMONYM_PERSON = _compile(
1316:     r"^\W*(?:national|nationals|citizen|citizens|immigrant|immigrants|migrant|migrants|"

1677:     # origin:<COUNTRY> – validated against the sovereign-state vocab.
1678:     for m in PATTERN_ORIGIN_CANDIDATE.finditer(text):
1679:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1680:         if resolved:
1681:             add(f"origin:{_normalize_country(resolved)}", span=m.span(1))
1682:
1683:     # country:<NAME> – broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:
1687:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:     # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:     # country, which the bare-name validators above can't see. A demonym
1691:     # followed by a person noun also earns origin:.
1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end(): m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

origin:Ecuador (25 (0.2%))

Precise plain-English rule

Emitted as origin:Ecuador when the text has from Ecuador, with a comma after the country, or a demonym for Ecuador followed immediately by a person-origin noun.

Code text for this tag

```

96:     "Ecuador",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]+(?:\s+(?!(?:{_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]+)?"
1225: # "from <Country>," – anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE})\s+({_PLACE})\b")

1238: DEMONYM_TO_COUNTRY: dict[str, str] = {
1239:     "iranian": "Iran",
1240:     "mexican": "Mexico",
1241:     "venezuelan": "Venezuela",
1242:     "chinese": "China",
1243:     "russian": "Russia",
1244:     "cuban": "Cuba",
1245:     "colombian": "Colombia",
1246:     "salvadoran": "El Salvador",
1247:     "salvadorian": "El Salvador",
1248:     "honduran": "Honduras",
1249:     "guatemalan": "Guatemala",
1250:     "haitian": "Haiti",
1251:     "somali": "Somalia",
1252:     "somalian": "Somalia",
1253:     "afghan": "Afghanistan",
1254:     "syrian": "Syria",
1255:     "nigerian": "Nigeria",
1256:     "pakistanian": "Pakistan",
1257:     "ukrainian": "Ukraine",
1258:     "israeli": "Israel",
1259:     "iraqi": "Iraq",
1260:     "egyptian": "Egypt",
1261:     "yemeni": "Yemen",
1262:     "brazilian": "Brazil",
1263:     "nicaraguan": "Nicaragua",
1264:     "ecuadorian": "Ecuador",
1265:     "peruvian": "Peru",
1266:     "jamaican": "Jamaica",
1267:     "filipino": "Philippines",
1268:     "vietnamese": "Vietnam",
1269:     "lebanese": "Lebanon",
1270:     "libyan": "Libya",
1271:     "sudanese": "Sudan",

```

```

1272: "ethiopian": "Ethiopia",
1273: "kenyan": "Kenya",
1274: "moroccan": "Morocco",
1275: "algerian": "Algeria",
1276: "bangladeshi": "Bangladesh",
1277: "cambodian": "Cambodia",
1278: "indonesian": "Indonesia",
1279: "dominican": "Dominican Republic",
1280: "panamanian": "Panama",
1281: "bolivian": "Bolivia",
1282: "chilean": "Chile",
1283: "argentine": "Argentina",
1284: "argentinian": "Argentina",
1285: "saudi": "Saudi Arabia",
1286: "turkish": "Turkey",
1287: }
1288: FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
1289:     "tehran": "Iran",
1290:     "caracas": "Venezuela",
1291:     "beijing": "China",
1292:     "moscow": "Russia",
1293:     "havana": "Cuba",
1294:     "kabul": "Afghanistan",
1295:     "damascus": "Syria",
1296:     "baghdad": "Iraq",
1297:     "tripoli": "Libya",
1298:     "mogadishu": "Somalia",
1299:     "bogota": "Colombia",
1300:     "managua": "Nicaragua",
1301:     "tegucigalpa": "Honduras",
1302:     "kyiv": "Ukraine",
1303: }
1304:
1305:
1306: def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:
1307:     alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
1308:     return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)
1309:
1310:
1311: PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))
1312: PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
1313: # A demonym that immediately precedes one of these nouns describes a person's
1314: # nationality, so it also earns origin:<country>, not just a contextual country:.
1315: PATTERN_DEMONYM_PERSON = _compile(
1316:     r"^\w*(?:national|nationals|citizen|citizens|immigrant|immigrants|migrant|migrants|"
1677: # origin:<COUNTRY> - validated against the sovereign-state vocab.
1678: for m in PATTERN_ORIGIN_CANDIDATE.finditer(text):
1679:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1680:     if resolved:
1681:         add(f"origin:{_normalize_country(resolved)}", span=m.span(1))
1682:
1683: # country:<NAME> - broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
2052: if low == "burma":
2053:     return "Myanmar"
2054: return s.replace(" ", ".")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", ".")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

origin:China (23 (0.2%))

Precise plain-English rule

Emitted as origin:China when the text has from China, with a comma after the country, or a demonym for China followed immediately by a person-origin noun.

Code text for this tag

```

82:     "China",
1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). '_resolve_vocab'
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").

```

```

1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!{:_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]+)?"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE}\s+({_PLACE})\b")

1238: DEMONYM_TO_COUNTRY: dict[str, str] = {
1239:     "iranian": "Iran",
1240:     "mexican": "Mexico",
1241:     "venezuelan": "Venezuela",
1242:     "chinese": "China",
1243:     "russian": "Russia",
1244:     "cuban": "Cuba",
1245:     "colombian": "Colombia",
1246:     "salvadoran": "El Salvador",
1247:     "salvadorian": "El Salvador",
1248:     "honduran": "Honduras",
1249:     "guatemalan": "Guatemala",
1250:     "haitian": "Haiti",
1251:     "somali": "Somalia",
1252:     "somalian": "Somalia",
1253:     "afghan": "Afghanistan",
1254:     "syrian": "Syria",
1255:     "nigerian": "Nigeria",
1256:     "pakistani": "Pakistan",
1257:     "ukrainian": "Ukraine",
1258:     "israeli": "Israel",
1259:     "iraqi": "Iraq",
1260:     "egyptian": "Egypt",
1261:     "yemeni": "Yemen",
1262:     "brazilian": "Brazil",
1263:     "nicaraguan": "Nicaragua",
1264:     "ecuadorian": "Ecuador",
1265:     "peruvian": "Peru",
1266:     "jamaican": "Jamaica",
1267:     "filipino": "Philippines",
1268:     "vietnamese": "Vietnam",
1269:     "lebanese": "Lebanon",
1270:     "libyan": "Libya",
1271:     "sudanese": "Sudan",
1272:     "ethiopian": "Ethiopia",
1273:     "kenyan": "Kenya",
1274:     "moroccan": "Morocco",
1275:     "algerian": "Algeria",
1276:     "bangladeshi": "Bangladesh",
1277:     "cambodian": "Cambodia",
1278:     "indonesian": "Indonesia",
1279:     "dominican": "Dominican Republic",
1280:     "panamanian": "Panama",
1281:     "bolivian": "Bolivia",
1282:     "chilean": "Chile",
1283:     "argentine": "Argentina",
1284:     "argentinian": "Argentina",
1285:     "saudi": "Saudi Arabia",
1286:     "turkish": "Turkey",
1287: }
1288: FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
1289:     "tehran": "Iran",
1290:     "caracas": "Venezuela",
1291:     "beijing": "China",
1292:     "moscow": "Russia",
1293:     "havana": "Cuba",
1294:     "kabul": "Afghanistan",
1295:     "damascus": "Syria",
1296:     "baghdad": "Iraq",
1297:     "tripoli": "Libya",
1298:     "mogadishu": "Somalia",
1299:     "bogota": "Colombia",
1300:     "managua": "Nicaragua",
1301:     "tegucigalpa": "Honduras",
1302:     "kyiv": "Ukraine",
1303: }
1304:
1305:
1306: def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:
1307:     alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
1308:     return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)
1309:
1310:
1311: PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))
1312: PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
1313: # A demonym that immediately precedes one of these nouns describes a person's
1314: # nationality, so it also earns origin:<country>, not just a contextual country:.
1315: PATTERN_DEMONYM_PERSON = _compile(
1316:     r"^\W*(?:national|nationals|citizen|citizens|immigrant|immigrants|migrant|migrants)"
1317: )

1677: # origin:<COUNTRY> - validated against the sovereign-state vocab.
1678: for m in PATTERN_ORIGIN_CANDIDATE.finditer(text):
1679:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1680:     if resolved:
1681:         add(f"origin:{_normalize_country(resolved)}", span=m.span(1))
1682:
1683: # country:<NAME> - broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))

```

```

1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
2046:         if low in ("usa", "united states"):
2047:             return "United-States"
2048:         if low == "uk":
2049:             return "United-Kingdom"
2050:         if low == "uae":
2051:             return "United-Arab-Emirates"
2052:         if low == "burma":
2053:             return "Myanmar"
2054:         return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

origin:Iran (23 (0.2%))

Precise plain-English rule

Emitted as origin:Iran when the text has from Iran, with a comma after the country, or a demonym for Iran followed immediately by a person-origin noun.

Code text for this tag

```

122:     "Iran",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). '_resolve_vocab'
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]+)?"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1238: DEMONYM_TO_COUNTRY: dict[str, str] = {
1239:     "iranian": "Iran",
1240:     "mexican": "Mexico",
1241:     "venezuelan": "Venezuela",
1242:     "chinese": "China",
1243:     "russian": "Russia",
1244:     "cuban": "Cuba",
1245:     "colombian": "Colombia",
1246:     "salvadoran": "El Salvador",
1247:     "salvadorian": "El Salvador",
1248:     "honduran": "Honduras",
1249:     "guatemalan": "Guatemala",
1250:     "haitian": "Haiti",
1251:     "somali": "Somalia",
1252:     "somalian": "Somalia",
1253:     "afghan": "Afghanistan",
1254:     "syrian": "Syria",
1255:     "nigerian": "Nigeria",
1256:     "pakistani": "Pakistan",
1257:     "ukrainian": "Ukraine",
1258:     "israeli": "Israel",
1259:     "iraqi": "Iraq",
1260:     "egyptian": "Egypt",
1261:     "yemeni": "Yemen",
1262:     "brazilian": "Brazil",
1263:     "nicaraguan": "Nicaragua",
1264:     "ecuadorian": "Ecuador",
1265:     "peruvian": "Peru",
1266:     "jamaican": "Jamaica",
1267:     "filipino": "Philippines",
1268:     "vietnamese": "Vietnam",
1269:     "lebanese": "Lebanon",
1270:     "libyan": "Libya",
1271:     "sudanese": "Sudan",
1272:     "ethiopian": "Ethiopia",
1273:     "kenyan": "Kenya",
1274:     "moroccan": "Morocco",
1275:     "algerian": "Algeria",
1276:     "bangladeshi": "Bangladesh",
1277:     "cambodian": "Cambodia",
1278:     "indonesian": "Indonesia",
1279:     "dominican": "Dominican Republic",
1280:     "panamanian": "Panama",
1281:     "bolivian": "Bolivia",
1282:     "chilean": "Chile",
1283:     "argentine": "Argentina",
1284:     "argentinian": "Argentina",
1285:     "saudi": "Saudi Arabia",
1286:     "turkish": "Turkey",
1287: }
1288: FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
1289:     "tehran": "Iran",
1290:     "caracas": "Venezuela",
1291:     "beijing": "China",
1292:     "moscow": "Russia",
1293:     "havana": "Cuba",
1294:     "kabul": "Afghanistan",
1295:     "damascus": "Syria",

```

```

1296:     "baghdad": "Iraq",
1297:     "tripoli": "Libya",
1298:     "mogadishu": "Somalia",
1299:     "bogota": "Colombia",
1300:     "managua": "Nicaragua",
1301:     "tegucigalpa": "Honduras",
1302:     "kyiv": "Ukraine",
1303: }
1304:
1305:
1306: def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:
1307:     alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
1308:     return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)
1309:
1310:
1311: PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))
1312: PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
1313: # A demonym that immediately precedes one of these nouns describes a person's
1314: # nationality, so it also earns origin:<country>, not just a contextual country:.
1315: PATTERN_DEMONYM_PERSON = _compile(
1316:     r"^\W*(?:national|nationals|citizen|citizens|immigrant|immigrants|migrant|migrants|"
1677:     # origin:<COUNTRY> – validated against the sovereign-state vocab.
1678:     for m in PATTERN_ORIGIN_CANDIDATE.finditer(text):
1679:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1680:         if resolved:
1681:             add(f"origin:{_normalize_country(resolved)}", span=m.span(1))
1682:
1683:     # country:<NAME> – broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:
1687:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:     # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:     # country, which the bare-name validators above can't see. A demonym
1691:     # followed by a person noun also earns origin:.
1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

origin:Laos (23 (0.2%))

Precise plain-English rule

Emitted as origin:Laos when the text has from Laos, with a comma after the country, or a demonym for Laos followed immediately by a person-origin noun.

Code text for this tag

```

136:     "Laos",
1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). ` _resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-Z][a-zA-Z]*(?:\s+(?!(?:{_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]+)?"
1225: # "from <Country>," – anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE})\s+({_PLACE})\b")
1238: DEMONYM_TO_COUNTRY: dict[str, str] = {
1239:     "iranian": "Iran",
1240:     "mexican": "Mexico",
1241:     "venezuelan": "Venezuela",
1242:     "chinese": "China",
1243:     "russian": "Russia",
1244:     "cuban": "Cuba",
1245:     "colombian": "Colombia",
1246:     "salvadoran": "El Salvador",
1247:     "salvadorian": "El Salvador",
1248:     "honduran": "Honduras",
1249:     "guatemalan": "Guatemala",
1250:     "haitian": "Haiti",
1251:     "somali": "Somalia",
1252:     "somalian": "Somalia",

```

```

1253: "afghan": "Afghanistan",
1254: "syrian": "Syria",
1255: "nigerian": "Nigeria",
1256: "pakistani": "Pakistan",
1257: "ukrainian": "Ukraine",
1258: "israeli": "Israel",
1259: "iraqi": "Iraq",
1260: "egyptian": "Egypt",
1261: "yemeni": "Yemen",
1262: "brazilian": "Brazil",
1263: "nicaraguan": "Nicaragua",
1264: "ecuadorian": "Ecuador",
1265: "peruvian": "Peru",
1266: "jamaican": "Jamaica",
1267: "filipino": "Philippines",
1268: "vietnamese": "Vietnam",
1269: "lebanese": "Lebanon",
1270: "libyan": "Libya",
1271: "sudanese": "Sudan",
1272: "ethiopian": "Ethiopia",
1273: "kenyan": "Kenya",
1274: "moroccan": "Morocco",
1275: "algerian": "Algeria",
1276: "bangladeshi": "Bangladesh",
1277: "cambodian": "Cambodia",
1278: "indonesian": "Indonesia",
1279: "dominican": "Dominican Republic",
1280: "panamanian": "Panama",
1281: "bolivian": "Bolivia",
1282: "chilean": "Chile",
1283: "argentine": "Argentina",
1284: "argentinian": "Argentina",
1285: "saudi": "Saudi Arabia",
1286: "turkish": "Turkey",
1287: }
1288: FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
1289:     "tehran": "Iran",
1290:     "caracas": "Venezuela",
1291:     "beijing": "China",
1292:     "moscow": "Russia",
1293:     "havana": "Cuba",
1294:     "kabul": "Afghanistan",
1295:     "damascus": "Syria",
1296:     "baghdad": "Iraq",
1297:     "tripoli": "Libya",
1298:     "mogadishu": "Somalia",
1299:     "bogota": "Colombia",
1300:     "managua": "Nicaragua",
1301:     "tegucigalpa": "Honduras",
1302:     "kyiv": "Ukraine",
1303: }
1304:
1305:
1306: def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:
1307:     alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
1308:     return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)
1309:
1310:
1311: PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))
1312: PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
1313: # A demonym that immediately precedes one of these nouns describes a person's
1314: # nationality, so it also earns origin:<country>, not just a contextual country:.
1315: PATTERN_DEMONYM_PERSON = _compile(
1316:     r"^\w*(?:national|nationals|citizen|citizens|immigrant|immigrants|migrant|migrants)"
1317: )
1318:
1319:
1320: # origin:<COUNTRY> – validated against the sovereign-state vocab.
1321: for m in PATTERN_ORIGIN_CANDIDATE.finditer(text):
1322:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1323:     if resolved:
1324:         add(f"origin:{_normalize_country(resolved)}", span=m.span(1))
1325:
1326:
1327: # country:<NAME> – broader: any contextual country mention.
1328: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1329:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1330:     if resolved:
1331:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1332:
1333:
1334: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1335: # country, which the bare-name validators above can't see. A demonym
1336: # followed by a person noun also earns origin:.
1337: for m in PATTERN_DEMONYM.finditer(text):
1338:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1339:     if country and country.lower() in COUNTRY_LOWER:
1340:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1341:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1342:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1343:
1344: for m in PATTERN_FOREIGN_CITY.finditer(text):
1345:
1346:
1347: if low in ("usa", "united states"):
1348:     return "United-States"
1349: if low == "uk":
1350:     return "United-Kingdom"
1351: if low == "uae":
1352:     return "United-Arab-Emirates"
1353: if low == "burma":
1354:     return "Myanmar"
1355: return s.replace(" ", "-")
1356:
1357:
1358: def _normalize_state(s: str) -> str:
1359:     return s.replace(" ", "-")
1360:
1361:
1362: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
1363:     """Resolve a candidate (any case, possibly two words) to its canonical
1364:     vocab spelling.
1365:
1366:     Tries the full phrase first, then its leading word, so a greedy
1367:     case-insensitive capture that swallowed a trailing lowercase token

```

origin:Colombia (22 (0.2%))

Precise plain-English rule

Emitted as origin:Colombia when the text has from Colombia, with a comma after the country, or a demonym for Colombia followed immediately by a person-origin noun.

Code text for this tag

```
83:     "Colombia",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the 'next' place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). `_resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*(?)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:{_PREPOSITIONS})\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE})\s+({_PLACE})\b")

1238: DEMONYM_TO_COUNTRY: dict[str, str] = {
1239:     "iranian": "Iran",
1240:     "mexican": "Mexico",
1241:     "venezuelan": "Venezuela",
1242:     "chinese": "China",
1243:     "russian": "Russia",
1244:     "cuban": "Cuba",
1245:     "colombian": "Colombia",
1246:     "salvadoran": "El Salvador",
1247:     "salvadorian": "El Salvador",
1248:     "honduran": "Honduras",
1249:     "guatemalan": "Guatemala",
1250:     "haitian": "Haiti",
1251:     "somali": "Somalia",
1252:     "somalian": "Somalia",
1253:     "afghan": "Afghanistan",
1254:     "syrian": "Syria",
1255:     "nigerian": "Nigeria",
1256:     "pakistani": "Pakistan",
1257:     "ukrainian": "Ukraine",
1258:     "israeli": "Israel",
1259:     "iraqi": "Iraq",
1260:     "egyptian": "Egypt",
1261:     "yemeni": "Yemen",
1262:     "brazilian": "Brazil",
1263:     "nicaraguan": "Nicaragua",
1264:     "ecuadorian": "Ecuador",
1265:     "peruvian": "Peru",
1266:     "jamaican": "Jamaica",
1267:     "filipino": "Philippines",
1268:     "vietnamese": "Vietnam",
1269:     "lebanese": "Lebanon",
1270:     "libyan": "Libya",
1271:     "sudanese": "Sudan",
1272:     "ethiopian": "Ethiopia",
1273:     "kenyan": "Kenya",
1274:     "moroccan": "Morocco",
1275:     "algerian": "Algeria",
1276:     "bangladeshi": "Bangladesh",
1277:     "cambodian": "Cambodia",
1278:     "indonesian": "Indonesia",
1279:     "dominican": "Dominican Republic",
1280:     "panamanian": "Panama",
1281:     "bolivian": "Bolivia",
1282:     "chilean": "Chile",
1283:     "argentine": "Argentina",
1284:     "argentinian": "Argentina",
1285:     "saudi": "Saudi Arabia",
1286:     "turkish": "Turkey",
1287: }
1288: FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
1289:     "tehran": "Iran",
1290:     "caracas": "Venezuela",
1291:     "beijing": "China",
1292:     "moscow": "Russia",
1293:     "havana": "Cuba",
1294:     "kabul": "Afghanistan",
1295:     "damascus": "Syria",
1296:     "baghdad": "Iraq",
1297:     "tripoli": "Libya",
1298:     "mogadishu": "Somalia",
1299:     "bogota": "Colombia",
1300:     "managua": "Nicaragua",
1301:     "tegucigalpa": "Honduras",
1302:     "kyiv": "Ukraine",
1303: }
1304:
1305:
1306: def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:
1307:     alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
1308:     return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)
1309:
1310:
1311: PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))
1312: PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
1313: # A demonym that immediately precedes one of these nouns describes a person's
1314: # nationality, so it also earns origin:<country>, not just a contextual country:.
1315: PATTERN_DEMONYM_PERSON = _compile(
1316:     r"^\W*(?:national|nationals|citizen|citizens|immigrant|immigrants|migrant|migrants|"
1677: # origin:<COUNTRY> - validated against the sovereign-state vocab.
1678: for m in PATTERN_ORIGIN_CANDIDATE.finditer(text):
1679:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
```

```

1680:         if resolved:
1681:             add(f"origin:{_normalize_country(resolved)}", span=m.span(1))
1682:
1683:         # country:<NAME> – broader: any contextual country mention.
1684:         for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:             resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:             if resolved:
1687:                 add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:         # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:         # country, which the bare-name validators above can't see. A demonym
1691:         # followed by a person noun also earns origin:.
1692:         for m in PATTERN_DEMONYM.finditer(text):
1693:             country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:             if country and country.lower() in COUNTRY_LOWER:
1695:                 add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:                 if PATTERN_DEMONYM_PERSON.search(text[m.end(): m.end() + 24]):
1697:                     add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:             for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:
2046:         if low in ("usa", "united states"):
2047:             return "United-States"
2048:         if low == "uk":
2049:             return "United-Kingdom"
2050:         if low == "uae":
2051:             return "United-Arab-Emirates"
2052:         if low == "burma":
2053:             return "Myanmar"
2054:         return s.replace(" ", ".")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", ".")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

origin:Vietnam (18 (0.2%))

Precise plain-English rule

Emitted as origin:Vietnam when the text has from Vietnam, with a comma after the country, or a demonym for Vietnam followed immediately by a person-origin noun.

Code text for this tag

```

226:     "Vietnam",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the *next* place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). ` _resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z](?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]?"
1225: # "from <Country>," – anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{ _PREPOSITIONS })\s+({_PLACE})\b")
1229: # "<Place>," <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE}\s+({_PLACE})\b")

1238: DEMONYM_TO_COUNTRY: dict[str, str] = {
1239:     "iranian": "Iran",
1240:     "mexican": "Mexico",
1241:     "venezuelan": "Venezuela",
1242:     "chinese": "China",
1243:     "russian": "Russia",
1244:     "cuban": "Cuba",
1245:     "colombian": "Colombia",
1246:     "salvadoran": "El Salvador",
1247:     "salvadorian": "El Salvador",
1248:     "honduran": "Honduras",
1249:     "guatemalan": "Guatemala",
1250:     "haitian": "Haiti",
1251:     "somali": "Somalia",
1252:     "somalian": "Somalia",
1253:     "afghan": "Afghanistan",
1254:     "syrian": "Syria",
1255:     "nigerian": "Nigeria",
1256:     "pakistani": "Pakistan",
1257:     "ukrainian": "Ukraine",
1258:     "israeli": "Israel",
1259:     "iraqi": "Iraq",
1260:     "egyptian": "Egypt",
1261:     "yemeni": "Yemen",
1262:     "brazilian": "Brazil",
1263:     "nicaraguan": "Nicaragua",
1264:     "ecuadorian": "Ecuador",
1265:     "peruvian": "Peru",
1266:     "jamaican": "Jamaica",
1267:     "filipino": "Philippines",
1268:     "vietnamese": "Vietnam",
1269:     "lebanese": "Lebanon",
1270:     "libyan": "Libya",
1271:     "sudanese": "Sudan",
1272:     "ethiopian": "Ethiopia",
1273:     "kenyan": "Kenya",
1274:     "moroccan": "Morocco",
1275:     "algerian": "Algeria",
1276:     "bangladeshi": "Bangladesh",
1277:     "cambodian": "Cambodia",

```

```

1278:     "indonesian": "Indonesia",
1279:     "dominican": "Dominican Republic",
1280:     "panamanian": "Panama",
1281:     "bolivian": "Bolivia",
1282:     "chilean": "Chile",
1283:     "argentine": "Argentina",
1284:     "argentinian": "Argentina",
1285:     "saudi": "Saudi Arabia",
1286:     "turkish": "Turkey",
1287: }
1288: FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
1289:     "tehran": "Iran",
1290:     "caracas": "Venezuela",
1291:     "beijing": "China",
1292:     "moscow": "Russia",
1293:     "havana": "Cuba",
1294:     "kabul": "Afghanistan",
1295:     "damascus": "Syria",
1296:     "baghdad": "Iraq",
1297:     "tripoli": "Libya",
1298:     "mogadishu": "Somalia",
1299:     "bogota": "Colombia",
1300:     "managua": "Nicaragua",
1301:     "tegucigalpa": "Honduras",
1302:     "kyiv": "Ukraine",
1303: }
1304:
1305:
1306: def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:
1307:     alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
1308:     return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)
1309:
1310:
1311: PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))
1312: PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
1313: # A demonym that immediately precedes one of these nouns describes a person's
1314: # nationality, so it also earns origin:<country>, not just a contextual country:.
1315: PATTERN_DEMONYM_PERSON = _compile(
1316:     r"^\w*(?:national|nationals|citizen|citizens|immigrant|immigrants|migrant|migrants|"
1677:     # origin:<COUNTRY> – validated against the sovereign-state vocab.
1678:     for m in PATTERN_ORIGIN_CANDIDATE.finditer(text):
1679:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1680:         if resolved:
1681:             add(f"origin:{_normalize_country(resolved)}", span=m.span(1))
1682:
1683:     # country:<NAME> – broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:
1687:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:     # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:     # country, which the bare-name validators above can't see. A demonym
1691:     # followed by a person noun also earns origin:.
1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
2046:
2047:     if low in ("usa", "united states"):
2048:         return "United-States"
2049:     if low == "uk":
2050:         return "United-Kingdom"
2051:     if low == "uae":
2052:         return "United-Arab-Emirates"
2053:     if low == "burma":
2054:         return "Myanmar"
2055:     return s.replace(" ", "-")
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

origin:Haiti (16 (0.2%))

Precise plain-English rule

Emitted as origin:Haiti when the text has from Haiti, with a comma after the country, or a demonym for Haiti followed immediately by a person-origin noun.

Code text for this tag

```

116:     "Haiti",
1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-Z][a-zA-Z]*(?:\s+(?!{:_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," – anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"^(?i:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"^(?i:{:_PREPOSITIONS})\s+({_PLACE})\b")

```

```

1229: # "<Place>, <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{[_PLACE]},\s+({[_PLACE]})\b")

1238: DEMONYM_TO_COUNTRY: dict[str, str] = {
1239:     "iranian": "Iran",
1240:     "mexican": "Mexico",
1241:     "venezuelan": "Venezuela",
1242:     "chinese": "China",
1243:     "russian": "Russia",
1244:     "cuban": "Cuba",
1245:     "colombian": "Colombia",
1246:     "salvadoran": "El Salvador",
1247:     "salvadorian": "El Salvador",
1248:     "honduran": "Honduras",
1249:     "guatemalan": "Guatemala",
1250:     "haitian": "Haiti",
1251:     "somali": "Somalia",
1252:     "somalian": "Somalia",
1253:     "afghan": "Afghanistan",
1254:     "syrian": "Syria",
1255:     "nigerian": "Nigeria",
1256:     "pakistanian": "Pakistan",
1257:     "ukrainian": "Ukraine",
1258:     "israeli": "Israel",
1259:     "iraqi": "Iraq",
1260:     "egyptian": "Egypt",
1261:     "yemeni": "Yemen",
1262:     "brazilian": "Brazil",
1263:     "nicaraguan": "Nicaragua",
1264:     "ecuadorian": "Ecuador",
1265:     "peruvian": "Peru",
1266:     "jamaican": "Jamaica",
1267:     "filipino": "Philippines",
1268:     "vietnamese": "Vietnam",
1269:     "lebanese": "Lebanon",
1270:     "libyan": "Libya",
1271:     "sudanese": "Sudan",
1272:     "ethiopian": "Ethiopia",
1273:     "kenyan": "Kenya",
1274:     "moroccan": "Morocco",
1275:     "algerian": "Algeria",
1276:     "bangladeshi": "Bangladesh",
1277:     "cambodian": "Cambodia",
1278:     "indonesian": "Indonesia",
1279:     "dominican": "Dominican Republic",
1280:     "panamanian": "Panama",
1281:     "bolivian": "Bolivia",
1282:     "chilean": "Chile",
1283:     "argentine": "Argentina",
1284:     "argentinian": "Argentina",
1285:     "saudi": "Saudi Arabia",
1286:     "turkish": "Turkey",
1287: }
1288: FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
1289:     "tehran": "Iran",
1290:     "caracas": "Venezuela",
1291:     "beijing": "China",
1292:     "moscow": "Russia",
1293:     "havana": "Cuba",
1294:     "kabul": "Afghanistan",
1295:     "damascus": "Syria",
1296:     "baghdad": "Iraq",
1297:     "tripoli": "Libya",
1298:     "mogadishu": "Somalia",
1299:     "bogota": "Colombia",
1300:     "managua": "Nicaragua",
1301:     "tegucigalpa": "Honduras",
1302:     "kyiv": "Ukraine",
1303: }
1304:
1305:
1306: def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:
1307:     alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
1308:     return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)
1309:
1310:
1311: PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))
1312: PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
1313: # A demonym that immediately precedes one of these nouns describes a person's
1314: # nationality, so it also earns origin:<country>, not just a contextual country:.
1315: PATTERN_DEMONYM_PERSON = _compile(
1316:     r"^\W*(?:national|nationals|citizen|citizens|immigrant|immigrants|migrant|migrants)"
1317: )
1318:
1319:
1320: # origin:<COUNTRY> – validated against the sovereign-state vocab.
1321: for m in PATTERN_ORIGIN_CANDIDATE.finditer(text):
1322:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1323:     if resolved:
1324:         add(f"origin:{_normalize_country(resolved)}", span=m.span(1))
1325:
1326:
1327: # country:<NAME> – broader: any contextual country mention.
1328: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1329:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1330:     if resolved:
1331:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1332:
1333:
1334: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1335: # country, which the bare-name validators above can't see. A demonym
1336: # followed by a person noun also earns origin:.
1337: for m in PATTERN_DEMONYM_PERSON.finditer(text):
1338:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1339:     if country and country.lower() in COUNTRY_LOWER:
1340:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1341:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1342:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1343:
1344: for m in PATTERN_FOREIGN_CITY.finditer(text):
1345:
1346:
1347: if low in ("usa", "united states"):
1348:     return "United-States"
1349:
1350: if low == "uk":
1351:     return "United-Kingdom"

```

```

2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

origin:Afghanistan (15 (0.1%))

Precise plain-English rule

Emitted as origin:Afghanistan when the text has from Afghanistan, with a comma after the country, or a demonym for Afghanistan followed immediately by a person-origin noun.

Code text for this tag

```

50:     "Afghanistan",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the 'next' place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]+(?:\s+(?!{_{PREPOSITIONS}})\b)[A-Za-z][a-zA-Z]+)?"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{_{PREPOSITIONS}})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_{PLACE}}\s+({_PLACE})\b")

1238: DEMONYM_TO_COUNTRY: dict[str, str] = {
1239:     "iranian": "Iran",
1240:     "mexican": "Mexico",
1241:     "venezuelan": "Venezuela",
1242:     "chinese": "China",
1243:     "russian": "Russia",
1244:     "cuban": "Cuba",
1245:     "colombian": "Colombia",
1246:     "salvadoran": "El Salvador",
1247:     "salvadorian": "El Salvador",
1248:     "honduran": "Honduras",
1249:     "guatemalan": "Guatemala",
1250:     "haitian": "Haiti",
1251:     "somali": "Somalia",
1252:     "somalian": "Somalia",
1253:     "afghan": "Afghanistan",
1254:     "syrian": "Syria",
1255:     "nigerian": "Nigeria",
1256:     "pakistanian": "Pakistan",
1257:     "ukrainian": "Ukraine",
1258:     "israeli": "Israel",
1259:     "iraqi": "Iraq",
1260:     "egyptian": "Egypt",
1261:     "yemeni": "Yemen",
1262:     "brazilian": "Brazil",
1263:     "nicaraguan": "Nicaragua",
1264:     "ecuadorian": "Ecuador",
1265:     "peruvian": "Peru",
1266:     "jamaican": "Jamaica",
1267:     "filipino": "Philippines",
1268:     "vietnamese": "Vietnam",
1269:     "lebanese": "Lebanon",
1270:     "libyan": "Libya",
1271:     "sudanese": "Sudan",
1272:     "ethiopian": "Ethiopia",
1273:     "kenyan": "Kenya",
1274:     "moroccan": "Morocco",
1275:     "algerian": "Algeria",
1276:     "bangladeshi": "Bangladesh",
1277:     "cambodian": "Cambodia",
1278:     "indonesian": "Indonesia",
1279:     "dominican": "Dominican Republic",
1280:     "panamanian": "Panama",
1281:     "bolivian": "Bolivia",
1282:     "chilean": "Chile",
1283:     "argentine": "Argentina",
1284:     "argentinian": "Argentina",
1285:     "saudi": "Saudi Arabia",
1286:     "turkish": "Turkey",
1287: }
1288: FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
1289:     "tehran": "Iran",
1290:     "caracas": "Venezuela",
1291:     "beijing": "China",
1292:     "moscow": "Russia",
1293:     "havana": "Cuba",
1294:     "kabul": "Afghanistan",
1295:     "damascus": "Syria",
1296:     "baghdad": "Iraq",
1297:     "tripoli": "Libya",
1298:     "mogadishu": "Somalia",
1299:     "bogota": "Colombia",
1300:     "managua": "Nicaragua",
1301:     "tegucigalpa": "Honduras",

```

```

1302:     "kyiv": "Ukraine",
1303: }
1304:
1305:
1306: def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:
1307:     alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
1308:     return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)
1309:
1310:
1311: PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))
1312: PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
1313: # A demonym that immediately precedes one of these nouns describes a person's
1314: # nationality, so it also earns origin:<country>, not just a contextual country:.
1315: PATTERN_DEMONYM_PERSON = _compile(
1316:     r"^\W*(?:national|nationals|citizen|citizens|immigrant|immigrants|migrant|migrants|"
1317:
1318:     # origin:<COUNTRY> – validated against the sovereign-state vocab.
1319:     for m in PATTERN_ORIGIN_CANDIDATE.finditer(text):
1320:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1321:         if resolved:
1322:             add(f"origin:{_normalize_country(resolved)}", span=m.span(1))
1323:
1324:     # country:<NAME> – broader: any contextual country mention.
1325:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1326:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1327:         if resolved:
1328:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1329:
1330:     # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1331:     # country, which the bare-name validators above can't see. A demonym
1332:     # followed by a person noun also earns origin:.
1333:     for m in PATTERN_DEMONYM.finditer(text):
1334:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1335:         if country and country.lower() in COUNTRY_LOWER:
1336:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1337:             if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1338:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1339:     for m in PATTERN_FOREIGN_CITY.finditer(text):
1340:
1341: if low in ("usa", "united states"):
1342:     return "United-States"
1343: if low == "uk":
1344:     return "United-Kingdom"
1345: if low == "uae":
1346:     return "United-Arab-Emirates"
1347: if low == "burma":
1348:     return "Myanmar"
1349: return s.replace(" ", ".")
1350:
1351:
1352: def _normalize_state(s: str) -> str:
1353:     return s.replace(" ", ".")
1354:
1355:
1356: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
1357:     """Resolve a candidate (any case, possibly two words) to its canonical
1358:     vocab spelling.
1359:
1360:     Tries the full phrase first, then its leading word, so a greedy
1361:     case-insensitive capture that swallowed a trailing lowercase token

```

origin:Dominican-Republic (14 (0.1%))

Precise plain-English rule

Emitted as origin:Dominican-Republic when the text has from Dominican Republic, with a comma after the country, or a demonym for Dominican Republic followed immediately by a person-origin noun.

Code text for this tag

```

95:     "Dominican Republic",
96:
97:
98:
99:
100:
101:
102:
103:
104:
105:
106:
107:
108:
109:
110:
111:
112:
113:
114:
115:
116:
117: # second word excludes those same prepositions via lookahead, so a greedy
118: # capture can't swallow the preposition that introduces the *next* place
119: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab`
120: # then validates against the vocab and falls back to the leading word for any
121: # other trailing token. Kept conservative because raw country names
122: # false-positive easily ("Chad", "Georgia").
123: _PREPOSITIONS = "from|in|to|of|with|by"
124: _PLACE = rf"[A-Za-z][a-zA-Z]+(?:\s+?(?!(?:{_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]+)?"
125: # "from <Country>," – anchors the ORIGIN validator.
126: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*", "")
127: # Any sovereign-state mention after a preposition.
128: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{_PREPOSITIONS})\s+({_PLACE})\b")
129: # "<Place>, <State>" – anchors the STATE validator.
130: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE})\s+({_PLACE})\b")
131:
132:
133: DEMONYM_TO_COUNTRY: dict[str, str] = {
134:     "iranian": "Iran",
135:     "mexican": "Mexico",
136:     "venezuelan": "Venezuela",
137:     "chinese": "China",
138:     "russian": "Russia",
139:     "cuban": "Cuba",
140:     "colombian": "Colombia",
141:     "salvadoran": "El Salvador",
142:     "salvadorian": "El Salvador",
143:     "honduran": "Honduras",
144:     "guatemalan": "Guatemala",
145:     "haitian": "Haiti",
146:     "somali": "Somalia",
147:     "somalian": "Somalia",
148:     "afghan": "Afghanistan",
149:     "syrian": "Syria",
150:     "nigerian": "Nigeria",
151:     "pakistani": "Pakistan",
152:     "ukrainian": "Ukraine",
153:     "israeli": "Israel",

```

```

1259: "iraqi": "Iraq",
1260: "egyptian": "Egypt",
1261: "yemeni": "Yemen",
1262: "brazilian": "Brazil",
1263: "nicaraguan": "Nicaragua",
1264: "ecuadorian": "Ecuador",
1265: "peruvian": "Peru",
1266: "jamaican": "Jamaica",
1267: "filipino": "Philippines",
1268: "vietnamese": "Vietnam",
1269: "lebanese": "Lebanon",
1270: "libyan": "Libya",
1271: "sudanese": "Sudan",
1272: "ethiopian": "Ethiopia",
1273: "kenyan": "Kenya",
1274: "moroccan": "Morocco",
1275: "algerian": "Algeria",
1276: "bangladeshi": "Bangladesh",
1277: "cambodian": "Cambodia",
1278: "indonesian": "Indonesia",
1279: "dominican": "Dominican Republic",
1280: "panamanian": "Panama",
1281: "bolivian": "Bolivia",
1282: "chilean": "Chile",
1283: "argentine": "Argentina",
1284: "argentinian": "Argentina",
1285: "saudi": "Saudi Arabia",
1286: "turkish": "Turkey",
1287: }
1288: FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
1289:     "tehran": "Iran",
1290:     "caracas": "Venezuela",
1291:     "beijing": "China",
1292:     "moscow": "Russia",
1293:     "havana": "Cuba",
1294:     "kabul": "Afghanistan",
1295:     "damascus": "Syria",
1296:     "baghdad": "Iraq",
1297:     "tripoli": "Libya",
1298:     "mogadishu": "Somalia",
1299:     "bogota": "Colombia",
1300:     "managua": "Nicaragua",
1301:     "tegucigalpa": "Honduras",
1302:     "kyiv": "Ukraine",
1303: }
1304:
1305:
1306: def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:
1307:     alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
1308:     return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)
1309:
1310:
1311: PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))
1312: PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
1313: # A demonym that immediately precedes one of these nouns describes a person's
1314: # nationality, so it also earns origin:<country>, not just a contextual country:.
1315: PATTERN_DEMONYM_PERSON = _compile(
1316:     r"^\w*(?:national|nationals|citizen|citizens|immigrant|immigrants|migrant|migrants|"
1677:     # origin:<COUNTRY> – validated against the sovereign-state vocab.
1678:     for m in PATTERN_ORIGIN_CANDIDATE.finditer(text):
1679:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1680:         if resolved:
1681:             add(f"origin:{_normalize_country(resolved)}", span=m.span(1))
1682:
1683:     # country:<NAME> – broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:
1687:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:     # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:     # country, which the bare-name validators above can't see. A demonym
1691:     # followed by a person noun also earns origin:.
1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
2052: if low == "burma":
2053:     return "Myanmar"
2054: return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

origin:Nicaragua (14 (0.1%))

Precise plain-English rule

Emitted as origin:Nicaragua when the text has from Nicaragua, with a comma after the country, or a demonym for Nicaragua followed immediately by a person-origin noun.

Code text for this tag

```
167:     "Nicaragua",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). `._resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z](?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]?"
1225: # "from <Country>" - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s+", "")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>, <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE}),\s+({_PLACE})\b")

1238: DEMONYM_TO_COUNTRY: dict[str, str] = {
1239:     "iranian": "Iran",
1240:     "mexican": "Mexico",
1241:     "venezuelan": "Venezuela",
1242:     "chinese": "China",
1243:     "russian": "Russia",
1244:     "cuban": "Cuba",
1245:     "colombian": "Colombia",
1246:     "salvadoran": "El Salvador",
1247:     "salvadorian": "El Salvador",
1248:     "honduran": "Honduras",
1249:     "guatemalan": "Guatemala",
1250:     "haitian": "Haiti",
1251:     "somali": "Somalia",
1252:     "somalian": "Somalia",
1253:     "afghan": "Afghanistan",
1254:     "syrian": "Syria",
1255:     "nigerian": "Nigeria",
1256:     "pakistani": "Pakistan",
1257:     "ukrainian": "Ukraine",
1258:     "israeli": "Israel",
1259:     "iraqi": "Iraq",
1260:     "egyptian": "Egypt",
1261:     "yemeni": "Yemen",
1262:     "brazilian": "Brazil",
1263:     "nicaraguan": "Nicaragua",
1264:     "ecuadorian": "Ecuador",
1265:     "peruvian": "Peru",
1266:     "jamaican": "Jamaica",
1267:     "filipino": "Philippines",
1268:     "vietnamese": "Vietnam",
1269:     "lebanese": "Lebanon",
1270:     "libyan": "Libya",
1271:     "sudanese": "Sudan",
1272:     "ethiopian": "Ethiopia",
1273:     "kenyan": "Kenya",
1274:     "moroccan": "Morocco",
1275:     "algerian": "Algeria",
1276:     "bangladeshi": "Bangladesh",
1277:     "cambodian": "Cambodia",
1278:     "indonesian": "Indonesia",
1279:     "dominican": "Dominican Republic",
1280:     "panamanian": "Panama",
1281:     "bolivian": "Bolivia",
1282:     "chilean": "Chile",
1283:     "argentine": "Argentina",
1284:     "argentinian": "Argentina",
1285:     "saudi": "Saudi Arabia",
1286:     "turkish": "Turkey",
1287: }
1288: FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
1289:     "tehran": "Iran",
1290:     "caracas": "Venezuela",
1291:     "beijing": "China",
1292:     "moscow": "Russia",
1293:     "havana": "Cuba",
1294:     "kabul": "Afghanistan",
1295:     "damascus": "Syria",
1296:     "baghdad": "Iraq",
1297:     "tripoli": "Libya",
1298:     "mogadishu": "Somalia",
1299:     "bogota": "Colombia",
1300:     "managua": "Nicaragua",
1301:     "tegucigalpa": "Honduras",
1302:     "kyiv": "Ukraine",
1303: }
1304:
1305:
1306: def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:
1307:     alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
1308:     return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)
1309:
1310:
1311: PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))
1312: PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
1313: # A demonym that immediately precedes one of these nouns describes a person's
1314: # nationality, so it also earns origin:<country>, not just a contextual country:.
1315: PATTERN_DEMONYM_PERSON = re.compile(
1316:     r"^\W*(?:national|national[s]|citizen|citizens|immigrant|immigrants|migrant|migrants|")

1677: # origin:<COUNTRY> - validated against the sovereign-state vocab.
1678: for m in PATTERN_ORIGIN_CANDIDATE.finditer(text):
1679:     resolved = _resolve_vocab(m.group(1)) or "", COUNTRY_BY_LOWER
1680:     if resolved:
1681:         add(f"origin:{_normalize_country(resolved)}", span=m.span(1))
1682:
1683: # country:<NAME> - broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
```

```

1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:
1687:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:         # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:         # country, which the bare-name validators above can't see. A demonym
1691:         # followed by a person noun also earns origin:.
1692:         for m in PATTERN_DEMONYM.finditer(text):
1693:             country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:             if country and country.lower() in COUNTRY_LOWER:
1695:                 add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:                 if PATTERN_DEMONYM_PERSON.search(text[m.end(): m.end() + 24]):
1697:                     add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:         for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:
2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

origin:India (13 (0.1%))

Precise plain-English rule

Emitted as origin:India when the text has from India, with a comma after the country, or a demonym for India followed immediately by a person-origin noun.

Code text for this tag

```

120:     "India",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!{:_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:{:_PREPOSITIONS})\s+({_PLACE})\s*",)
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{:_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE}\s+({_PLACE})\b")

1238: DEMONYM_TO_COUNTRY: dict[str, str] = {
1239:     "iranian": "Iran",
1240:     "mexican": "Mexico",
1241:     "venezuelan": "Venezuela",
1242:     "chinese": "China",
1243:     "russian": "Russia",
1244:     "cuban": "Cuba",
1245:     "colombian": "Colombia",
1246:     "salvadoran": "El Salvador",
1247:     "salvadorian": "El Salvador",
1248:     "honduran": "Honduras",
1249:     "guatemalan": "Guatemala",
1250:     "haitian": "Haiti",
1251:     "somali": "Somalia",
1252:     "somalian": "Somalia",
1253:     "afghan": "Afghanistan",
1254:     "syrian": "Syria",
1255:     "nigerian": "Nigeria",
1256:     "pakistani": "Pakistan",
1257:     "ukrainian": "Ukraine",
1258:     "israeli": "Israel",
1259:     "iraqi": "Iraq",
1260:     "egyptian": "Egypt",
1261:     "yemeni": "Yemen",
1262:     "brazilian": "Brazil",
1263:     "nicaraguan": "Nicaragua",
1264:     "ecuadorian": "Ecuador",
1265:     "peruvian": "Peru",
1266:     "jamaican": "Jamaica",
1267:     "filipino": "Philippines",
1268:     "vietnamese": "Vietnam",
1269:     "lebanese": "Lebanon",
1270:     "libyan": "Libya",
1271:     "sudanese": "Sudan",
1272:     "ethiopian": "Ethiopia",
1273:     "kenyan": "Kenya",
1274:     "moroccan": "Morocco",
1275:     "algerian": "Algeria",
1276:     "bangladeshi": "Bangladesh",
1277:     "cambodian": "Cambodia",
1278:     "indonesian": "Indonesia",
1279:     "dominican": "Dominican Republic",
1280:     "panamanian": "Panama",
1281:     "bolivian": "Bolivia",
1282:     "chilean": "Chile",

```

```

1283:     "argentine": "Argentina",
1284:     "argentinian": "Argentina",
1285:     "saudi": "Saudi Arabia",
1286:     "turkish": "Turkey",
1287: }
1288: FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
1289:     "tehran": "Iran",
1290:     "caracas": "Venezuela",
1291:     "beijing": "China",
1292:     "moscow": "Russia",
1293:     "havana": "Cuba",
1294:     "kabul": "Afghanistan",
1295:     "damascus": "Syria",
1296:     "baghdad": "Iraq",
1297:     "tripoli": "Libya",
1298:     "mogadishu": "Somalia",
1299:     "bogota": "Colombia",
1300:     "managua": "Nicaragua",
1301:     "tegucigalpa": "Honduras",
1302:     "kyiv": "Ukraine",
1303: }
1304:
1305:
1306: def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:
1307:     alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
1308:     return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)
1309:
1310:
1311: PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))
1312: PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
1313: # A demonym that immediately precedes one of these nouns describes a person's
1314: # nationality, so it also earns origin:<country>, not just a contextual country:.
1315: PATTERN_DEMONYM_PERSON = _compile(
1316:     r"^\W*(?:national|nationals|citizen|citizens|immigrant|immigrants|migrant|migrants|"
1677:     # origin:<COUNTRY> – validated against the sovereign-state vocab.
1678:     for m in PATTERN_ORIGIN_CANDIDATE.finditer(text):
1679:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1680:         if resolved:
1681:             add(f"origin:{_normalize_country(resolved)}", span=m.span(1))
1682:
1683:     # country:<NAME> – broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:
1687:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:     # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:     # country, which the bare-name validators above can't see. A demonym
1691:     # followed by a person noun also earns origin:.
1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", ".")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", ".")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

origin:Jamaica (13 (0.1%))

Precise plain-English rule

Emitted as origin:Jamaica when the text has from Jamaica, with a comma after the country, or a demonym for Jamaica followed immediately by a person-origin noun.

Code text for this tag

```

127:     "Jamaica",
1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the *next* place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). `_resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]+(?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]+)?"
1225: # "from <country>." – anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{[_PREPOSITIONS]})\s+({_PLACE})\b")
1229: # "<Place>, <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE})\s+({_PLACE})\b")
1238: DEMONYM_TO_COUNTRY: dict[str, str] = {
1239:     "iranian": "Iran",

```

```

1240: "mexican": "Mexico",
1241: "venezuelan": "Venezuela",
1242: "chinese": "China",
1243: "russian": "Russia",
1244: "cuban": "Cuba",
1245: "colombian": "Colombia",
1246: "salvadoran": "El Salvador",
1247: "salvadorian": "El Salvador",
1248: "honduran": "Honduras",
1249: "guatemalan": "Guatemala",
1250: "haitian": "Haiti",
1251: "somali": "Somalia",
1252: "somalian": "Somalia",
1253: "afghan": "Afghanistan",
1254: "syrian": "Syria",
1255: "nigerian": "Nigeria",
1256: "pakistani": "Pakistan",
1257: "ukrainian": "Ukraine",
1258: "israeli": "Israel",
1259: "iraqi": "Iraq",
1260: "egyptian": "Egypt",
1261: "yemeni": "Yemen",
1262: "brazilian": "Brazil",
1263: "nicaraguan": "Nicaragua",
1264: "ecuadorian": "Ecuador",
1265: "peruvian": "Peru",
1266: "jamaican": "Jamaica",
1267: "filipino": "Philippines",
1268: "vietnamese": "Vietnam",
1269: "lebanese": "Lebanon",
1270: "libyan": "Libya",
1271: "sudanese": "Sudan",
1272: "ethiopian": "Ethiopia",
1273: "kenyan": "Kenya",
1274: "moroccan": "Morocco",
1275: "algerian": "Algeria",
1276: "bangladeshi": "Bangladesh",
1277: "cambodian": "Cambodia",
1278: "indonesian": "Indonesia",
1279: "dominican": "Dominican Republic",
1280: "panamanian": "Panama",
1281: "bolivian": "Bolivia",
1282: "chilean": "Chile",
1283: "argentine": "Argentina",
1284: "argentinian": "Argentina",
1285: "saudi": "Saudi Arabia",
1286: "turkish": "Turkey",
1287: }
1288: FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
1289: "tehran": "Iran",
1290: "caracas": "Venezuela",
1291: "beijing": "China",
1292: "moscow": "Russia",
1293: "havana": "Cuba",
1294: "kabul": "Afghanistan",
1295: "damascus": "Syria",
1296: "baghdad": "Iraq",
1297: "tripoli": "Libya",
1298: "mogadishu": "Somalia",
1299: "bogota": "Colombia",
1300: "managua": "Nicaragua",
1301: "tegucigalpa": "Honduras",
1302: "kyiv": "Ukraine",
1303: }
1304:
1305:
1306: def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:
1307:     alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
1308:     return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)
1309:
1310:
1311: PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))
1312: PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
1313: # A demonym that immediately precedes one of these nouns describes a person's
1314: # nationality, so it also earns origin:<country>, not just a contextual country:.
1315: PATTERN_DEMONYM_PERSON = _compile(
1316:     r"^\w*(?:national|nationals|citizen|citizens|immigrant|immigrants|migrant|migrants|
1677: # origin:<COUNTRY> - validated against the sovereign-state vocab.
1678: for m in PATTERN_ORIGIN_CANDIDATE.finditer(text):
1679:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1680:     if resolved:
1681:         add(f"origin:{_normalize_country(resolved)}", span=m.span(1))
1682:
1683: # country:<NAME> - broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end(): m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
2052: if low == "burma":
2053:     return "Myanmar"
2054: return s.replace(" ", "-")

```

```

2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

origin:Somalia (13 (0.1%))

Precise plain-English rule

Emitted as origin:Somalia when the text has from Somalia, with a comma after the country, or a demonym for Somalia followed immediately by a person-origin noun.

Code text for this tag

```

192:     "Somalia",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!(?:{_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]+)?"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE})\s+({_PLACE})\b")

1238: DEMONYM_TO_COUNTRY: dict[str, str] = {
1239:     "iranian": "Iran",
1240:     "mexican": "Mexico",
1241:     "venezuelan": "Venezuela",
1242:     "chinese": "China",
1243:     "russian": "Russia",
1244:     "cuban": "Cuba",
1245:     "colombian": "Colombia",
1246:     "salvadoran": "El Salvador",
1247:     "salvadorian": "El Salvador",
1248:     "honduran": "Honduras",
1249:     "guatemalan": "Guatemala",
1250:     "haitian": "Haiti",
1251:     "somali": "Somalia",
1252:     "somalian": "Somalia",
1253:     "afghan": "Afghanistan",
1254:     "syrian": "Syria",
1255:     "nigerian": "Nigeria",
1256:     "pakistani": "Pakistan",
1257:     "ukrainian": "Ukraine",
1258:     "israeli": "Israel",
1259:     "iraqi": "Iraq",
1260:     "egyptian": "Egypt",
1261:     "yemeni": "Yemen",
1262:     "brazilian": "Brazil",
1263:     "nicaraguan": "Nicaragua",
1264:     "ecuadorian": "Ecuador",
1265:     "peruvian": "Peru",
1266:     "jamaican": "Jamaica",
1267:     "filipino": "Philippines",
1268:     "vietnamese": "Vietnam",
1269:     "lebanese": "Lebanon",
1270:     "libyan": "Libya",
1271:     "sudanese": "Sudan",
1272:     "ethiopian": "Ethiopia",
1273:     "kenyan": "Kenya",
1274:     "moroccan": "Morocco",
1275:     "algerian": "Algeria",
1276:     "bangladeshi": "Bangladesh",
1277:     "cambodian": "Cambodia",
1278:     "indonesian": "Indonesia",
1279:     "dominican": "Dominican Republic",
1280:     "panamanian": "Panama",
1281:     "bolivian": "Bolivia",
1282:     "chilean": "Chile",
1283:     "argentine": "Argentina",
1284:     "argentinian": "Argentina",
1285:     "saudi": "Saudi Arabia",
1286:     "turkish": "Turkey",
1287: }
1288: FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
1289:     "tehran": "Iran",
1290:     "caracas": "Venezuela",
1291:     "beijing": "China",
1292:     "moscow": "Russia",
1293:     "havana": "Cuba",
1294:     "kabul": "Afghanistan",
1295:     "damascus": "Syria",
1296:     "baghdad": "Iraq",
1297:     "tripoli": "Libya",
1298:     "mogadishu": "Somalia",
1299:     "bogota": "Colombia",
1300:     "managua": "Nicaragua",
1301:     "tegucigalpa": "Honduras",
1302:     "kyiv": "Ukraine",
1303: }
1304:
1305:
1306: def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:

```

```

1307:     alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
1308:     return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)
1309:
1310:
1311: PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))
1312: PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
1313: # A demonym that immediately precedes one of these nouns describes a person's
1314: # nationality, so it also earns origin:<country>, not just a contextual country:.
1315: PATTERN_DEMONYM_PERSON = _compile(
1316:     r"^\w*(?:national|nationals|citizen|citizens|immigrant|immigrants|migrant|migrants|"
1677:     # origin:<COUNTRY> – validated against the sovereign-state vocab.
1678:     for m in PATTERN_ORIGIN_CANDIDATE.finditer(text):
1679:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1680:         if resolved:
1681:             add(f"origin:{_normalize_country(resolved)}", span=m.span(1))
1682:
1683:     # country:<NAME> – broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:
1687:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:     # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:     # country, which the bare-name validators above can't see. A demonym
1691:     # followed by a person noun also earns origin:.
1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

origin:Brazil (12 (0.1%))

Precise plain-English rule

Emitted as origin:Brazil when the text has from Brazil, with a comma after the country, or a demonym for Brazil followed immediately by a person-origin noun.

Code text for this tag

```

72:     "Brazil",
1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). '_resolve_vocab'
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]+)?"
1225: # "from <Country>," – anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>, <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE}\s+({_PLACE})\b")
1238: DEMONYM_TO_COUNTRY: dict[str, str] = {
1239:     "iranian": "Iran",
1240:     "mexican": "Mexico",
1241:     "venezuelan": "Venezuela",
1242:     "chinese": "China",
1243:     "russian": "Russia",
1244:     "cuban": "Cuba",
1245:     "colombian": "Colombia",
1246:     "salvadoran": "El Salvador",
1247:     "salvadorian": "El Salvador",
1248:     "honduran": "Honduras",
1249:     "guatemalan": "Guatemala",
1250:     "haitian": "Haiti",
1251:     "somali": "Somalia",
1252:     "somalian": "Somalia",
1253:     "afghan": "Afghanistan",
1254:     "syrian": "Syria",
1255:     "nigerian": "Nigeria",
1256:     "pakistani": "Pakistan",
1257:     "ukrainian": "Ukraine",
1258:     "israeli": "Israel",
1259:     "iraqi": "Iraq",
1260:     "egyptian": "Egypt",
1261:     "yemeni": "Yemen",
1262:     "brazilian": "Brazil",
1263:     "nicaraguan": "Nicaragua",

```

```

1264: "ecuadorian": "Ecuador",
1265: "peruvian": "Peru",
1266: "jamaican": "Jamaica",
1267: "filipino": "Philippines",
1268: "vietnamese": "Vietnam",
1269: "lebanese": "Lebanon",
1270: "libyan": "Libya",
1271: "sudanese": "Sudan",
1272: "ethiopian": "Ethiopia",
1273: "kenyan": "Kenya",
1274: "moroccan": "Morocco",
1275: "algerian": "Algeria",
1276: "bangladeshi": "Bangladesh",
1277: "cambodian": "Cambodia",
1278: "indonesian": "Indonesia",
1279: "dominican": "Dominican Republic",
1280: "panamanian": "Panama",
1281: "bolivian": "Bolivia",
1282: "chilean": "Chile",
1283: "argentine": "Argentina",
1284: "argentinian": "Argentina",
1285: "saudi": "Saudi Arabia",
1286: "turkish": "Turkey",
1287: }
1288: FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
1289:     "tehran": "Iran",
1290:     "caracas": "Venezuela",
1291:     "beijing": "China",
1292:     "moscow": "Russia",
1293:     "havana": "Cuba",
1294:     "kabul": "Afghanistan",
1295:     "damascus": "Syria",
1296:     "baghdad": "Iraq",
1297:     "tripoli": "Libya",
1298:     "mogadishu": "Somalia",
1299:     "bogota": "Colombia",
1300:     "managua": "Nicaragua",
1301:     "tegucigalpa": "Honduras",
1302:     "kyiv": "Ukraine",
1303: }
1304:
1305:
1306: def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:
1307:     alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
1308:     return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)
1309:
1310:
1311: PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))
1312: PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
1313: # A demonym that immediately precedes one of these nouns describes a person's
1314: # nationality, so it also earns origin:<country>, not just a contextual country:.
1315: PATTERN_DEMONYM_PERSON = _compile(
1316:     r"^\W(?:national|nationals|citizen|citizens|immigrant|immigrants|migrant|migrants|"
1317:
1318:
1319:
1320:
1321:
1322:
1323:
1324:
1325:
1326:
1327:
1328:
1329:
1330:
1331:
1332:
1333:
1334:
1335:
1336:
1337:
1338:
1339:
1340:
1341:
1342:
1343:
1344:
1345:
1346:
1347:
1348:
1349:
1350:
1351:
1352:
1353:
1354:
1355:
1356:
1357:
1358:
1359:
1360:
1361:
1362:
1363:
1364:
1365:
1366:
1367:
1368:
1369:
1370:
1371:
1372:
1373:
1374:
1375:
1376:
1377:
1378:
1379:
1380:
1381:
1382:
1383:
1384:
1385:
1386:
1387:
1388:
1389:
1390:
1391:
1392:
1393:
1394:
1395:
1396:
1397:
1398:
1399:
1400:
1401:
1402:
1403:
1404:
1405:
1406:
1407:
1408:
1409:
1410:
1411:
1412:
1413:
1414:
1415:
1416:
1417:
1418:
1419:
1420:
1421:
1422:
1423:
1424:
1425:
1426:
1427:
1428:
1429:
1430:
1431:
1432:
1433:
1434:
1435:
1436:
1437:
1438:
1439:
1440:
1441:
1442:
1443:
1444:
1445:
1446:
1447:
1448:
1449:
1450:
1451:
1452:
1453:
1454:
1455:
1456:
1457:
1458:
1459:
1460:
1461:
1462:
1463:
1464:
1465:
1466:
1467:
1468:
1469:
1470:
1471:
1472:
1473:
1474:
1475:
1476:
1477:
1478:
1479:
1480:
1481:
1482:
1483:
1484:
1485:
1486:
1487:
1488:
1489:
1490:
1491:
1492:
1493:
1494:
1495:
1496:
1497:
1498:
1499:
1500:
1501:
1502:
1503:
1504:
1505:
1506:
1507:
1508:
1509:
1510:
1511:
1512:
1513:
1514:
1515:
1516:
1517:
1518:
1519:
1520:
1521:
1522:
1523:
1524:
1525:
1526:
1527:
1528:
1529:
1530:
1531:
1532:
1533:
1534:
1535:
1536:
1537:
1538:
1539:
1540:
1541:
1542:
1543:
1544:
1545:
1546:
1547:
1548:
1549:
1550:
1551:
1552:
1553:
1554:
1555:
1556:
1557:
1558:
1559:
1560:
1561:
1562:
1563:
1564:
1565:
1566:
1567:
1568:
1569:
1570:
1571:
1572:
1573:
1574:
1575:
1576:
1577:
1578:
1579:
1580:
1581:
1582:
1583:
1584:
1585:
1586:
1587:
1588:
1589:
1590:
1591:
1592:
1593:
1594:
1595:
1596:
1597:
1598:
1599:
1600:
1601:
1602:
1603:
1604:
1605:
1606:
1607:
1608:
1609:
1610:
1611:
1612:
1613:
1614:
1615:
1616:
1617:
1618:
1619:
1620:
1621:
1622:
1623:
1624:
1625:
1626:
1627:
1628:
1629:
1630:
1631:
1632:
1633:
1634:
1635:
1636:
1637:
1638:
1639:
1640:
1641:
1642:
1643:
1644:
1645:
1646:
1647:
1648:
1649:
1650:
1651:
1652:
1653:
1654:
1655:
1656:
1657:
1658:
1659:
1660:
1661:
1662:
1663:
1664:
1665:
1666:
1667:
1668:
1669:
1670:
1671:
1672:
1673:
1674:
1675:
1676:
1677:
1678:
1679:
1680:
1681:
1682:
1683:
1684:
1685:
1686:
1687:
1688:
1689:
1690:
1691:
1692:
1693:
1694:
1695:
1696:
1697:
1698:
1699:
1700:
1701:
1702:
1703:
1704:
1705:
1706:
1707:
1708:
1709:
1710:
1711:
1712:
1713:
1714:
1715:
1716:
1717:
1718:
1719:
1720:
1721:
1722:
1723:
1724:
1725:
1726:
1727:
1728:
1729:
1730:
1731:
1732:
1733:
1734:
1735:
1736:
1737:
1738:
1739:
1740:
1741:
1742:
1743:
1744:
1745:
1746:
1747:
1748:
1749:
1750:
1751:
1752:
1753:
1754:
1755:
1756:
1757:
1758:
1759:
1760:
1761:
1762:
1763:
1764:
1765:
1766:
1767:
1768:
1769:
1770:
1771:
1772:
1773:
1774:
1775:
1776:
1777:
1778:
1779:
1780:
1781:
1782:
1783:
1784:
1785:
1786:
1787:
1788:
1789:
1790:
1791:
1792:
1793:
1794:
1795:
1796:
1797:
1798:
1799:
1800:
1801:
1802:
1803:
1804:
1805:
1806:
1807:
1808:
1809:
1810:
1811:
1812:
1813:
1814:
1815:
1816:
1817:
1818:
1819:
1820:
1821:
1822:
1823:
1824:
1825:
1826:
1827:
1828:
1829:
1830:
1831:
1832:
1833:
1834:
1835:
1836:
1837:
1838:
1839:
1840:
1841:
1842:
1843:
1844:
1845:
1846:
1847:
1848:
1849:
1850:
1851:
1852:
1853:
1854:
1855:
1856:
1857:
1858:
1859:
1860:
1861:
1862:
1863:
1864:
1865:
1866:
1867:
1868:
1869:
1870:
1871:
1872:
1873:
1874:
1875:
1876:
1877:
1878:
1879:
1880:
1881:
1882:
1883:
1884:
1885:
1886:
1887:
1888:
1889:
1890:
1891:
1892:
1893:
1894:
1895:
1896:
1897:
1898:
1899:
1900:
1901:
1902:
1903:
1904:
1905:
1906:
1907:
1908:
1909:
1910:
1911:
1912:
1913:
1914:
1915:
1916:
1917:
1918:
1919:
1920:
1921:
1922:
1923:
1924:
1925:
1926:
1927:
1928:
1929:
1930:
1931:
1932:
1933:
1934:
1935:
1936:
1937:
1938:
1939:
1940:
1941:
1942:
1943:
1944:
1945:
1946:
1947:
1948:
1949:
1950:
1951:
1952:
1953:
1954:
1955:
1956:
1957:
1958:
1959:
1960:
1961:
1962:
1963:
1964:
1965:
1966:
1967:
1968:
1969:
1970:
1971:
1972:
1973:
1974:
1975:
1976:
1977:
1978:
1979:
1980:
1981:
1982:
1983:
1984:
1985:
1986:
1987:
1988:
1989:
1990:
1991:
1992:
1993:
1994:
1995:
1996:
1997:
1998:
1999:
2000:
2001:
2002:
2003:
2004:
2005:
2006:
2007:
2008:
2009:
2010:
2011:
2012:
2013:
2014:
2015:
2016:
2017:
2018:
2019:
2020:
2021:
2022:
2023:
2024:
2025:
2026:
2027:
2028:
2029:
2030:
2031:
2032:
2033:
2034:
2035:
2036:
2037:
2038:
2039:
2040:
2041:
2042:
2043:
2044:
2045:
2046:
2047:
2048:
2049:
2050:
2051:
2052:
2053:
2054:
2055:
2056:
2057:
2058:
2059:
2060:
2061:
2062:
2063:
2064:
2065:
2066:

```

origin:Peru (6 (0.1%))

Precise plain-English rule

Emitted as origin:Peru when the text has from Peru, with a comma after the country, or a demonym for Peru followed immediately by a person-origin noun.

Code text for this tag

```

176:     "Peru",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the *next* place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). `resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>" - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*", "")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:({_PREPOSITIONS}))\s+({_PLACE})\b")
1229: # "<Place>, <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE}),\s+({_PLACE})\b")

1238: DEMONYM_TO_COUNTRY: dict[str, str] = {
1239:     "iranian": "Iran",
1240:     "mexican": "Mexico",
1241:     "venezuelan": "Venezuela",
1242:     "chinese": "China",
1243:     "russian": "Russia",
1244:     "cuban": "Cuba",
1245:     "colombian": "Colombia",
1246:     "salvadoran": "El Salvador",
1247:     "salvadorian": "El Salvador",
1248:     "honduran": "Honduras",
1249:     "guatemalan": "Guatemala",
1250:     "haitian": "Haiti",
1251:     "somali": "Somalia",
1252:     "somalian": "Somalia",
1253:     "afghan": "Afghanistan",
1254:     "syrian": "Syria",
1255:     "nigerian": "Nigeria",
1256:     "pakistani": "Pakistan",
1257:     "ukrainian": "Ukraine",
1258:     "israeli": "Israel",
1259:     "iraqi": "Iraq",
1260:     "egyptian": "Egypt",
1261:     "yemeni": "Yemen",
1262:     "brazilian": "Brazil",
1263:     "nicaraguan": "Nicaragua",
1264:     "ecuadorian": "Ecuador",
1265:     "peruvian": "Peru",
1266:     "jamaican": "Jamaica",
1267:     "filipino": "Philippines",
1268:     "vietnamese": "Vietnam",
1269:     "lebanese": "Lebanon",
1270:     "libyan": "Libya",
1271:     "sudanese": "Sudan",
1272:     "ethiopian": "Ethiopia",
1273:     "kenyan": "Kenya",
1274:     "moroccan": "Morocco",
1275:     "algerian": "Algeria",
1276:     "bangladeshi": "Bangladesh",
1277:     "cambodian": "Cambodia",
1278:     "indonesian": "Indonesia",
1279:     "dominican": "Dominican Republic",
1280:     "panamanian": "Panama",
1281:     "bolivian": "Bolivia",
1282:     "chilean": "Chile",
1283:     "argentine": "Argentina",
1284:     "argentinian": "Argentina",
1285:     "saudi": "Saudi Arabia",
1286:     "turkish": "Turkey",
1287: }
1288: FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
1289:     "tehran": "Iran",
1290:     "caracas": "Venezuela",
1291:     "beijing": "China",
1292:     "moscow": "Russia",
1293:     "havana": "Cuba",
1294:     "kabul": "Afghanistan",
1295:     "damascus": "Syria",
1296:     "baghdad": "Iraq",
1297:     "tripoli": "Libya",
1298:     "mogadishu": "Somalia",
1299:     "bogota": "Colombia",
1300:     "managua": "Nicaragua",
1301:     "tegucigalpa": "Honduras",
1302:     "kyiv": "Ukraine",
1303: }
1304:
1305: def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:
1306:     alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
1307:     return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)

1310:
1311: PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))
1312: PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
1313: # A demonym that immediately precedes one of these nouns describes a person's
1314: # nationality, so it also earns origin:<country>, not just a contextual country:.
1315: PATTERN_DEMONYM_PERSON = re.compile(
1316:     r"^\W*(?:national|nationals|citizen|citizens|immigrant|immigrants|migrant|migrants|"
1677: # origin:<COUNTRY> - validated against the sovereign-state vocab.
1678: for m in PATTERN_ORIGIN_CANDIDATE.finditer(text):
1679:     resolved = resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1680:     if resolved:
1681:         add(f"origin:{normalize_country(resolved)}", span=m.span(1))
1682:
1683: # country:<NAME> - broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->

```

```

1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):

2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
2052: if low == "burma":
2053:     return "Myanmar"
2054: return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

origin:Guyana (5 (0.0%))

Precise plain-English rule

Emitted as origin:Guyana when the text has from Guyana, with a comma after the country, or a demonym for Guyana followed immediately by a person-origin noun.

Code text for this tag

```

115:     "Guyana",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the *next* place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). `_resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-Z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"b(?:{:_PREPOSITIONS})\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"b(?:{:_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"b({_PLACE})\s+({_PLACE})\b")

1238: DEMONYM_TO_COUNTRY: dict[str, str] = {
1239:     "iranian": "Iran",
1240:     "mexican": "Mexico",
1241:     "venezuelan": "Venezuela",
1242:     "chinese": "China",
1243:     "russian": "Russia",
1244:     "cuban": "Cuba",
1245:     "colombian": "Colombia",
1246:     "salvadoran": "El Salvador",
1247:     "salvadorian": "El Salvador",
1248:     "honduran": "Honduras",
1249:     "guatemalan": "Guatemala",
1250:     "haitian": "Haiti",
1251:     "somali": "Somalia",
1252:     "somalian": "Somalia",
1253:     "afghan": "Afghanistan",
1254:     "syrian": "Syria",
1255:     "nigerian": "Nigeria",
1256:     "pakistani": "Pakistan",
1257:     "ukrainian": "Ukraine",
1258:     "israeli": "Israel",
1259:     "iraqi": "Iraq",
1260:     "egyptian": "Egypt",
1261:     "yemeni": "Yemen",
1262:     "brazilian": "Brazil",
1263:     "nicaraguan": "Nicaragua",
1264:     "ecuadorian": "Ecuador",
1265:     "peruvian": "Peru",
1266:     "jamaican": "Jamaica",
1267:     "filipino": "Philippines",
1268:     "vietnamese": "Vietnam",
1269:     "lebanese": "Lebanon",
1270:     "libyan": "Libya",
1271:     "sudanese": "Sudan",
1272:     "ethiopian": "Ethiopia",
1273:     "kenyan": "Kenya",
1274:     "moroccan": "Morocco",
1275:     "algerian": "Algeria",
1276:     "bangladeshi": "Bangladesh",
1277:     "cambodian": "Cambodia",
1278:     "indonesian": "Indonesia",
1279:     "dominican": "Dominican Republic",
1280:     "panamanian": "Panama",
1281:     "bolivian": "Bolivia",
1282:     "chilean": "Chile",
1283:     "argentine": "Argentina",
1284:     "argentinian": "Argentina",
1285:     "saudi": "Saudi Arabia",
1286:     "turkish": "Turkey",
1287: }

```

```

1288: FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
1289:     "tehran": "Iran",
1290:     "caracas": "Venezuela",
1291:     "beijing": "China",
1292:     "moscow": "Russia",
1293:     "havana": "Cuba",
1294:     "kabul": "Afghanistan",
1295:     "damascus": "Syria",
1296:     "baghdad": "Iraq",
1297:     "tripoli": "Libya",
1298:     "mogadishu": "Somalia",
1299:     "bogota": "Colombia",
1300:     "managua": "Nicaragua",
1301:     "tegucigalpa": "Honduras",
1302:     "kyiv": "Ukraine",
1303: }
1304:
1305:
1306: def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:
1307:     alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
1308:     return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)
1309:
1310:
1311: PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))
1312: PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
1313: # A demonym that immediately precedes one of these nouns describes a person's
1314: # nationality, so it also earns origin:<country>, not just a contextual country:.
1315: PATTERN_DEMONYM_PERSON = _compile(
1316:     r"^\w*(?:national|nationals|citizen|citizens|immigrant|immigrants|migrant|migrants|"
1677:     # origin:<COUNTRY> – validated against the sovereign-state vocab.
1678:     for m in PATTERN_ORIGIN_CANDIDATE.finditer(text):
1679:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1680:         if resolved:
1681:             add(f"origin:{_normalize_country(resolved)}", span=m.span(1))
1682:
1683:     # country:<NAME> – broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:
1687:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:     # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:     # country, which the bare-name validators above can't see. A demonym
1691:     # followed by a person noun also earns origin:.
1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", ".")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", ".")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

origin:Nigeria (5 (0.0%))

Precise plain-English rule

Emitted as origin:Nigeria when the text has from Nigeria, with a comma after the country, or a demonym for Nigeria followed immediately by a person-origin noun.

Code text for this tag

```

169:     "Nigeria",
1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). '_resolve_vocab'
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?:{:_PREPOSITIONS_})\b)[A-Za-z][a-zA-Z]*(?)"
1225: # "from <Country>," – anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"^(?:{:_PLACE_})\s*(?!\s+)"
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"^(?:{:_PREPOSITIONS_})\s*(?!\s+)"
1229: # "<Place>," <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"^(?:{:_PLACE_})\s*(?!\s+)"
1238: DEMONYM_TO_COUNTRY: dict[str, str] = {
1239:     "iranian": "Iran",
1240:     "mexican": "Mexico",
1241:     "venezuelan": "Venezuela",
1242:     "chinese": "China",
1243:     "russian": "Russia",
1244:     "cuban": "Cuba",

```

```

1245: "colombian": "Colombia",
1246: "salvadoran": "El Salvador",
1247: "salvadorian": "El Salvador",
1248: "honduran": "Honduras",
1249: "guatemalan": "Guatemala",
1250: "haitian": "Haiti",
1251: "somali": "Somalia",
1252: "somalian": "Somalia",
1253: "afghan": "Afghanistan",
1254: "syrian": "Syria",
1255: "nigerian": "Nigeria",
1256: "pakistani": "Pakistan",
1257: "ukrainian": "Ukraine",
1258: "israeli": "Israel",
1259: "iraqi": "Iraq",
1260: "egyptian": "Egypt",
1261: "yemeni": "Yemen",
1262: "brazilian": "Brazil",
1263: "nicaraguan": "Nicaragua",
1264: "ecuadorian": "Ecuador",
1265: "peruvian": "Peru",
1266: "jamaican": "Jamaica",
1267: "filipino": "Philippines",
1268: "vietnamese": "Vietnam",
1269: "lebanese": "Lebanon",
1270: "libyan": "Libya",
1271: "sudanese": "Sudan",
1272: "ethiopian": "Ethiopia",
1273: "kenyan": "Kenya",
1274: "moroccan": "Morocco",
1275: "algerian": "Algeria",
1276: "bangladeshi": "Bangladesh",
1277: "cambodian": "Cambodia",
1278: "indonesian": "Indonesia",
1279: "dominican": "Dominican Republic",
1280: "panamanian": "Panama",
1281: "bolivian": "Bolivia",
1282: "chilean": "Chile",
1283: "argentine": "Argentina",
1284: "argentinian": "Argentina",
1285: "saudi": "Saudi Arabia",
1286: "turkish": "Turkey",
1287: }
1288: FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
1289:     "tehran": "Iran",
1290:     "caracas": "Venezuela",
1291:     "beijing": "China",
1292:     "moscow": "Russia",
1293:     "havana": "Cuba",
1294:     "kabul": "Afghanistan",
1295:     "damascus": "Syria",
1296:     "baghdad": "Iraq",
1297:     "tripoli": "Libya",
1298:     "mogadishu": "Somalia",
1299:     "bogota": "Colombia",
1300:     "managua": "Nicaragua",
1301:     "tegucigalpa": "Honduras",
1302:     "kyiv": "Ukraine",
1303: }
1304:
1305:
1306: def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:
1307:     alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
1308:     return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)
1309:
1310:
1311: PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))
1312: PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
1313: # A demonym that immediately precedes one of these nouns describes a person's
1314: # nationality, so it also earns origin:<country>, not just a contextual country:.
1315: PATTERN_DEMONYM_PERSON = _compile(
1316:     r"^\W*(?:national|nationals|citizen|citizens|immigrant|immigrants|migrant|migrants|"
1317:
1318:
1319:
1320:
1321:
1322:
1323:
1324:
1325:
1326:
1327:
1328:
1329:
1330:
1331:
1332:
1333:
1334:
1335:
1336:
1337:
1338:
1339:
1340:
1341:
1342:
1343:
1344:
1345:
1346:
1347:
1348:
1349:
1350:
1351:
1352:
1353:
1354:
1355:
1356:
1357:
1358:
1359:
1360:
1361:
1362:
1363:
1364:
1365:
1366:
1367:
1368:
1369:
1370:
1371:
1372:
1373:
1374:
1375:
1376:
1377:
1378:
1379:
1380:
1381:
1382:
1383:
1384:
1385:
1386:
1387:
1388:
1389:
1390:
1391:
1392:
1393:
1394:
1395:
1396:
1397:
1398:
1399:
1400:
1401:
1402:
1403:
1404:
1405:
1406:
1407:
1408:
1409:
1410:
1411:
1412:
1413:
1414:
1415:
1416:
1417:
1418:
1419:
1420:
1421:
1422:
1423:
1424:
1425:
1426:
1427:
1428:
1429:
1430:
1431:
1432:
1433:
1434:
1435:
1436:
1437:
1438:
1439:
1440:
1441:
1442:
1443:
1444:
1445:
1446:
1447:
1448:
1449:
1450:
1451:
1452:
1453:
1454:
1455:
1456:
1457:
1458:
1459:
1460:
1461:
1462:
1463:
1464:
1465:
1466:
1467:
1468:
1469:
1470:
1471:
1472:
1473:
1474:
1475:
1476:
1477:
1478:
1479:
1480:
1481:
1482:
1483:
1484:
1485:
1486:
1487:
1488:
1489:
1490:
1491:
1492:
1493:
1494:
1495:
1496:
1497:
1498:
1499:
1500:
1501:
1502:
1503:
1504:
1505:
1506:
1507:
1508:
1509:
1510:
1511:
1512:
1513:
1514:
1515:
1516:
1517:
1518:
1519:
1520:
1521:
1522:
1523:
1524:
1525:
1526:
1527:
1528:
1529:
1530:
1531:
1532:
1533:
1534:
1535:
1536:
1537:
1538:
1539:
1540:
1541:
1542:
1543:
1544:
1545:
1546:
1547:
1548:
1549:
1550:
1551:
1552:
1553:
1554:
1555:
1556:
1557:
1558:
1559:
1560:
1561:
1562:
1563:
1564:
1565:
1566:
1567:
1568:
1569:
1570:
1571:
1572:
1573:
1574:
1575:
1576:
1577:
1578:
1579:
1580:
1581:
1582:
1583:
1584:
1585:
1586:
1587:
1588:
1589:
1590:
1591:
1592:
1593:
1594:
1595:
1596:
1597:
1598:
1599:
1600:
1601:
1602:
1603:
1604:
1605:
1606:
1607:
1608:
1609:
1610:
1611:
1612:
1613:
1614:
1615:
1616:
1617:
1618:
1619:
1620:
1621:
1622:
1623:
1624:
1625:
1626:
1627:
1628:
1629:
1630:
1631:
1632:
1633:
1634:
1635:
1636:
1637:
1638:
1639:
1640:
1641:
1642:
1643:
1644:
1645:
1646:
1647:
1648:
1649:
1650:
1651:
1652:
1653:
1654:
1655:
1656:
1657:
1658:
1659:
1660:
1661:
1662:
1663:
1664:
1665:
1666:
1667:
1668:
1669:
1670:
1671:
1672:
1673:
1674:
1675:
1676:
1677:
1678:
1679:
1680:
1681:
1682:
1683:
1684:
1685:
1686:
1687:
1688:
1689:
1690:
1691:
1692:
1693:
1694:
1695:
1696:
1697:
1698:
1699:
1700:
1701:
1702:
1703:
1704:
1705:
1706:
1707:
1708:
1709:
1710:
1711:
1712:
1713:
1714:
1715:
1716:
1717:
1718:
1719:
1720:
1721:
1722:
1723:
1724:
1725:
1726:
1727:
1728:
1729:
1730:
1731:
1732:
1733:
1734:
1735:
1736:
1737:
1738:
1739:
1740:
1741:
1742:
1743:
1744:
1745:
1746:
1747:
1748:
1749:
1750:
1751:
1752:
1753:
1754:
1755:
1756:
1757:
1758:
1759:
1760:
1761:
1762:
1763:
1764:
1765:
1766:
1767:
1768:
1769:
1770:
1771:
1772:
1773:
1774:
1775:
1776:
1777:
1778:
1779:
1780:
1781:
1782:
1783:
1784:
1785:
1786:
1787:
1788:
1789:
1790:
1791:
1792:
1793:
1794:
1795:
1796:
1797:
1798:
1799:
1800:
1801:
1802:
1803:
1804:
1805:
1806:
1807:
1808:
1809:
1810:
1811:
1812:
1813:
1814:
1815:
1816:
1817:
1818:
1819:
1820:
1821:
1822:
1823:
1824:
1825:
1826:
1827:
1828:
1829:
1830:
1831:
1832:
1833:
1834:
1835:
1836:
1837:
1838:
1839:
1840:
1841:
1842:
1843:
1844:
1845:
1846:
1847:
1848:
1849:
1850:
1851:
1852:
1853:
1854:
1855:
1856:
1857:
1858:
1859:
1860:
1861:
1862:
1863:
1864:
1865:
1866:
1867:
1868:
1869:
1870:
1871:
1872:
1873:
1874:
1875:
1876:
1877:
1878:
1879:
1880:
1881:
1882:
1883:
1884:
1885:
1886:
1887:
1888:
1889:
1890:
1891:
1892:
1893:
1894:
1895:
1896:
1897:
1898:
1899:
1900:
1901:
1902:
1903:
1904:
1905:
1906:
1907:
1908:
1909:
1910:
1911:
1912:
1913:
1914:
1915:
1916:
1917:
1918:
1919:
1920:
1921:
1922:
1923:
1924:
1925:
1926:
1927:
1928:
1929:
1930:
1931:
1932:
1933:
1934:
1935:
1936:
1937:
1938:
1939:
1940:
1941:
1942:
1943:
1944:
1945:
1946:
1947:
1948:
1949:
1950:
1951:
1952:
1953:
1954:
1955:
1956:
1957:
1958:
1959:
1960:
1961:
1962:
1963:
1964:
1965:
1966:
1967:
1968:
1969:
1970:
1971:
1972:
1973:
1974:
1975:
1976:
1977:
1978:
1979:
1980:
1981:
1982:
1983:
1984:
1985:
1986:
1987:
1988:
1989:
1990:
1991:
1992:
1993:
1994:
1995:
1996:
1997:
1998:
1999:
2000:
2001:
2002:
2003:
2004:
2005:
2006:
2007:
2008:
2009:
2010:
2011:
2012:
2013:
2014:
2015:
2016:
2017:
2018:
2019:
2020:
2021:
2022:
2023:
2024:
2025:
2026:
2027:
2028:
2029:
2030:
2031:
2032:
2033:
2034:
2035:
2036:
2037:
2038:
2039:
2040:
2041:
2042:
2043:
2044:
2045:
2046:
2047:
2048:
2049:
2050:
2051:
2052:
2053:
2054:
2055:
2056:
2057:
2058:
2059:

```

```

2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

origin:Russia (5 (0.0%))

Precise plain-English rule

Emitted as origin:Russia when the text has from Russia, with a comma after the country, or a demonym for Russia followed immediately by a person-origin noun.

Code text for this tag

```

182:     "Russia",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z](?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]?"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:({_PREPOSITIONS})\s+({_PLACE}))\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE}),\s+({_PLACE})\b")

1238: DEMONYM_TO_COUNTRY: dict[str, str] = {
1239:     "iranian": "Iran",
1240:     "mexican": "Mexico",
1241:     "venezuelan": "Venezuela",
1242:     "chinese": "China",
1243:     "russian": "Russia",
1244:     "cuban": "Cuba",
1245:     "colombian": "Colombia",
1246:     "salvadoran": "El Salvador",
1247:     "salvadorian": "El Salvador",
1248:     "honduran": "Honduras",
1249:     "guatemalan": "Guatemala",
1250:     "haitian": "Haiti",
1251:     "somali": "Somalia",
1252:     "somalian": "Somalia",
1253:     "afghan": "Afghanistan",
1254:     "syrian": "Syria",
1255:     "nigerian": "Nigeria",
1256:     "pakistani": "Pakistan",
1257:     "ukrainian": "Ukraine",
1258:     "israeli": "Israel",
1259:     "iraqi": "Iraq",
1260:     "egyptian": "Egypt",
1261:     "yemeni": "Yemen",
1262:     "brazilian": "Brazil",
1263:     "nicaraguan": "Nicaragua",
1264:     "ecuadorian": "Ecuador",
1265:     "peruvian": "Peru",
1266:     "jamaican": "Jamaica",
1267:     "filipino": "Philippines",
1268:     "vietnamese": "Vietnam",
1269:     "lebanese": "Lebanon",
1270:     "libyan": "Libya",
1271:     "sudanese": "Sudan",
1272:     "ethiopian": "Ethiopia",
1273:     "kenyan": "Kenya",
1274:     "moroccan": "Morocco",
1275:     "algerian": "Algeria",
1276:     "bangladeshi": "Bangladesh",
1277:     "cambodian": "Cambodia",
1278:     "indonesian": "Indonesia",
1279:     "dominican": "Dominican Republic",
1280:     "panamanian": "Panama",
1281:     "bolivian": "Bolivia",
1282:     "chilean": "Chile",
1283:     "argentine": "Argentina",
1284:     "argentinian": "Argentina",
1285:     "saudi": "Saudi Arabia",
1286:     "turkish": "Turkey",
1287: }
1288: FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
1289:     "tehran": "Iran",
1290:     "caracas": "Venezuela",
1291:     "beijing": "China",
1292:     "moscow": "Russia",
1293:     "havana": "Cuba",
1294:     "kabul": "Afghanistan",
1295:     "damascus": "Syria",
1296:     "baghdad": "Iraq",
1297:     "tripoli": "Libya",
1298:     "mogadishu": "Somalia",
1299:     "bogota": "Colombia",
1300:     "managua": "Nicaragua",
1301:     "tegucigalpa": "Honduras",
1302:     "kyiv": "Ukraine",
1303: }
1304:
1305:
1306: def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:
1307:     alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
1308:     return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)
1309:
1310:
1311: PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))

```

```

1312: PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
1313: # A demonym that immediately precedes one of these nouns describes a person's
1314: # nationality, so it also earns origin:<country>, not just a contextual country:.
1315: PATTERN_DEMONYM_PERSON = _compile(
1316:     r"^\W*(?:national|nationals|citizen|citizens|immigrant|immigrants|migrant|migrants|"
1677:     # origin:<COUNTRY> – validated against the sovereign-state vocab.
1678:     for m in PATTERN_ORIGIN_CANDIDATE.finditer(text):
1679:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1680:         if resolved:
1681:             add(f"origin:{_normalize_country(resolved)}", span=m.span(1))
1682:
1683:     # country:<NAME> – broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:
1687:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:     # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:     # country, which the bare-name validators above can't see. A demonym
1691:     # followed by a person noun also earns origin:.
1692:     for m in PATTERN_DEMONYM_PERSON.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

origin:Ukraine (5 (0.0%))

Precise plain-English rule

Emitted as origin:Ukraine when the text has from Ukraine, with a comma after the country, or a demonym for Ukraine followed immediately by a person-origin noun.

Code text for this tag

```

215:     "Ukraine",
1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the *next* place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). `_resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," – anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{[_PREPOSITIONS]})\s+({_PLACE})\b")
1229: # "<Place>," <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE})\s+({_PLACE})\b")
1238: DEMONYM_TO_COUNTRY: dict[str, str] = {
1239:     "iranian": "Iran",
1240:     "mexican": "Mexico",
1241:     "venezuelan": "Venezuela",
1242:     "chinese": "China",
1243:     "russian": "Russia",
1244:     "cuban": "Cuba",
1245:     "colombian": "Colombia",
1246:     "salvadoran": "El Salvador",
1247:     "salvadorian": "El Salvador",
1248:     "honduran": "Honduras",
1249:     "guatemalan": "Guatemala",
1250:     "haitian": "Haiti",
1251:     "somali": "Somalia",
1252:     "somalian": "Somalia",
1253:     "afghan": "Afghanistan",
1254:     "syrian": "Syria",
1255:     "nigerian": "Nigeria",
1256:     "pakistani": "Pakistan",
1257:     "ukrainian": "Ukraine",
1258:     "israeli": "Israel",
1259:     "iraqi": "Iraq",
1260:     "egyptian": "Egypt",
1261:     "yemeni": "Yemen",
1262:     "brazilian": "Brazil",
1263:     "nicaraguan": "Nicaragua",
1264:     "ecuadorian": "Ecuador",
1265:     "peruvian": "Peru",
1266:     "jamaican": "Jamaica",
1267:     "filipino": "Philippines",
1268:     "vietnamese": "Vietnam",

```

```

1269: "lebanese": "Lebanon",
1270: "libyan": "Libya",
1271: "sudanese": "Sudan",
1272: "ethiopian": "Ethiopia",
1273: "kenyan": "Kenya",
1274: "moroccan": "Morocco",
1275: "algerian": "Algeria",
1276: "bangladeshi": "Bangladesh",
1277: "cambodian": "Cambodia",
1278: "indonesian": "Indonesia",
1279: "dominican": "Dominican Republic",
1280: "panamanian": "Panama",
1281: "bolivian": "Bolivia",
1282: "chilean": "Chile",
1283: "argentine": "Argentina",
1284: "argentinian": "Argentina",
1285: "saudi": "Saudi Arabia",
1286: "turkish": "Turkey",
1287: }
1288: FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
1289:     "tehran": "Iran",
1290:     "caracas": "Venezuela",
1291:     "beijing": "China",
1292:     "moscow": "Russia",
1293:     "havana": "Cuba",
1294:     "kabul": "Afghanistan",
1295:     "damascus": "Syria",
1296:     "baghdad": "Iraq",
1297:     "tripoli": "Libya",
1298:     "mogadishu": "Somalia",
1299:     "bogota": "Colombia",
1300:     "managua": "Nicaragua",
1301:     "tegucigalpa": "Honduras",
1302:     "kyiv": "Ukraine",
1303: }
1304:
1305:
1306: def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:
1307:     alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
1308:     return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)
1309:
1310:
1311: PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))
1312: PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
1313: # A demonym that immediately precedes one of these nouns describes a person's
1314: # nationality, so it also earns origin:<country>, not just a contextual country:.
1315: PATTERN_DEMONYM_PERSON = _compile(
1316:     r"^\w*(?:national|nationals|citizen|citizens|immigrant|immigrants|migrant|migrants)"
1677: # origin:<COUNTRY> – validated against the sovereign-state vocab.
1678: for m in PATTERN_ORIGIN_CANDIDATE.finditer(text):
1679:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1680:     if resolved:
1681:         add(f"origin:{_normalize_country(resolved)}", span=m.span(1))
1682:
1683: # country:<NAME> – broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
2052: if low == "burma":
2053:     return "Myanmar"
2054: return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

origin:Cambodia (4 (0.0%))

Precise plain-English rule

Emitted as origin:Cambodia when the text has from Cambodia, with a comma after the country, or a demonym for Cambodia followed immediately by a person-origin noun.

Code text for this tag

```

77:     "Cambodia",
1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the *next* place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). `_resolve_vocab`

```

```

1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]+)?"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*", "")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:({_PREPOSITIONS})\s+({_PLACE})\b)")
1229: # "<Place>," - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE})\s+({_PLACE})\b")

1238: DEMONYM_TO_COUNTRY: dict[str, str] = {
1239:     "iranian": "Iran",
1240:     "mexican": "Mexico",
1241:     "venezuelan": "Venezuela",
1242:     "chinese": "China",
1243:     "russian": "Russia",
1244:     "cuban": "Cuba",
1245:     "colombian": "Colombia",
1246:     "salvadoran": "El Salvador",
1247:     "salvadorian": "El Salvador",
1248:     "honduran": "Honduras",
1249:     "guatemalan": "Guatemala",
1250:     "haitian": "Haiti",
1251:     "somali": "Somalia",
1252:     "somalian": "Somalia",
1253:     "afghan": "Afghanistan",
1254:     "syrian": "Syria",
1255:     "nigerian": "Nigeria",
1256:     "pakistani": "Pakistan",
1257:     "ukrainian": "Ukraine",
1258:     "israeli": "Israel",
1259:     "iraqi": "Iraq",
1260:     "egyptian": "Egypt",
1261:     "yemeni": "Yemen",
1262:     "brazilian": "Brazil",
1263:     "nicaraguan": "Nicaragua",
1264:     "ecuadorian": "Ecuador",
1265:     "peruvian": "Peru",
1266:     "jamaican": "Jamaica",
1267:     "filipino": "Philippines",
1268:     "vietnamese": "Vietnam",
1269:     "lebanese": "Lebanon",
1270:     "libyan": "Libya",
1271:     "sudanese": "Sudan",
1272:     "ethiopian": "Ethiopia",
1273:     "kenyan": "Kenya",
1274:     "moroccan": "Morocco",
1275:     "algerian": "Algeria",
1276:     "bangladeshi": "Bangladesh",
1277:     "cambodian": "Cambodia",
1278:     "indonesian": "Indonesia",
1279:     "dominican": "Dominican Republic",
1280:     "panamanian": "Panama",
1281:     "bolivian": "Bolivia",
1282:     "chilean": "Chile",
1283:     "argentine": "Argentina",
1284:     "argentinian": "Argentina",
1285:     "saudi": "Saudi Arabia",
1286:     "turkish": "Turkey",
1287: }
1288: FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
1289:     "tehran": "Iran",
1290:     "caracas": "Venezuela",
1291:     "beijing": "China",
1292:     "moscow": "Russia",
1293:     "havana": "Cuba",
1294:     "kabul": "Afghanistan",
1295:     "damascus": "Syria",
1296:     "baghdad": "Iraq",
1297:     "tripoli": "Libya",
1298:     "mogadishu": "Somalia",
1299:     "bogota": "Colombia",
1300:     "managua": "Nicaragua",
1301:     "tegucigalpa": "Honduras",
1302:     "kyiv": "Ukraine",
1303: }
1304:
1305:
1306: def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:
1307:     alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
1308:     return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)
1309:
1310:
1311: PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))
1312: PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
1313: # A demonym that immediately precedes one of these nouns describes a person's
1314: # nationality, so it also earns origin:<country>, not just a contextual country:.
1315: PATTERN_DEMONYM_PERSON = re.compile(
1316:     r"^\W*(?:national|nationals|citizen|citizens|immigrant|immigrants|migrant|migrants|"
1317:
1318:
1319:
1320:
1321:
1322:
1323:
1324:
1325:
1326:
1327:
1328:
1329:
1330:
1331:
1332:
1333:
1334:
1335:
1336:
1337:
1338:
1339:
1340:
1341:
1342:
1343:
1344:
1345:
1346:
1347:
1348:
1349:
1350:
1351:
1352:
1353:
1354:
1355:
1356:
1357:
1358:
1359:
1360:
1361:
1362:
1363:
1364:
1365:
1366:
1367:
1368:
1369:
1370:
1371:
1372:
1373:
1374:
1375:
1376:
1377:
1378:
1379:
1380:
1381:
1382:
1383:
1384:
1385:
1386:
1387:
1388:
1389:
1390:
1391:
1392:
1393:
1394:
1395:
1396:
1397:
1398:
1399:
1400:
1401:
1402:
1403:
1404:
1405:
1406:
1407:
1408:
1409:
1410:
1411:
1412:
1413:
1414:
1415:
1416:
1417:
1418:
1419:
1420:
1421:
1422:
1423:
1424:
1425:
1426:
1427:
1428:
1429:
1430:
1431:
1432:
1433:
1434:
1435:
1436:
1437:
1438:
1439:
1440:
1441:
1442:
1443:
1444:
1445:
1446:
1447:
1448:
1449:
1450:
1451:
1452:
1453:
1454:
1455:
1456:
1457:
1458:
1459:
1460:
1461:
1462:
1463:
1464:
1465:
1466:
1467:
1468:
1469:
1470:
1471:
1472:
1473:
1474:
1475:
1476:
1477:
1478:
1479:
1480:
1481:
1482:
1483:
1484:
1485:
1486:
1487:
1488:
1489:
1490:
1491:
1492:
1493:
1494:
1495:
1496:
1497:
1498:
1499:
1500:
1501:
1502:
1503:
1504:
1505:
1506:
1507:
1508:
1509:
1510:
1511:
1512:
1513:
1514:
1515:
1516:
1517:
1518:
1519:
1520:
1521:
1522:
1523:
1524:
1525:
1526:
1527:
1528:
1529:
1530:
1531:
1532:
1533:
1534:
1535:
1536:
1537:
1538:
1539:
1540:
1541:
1542:
1543:
1544:
1545:
1546:
1547:
1548:
1549:
1550:
1551:
1552:
1553:
1554:
1555:
1556:
1557:
1558:
1559:
1560:
1561:
1562:
1563:
1564:
1565:
1566:
1567:
1568:
1569:
1570:
1571:
1572:
1573:
1574:
1575:
1576:
1577:
1578:
1579:
1580:
1581:
1582:
1583:
1584:
1585:
1586:
1587:
1588:
1589:
1590:
1591:
1592:
1593:
1594:

```

```

1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:
2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", ".")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", ".")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

origin:Syria (4 (0.0%))

Precise plain-English rule

Emitted as origin:Syria when the text has from Syria, with a comma after the country, or a demonym for Syria followed immediately by a person-origin noun.

Code text for this tag

```

202:     "Syria",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the *next* place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]+)?"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*", "")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:({_PREPOSITIONS})\s+({_PLACE})\b)"
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE})\s+({_PLACE})\b)"

1238: DEMONYM_TO_COUNTRY: dict[str, str] = {
1239:     "iranian": "Iran",
1240:     "mexican": "Mexico",
1241:     "venezuelan": "Venezuela",
1242:     "chinese": "China",
1243:     "russian": "Russia",
1244:     "cuban": "Cuba",
1245:     "colombian": "Colombia",
1246:     "salvadoran": "El Salvador",
1247:     "salvadorian": "El Salvador",
1248:     "honduran": "Honduras",
1249:     "guatemalan": "Guatemala",
1250:     "haitian": "Haiti",
1251:     "somali": "Somalia",
1252:     "somalian": "Somalia",
1253:     "afghan": "Afghanistan",
1254:     "syrian": "Syria",
1255:     "nigerian": "Nigeria",
1256:     "pakistani": "Pakistan",
1257:     "ukrainian": "Ukraine",
1258:     "israeli": "Israel",
1259:     "iraqi": "Iraq",
1260:     "egyptian": "Egypt",
1261:     "yemeni": "Yemen",
1262:     "brazilian": "Brazil",
1263:     "nicaraguan": "Nicaragua",
1264:     "ecuadorian": "Ecuador",
1265:     "peruvian": "Peru",
1266:     "jamaican": "Jamaica",
1267:     "filipino": "Philippines",
1268:     "vietnamese": "Vietnam",
1269:     "lebanese": "Lebanon",
1270:     "libyan": "Libya",
1271:     "sudanese": "Sudan",
1272:     "ethiopian": "Ethiopia",
1273:     "kenyan": "Kenya",
1274:     "moroccan": "Morocco",
1275:     "algerian": "Algeria",
1276:     "bangladeshi": "Bangladesh",
1277:     "cambodian": "Cambodia",
1278:     "indonesian": "Indonesia",
1279:     "dominican": "Dominican Republic",
1280:     "panamanian": "Panama",
1281:     "bolivian": "Bolivia",
1282:     "chilean": "Chile",
1283:     "argentine": "Argentina",
1284:     "argentinian": "Argentina",
1285:     "saudi": "Saudi Arabia",
1286:     "turkish": "Turkey",
1287: }
1288: FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
1289:     "tehran": "Iran",
1290:     "caracas": "Venezuela",
1291:     "beijing": "China",
1292:     "moscow": "Russia",

```

```

1293:     "havana": "Cuba",
1294:     "kabul": "Afghanistan",
1295:     "damascus": "Syria",
1296:     "baghdad": "Iraq",
1297:     "tripoli": "Libya",
1298:     "mogadishu": "Somalia",
1299:     "bogota": "Colombia",
1300:     "managua": "Nicaragua",
1301:     "tegucigalpa": "Honduras",
1302:     "kyiv": "Ukraine",
1303: }
1304:
1305:
1306: def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:
1307:     alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
1308:     return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)
1309:
1310:
1311: PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))
1312: PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
1313: # A demonym that immediately precedes one of these nouns describes a person's
1314: # nationality, so it also earns origin:<country>, not just a contextual country:.
1315: PATTERN_DEMONYM_PERSON = _compile(
1316:     r"^\W*(?:national|nationals|citizen|citizens|immigrant|immigrants|migrant|migrants|"
1677:     # origin:<COUNTRY> – validated against the sovereign-state vocab.
1678:     for m in PATTERN_ORIGIN_CANDIDATE.finditer(text):
1679:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1680:         if resolved:
1681:             add(f"origin:{_normalize_country(resolved)}", span=m.span(1))
1682:
1683:     # country:<NAME> – broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:
1687:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:     # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:     # country, which the bare-name validators above can't see. A demonym
1691:     # followed by a person noun also earns origin:.
1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

origin:Kenya (3 (0.0%))

Precise plain-English rule

Emitted as origin:Kenya when the text has from Kenya, with a comma after the country, or a demonym for Kenya followed immediately by a person-origin noun.

Code text for this tag

```

131:     "Kenya",
1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). `_resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!{i:_PREPOSITIONS}})\b)[A-Za-z][a-zA-Z]*(?)"
1225: # "from <Country>," – anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:{i:_PREPOSITIONS}})\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{i:_PREPOSITIONS}})\s+({_PLACE})\b")
1229: # "<Place>," <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{PLACE}\s+({_PLACE})\b")
1238: DEMONYM_TO_COUNTRY: dict[str, str] = {
1239:     "iranian": "Iran",
1240:     "mexican": "Mexico",
1241:     "venezuelan": "Venezuela",
1242:     "chinese": "China",
1243:     "russian": "Russia",
1244:     "cuban": "Cuba",
1245:     "colombian": "Colombia",
1246:     "salvadoran": "El Salvador",
1247:     "salvadorian": "El Salvador",
1248:     "honduran": "Honduras",
1249:     "guatemalan": "Guatemala",

```

```

1250: "haitian": "Haiti",
1251: "somali": "Somalia",
1252: "somalian": "Somalia",
1253: "afghan": "Afghanistan",
1254: "syrian": "Syria",
1255: "nigerian": "Nigeria",
1256: "pakistanian": "Pakistan",
1257: "ukrainian": "Ukraine",
1258: "israeli": "Israel",
1259: "iraqi": "Iraq",
1260: "egyptian": "Egypt",
1261: "yemeni": "Yemen",
1262: "brazilian": "Brazil",
1263: "nicaraguan": "Nicaragua",
1264: "ecuadorian": "Ecuador",
1265: "peruvian": "Peru",
1266: "jamaican": "Jamaica",
1267: "filipino": "Philippines",
1268: "vietnamese": "Vietnam",
1269: "lebanese": "Lebanon",
1270: "libyan": "Libya",
1271: "sudanese": "Sudan",
1272: "ethiopian": "Ethiopia",
1273: "kenyan": "Kenya",
1274: "moroccan": "Morocco",
1275: "algerian": "Algeria",
1276: "bangladeshi": "Bangladesh",
1277: "cambodian": "Cambodia",
1278: "indonesian": "Indonesia",
1279: "dominican": "Dominican Republic",
1280: "panamanian": "Panama",
1281: "bolivian": "Bolivia",
1282: "chilean": "Chile",
1283: "argentine": "Argentina",
1284: "argentinian": "Argentina",
1285: "saudi": "Saudi Arabia",
1286: "turkish": "Turkey",
1287: }
1288: FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
1289:     "tehran": "Iran",
1290:     "caracas": "Venezuela",
1291:     "beijing": "China",
1292:     "moscow": "Russia",
1293:     "havana": "Cuba",
1294:     "kabul": "Afghanistan",
1295:     "damascus": "Syria",
1296:     "baghdad": "Iraq",
1297:     "tripoli": "Libya",
1298:     "mogadishu": "Somalia",
1299:     "bogota": "Colombia",
1300:     "managua": "Nicaragua",
1301:     "tegucigalpa": "Honduras",
1302:     "kyiv": "Ukraine",
1303: }
1304:
1305:
1306: def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:
1307:     alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
1308:     return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)
1309:
1310:
1311: PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))
1312: PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
1313: # A demonym that immediately precedes one of these nouns describes a person's
1314: # nationality, so it also earns origin:<country>, not just a contextual country:.
1315: PATTERN_DEMONYM_PERSON = _compile(
1316:     r"^\w*(?:national|nationals|citizen|citizens|immigrant|immigrants|migrant|migrants)"
1677: # origin:<COUNTRY> – validated against the sovereign-state vocab.
1678: for m in PATTERN_ORIGIN_CANDIDATE.finditer(text):
1679:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1680:     if resolved:
1681:         add(f"origin:{_normalize_country(resolved)}", span=m.span(1))
1682:
1683: # country:<NAME> – broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
2052: if low == "burma":
2053:     return "Myanmar"
2054: return s.replace(" ", ".")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", ".")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:

```

2065: Tries the full phrase first, then its leading word, so a greedy
2066: case-insensitive capture that swallowed a trailing lowercase token

origin:Sierra-Leone (3 (0.0%))

Precise plain-English rule

Emitted as origin:Sierra-Leone when the text has from Sierra Leone, with a comma after the country, or a demonym for Sierra Leone followed immediately by a person-origin noun.

Code text for this tag

```
188:     "Sierra Leone",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z](?:\s+(?!{_{PREPOSITIONS}}\b)[A-Za-z][a-zA-Z]+)?"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{_{PREPOSITIONS}})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_{PLACE}}\s+({_PLACE})\b")

1238: DEMONYM_TO_COUNTRY: dict[str, str] = {
1239:     "iranian": "Iran",
1240:     "mexican": "Mexico",
1241:     "venezuelan": "Venezuela",
1242:     "chinese": "China",
1243:     "russian": "Russia",
1244:     "cuban": "Cuba",
1245:     "colombian": "Colombia",
1246:     "salvadoran": "El Salvador",
1247:     "salvadorian": "El Salvador",
1248:     "honduran": "Honduras",
1249:     "guatemalan": "Guatemala",
1250:     "haitian": "Haiti",
1251:     "somali": "Somalia",
1252:     "somalian": "Somalia",
1253:     "afghan": "Afghanistan",
1254:     "syrrian": "Syria",
1255:     "nigerian": "Nigeria",
1256:     "pakistani": "Pakistan",
1257:     "ukrainian": "Ukraine",
1258:     "israeli": "Israel",
1259:     "iraqi": "Iraq",
1260:     "egyptian": "Egypt",
1261:     "yemeni": "Yemen",
1262:     "brazilian": "Brazil",
1263:     "nicaraguan": "Nicaragua",
1264:     "ecuadorian": "Ecuador",
1265:     "peruvian": "Peru",
1266:     "jamaican": "Jamaica",
1267:     "filipino": "Philippines",
1268:     "vietnamese": "Vietnam",
1269:     "lebanese": "Lebanon",
1270:     "libyan": "Libya",
1271:     "sudanese": "Sudan",
1272:     "ethiopian": "Ethiopia",
1273:     "kenyan": "Kenya",
1274:     "moroccan": "Morocco",
1275:     "algerian": "Algeria",
1276:     "bangladeshi": "Bangladesh",
1277:     " cambodian": "Cambodia",
1278:     "indonesian": "Indonesia",
1279:     "dominican": "Dominican Republic",
1280:     "panamanian": "Panama",
1281:     "bolivian": "Bolivia",
1282:     "chilean": "Chile",
1283:     "argentine": "Argentina",
1284:     "argentinian": "Argentina",
1285:     "saudi": "Saudi Arabia",
1286:     "turkish": "Turkey",
1287: }
1288: FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
1289:     "tehran": "Iran",
1290:     "caracas": "Venezuela",
1291:     "beijing": "China",
1292:     "moscow": "Russia",
1293:     "havana": "Cuba",
1294:     "kabul": "Afghanistan",
1295:     "damascus": "Syria",
1296:     "baghdad": "Iraq",
1297:     "tripoli": "Libya",
1298:     "mogadishu": "Somalia",
1299:     "bogota": "Colombia",
1300:     "managua": "Nicaragua",
1301:     "tegucigalpa": "Honduras",
1302:     "kyiv": "Ukraine",
1303: }
1304:
1305:
1306: def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:
1307:     alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
1308:     return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)
1309:
1310:
1311: PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))
1312: PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
1313: # A demonym that immediately precedes one of these nouns describes a person's
1314: # nationality, so it also earns origin:<country>, not just a contextual country:.
1315: PATTERN_DEMONYM_PERSON = _compile(
1316:     r"^\W*(?:national|nationals|citizen|citizens|immigrant|immigrants|migrant|migrants|"
```

```

1677: # origin:<COUNTRY> – validated against the sovereign-state vocab.
1678: for m in PATTERN_ORIGIN_CANDIDATE.finditer(text):
1679:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1680:     if resolved:
1681:         add(f"origin:{_normalize_country(resolved)}", span=m.span(1))
1682:
1683: # country:<NAME> – broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:
2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
2052: if low == "burma":
2053:     return "Myanmar"
2054: return s.replace(" ", ".")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", ".")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

origin:Sudan (3 (0.0%))

Precise plain-English rule

Emitted as origin:Sudan when the text has from Sudan, with a comma after the country, or a demonym for Sudan followed immediately by a person-origin noun.

Code text for this tag

```

198:     "Sudan",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]+)?"
1225: # "from <Country>" – anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*", "")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>", <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE})\s+({_PLACE})\b")

1238: DEMONYM_TO_COUNTRY: dict[str, str] = {
1239:     "iranian": "Iran",
1240:     "mexican": "Mexico",
1241:     "venezuelan": "Venezuela",
1242:     "chinese": "China",
1243:     "russian": "Russia",
1244:     "cuban": "Cuba",
1245:     "colombian": "Colombia",
1246:     "salvadoran": "El Salvador",
1247:     "salvadorian": "El Salvador",
1248:     "honduran": "Honduras",
1249:     "guatemalan": "Guatemala",
1250:     "haitian": "Haiti",
1251:     "somali": "Somalia",
1252:     "somalian": "Somalia",
1253:     "afghan": "Afghanistan",
1254:     "syrian": "Syria",
1255:     "nigerian": "Nigeria",
1256:     "pakistani": "Pakistan",
1257:     "ukrainian": "Ukraine",
1258:     "israeli": "Israel",
1259:     "iraqi": "Iraq",
1260:     "egyptian": "Egypt",
1261:     "yemeni": "Yemen",
1262:     "brazilian": "Brazil",
1263:     "nicaraguan": "Nicaragua",
1264:     "ecuadorian": "Ecuador",
1265:     "peruvian": "Peru",
1266:     "jamaican": "Jamaica",
1267:     "filipino": "Philippines",
1268:     "vietnamese": "Vietnam",
1269:     "lebanese": "Lebanon",
1270:     "libyan": "Libya",
1271:     "sudanese": "Sudan",
1272:     "ethiopian": "Ethiopia",
1273:     "kenyan": "Kenya",
1274:     "moroccan": "Morocco",

```

```

1275: "algerian": "Algeria",
1276: "bangladeshi": "Bangladesh",
1277: "cambodian": "Cambodia",
1278: "indonesian": "Indonesia",
1279: "dominican": "Dominican Republic",
1280: "panamanian": "Panama",
1281: "bolivian": "Bolivia",
1282: "chilean": "Chile",
1283: "argentine": "Argentina",
1284: "argentinian": "Argentina",
1285: "saudi": "Saudi Arabia",
1286: "turkish": "Turkey",
1287: }
1288: FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
1289:     "tehran": "Iran",
1290:     "caracas": "Venezuela",
1291:     "beijing": "China",
1292:     "moscow": "Russia",
1293:     "havana": "Cuba",
1294:     "kabul": "Afghanistan",
1295:     "damascus": "Syria",
1296:     "baghdad": "Iraq",
1297:     "tripoli": "Libya",
1298:     "mogadishu": "Somalia",
1299:     "bogota": "Colombia",
1300:     "managua": "Nicaragua",
1301:     "tegucigalpa": "Honduras",
1302:     "kyiv": "Ukraine",
1303: }
1304:
1305:
1306: def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:
1307:     alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
1308:     return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)
1309:
1310:
1311: PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))
1312: PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
1313: # A demonym that immediately precedes one of these nouns describes a person's
1314: # nationality, so it also earns origin:<country>, not just a contextual country:.
1315: PATTERN_DEMONYM_PERSON = _compile(
1316:     r"^\w*(?:national|nationals|citizen|citizens|immigrant|immigrants|migrant|migrants|"
1677:     # origin:<COUNTRY> – validated against the sovereign-state vocab.
1678:     for m in PATTERN_ORIGIN_CANDIDATE.finditer(text):
1679:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1680:         if resolved:
1681:             add(f"origin:{_normalize_country(resolved)}", span=m.span(1))
1682:
1683:     # country:<NAME> – broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:
1687:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:     # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:     # country, which the bare-name validators above can't see. A demonym
1691:     # followed by a person noun also earns origin:.
1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

origin:Angola (2 (0.0%))

Precise plain-English rule

Emitted as origin:Angola when the text has from Angola, with a comma after the country, or a demonym for Angola followed immediately by a person-origin noun.

Code text for this tag

```

54:     "Angola",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). `_resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?:{:_PREPOSITIONS_})\b)[A-Za-z][a-zA-Z]*(?)"
1225: # "from <Country>," – anchors the ORIGIN validator.

```

```

1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"(?i:from)\s+({_PLACE})\s*")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"(?i:{_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>, <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1238: DEMONYM_TO_COUNTRY: dict[str, str] = {
1239:     "iranian": "Iran",
1240:     "mexican": "Mexico",
1241:     "venezuelan": "Venezuela",
1242:     "chinese": "China",
1243:     "russian": "Russia",
1244:     "cuban": "Cuba",
1245:     "colombian": "Colombia",
1246:     "salvadoran": "El Salvador",
1247:     "salvadorian": "El Salvador",
1248:     "honduran": "Honduras",
1249:     "guatemalan": "Guatemala",
1250:     "haitian": "Haiti",
1251:     "somali": "Somalia",
1252:     "somalian": "Somalia",
1253:     "afghan": "Afghanistan",
1254:     "syrian": "Syria",
1255:     "nigerian": "Nigeria",
1256:     "pakistani": "Pakistan",
1257:     "ukrainian": "Ukraine",
1258:     "israeli": "Israel",
1259:     "iraqi": "Iraq",
1260:     "egyptian": "Egypt",
1261:     "yemeni": "Yemen",
1262:     "brazilian": "Brazil",
1263:     "nicaraguan": "Nicaragua",
1264:     "ecuadorian": "Ecuador",
1265:     "peruvian": "Peru",
1266:     "jamaican": "Jamaica",
1267:     "filipino": "Philippines",
1268:     "vietnamese": "Vietnam",
1269:     "lebanese": "Lebanon",
1270:     "libyan": "Libya",
1271:     "sudanese": "Sudan",
1272:     "ethiopian": "Ethiopia",
1273:     "kenyan": "Kenya",
1274:     "moroccan": "Morocco",
1275:     "algerian": "Algeria",
1276:     "bangladeshi": "Bangladesh",
1277:     "cambodian": "Cambodia",
1278:     "indonesian": "Indonesia",
1279:     "dominican": "Dominican Republic",
1280:     "panamanian": "Panama",
1281:     "bolivian": "Bolivia",
1282:     "chilean": "Chile",
1283:     "argentine": "Argentina",
1284:     "argentinian": "Argentina",
1285:     "saudi": "Saudi Arabia",
1286:     "turkish": "Turkey",
1287: }
1288: FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
1289:     "tehran": "Iran",
1290:     "caracas": "Venezuela",
1291:     "beijing": "China",
1292:     "moscow": "Russia",
1293:     "havana": "Cuba",
1294:     "kabul": "Afghanistan",
1295:     "damascus": "Syria",
1296:     "baghdad": "Iraq",
1297:     "tripoli": "Libya",
1298:     "mogadishu": "Somalia",
1299:     "bogota": "Colombia",
1300:     "managua": "Nicaragua",
1301:     "tegucigalpa": "Honduras",
1302:     "kyiv": "Ukraine",
1303: }
1304:
1305:
1306: def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:
1307:     alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
1308:     return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)
1309:
1310:
1311: PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))
1312: PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
1313: # A demonym that immediately precedes one of these nouns describes a person's
1314: # nationality, so it also earns origin:<country>, not just a contextual country:.
1315: PATTERN_DEMONYM_PERSON = _compile(
1316:     r"^\W*(?:national|nationals|citizen|citizens|immigrant|immigrants|migrant|migrants|"
1677:     # origin:<COUNTRY> – validated against the sovereign-state vocab.
1678:     for m in PATTERN_ORIGIN_CANDIDATE.finditer(text):
1679:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1680:         if resolved:
1681:             add(f"origin:{_normalize_country(resolved)}", span=m.span(1))
1682:
1683:     # country:<NAME> – broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:
1687:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:     # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:     # country, which the bare-name validators above can't see. A demonym
1691:     # followed by a person noun also earns origin:.
1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
2046:     if low in ("usa", "united states"):
```

```

2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

origin:Barbados (2 (0.0%))

Precise plain-English rule

Emitted as origin:Barbados when the text has from Barbados, with a comma after the country, or a demonym for Barbados followed immediately by a person-origin noun.

Code text for this tag

```

63:     "Barbados",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]+(?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]+)?"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:{_PREPOSITIONS})\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE}\s+({_PLACE})\b")

1238: DEMONYM_TO_COUNTRY: dict[str, str] = {
1239:     "iranian": "Iran",
1240:     "mexican": "Mexico",
1241:     "venezuelan": "Venezuela",
1242:     "chinese": "China",
1243:     "russian": "Russia",
1244:     "cuban": "Cuba",
1245:     "colombian": "Colombia",
1246:     "salvadoran": "El Salvador",
1247:     "salvadorian": "El Salvador",
1248:     "honduran": "Honduras",
1249:     "guatemalan": "Guatemala",
1250:     "haitian": "Haiti",
1251:     "somali": "Somalia",
1252:     "somalian": "Somalia",
1253:     "afghan": "Afghanistan",
1254:     "syrian": "Syria",
1255:     "nigerian": "Nigeria",
1256:     "pakistani": "Pakistan",
1257:     "ukrainian": "Ukraine",
1258:     "israeli": "Israel",
1259:     "iraqi": "Iraq",
1260:     "egyptian": "Egypt",
1261:     "yemeni": "Yemen",
1262:     "brazilian": "Brazil",
1263:     "nicaraguan": "Nicaragua",
1264:     "ecuadorian": "Ecuador",
1265:     "peruvian": "Peru",
1266:     "jamaican": "Jamaica",
1267:     "filipino": "Philippines",
1268:     "vietnamese": "Vietnam",
1269:     "lebanese": "Lebanon",
1270:     "libyan": "Libya",
1271:     "sudanese": "Sudan",
1272:     "ethiopian": "Ethiopia",
1273:     "kenyan": "Kenya",
1274:     "moroccan": "Morocco",
1275:     "algerian": "Algeria",
1276:     "bangladeshi": "Bangladesh",
1277:     "cambodian": "Cambodia",
1278:     "indonesian": "Indonesia",
1279:     "dominican": "Dominican Republic",
1280:     "panamanian": "Panama",
1281:     "bolivian": "Bolivia",
1282:     "chilean": "Chile",
1283:     "argentine": "Argentina",
1284:     "argentinian": "Argentina",
1285:     "saudi": "Saudi Arabia",
1286:     "turkish": "Turkey",
1287: }
1288: FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
1289:     "tehran": "Iran",
1290:     "caracas": "Venezuela",
1291:     "beijing": "China",
1292:     "moscow": "Russia",
1293:     "havana": "Cuba",
1294:     "kabul": "Afghanistan",
1295:     "damascus": "Syria",
1296:     "baghdad": "Iraq",
1297:     "tripoli": "Libya",
1298:     "mogadishu": "Somalia",

```

```

1299:     "bogota": "Colombia",
1300:     "managua": "Nicaragua",
1301:     "tegucigalpa": "Honduras",
1302:     "kyiv": "Ukraine",
1303: }
1304:
1305:
1306: def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:
1307:     alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
1308:     return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)
1309:
1310:
1311: PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))
1312: PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
1313: # A demonym that immediately precedes one of these nouns describes a person's
1314: # nationality, so it also earns origin:<country>, not just a contextual country:.
1315: PATTERN_DEMONYM_PERSON = _compile(
1316:     r"^\W*(?:national|nationals|citizen|citizens|immigrant|immigrants|migrant|migrants|"
1677:     # origin:<COUNTRY> – validated against the sovereign-state vocab.
1678:     for m in PATTERN_ORIGIN_CANDIDATE.finditer(text):
1679:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1680:         if resolved:
1681:             add(f"origin:{_normalize_country(resolved)}", span=m.span(1))
1682:
1683:     # country:<NAME> – broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:
1687:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:     # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:     # country, which the bare-name validators above can't see. A demonym
1691:     # followed by a person noun also earns origin:.
1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", ".")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", ".")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

origin:Canada (2 (0.0%))

Precise plain-English rule

Emitted as origin:Canada when the text has from Canada, with a comma after the country, or a demonym for Canada followed immediately by a person-origin noun.

Code text for this tag

```

79:     "Canada",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the *next* place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!(?:{_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <country>," – anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"b(?:{_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"b({_PLACE})\s+({_PLACE})\b")

1238: DEMONYM_TO_COUNTRY: dict[str, str] = {
1239:     "iranian": "Iran",
1240:     "mexican": "Mexico",
1241:     "venezuelan": "Venezuela",
1242:     "chinese": "China",
1243:     "russian": "Russia",
1244:     "cuban": "Cuba",
1245:     "colombian": "Colombia",
1246:     "salvadoran": "El Salvador",
1247:     "salvadorian": "El Salvador",
1248:     "honduran": "Honduras",
1249:     "guatemalan": "Guatemala",
1250:     "haitian": "Haiti",
1251:     "somali": "Somalia",
1252:     "somalian": "Somalia",
1253:     "afghan": "Afghanistan",
1254:     "syrian": "Syria",
1255:     "nigerian": "Nigeria",

```

```

1256: "pakistani": "Pakistan",
1257: "ukrainian": "Ukraine",
1258: "israeli": "Israel",
1259: "iraqi": "Iraq",
1260: "egyptian": "Egypt",
1261: "yemeni": "Yemen",
1262: "brazilian": "Brazil",
1263: "nicaraguan": "Nicaragua",
1264: "ecuadorian": "Ecuador",
1265: "peruvian": "Peru",
1266: "jamaican": "Jamaica",
1267: "filipino": "Philippines",
1268: "vietnamese": "Vietnam",
1269: "lebanese": "Lebanon",
1270: "libyan": "Libya",
1271: "sudanese": "Sudan",
1272: "ethiopian": "Ethiopia",
1273: "kenyan": "Kenya",
1274: "moroccan": "Morocco",
1275: "algerian": "Algeria",
1276: "bangladeshi": "Bangladesh",
1277: "cambodian": "Cambodia",
1278: "indonesian": "Indonesia",
1279: "dominican": "Dominican Republic",
1280: "panamanian": "Panama",
1281: "bolivian": "Bolivia",
1282: "chilean": "Chile",
1283: "argentine": "Argentina",
1284: "argentinian": "Argentina",
1285: "saudi": "Saudi Arabia",
1286: "turkish": "Turkey",
1287: }
1288: FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
1289:     "tehran": "Iran",
1290:     "caracas": "Venezuela",
1291:     "beijing": "China",
1292:     "moscow": "Russia",
1293:     "havana": "Cuba",
1294:     "kabul": "Afghanistan",
1295:     "damascus": "Syria",
1296:     "baghdad": "Iraq",
1297:     "tripoli": "Libya",
1298:     "mogadishu": "Somalia",
1299:     "bogota": "Colombia",
1300:     "managua": "Nicaragua",
1301:     "tegucigalpa": "Honduras",
1302:     "kyiv": "Ukraine",
1303: }
1304:
1305:
1306: def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:
1307:     alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
1308:     return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)
1309:
1310:
1311: PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))
1312: PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
1313: # A demonym that immediately precedes one of these nouns describes a person's
1314: # nationality, so it also earns origin:<country>, not just a contextual country:.
1315: PATTERN_DEMONYM_PERSON = _compile(
1316:     r"^\w*(?:national|nationals|citizen|citizens|immigrant|immigrants|migrant|migrants|"
1677:     # origin:<COUNTRY> – validated against the sovereign-state vocab.
1678:     for m in PATTERN_ORIGIN_CANDIDATE.finditer(text):
1679:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1680:         if resolved:
1681:             add(f"origin:{_normalize_country(resolved)}", span=m.span(1))
1682:
1683:     # country:<NAME> – broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:
1687:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:     # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:     # country, which the bare-name validators above can't see. A demonym
1691:     # followed by a person noun also earns origin:.
1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

origin:Chile (2 (0.0%))

Precise plain-English rule

Emitted as origin:Chile when the text has from Chile, with a comma after the country, or a demonym for Chile followed immediately by a person-origin noun.

Code text for this tag

```
81:      "Chile",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). `_resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{:_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE})\s+({_PLACE})\b")

1238: DEMONYM_TO_COUNTRY: dict[str, str] = {
1239:     "iranian": "Iran",
1240:     "mexican": "Mexico",
1241:     "venezuelan": "Venezuela",
1242:     "chinese": "China",
1243:     "russian": "Russia",
1244:     "cuban": "Cuba",
1245:     "colombian": "Colombia",
1246:     "salvadoran": "El Salvador",
1247:     "salvadorian": "El Salvador",
1248:     "honduran": "Honduras",
1249:     "guatemalan": "Guatemala",
1250:     "haitian": "Haiti",
1251:     "somali": "Somalia",
1252:     "somalian": "Somalia",
1253:     "afghan": "Afghanistan",
1254:     "syrian": "Syria",
1255:     "nigerian": "Nigeria",
1256:     "pakistani": "Pakistan",
1257:     "ukrainian": "Ukraine",
1258:     "israeli": "Israel",
1259:     "iraqi": "Iraq",
1260:     "egyptian": "Egypt",
1261:     "yemeni": "Yemen",
1262:     "brazilian": "Brazil",
1263:     "nicaraguan": "Nicaragua",
1264:     "ecuadorian": "Ecuador",
1265:     "peruvian": "Peru",
1266:     "jamaican": "Jamaica",
1267:     "filipino": "Philippines",
1268:     "vietnamese": "Vietnam",
1269:     "lebanese": "Lebanon",
1270:     "libyan": "Libya",
1271:     "sudanese": "Sudan",
1272:     "ethiopian": "Ethiopia",
1273:     "kenyan": "Kenya",
1274:     "moroccan": "Morocco",
1275:     "algerian": "Algeria",
1276:     "bangladeshi": "Bangladesh",
1277:     "cambodian": "Cambodia",
1278:     "indonesian": "Indonesia",
1279:     "dominican": "Dominican Republic",
1280:     "panamanian": "Panama",
1281:     "bolivian": "Bolivia",
1282:     "chilean": "Chile",
1283:     "argentine": "Argentina",
1284:     "argentinian": "Argentina",
1285:     "saudi": "Saudi Arabia",
1286:     "turkish": "Turkey",
1287: }
1288: FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
1289:     "tehran": "Iran",
1290:     "caracas": "Venezuela",
1291:     "beijing": "China",
1292:     "moscow": "Russia",
1293:     "havana": "Cuba",
1294:     "kabul": "Afghanistan",
1295:     "damascus": "Syria",
1296:     "baghdad": "Iraq",
1297:     "tripoli": "Libya",
1298:     "mogadishu": "Somalia",
1299:     "bogota": "Colombia",
1300:     "managua": "Nicaragua",
1301:     "tegucigalpa": "Honduras",
1302:     "kyiv": "Ukraine",
1303: }
1304:
1305:
1306: def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:
1307:     alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
1308:     return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)
1309:
1310:
1311: PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))
1312: PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
1313: # A demonym that immediately precedes one of these nouns describes a person's
1314: # nationality, so it also earns origin:<country>, not just a contextual country:.
1315: PATTERN_DEMONYM_PERSON = _compile(
1316:     r"^\W(?:national|nationals|citizen|citizens|immigrant|immigrants|migrant|migrants|"
1317:
1318:
1319:
1320:
1321:
1322:
1323:
1324:
1325:
1326:
1327:
1328:
1329:
1330:
1331:
1332:
1333:
1334:
1335:
1336:
1337:
1338:
1339:
1340:
1341:
1342:
1343:
1344:
1345:
1346:
1347:
1348:
1349:
1350:
1351:
1352:
1353:
1354:
1355:
1356:
1357:
1358:
1359:
1360:
1361:
1362:
1363:
1364:
1365:
1366:
1367:
1368:
1369:
1370:
1371:
1372:
1373:
1374:
1375:
1376:
1377:
1378:
1379:
1380:
1381:
1382:
1383:
1384:
1385:
1386:
1387:
1388:
1389:
1390:
1391:
1392:
1393:
1394:
1395:
1396:
1397:
1398:
1399:
1400:
1401:
1402:
1403:
1404:
1405:
1406:
1407:
1408:
1409:
1410:
1411:
1412:
1413:
1414:
1415:
1416:
1417:
1418:
1419:
1420:
1421:
1422:
1423:
1424:
1425:
1426:
1427:
1428:
1429:
1430:
1431:
1432:
1433:
1434:
1435:
1436:
1437:
1438:
1439:
1440:
1441:
1442:
1443:
1444:
1445:
1446:
1447:
1448:
1449:
1450:
1451:
1452:
1453:
1454:
1455:
1456:
1457:
1458:
1459:
1460:
1461:
1462:
1463:
1464:
1465:
1466:
1467:
1468:
1469:
1470:
1471:
1472:
1473:
1474:
1475:
1476:
1477:
1478:
1479:
1480:
1481:
1482:
1483:
1484:
1485:
1486:
1487:
1488:
1489:
1490:
1491:
1492:
1493:
1494:
1495:
1496:
1497:
1498:
1499:
1500:
1501:
1502:
1503:
1504:
1505:
1506:
1507:
1508:
1509:
1510:
1511:
1512:
1513:
1514:
1515:
1516:
1517:
1518:
1519:
1520:
1521:
1522:
1523:
1524:
1525:
1526:
1527:
1528:
1529:
1530:
1531:
1532:
1533:
1534:
1535:
1536:
1537:
1538:
1539:
1540:
1541:
1542:
1543:
1544:
1545:
1546:
1547:
1548:
1549:
1550:
1551:
1552:
1553:
1554:
1555:
1556:
1557:
1558:
1559:
1560:
1561:
1562:
1563:
1564:
1565:
1566:
1567:
1568:
1569:
1570:
1571:
1572:
1573:
1574:
1575:
1576:
1577:
1578:
1579:
1580:
1581:
1582:
1583:
1584:
1585:
1586:
1587:
1588:
1589:
1590:
1591:
1592:
1593:
1594:
1595:
1596:
1597:
1598:
1599:
1600:
1601:
1602:
1603:
1604:
1605:
1606:
1607:
1608:
1609:
1610:
1611:
1612:
1613:
1614:
1615:
1616:
1617:
1618:
1619:
1620:
1621:
1622:
1623:
1624:
1625:
1626:
1627:
1628:
1629:
1630:
1631:
1632:
1633:
1634:
1635:
1636:
1637:
1638:
1639:
1640:
1641:
1642:
1643:
1644:
1645:
1646:
1647:
1648:
1649:
1650:
1651:
1652:
1653:
1654:
1655:
1656:
1657:
1658:
1659:
1660:
1661:
1662:
1663:
1664:
1665:
1666:
1667:
1668:
1669:
1670:
1671:
1672:
1673:
1674:
1675:
1676:
1677:
1678:
1679:
1680:
1681:
1682:
1683:
1684:
1685:
1686:
1687:
1688:
1689:
1690:
1691:
1692:
1693:
1694:
1695:
1696:
1697:
1698:
1699:
1700:
1701:
1702:
1703:
1704:
1705:
1706:
1707:
1708:
1709:
1710:
1711:
1712:
1713:
1714:
1715:
1716:
1717:
1718:
1719:
1720:
1721:
1722:
1723:
1724:
1725:
1726:
1727:
1728:
1729:
1730:
1731:
1732:
1733:
1734:
1735:
1736:
1737:
1738:
1739:
1740:
1741:
1742:
1743:
1744:
1745:
1746:
1747:
1748:
1749:
1750:
1751:
1752:
1753:
1754:
1755:
1756:
1757:
1758:
1759:
1760:
1761:
1762:
1763:
1764:
1765:
1766:
1767:
1768:
1769:
1770:
1771:
1772:
1773:
1774:
1775:
1776:
1777:
1778:
1779:
1780:
1781:
1782:
1783:
1784:
1785:
1786:
1787:
1788:
1789:
1790:
1791:
1792:
1793:
1794:
1795:
1796:
1797:
1798:
1799:
1800:
1801:
1802:
1803:
1804:
1805:
1806:
1807:
1808:
1809:
1810:
1811:
1812:
1813:
1814:
1815:
1816:
1817:
1818:
1819:
1820:
1821:
1822:
1823:
1824:
1825:
1826:
1827:
1828:
1829:
1830:
1831:
1832:
1833:
1834:
1835:
1836:
1837:
1838:
1839:
1840:
1841:
1842:
1843:
1844:
1845:
1846:
1847:
1848:
1849:
1850:
1851:
1852:
1853:
1854:
1855:
1856:
1857:
1858:
1859:
1860:
1861:
1862:
1863:
1864:
1865:
1866:
1867:
1868:
1869:
1870:
1871:
1872:
1873:
1874:
1875:
1876:
1877:
1878:
1879:
1880:
1881:
1882:
1883:
1884:
1885:
1886:
1887:
1888:
1889:
1890:
1891:
1892:
1893:
1894:
1895:
1896:
1897:
1898:
1899:
1900:
1901:
1902:
1903:
1904:
1905:
1906:
1907:
1908:
1909:
1910:
1911:
1912:
1913:
1914:
1915:
1916:
1917:
1918:
1919:
1920:
1921:
1922:
1923:
1924:
1925:
1926:
1927:
1928:
1929:
1930:
1931:
1932:
1933:
1934:
1935:
1936:
1937:
1938:
1939:
1940:
1941:
1942:
1943:
1944:
1945:
1946:
1947:
1948:
1949:
1950:
1951:
1952:
1953:
1954:
1955:
1956:
1957:
1958:
1959:
1960:
1961:
1962:
1963:
1964:
1965:
1966:
1967:
1968:
1969:
1970:
1971:
1972:
1973:
1974:
1975:
1976:
1977:
1978:
1979:
1980:
1981:
1982:
1983:
1984:
1985:
1986:
1987:
1988:
1989:
1990:
1991:
1992:
1993:
1994:
1995:
1996:
1997:
1998:
1999:
2000:
2001:
2002:
2003:
2004:
2005:
2006:
2007:
2008:
2009:
2010:
2011:
2012:
2013:
2014:
2015:
2016:
2017:
2018:
2019:
2020:
2021:
2022:
2023:
2024:
2025:
2026:
2027:
2028:
2029:
2030:
2031:
2032:
2033:
2034:
2035:
2036:
2037:
2038:
2039:
2040:
2041:
2042:
2043:
2044:
2045:
2046:
2047:
2048:
2049:
2050:
2051:
2052:
2053:
2054:
2055:
2056:
2057:
2058:
2059:
2060:
2061:
2062:
2063:
2064:
2065:
2066:
2067:
2068:
2069:
2070:
2071:
2072:
2073:
2074:
2075:
2076:
2077:
2078:
2079:
2080:
2081:
2082:
2083:
2084:
2085:
2086:
2087:
2088:
2089:
2090:
2091:
2092:
2093:
2094:
2095:
2096:
2097:
2098:
2099:
2100:
2101:
2102:
2103:
2104:
2105:
2106:
2107:
2108:
2109:
2110:
2111:
2112:
2113:
2114:
2115:
2116:
2117:
2118:
2119:
2120:
2121:
2122:
2123:
2124:
2125:
2126:
2127:
2128:
2129:
2130:
2131:
2132:
2133:
2134:
2135:
2136:
2137:
2138:
2139:
2140:
2141:
2142:
2143:
2144:
2145:
2146:
2147:
2148:
2149:
2150:
2151:
2152:
2153:
2154:
2155:
2156:
2157:
2158:
2159:
2160:
2161:
2162:
2163:
2164:
2165:
2166:
2167:
2168:
2169:
2170:
2171:
2172:
2173:
2174:
2175:
2176:
2177:
2178:
2179:
2180:
2181:
2182:
2183:
2184:
2185:
2186:
2187:
2188:
2189:
2190:
2191:
2192:
2193:
2194:
2195:
2196:
2197:
2198:
2199:
2200:
2201:
2202:
2203:
2204:
2205:
2206:
2207:
2208:
2209:
2210:
2211:
2212:
2213:
2214:
2215:
2216:
2217:
2218:
2219:
2220:
2221:
2222:
2223:
2224:
2225:
2226:
2227:
2228:
2229:
2230:
2231:
2232:
2233:
2234:
2235:
2236:
2237:
2238:
2239:
2240:
2241:
2242:
2243:
2244:
2245:
2246:
2247:
2248:
2249:
2250:
2251:
2252:
2253:
2254:
2255:
2256:
2257:
2258:
2259:
2260:
2261:
2262:
2263:
2264:
2265:
2266:
2267:
2268:
2269:
2270:
2271:
2272:
2273:
2274:
2275:
2276:
2277:
2278:
2279:
2280:
2281:
2282:
2283:
2284:
2285:
2286:
2287:
2288:
2289:
2290:
2291:
2292:
2293:
2294:
2295:
2296:
2297:
2298:
2299:
2300:
2301:
2302:
2303:
2304:
2305:
2306:
2307:
2308:
2309:
2310:
2311:
2312:
2313:
2314:
2315:
2316:
2317:
2318:
2319:
2320:
2321:
2322:
2323:
2324:
2325:
2326:
2327:
2328:
2329:
2330:
2331:
2332:
2333:
2334:
2335:
2336:
2337:
2338:
2339:
2340:
2341:
2342:
2343:
2344:
2345:
2346:
2347:
2348:
2349:
2350:
2351:
2352:
2353:
2354:
2355:
2356:
2357:
2358:
2359:
2360:
2361:
2362:
2363:
2364:
2365:
2366:
2367:
2368:
2369:
2370:
2371:
2372:
2373:
2374:
2375:
2376:
2377:
2378:
2379:
2380:
2381:
2382:
2383:
2384:
2385:
2386:
2387:
2388:
2389:
2390:
2391:
2392:
2393:
2394:
2395:
2396:
2397:
2398:
2399:
2400:
2401:
2402:
2403:
2404:
2405:
2406:
2407:
2408:
2409:
2410:
2411:
2412:
2413:
2414:
2415:
2416:
2417:
2418:
2419:
2420:
2421:
2422:
2423:
2424:
2425:
2426:
2427:
2428:
2429:
2430:
2431:
2432:
2433:
2434:
2435:
2436:
2437:
2438:
2439:
2440:
2441:
2442:
2443:
2444:
2445:
2446:
2447:
2448:
2449:
2450:
2451:
2452:
2453:
2454:
2455:
2456:
2457:
2458:
2459:
2460:
2461:
2462:
2463:
2464:
2465:
2466:
2467:
2468:
2469:
2470:
2471:
2472:
2473:
2474:
2475:
2476:
2477:
2478:
2479:
2480:
2481:
2482:
2483:
2484:
2485:
2486:
2487:
2488:
2489:
2490:
2491:
2492:
2493:
2494:
2495:
2496:
2497:
2498:
2499:
2500:
2501:
2502:
2503:
2504:
2505:
2506:
2507:
2508:
2509:
2510:
2511:
2512:
2513:
2514:
2515:
2516:
2517:
2518:
2519:
2520:
2521:
2522:
2523:
2524:
2525:
2526:
2527:
2528:
2529:
2530:
2531:
2532:
2533:
2534:
2535:
2536:
2537:
2538:
2539:
2540:
2541:
2542:
2543:
2544:
2545:
2546:
2547:
2548:
2549:
2550:
2551:
2552:
2553:
2554:
2555:
2556:
2557:
2558:
2559:
2560:
2561:
2562:
2563:
2564:
2565:
2566:
2567:
2568:
2569:
2570:
2571:
2572:
2573:
2574:
2575:
2576:
2577:
2578:
2579:
2580:
2581:
2582:
2583:
2584:
2585:
2586:
2587:
2588:
2589:
2590:
2591:
2592:
2593:
2594:
2595:
2596:
2597:
2598:
2599:
2600:
2601:
2602:
2603:
2604:
2605:
2606:
2607:
2608:
2609:
2610:
2611:
2612:
2613:
2614:
2615:
2616:
2617:
2618:
2619:
2620:
2621:
2622:
2623:
2624:
2625:
2626:
2627:
2628:
2629:
2630:
2631:
2632:
2633:
2634:
2635:
2636:
2637:
2638:
2639:
2640:
2641:
2642:
2643:
2644:
2645:
2646:
2647:
2648:
2649:
2650:
2651:
2652:
2653:
2654:
2655:
2656:
2657:
2658:
2659:
2660:
2661:
2662:
2663:
2664:
2665:
2666:
2667:
2668:
2669:
2670:
2671:
2672:
2673:
2674:
2675:
2676:
2677:
2678:
2679:
2680:
2681:
2682:
2683:
2684:
2685:
2686:
2687:
2688:
2689:
2690:
2691:
2692:
2693:
2694:
2695:
2696:
2697:
2698:
2699:
2700:
2701:
2702:
2703:
2704:
2705:
2706:
2707:
2708:
2709:
2710:
2711:
2712:
2713:
2714:
2715:
2716:
2717:
2718:
2719:
2720:
2721:
2722:
2723:
2724:
2725:
2726:
2727:
2728:
2729:
2730:
2731:
2732:
2733:
2734:
2735:
2736:
2737:
2738:
2739:
2740:
2741:
2742:
2743:
2744:
2745:
2746:
2747:
2748:
2749:
2750:
2751:
2752:
2753:
2754:
2755:
2756:
2757:
2758:
2759:
2760:
2761:
2762:
2763:
2764:
2765:
2766:
2767:
2768:
2769:
2770:
2771:
2772:
2773:
2774:
2775:
2776:
2777:
2778:
2779:
2780:
2781:
2782:
2783:
2784:
2785:
2786:
2787:
2788:
2789:
2790:
2791:
2792:
2793:
2794:
2795:
2796:
2797:
2798:
2799:
2800:
2801:
2802:
2803:
2804:
2805:
2806:
2807:
2808:
2809:
2810:
2811:
2812:
2813:
2814:
2815:
2816:
2817:
2818:
2819:
2820:
2821:
2822:
2823:
2824:
2825:
2826:
2827:
2828:
2829:
2830:
2831:
2832:
2833:
2834:
2835:
2836:
2837:
2838:
2839:
2840:
2841:
2842:
2843:
2844:
2845:
2846:
2847:
2848:
2849:
2850:
2851:
2852:
2853:
2854:
2855:
2856:
2857:
2858:
2859:
2860:
2861:
2862:
2863:
2864:
2865:
2866:
2867:
2868:
2869:
2870:
2871:
2872:
2873:
2874:
2875:
2876:
2877:
2878:
2879:
2880:
2881:
2882:
2883:
2884:
2885:
2886:
2887:
2888:
2889:
2890:
2891:
2892:
2893:
2894:
2895:
2896:
2897:
2898:
2899:
2900:
2901:
2902:
2903:
2904:
2905:
2906:
2907:
2908:
2909:
2910:
2911:
2912:
2913:
2914:
2915:
2916:
2917:
2918:
2919:
2920:
2921:
2922:
2923:
2924:
2925:
2926:
2927:
2928:
2929:
2930:
2931:
2932:
2933:
2934:
2935:
2936:
2937:
2938:
2939:
2940:
2941:
2942:
2943:
2944:
2945:
2946:
2947:
2948:
2949:
2950:
2951:
2952:
2953:
2954:
2955:
2956:
2957:
2958:
2959:
2960:
2961:
2962:
2963:
2964:
2965:
2966:
2967:
2968:
2969:
2970:
2971:
2972:
2973:
2974:
2975:
2976:
2977:
2978:
2979:
2980:
2981:
2982:
2983:
2984:
2985:
2986:
2987:
2988:
2989:
2990:
2991:
2992:
2993:
2994:
2995:
2996:
2997:
2998:
2999:
3000:
3001:
3002:
3003:
3004:
3005:
3006:
3007:
3008:
3009:
3010:
3011:
3012:
3013:
3014:
3015:
3016:
3017:
3018:
3019:
3020:
3021:
3022:
3023:
3024:
3025:
3026:
3027:
3028:
3029:
3030:
3031:
3032:
3033:
3034:
3035:
3036:
3037:
3038:
3039:
3040:
3041:
3042:
3043:
3044:
3045:
3046:
3047:
3048:
3049:
3050:
3051:
3052:
3053:
3054:
3055:
3056:
3057:
3058:
3059:
3060:
3061:
3062:
3063:
3064:
3065:
3066:
3067:
3068:
3069:
3070:
3071:
3072:
3073:
3074:
3075:
3076:
3077:
3078:
3079:
3
```

```

1682:
1683: # country:<NAME> – broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:
2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
2052: if low == "burma":
2053:     return "Myanmar"
2054: return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

origin:Egypt (2 (0.0%))

Precise plain-English rule

Emitted as origin:Egypt when the text has from Egypt, with a comma after the country, or a demonym for Egypt followed immediately by a person-origin noun.

Code text for this tag

```

97:     "Egypt",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]+)?"
1225: # "from <Country>," – anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{[_PREPOSITIONS]})\s+({_PLACE})\b")
1229: # "<Place>," <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b[_PLACE],\s+({_PLACE})\b")

1238: DEMONYM_TO_COUNTRY: dict[str, str] = {
1239:     "iranian": "Iran",
1240:     "mexican": "Mexico",
1241:     "venezuelan": "Venezuela",
1242:     "chinese": "China",
1243:     "russian": "Russia",
1244:     "cuban": "Cuba",
1245:     "colombian": "Colombia",
1246:     "salvadoran": "El Salvador",
1247:     "salvadorian": "El Salvador",
1248:     "honduran": "Honduras",
1249:     "guatemalan": "Guatemala",
1250:     "haitian": "Haiti",
1251:     "somali": "Somalia",
1252:     "somalian": "Somalia",
1253:     "afghan": "Afghanistan",
1254:     "syrian": "Syria",
1255:     "nigerian": "Nigeria",
1256:     "pakistani": "Pakistan",
1257:     "ukrainian": "Ukraine",
1258:     "israeli": "Israel",
1259:     "iraqi": "Iraq",
1260:     "egyptian": "Egypt",
1261:     "yemeni": "Yemen",
1262:     "brazilian": "Brazil",
1263:     "nicaraguan": "Nicaragua",
1264:     "ecuadorian": "Ecuador",
1265:     "peruvian": "Peru",
1266:     "jamaican": "Jamaica",
1267:     "filipino": "Philippines",
1268:     "vietnamese": "Vietnam",
1269:     "lebanese": "Lebanon",
1270:     "libyan": "Libya",
1271:     "sudanese": "Sudan",
1272:     "ethiopian": "Ethiopia",
1273:     "kenyan": "Kenya",
1274:     "moroccan": "Morocco",
1275:     "algerian": "Algeria",
1276:     "bangladeshi": "Bangladesh",
1277:     "cambodian": "Cambodia",
1278:     "indonesian": "Indonesia",
1279:     "dominican": "Dominican Republic",

```

```

1280: "panamanian": "Panama",
1281: "bolivian": "Bolivia",
1282: "chilean": "Chile",
1283: "argentine": "Argentina",
1284: "argentinian": "Argentina",
1285: "saudi": "Saudi Arabia",
1286: "turkish": "Turkey",
1287: }
1288: FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
1289:     "tehran": "Iran",
1290:     "caracas": "Venezuela",
1291:     "beijing": "China",
1292:     "moscow": "Russia",
1293:     "havana": "Cuba",
1294:     "kabul": "Afghanistan",
1295:     "damascus": "Syria",
1296:     "baghdad": "Iraq",
1297:     "tripoli": "Libya",
1298:     "mogadishu": "Somalia",
1299:     "bogota": "Colombia",
1300:     "managua": "Nicaragua",
1301:     "tegucigalpa": "Honduras",
1302:     "kyiv": "Ukraine",
1303: }
1304:
1305:
1306: def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:
1307:     alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
1308:     return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)
1309:
1310:
1311: PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))
1312: PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
1313: # A demonym that immediately precedes one of these nouns describes a person's
1314: # nationality, so it also earns origin:<country>, not just a contextual country:.
1315: PATTERN_DEMONYM_PERSON = _compile(
1316:     r"^\W*(?:national|nationals|citizen|citizens|immigrant|immigrants|migrant|migrants)"
1677:
1678: # origin:<COUNTRY> – validated against the sovereign-state vocab.
1679: for m in PATTERN_ORIGIN_CANDIDATE.finditer(text):
1680:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1681:     if resolved:
1682:         add(f"origin:{_normalize_country(resolved)}", span=m.span(1))
1683:
1684: # country:<NAME> – broader: any contextual country mention.
1685: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1686:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1687:     if resolved:
1688:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1689:
1690: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1691: # country, which the bare-name validators above can't see. A demonym
1692: # followed by a person noun also earns origin:.
1693: for m in PATTERN_DEMONYM.finditer(text):
1694:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1695:     if country and country.lower() in COUNTRY_LOWER:
1696:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1697:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1698:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1699:
1700: for m in PATTERN_FOREIGN_CITY.finditer(text):
2046:
2047: if low in ("usa", "united states"):
2048:     return "United-States"
2049:
2050: if low == "uk":
2051:     return "United-Kingdom"
2052:
2053: if low == "uae":
2054:     return "United-Arab-Emirates"
2055:
2056: if low == "burma":
2057:     return "Myanmar"
2058:
2059: return s.replace(" ", "-")
2060:
2061: def _normalize_state(s: str) -> str:
2062:     return s.replace(" ", "-")
2063:
2064:
2065: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2066:     """Resolve a candidate (any case, possibly two words) to its canonical
2067:     vocab spelling.
2068:
2069:     Tries the full phrase first, then its leading word, so a greedy
2070:     case-insensitive capture that swallowed a trailing lowercase token

```

origin:Ghana (2 (0.0%))

Precise plain-English rule

Emitted as origin:Ghana when the text has from Ghana, with a comma after the country, or a demonym for Ghana followed immediately by a person-origin noun.

Code text for this tag

```

110:     "Ghana",
1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the *next* place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-Z][a-zA-Z]{0,20}(?:\s+(?!{PREPOSITIONS})\b)[A-Za-z][a-zA-Z]{0,20}?"
1225: # "from <Country>," – anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"^(?i:{PREPOSITIONS})\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"^(?i:{PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"^(?i:{PREPOSITIONS})\s+({_PLACE})\b")

```

```

1238: DEMONYM_TO_COUNTRY: dict[str, str] = {
1239:     "iranian": "Iran",
1240:     "mexican": "Mexico",
1241:     "venezuelan": "Venezuela",
1242:     "chinese": "China",
1243:     "russian": "Russia",
1244:     "cuban": "Cuba",
1245:     "colombian": "Colombia",
1246:     "salvadoran": "El Salvador",
1247:     "salvadorian": "El Salvador",
1248:     "honduran": "Honduras",
1249:     "guatemalan": "Guatemala",
1250:     "haitian": "Haiti",
1251:     "somali": "Somalia",
1252:     "somalian": "Somalia",
1253:     "afghan": "Afghanistan",
1254:     "syrian": "Syria",
1255:     "nigerian": "Nigeria",
1256:     "pakistani": "Pakistan",
1257:     "ukrainian": "Ukraine",
1258:     "israeli": "Israel",
1259:     "iraqi": "Iraq",
1260:     "egyptian": "Egypt",
1261:     "yemeni": "Yemen",
1262:     "brazilian": "Brazil",
1263:     "nicaraguan": "Nicaragua",
1264:     "ecuadorian": "Ecuador",
1265:     "peruvian": "Peru",
1266:     "jamaican": "Jamaica",
1267:     "filipino": "Philippines",
1268:     "vietnamese": "Vietnam",
1269:     "lebanese": "Lebanon",
1270:     "libyan": "Libya",
1271:     "sudanese": "Sudan",
1272:     "ethiopian": "Ethiopia",
1273:     "kenyan": "Kenya",
1274:     "moroccan": "Morocco",
1275:     "algerian": "Algeria",
1276:     "bangladeshi": "Bangladesh",
1277:     "cambodian": "Cambodia",
1278:     "indonesian": "Indonesia",
1279:     "dominican": "Dominican Republic",
1280:     "panamanian": "Panama",
1281:     "bolivian": "Bolivia",
1282:     "chilean": "Chile",
1283:     "argentine": "Argentina",
1284:     "argentinian": "Argentina",
1285:     "saudi": "Saudi Arabia",
1286:     "turkish": "Turkey",
1287: }
1288: FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
1289:     "tehran": "Iran",
1290:     "caracas": "Venezuela",
1291:     "beijing": "China",
1292:     "moscow": "Russia",
1293:     "havana": "Cuba",
1294:     "kabul": "Afghanistan",
1295:     "damascus": "Syria",
1296:     "baghdad": "Iraq",
1297:     "tripoli": "Libya",
1298:     "mogadishu": "Somalia",
1299:     "bogota": "Colombia",
1300:     "managua": "Nicaragua",
1301:     "tegucigalpa": "Honduras",
1302:     "kyiv": "Ukraine",
1303: }
1304:
1305:
1306: def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:
1307:     alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
1308:     return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)
1309:
1310:
1311: PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))
1312: PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
1313: # A demonym that immediately precedes one of these nouns describes a person's
1314: # nationality, so it also earns origin:country, not just a contextual country:.
1315: PATTERN_DEMONYM_PERSON = re.compile(
1316:     r"^\W*(?:national|nationals|citizen|citizens|immigrant|immigrants|migrant|migrants|"
1317:
1318:
1319:
1320:
1321:
1322:
1323:
1324:
1325:
1326:
1327:
1328:
1329:
1330:
1331:
1332:
1333:
1334:
1335:
1336:
1337:
1338:
1339:
1340:
1341:
1342:
1343:
1344:
1345:
1346:
1347:
1348:
1349:
1350:
1351:
1352:
1353:
1354:
1355:
1356:
1357:
1358:
1359:
1360:
1361:
1362:
1363:
1364:
1365:
1366:
1367:
1368:
1369:
1370:
1371:
1372:
1373:
1374:
1375:
1376:
1377:
1378:
1379:
1380:
1381:
1382:
1383:
1384:
1385:
1386:
1387:
1388:
1389:
1390:
1391:
1392:
1393:
1394:
1395:
1396:
1397:
1398:
1399:
1400:
1401:
1402:
1403:
1404:
1405:
1406:
1407:
1408:
1409:
1410:
1411:
1412:
1413:
1414:
1415:
1416:
1417:
1418:
1419:
1420:
1421:
1422:
1423:
1424:
1425:
1426:
1427:
1428:
1429:
1430:
1431:
1432:
1433:
1434:
1435:
1436:
1437:
1438:
1439:
1440:
1441:
1442:
1443:
1444:
1445:
1446:
1447:
1448:
1449:
1450:
1451:
1452:
1453:
1454:
1455:
1456:
1457:
1458:
1459:
1460:
1461:
1462:
1463:
1464:
1465:
1466:
1467:
1468:
1469:
1470:
1471:
1472:
1473:
1474:
1475:
1476:
1477:
1478:
1479:
1480:
1481:
1482:
1483:
1484:
1485:
1486:
1487:
1488:
1489:
1490:
1491:
1492:
1493:
1494:
1495:
1496:
1497:
1498:
1499:
1500:
1501:
1502:
1503:
1504:
1505:
1506:
1507:
1508:
1509:
1510:
1511:
1512:
1513:
1514:
1515:
1516:
1517:
1518:
1519:
1520:
1521:
1522:
1523:
1524:
1525:
1526:
1527:
1528:
1529:
1530:
1531:
1532:
1533:
1534:
1535:
1536:
1537:
1538:
1539:
1540:
1541:
1542:
1543:
1544:
1545:
1546:
1547:
1548:
1549:
1550:
1551:
1552:
1553:
1554:
1555:
1556:
1557:
1558:
1559:
1560:
1561:
1562:
1563:
1564:
1565:
1566:
1567:
1568:
1569:
1570:
1571:
1572:
1573:
1574:
1575:
1576:
1577:
1578:
1579:
1580:
1581:
1582:
1583:
1584:
1585:
1586:
1587:
1588:
1589:
1590:
1591:
1592:
1593:
1594:
1595:
1596:
1597:
1598:
1599:
1600:
1601:
1602:
1603:
1604:
1605:
1606:
1607:
1608:
1609:
1610:
1611:
1612:
1613:
1614:
1615:
1616:
1617:
1618:
1619:
1620:
1621:
1622:
1623:
1624:
1625:
1626:
1627:
1628:
1629:
1630:
1631:
1632:
1633:
1634:
1635:
1636:
1637:
1638:
1639:
1640:
1641:
1642:
1643:
1644:
1645:
1646:
1647:
1648:
1649:
1650:
1651:
1652:
1653:
1654:
1655:
1656:
1657:
1658:
1659:
1660:
1661:
1662:
1663:
1664:
1665:
1666:
1667:
1668:
1669:
1670:
1671:
1672:
1673:
1674:
1675:
1676:
1677:
1678:
1679:
1680:
1681:
1682:
1683:
1684:
1685:
1686:
1687:
1688:
1689:
1690:
1691:
1692:
1693:
1694:
1695:
1696:
1697:
1698:
1699:
1700:
1701:
1702:
1703:
1704:
1705:
1706:
1707:
1708:
1709:
1710:
1711:
1712:
1713:
1714:
1715:
1716:
1717:
1718:
1719:
1720:
1721:
1722:
1723:
1724:
1725:
1726:
1727:
1728:
1729:
1730:
1731:
1732:
1733:
1734:
1735:
1736:
1737:
1738:
1739:
1740:
1741:
1742:
1743:
1744:
1745:
1746:
1747:
1748:
1749:
1750:
1751:
1752:
1753:
1754:
1755:
1756:
1757:
1758:
1759:
1760:
1761:
1762:
1763:
1764:
1765:
1766:
1767:
1768:
1769:
1770:
1771:
1772:
1773:
1774:
1775:
1776:
1777:
1778:
1779:
1780:
1781:
1782:
1783:
1784:
1785:
1786:
1787:
1788:
1789:
1790:
1791:
1792:
1793:
1794:
1795:
1796:
1797:
1798:
1799:
1800:
1801:
1802:
1803:
1804:
1805:
1806:
1807:
1808:
1809:
1810:
1811:
1812:
1813:
1814:
1815:
1816:
1817:
1818:
1819:
1820:
1821:
1822:
1823:
1824:
1825:
1826:
1827:
1828:
1829:
1830:
1831:
1832:
1833:
1834:
1835:
1836:
1837:
1838:
1839:
1840:
1841:
1842:
1843:
1844:
1845:
1846:
1847:
1848:
1849:
1850:
1851:
1852:
1853:
1854:
1855:
1856:
1857:
1858:
1859:
1860:
1861:
1862:
1863:
1864:
1865:
1866:
1867:
1868:
1869:
1870:
1871:
1872:
1873:
1874:
1875:
1876:
1877:
1878:
1879:
1880:
1881:
1882:
1883:
1884:
1885:
1886:
1887:
1888:
1889:
1890:
1891:
1892:
1893:
1894:
1895:
1896:
1897:
1898:
1899:
1900:
1901:
1902:
1903:
1904:
1905:
1906:
1907:
1908:
1909:
1910:
1911:
1912:
1913:
1914:
1915:
1916:
1917:
1918:
1919:
1920:
1921:
1922:
1923:
1924:
1925:
1926:
1927:
1928:
1929:
1930:
1931:
1932:
1933:
1934:
1935:
1936:
1937:
1938:
1939:
1940:
1941:
1942:
1943:
1944:
1945:
1946:
1947:
1948:
1949:
1950:
1951:
1952:
1953:
1954:
1955:
1956:
1957:
1958:
1959:
1960:
1961:
1962:
1963:
1964:
1965:
1966:
1967:
1968:
1969:
1970:
1971:
1972:
1973:
1974:
1975:
1976:
1977:
1978:
1979:
1980:
1981:
1982:
1983:
1984:
1985:
1986:
1987:
1988:
1989:
1990:
1991:
1992:
1993:
1994:
1995:
1996:
1997:
1998:
1999:
2000:
2001:
2002:
2003:
2004:
2005:
2006:
2007:
2008:
2009:
2010:
2011:
2012:
2013:
2014:
2015:
2016:
2017:
2018:
2019:
2020:
2021:
2022:
2023:
2024:
2025:
2026:
2027:
2028:
2029:
2030:
2031:
2032:
2033:
2034:
2035:
2036:
2037:
2038:
2039:
2040:
2041:
2042:
2043:
2044:
2045:
2046:
2047:
2048:
2049:
2050:
2051:
2052:

```

```

2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

origin:Iraq (2 (0.0%))

Precise plain-English rule

Emitted as origin:Iraq when the text has from Iraq, with a comma after the country, or a demonym for Iraq followed immediately by a person-origin noun.

Code text for this tag

```

123:     "Iraq",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-Z][a-zA-Z]*(?:\s+(?!(?:{_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE})\s+({_PLACE})\b")

1238: DEMONYM_TO_COUNTRY: dict[str, str] = {
1239:     "iranian": "Iran",
1240:     "mexican": "Mexico",
1241:     "venezuelan": "Venezuela",
1242:     "chinese": "China",
1243:     "russian": "Russia",
1244:     "cuban": "Cuba",
1245:     "colombian": "Colombia",
1246:     "salvadoran": "El Salvador",
1247:     "salvadorian": "El Salvador",
1248:     "honduran": "Honduras",
1249:     "guatemalan": "Guatemala",
1250:     "haitian": "Haiti",
1251:     "somali": "Somalia",
1252:     "somalian": "Somalia",
1253:     "afghan": "Afghanistan",
1254:     "syrian": "Syria",
1255:     "nigerian": "Nigeria",
1256:     "pakistani": "Pakistan",
1257:     "ukrainian": "Ukraine",
1258:     "israeli": "Israel",
1259:     "iraqi": "Iraq",
1260:     "egyptian": "Egypt",
1261:     "yemeni": "Yemen",
1262:     "brazilian": "Brazil",
1263:     "nicaraguan": "Nicaragua",
1264:     "ecuadorian": "Ecuador",
1265:     "peruvian": "Peru",
1266:     "jamaican": "Jamaica",
1267:     "filipino": "Philippines",
1268:     "vietnamese": "Vietnam",
1269:     "lebanese": "Lebanon",
1270:     "libyan": "Libya",
1271:     "sudanese": "Sudan",
1272:     "ethiopian": "Ethiopia",
1273:     "kenyan": "Kenya",
1274:     "moroccan": "Morocco",
1275:     "algerian": "Algeria",
1276:     "bangladeshi": "Bangladesh",
1277:     "cambodian": "Cambodia",
1278:     "indonesian": "Indonesia",
1279:     "dominican": "Dominican Republic",
1280:     "panamanian": "Panama",
1281:     "bolivian": "Bolivia",
1282:     "chilean": "Chile",
1283:     "argentine": "Argentina",
1284:     "argentinian": "Argentina",
1285:     "saudi": "Saudi Arabia",
1286:     "turkish": "Turkey",
1287: }
1288: FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
1289:     "tehran": "Iran",
1290:     "caracas": "Venezuela",
1291:     "beijing": "China",
1292:     "moscow": "Russia",
1293:     "havana": "Cuba",
1294:     "kabul": "Afghanistan",
1295:     "damascus": "Syria",
1296:     "baghdad": "Iraq",
1297:     "tripoli": "Libya",
1298:     "mogadishu": "Somalia",
1299:     "bogota": "Colombia",
1300:     "managua": "Nicaragua",
1301:     "tegucigalpa": "Honduras",
1302:     "kyiv": "Ukraine",
1303: }
1304:

```

```

1305:
1306: def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:
1307:     alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
1308:     return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)
1309:
1310:
1311: PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))
1312: PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
1313: # A demonym that immediately precedes one of these nouns describes a person's
1314: # nationality, so it also earns origin:<country>, not just a contextual country:.
1315: PATTERN_DEMONYM_PERSON = _compile(
1316:     r"^\w*(?:national|nationals|citizen|citizens|immigrant|immigrants|migrant|migrants|"
1677:     # origin:<COUNTRY> – validated against the sovereign-state vocab.
1678:     for m in PATTERN_ORIGIN_CANDIDATE.finditer(text):
1679:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1680:         if resolved:
1681:             add(f"origin:{_normalize_country(resolved)}", span=m.span(1))
1682:
1683:     # country:<NAME> – broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:
1687:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:     # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:     # country, which the bare-name validators above can't see. A demonym
1691:     # followed by a person noun also earns origin:.
1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056: def _normalize_state(s: str) -> str:
2057:     return s.replace(" ", "-")
2058:
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

origin:Pakistan (2 (0.0%))

Precise plain-English rule

Emitted as origin:Pakistan when the text has from Pakistan, with a comma after the country, or a demonym for Pakistan followed immediately by a person-origin noun.

Code text for this tag

```

172:     "Pakistan",
1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). '_resolve_vocab'
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?:!(?:{_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]+)?"
1225: # "from <Country>" – anchors the ORIGIN validator
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*",)
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:!:_PREPOSITIONS)\s+({_PLACE})\b")
1229: # "<Place>, <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE})\s+({_PLACE})\b")
1238: DEMONYM_TO_COUNTRY: dict[str, str] = {
1239:     "iranian": "Iran",
1240:     "mexican": "Mexico",
1241:     "venezuelan": "Venezuela",
1242:     "chinese": "China",
1243:     "russian": "Russia",
1244:     "cuban": "Cuba",
1245:     "colombian": "Colombia",
1246:     "salvadoran": "El Salvador",
1247:     "salvadorian": "El Salvador",
1248:     "honduran": "Honduras",
1249:     "guatemalan": "Guatemala",
1250:     "haitian": "Haiti",
1251:     "somali": "Somalia",
1252:     "somalian": "Somalia",
1253:     "afghan": "Afghanistan",
1254:     "syrian": "Syria",
1255:     "nigerian": "Nigeria",
1256:     "pakistani": "Pakistan",
1257:     "ukrainian": "Ukraine",
1258:     "israeli": "Israel",
1259:     "iraqi": "Iraq",
1260:     "egyptian": "Egypt",
1261:     "yemeni": "Yemen",

```

```

1262: "brazilian": "Brazil",
1263: "nicaraguan": "Nicaragua",
1264: "ecuadorian": "Ecuador",
1265: "peruvian": "Peru",
1266: "jamaican": "Jamaica",
1267: "filipino": "Philippines",
1268: "vietnamese": "Vietnam",
1269: "lebanese": "Lebanon",
1270: "libyan": "Libya",
1271: "sudanese": "Sudan",
1272: "ethiopian": "Ethiopia",
1273: "kenyan": "Kenya",
1274: "moroccan": "Morocco",
1275: "algerian": "Algeria",
1276: "bangladeshi": "Bangladesh",
1277: "cambodian": "Cambodia",
1278: "indonesian": "Indonesia",
1279: "dominican": "Dominican Republic",
1280: "panamanian": "Panama",
1281: "bolivian": "Bolivia",
1282: "chilean": "Chile",
1283: "argentine": "Argentina",
1284: "argentinian": "Argentina",
1285: "saudi": "Saudi Arabia",
1286: "turkish": "Turkey",
1287: }
1288: FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
1289:     "tehran": "Iran",
1290:     "caracas": "Venezuela",
1291:     "beijing": "China",
1292:     "moscow": "Russia",
1293:     "havana": "Cuba",
1294:     "kabul": "Afghanistan",
1295:     "damascus": "Syria",
1296:     "baghdad": "Iraq",
1297:     "tripoli": "Libya",
1298:     "mogadishu": "Somalia",
1299:     "bogota": "Colombia",
1300:     "managua": "Nicaragua",
1301:     "tegucigalpa": "Honduras",
1302:     "kyiv": "Ukraine",
1303: }
1304:
1305:
1306: def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:
1307:     alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
1308:     return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)
1309:
1310:
1311: PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))
1312: PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
1313: # A demonym that immediately precedes one of these nouns describes a person's
1314: # nationality, so it also earns origin:<country>, not just a contextual country:.
1315: PATTERN_DEMONYM_PERSON = _compile(
1316:     r"^\W*(?:national|nationals|citizen|citizens|immigrant|immigrants|migrant|migrants|"
1317:
1318:
1319:
1320:
1321:
1322:
1323:
1324:
1325:
1326:
1327:
1328:
1329:
1330:
1331:
1332:
1333:
1334:
1335:
1336:
1337:
1338:
1339:
1340:
1341:
1342:
1343:
1344:
1345:
1346:
1347:
1348:
1349:
1350:
1351:
1352:
1353:
1354:
1355:
1356:
1357:
1358:
1359:
1360:
1361:
1362:
1363:
1364:
1365:
1366:
1367:
1368:
1369:
1370:
1371:
1372:
1373:
1374:
1375:
1376:
1377:
1378:
1379:
1380:
1381:
1382:
1383:
1384:
1385:
1386:
1387:
1388:
1389:
1390:
1391:
1392:
1393:
1394:
1395:
1396:
1397:
1398:
1399:
1400:
1401:
1402:
1403:
1404:
1405:
1406:
1407:
1408:
1409:
1410:
1411:
1412:
1413:
1414:
1415:
1416:
1417:
1418:
1419:
1420:
1421:
1422:
1423:
1424:
1425:
1426:
1427:
1428:
1429:
1430:
1431:
1432:
1433:
1434:
1435:
1436:
1437:
1438:
1439:
1440:
1441:
1442:
1443:
1444:
1445:
1446:
1447:
1448:
1449:
1450:
1451:
1452:
1453:
1454:
1455:
1456:
1457:
1458:
1459:
1460:
1461:
1462:
1463:
1464:
1465:
1466:
1467:
1468:
1469:
1470:
1471:
1472:
1473:
1474:
1475:
1476:
1477:
1478:
1479:
1480:
1481:
1482:
1483:
1484:
1485:
1486:
1487:
1488:
1489:
1490:
1491:
1492:
1493:
1494:
1495:
1496:
1497:
1498:
1499:
1500:
1501:
1502:
1503:
1504:
1505:
1506:
1507:
1508:
1509:
1510:
1511:
1512:
1513:
1514:
1515:
1516:
1517:
1518:
1519:
1520:
1521:
1522:
1523:
1524:
1525:
1526:
1527:
1528:
1529:
1530:
1531:
1532:
1533:
1534:
1535:
1536:
1537:
1538:
1539:
1540:
1541:
1542:
1543:
1544:
1545:
1546:
1547:
1548:
1549:
1550:
1551:
1552:
1553:
1554:
1555:
1556:
1557:
1558:
1559:
1560:
1561:
1562:
1563:
1564:
1565:
1566:
1567:
1568:
1569:
1570:
1571:
1572:
1573:
1574:
1575:
1576:
1577:
1578:
1579:
1580:
1581:
1582:
1583:
1584:
1585:
1586:
1587:
1588:
1589:
1590:
1591:
1592:
1593:
1594:
1595:
1596:
1597:
1598:
1599:
1600:
1601:
1602:
1603:
1604:
1605:
1606:
1607:
1608:
1609:
1610:
1611:
1612:
1613:
1614:
1615:
1616:
1617:
1618:
1619:
1620:
1621:
1622:
1623:
1624:
1625:
1626:
1627:
1628:
1629:
1630:
1631:
1632:
1633:
1634:
1635:
1636:
1637:
1638:
1639:
1640:
1641:
1642:
1643:
1644:
1645:
1646:
1647:
1648:
1649:
1650:
1651:
1652:
1653:
1654:
1655:
1656:
1657:
1658:
1659:
1660:
1661:
1662:
1663:
1664:
1665:
1666:
1667:
1668:
1669:
1670:
1671:
1672:
1673:
1674:
1675:
1676:
1677:
1678:
1679:
1680:
1681:
1682:
1683:
1684:
1685:
1686:
1687:
1688:
1689:
1690:
1691:
1692:
1693:
1694:
1695:
1696:
1697:
1698:
1699:
1700:
1701:
1702:
1703:
1704:
1705:
1706:
1707:
1708:
1709:
1710:
1711:
1712:
1713:
1714:
1715:
1716:
1717:
1718:
1719:
1720:
1721:
1722:
1723:
1724:
1725:
1726:
1727:
1728:
1729:
1730:
1731:
1732:
1733:
1734:
1735:
1736:
1737:
1738:
1739:
1740:
1741:
1742:
1743:
1744:
1745:
1746:
1747:
1748:
1749:
1750:
1751:
1752:
1753:
1754:
1755:
1756:
1757:
1758:
1759:
1760:
1761:
1762:
1763:
1764:
1765:
1766:
1767:
1768:
1769:
1770:
1771:
1772:
1773:
1774:
1775:
1776:
1777:
1778:
1779:
1780:
1781:
1782:
1783:
1784:
1785:
1786:
1787:
1788:
1789:
1790:
1791:
1792:
1793:
1794:
1795:
1796:
1797:
1798:
1799:
1800:
1801:
1802:
1803:
1804:
1805:
1806:
1807:
1808:
1809:
1810:
1811:
1812:
1813:
1814:
1815:
1816:
1817:
1818:
1819:
1820:
1821:
1822:
1823:
1824:
1825:
1826:
1827:
1828:
1829:
1830:
1831:
1832:
1833:
1834:
1835:
1836:
1837:
1838:
1839:
1840:
1841:
1842:
1843:
1844:
1845:
1846:
1847:
1848:
1849:
1850:
1851:
1852:
1853:
1854:
1855:
1856:
1857:
1858:
1859:
1860:
1861:
1862:
1863:
1864:
1865:
1866:
1867:
1868:
1869:
1870:
1871:
1872:
1873:
1874:
1875:
1876:
1877:
1878:
1879:
1880:
1881:
1882:
1883:
1884:
1885:
1886:
1887:
1888:
1889:
1890:
1891:
1892:
1893:
1894:
1895:
1896:
1897:
1898:
1899:
1900:
1901:
1902:
1903:
1904:
1905:
1906:
1907:
1908:
1909:
1910:
1911:
1912:
1913:
1914:
1915:
1916:
1917:
1918:
1919:
1920:
1921:
1922:
1923:
1924:
1925:
1926:
1927:
1928:
1929:
1930:
1931:
1932:
1933:
1934:
1935:
1936:
1937:
1938:
1939:
1940:
1941:
1942:
1943:
1944:
1945:
1946:
1947:
1948:
1949:
1950:
1951:
1952:
1953:
1954:
1955:
1956:
1957:
1958:
1959:
1960:
1961:
1962:
1963:
1964:
1965:
1966:
1967:
1968:
1969:
1970:
1971:
1972:
1973:
1974:
1975:
1976:
1977:
1978:
1979:
1980:
1981:
1982:
1983:
1984:
1985:
1986:
1987:
1988:
1989:
1990:
1991:
1992:
1993:
1994:
1995:
1996:
1997:
1998:
1999:
2000:
2001:
2002:
2003:
2004:
2005:
2006:
2007:
2008:
2009:
2010:
2011:
2012:
2013:
2014:
2015:
2016:
2017:
2018:
2019:
2020:
2021:
2022:
2023:
2024:
2025:
2026:
2027:
2028:
2029:
2030:
2031:
2032:
2033:
2034:
2035:
2036:
2037:
2038:
2039:
2040:
2041:
2042:
2043:
2044:
2045:
2046:
2047:
2048:
2049:
2050:
2051:
2052:
2053:
2054:
2055:
2056:
2057:
2058:
2059:
2060:
2061:
2062:
2063:
2064:
2065:
2066:

```

origin:Panama (2 (0.0%))

Precise plain-English rule

Emitted as origin:Panama when the text has from Panama, with a comma after the country, or a demonym for Panama followed immediately by a person-origin noun.

Code text for this tag

```

174:     "Panama",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the 'next' place
1219: # ("From USA to Mexico" -> "USA" + "Mexico", not "USA to"). `resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!{:_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]*(?)"
1225: # "from <Country>" - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*",)
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{:_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>, <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1238: DEMONYM_TO_COUNTRY: dict[str, str] = {
1239:     "iranian": "Iran",
1240:     "mexican": "Mexico",
1241:     "venezuelan": "Venezuela",
1242:     "chinese": "China",
1243:     "russian": "Russia",
1244:     "cuban": "Cuba",
1245:     "colombian": "Colombia",
1246:     "salvadoran": "El Salvador",
1247:     "salvadorian": "El Salvador",
1248:     "honduran": "Honduras",
1249:     "guatemalan": "Guatemala",
1250:     "haitian": "Haiti",
1251:     "somali": "Somalia",
1252:     "somalian": "Somalia",
1253:     "afghan": "Afghanistan",
1254:     "syrian": "Syria",
1255:     "nigerian": "Nigeria",
1256:     "pakistani": "Pakistan",
1257:     "ukrainian": "Ukraine",
1258:     "israeli": "Israel",
1259:     "iraqi": "Iraq",
1260:     "egyptian": "Egypt",
1261:     "yemeni": "Yemen",
1262:     "brazilian": "Brazil",
1263:     "nicaraguan": "Nicaragua",
1264:     "ecuadorian": "Ecuador",
1265:     "peruvian": "Peru",
1266:     "jamaican": "Jamaica",
1267:     "filipino": "Philippines",
1268:     "vietnamese": "Vietnam",
1269:     "lebanese": "Lebanon",
1270:     "libyan": "Libya",
1271:     "sudanese": "Sudan",
1272:     "ethiopian": "Ethiopia",
1273:     "kenyan": "Kenya",
1274:     "moroccan": "Morocco",
1275:     "algerian": "Algeria",
1276:     "bangladeshi": "Bangladesh",
1277:     "cambodian": "Cambodia",
1278:     "indonesian": "Indonesia",
1279:     "dominican": "Dominican Republic",
1280:     "panamanian": "Panama",
1281:     "bolivian": "Bolivia",
1282:     "chilean": "Chile",
1283:     "argentine": "Argentina",
1284:     "argentinian": "Argentina",
1285:     "saudi": "Saudi Arabia",
1286:     "turkish": "Turkey",
1287: }
1288: FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
1289:     "tehran": "Iran",
1290:     "caracas": "Venezuela",
1291:     "beijing": "China",
1292:     "moscow": "Russia",
1293:     "havana": "Cuba",
1294:     "kabul": "Afghanistan",
1295:     "damascus": "Syria",
1296:     "baghdad": "Iraq",
1297:     "tripoli": "Libya",
1298:     "mogadishu": "Somalia",
1299:     "bogota": "Colombia",
1300:     "managua": "Nicaragua",
1301:     "tegucigalpa": "Honduras",
1302:     "kyiv": "Ukraine",
1303: }
1304:
1305:
1306: def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:
1307:     alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
1308:     return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)
1309:
1310:
1311: PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))
1312: PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
1313: # A demonym that immediately precedes one of these nouns describes a person's
1314: # nationality, so it also earns origin:<country>, not just a contextual country:.
1315: PATTERN_DEMONYM_PERSON = _compile(
1316:     r"^\W*(?:national|nationals|citizen|citizens|immigrant|immigrants|migrant|migrants|"
1677: # origin:<COUNTRY> - validated against the sovereign-state vocab.
1678: for m in PATTERN_ORIGIN_CANDIDATE.finditer(text):
1679:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1680:     if resolved:
1681:         add(f"origin:{_normalize_country(resolved)}", span=m.span(1))
1682:
1683: # country:<NAME> - broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:

```

```

1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):

2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
2052: if low == "burma":
2053:     return "Myanmar"
2054: return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

origin:Philippines (2 (0.0%))

Precise plain-English rule

Emitted as origin:Philippines when the text has from Philippines, with a comma after the country, or a demonym for Philippines followed immediately by a person-origin noun.

Code text for this tag

```

177:     "Philippines",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]+)?"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*", "")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{[_PREPOSITIONS]})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE})\s+({_PLACE})\b")

1238: DEMONYM_TO_COUNTRY: dict[str, str] = {
1239:     "iranian": "Iran",
1240:     "mexican": "Mexico",
1241:     "venezuelan": "Venezuela",
1242:     "chinese": "China",
1243:     "russian": "Russia",
1244:     "cuban": "Cuba",
1245:     "colombian": "Colombia",
1246:     "salvadoran": "El Salvador",
1247:     "salvadorian": "El Salvador",
1248:     "honduran": "Honduras",
1249:     "guatemalan": "Guatemala",
1250:     "haitian": "Haiti",
1251:     "somali": "Somalia",
1252:     "somalian": "Somalia",
1253:     "afghan": "Afghanistan",
1254:     "syrian": "Syria",
1255:     "nigerian": "Nigeria",
1256:     "pakistani": "Pakistan",
1257:     "ukrainian": "Ukraine",
1258:     "israeli": "Israel",
1259:     "iraqi": "Iraq",
1260:     "egyptian": "Egypt",
1261:     "yemeni": "Yemen",
1262:     "brazilian": "Brazil",
1263:     "nicaraguan": "Nicaragua",
1264:     "ecuadorian": "Ecuador",
1265:     "peruvian": "Peru",
1266:     "jamaican": "Jamaica",
1267:     "filipino": "Philippines",
1268:     "vietnamese": "Vietnam",
1269:     "lebanese": "Lebanon",
1270:     "libyan": "Libya",
1271:     "sudanese": "Sudan",
1272:     "ethiopian": "Ethiopia",
1273:     "kenyan": "Kenya",
1274:     "moroccan": "Morocco",
1275:     "algerian": "Algeria",
1276:     "bangladeshi": "Bangladesh",
1277:     "cambodian": "Cambodia",
1278:     "indonesian": "Indonesia",
1279:     "dominican": "Dominican Republic",
1280:     "panamanian": "Panama",
1281:     "bolivian": "Bolivia",
1282:     "chilean": "Chile",
1283:     "argentine": "Argentina",
1284:     "argentinian": "Argentina",
1285:     "saudi": "Saudi Arabia",
1286:     "turkish": "Turkey",

```

```

1287: }
1288: FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
1289:     "tehran": "Iran",
1290:     "caracas": "Venezuela",
1291:     "beijing": "China",
1292:     "moscow": "Russia",
1293:     "havana": "Cuba",
1294:     "kabul": "Afghanistan",
1295:     "damascus": "Syria",
1296:     "baghdad": "Iraq",
1297:     "tripoli": "Libya",
1298:     "mogadishu": "Somalia",
1299:     "bogota": "Colombia",
1300:     "managua": "Nicaragua",
1301:     "tegucigalpa": "Honduras",
1302:     "kyiv": "Ukraine",
1303: }
1304:
1305:
1306: def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:
1307:     alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
1308:     return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)
1309:
1310:
1311: PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))
1312: PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
1313: # A demonym that immediately precedes one of these nouns describes a person's
1314: # nationality, so it also earns origin:<country>, not just a contextual country:.
1315: PATTERN_DEMONYM_PERSON = _compile(
1316:     r"^\w*(?national|nationals|citizen|citizens|immigrant|immigrants|migrant|migrants)"
1317:
1318:
1319: # origin:<COUNTRY> – validated against the sovereign-state vocab.
1320: for m in PATTERN_ORIGIN_CANDIDATE.finditer(text):
1321:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1322:     if resolved:
1323:         add(f"origin:{_normalize_country(resolved)}", span=m.span(1))
1324:
1325:
1326: # country:<NAME> – broader: any contextual country mention.
1327: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1328:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1329:     if resolved:
1330:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1331:
1332:
1333: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1334: # country, which the bare-name validators above can't see. A demonym
1335: # followed by a person noun also earns origin:.
1336: for m in PATTERN_DEMONYM.finditer(text):
1337:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1338:     if country and country.lower() in COUNTRY_LOWER:
1339:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1340:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1341:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1342:     for m in PATTERN_FOREIGN_CITY.finditer(text):
1343:
1344:
1345: if low in ("usa", "united states"):
1346:     return "United-States"
1347:
1348: if low == "uk":
1349:     return "United-Kingdom"
1350:
1351: if low == "uae":
1352:     return "United-Arab-Emirates"
1353:
1354: if low == "burma":
1355:     return "Myanmar"
1356:
1357: return s.replace(" ", "-")
1358:
1359:
1360: def _normalize_state(s: str) -> str:
1361:     return s.replace(" ", "-")
1362:
1363:
1364: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
1365:     """Resolve a candidate (any case, possibly two words) to its canonical
1366:     vocab spelling.
1367:
1368:     Tries the full phrase first, then its leading word, so a greedy
1369:     case-insensitive capture that swallowed a trailing lowercase token

```

origin:Thailand (2 (0.0%))

Precise plain-English rule

Emitted as origin:Thailand when the text has from Thailand, with a comma after the country, or a demonym for Thailand followed immediately by a person-origin noun.

Code text for this tag

```

206:     "Thailand",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). ` _resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," – anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{ _PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1238: DEMONYM_TO_COUNTRY: dict[str, str] = {
1239:     "iranian": "Iran",
1240:     "mexican": "Mexico",
1241:     "venezuelan": "Venezuela",
1242:     "chinese": "China",
1243:     "russian": "Russia",

```

```

1244: "cuban": "Cuba",
1245: "colombian": "Colombia",
1246: "salvadoran": "El Salvador",
1247: "salvadorian": "El Salvador",
1248: "honduran": "Honduras",
1249: "guatemalan": "Guatemala",
1250: "haitian": "Haiti",
1251: "somali": "Somalia",
1252: "somalian": "Somalia",
1253: "afghan": "Afghanistan",
1254: "syrian": "Syria",
1255: "nigerian": "Nigeria",
1256: "pakistani": "Pakistan",
1257: "ukrainian": "Ukraine",
1258: "israeli": "Israel",
1259: "iraqi": "Iraq",
1260: "egyptian": "Egypt",
1261: "yemeni": "Yemen",
1262: "brazilian": "Brazil",
1263: "nicaraguan": "Nicaragua",
1264: "ecuadorian": "Ecuador",
1265: "peruvian": "Peru",
1266: "jamaican": "Jamaica",
1267: "filipino": "Philippines",
1268: "vietnamese": "Vietnam",
1269: "lebanese": "Lebanon",
1270: "libyan": "Libya",
1271: "sudanese": "Sudan",
1272: "ethiopian": "Ethiopia",
1273: "kenyan": "Kenya",
1274: "moroccan": "Morocco",
1275: "algerian": "Algeria",
1276: "bangladeshi": "Bangladesh",
1277: "cambodian": "Cambodia",
1278: "indonesian": "Indonesia",
1279: "dominican": "Dominican Republic",
1280: "panamanian": "Panama",
1281: "bolivian": "Bolivia",
1282: "chilean": "Chile",
1283: "argentine": "Argentina",
1284: "argentinian": "Argentina",
1285: "saudi": "Saudi Arabia",
1286: "turkish": "Turkey",
1287: }
1288: FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
1289:     "tehran": "Iran",
1290:     "caracas": "Venezuela",
1291:     "beijing": "China",
1292:     "moscow": "Russia",
1293:     "havana": "Cuba",
1294:     "kabul": "Afghanistan",
1295:     "damascus": "Syria",
1296:     "baghdad": "Iraq",
1297:     "tripoli": "Libya",
1298:     "mogadishu": "Somalia",
1299:     "bogota": "Colombia",
1300:     "managua": "Nicaragua",
1301:     "tegucigalpa": "Honduras",
1302:     "kyiv": "Ukraine",
1303: }
1304:
1305:
1306: def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:
1307:     alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
1308:     return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)
1309:
1310:
1311: PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))
1312: PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
1313: # A demonym that immediately precedes one of these nouns describes a person's
1314: # nationality, so it also earns origin:<country>, not just a contextual country:.
1315: PATTERN_DEMONYM_PERSON = _compile(
1316:     r"^\w*(?:national|nationals|citizen|citizens|immigrant|immigrants|migrant|migrants|"
1677: # origin:<COUNTRY> – validated against the sovereign-state vocab.
1678: for m in PATTERN_ORIGIN_CANDIDATE.finditer(text):
1679:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1680:     if resolved:
1681:         add(f"origin:{_normalize_country(resolved)}", span=m.span(1))
1682:
1683: # country:<NAME> – broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
2052: if low == "burma":
2053:     return "Myanmar"
2054: return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")

```

```

2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

origin:Turkey (2 (0.0%))

Precise plain-English rule

Emitted as origin:Turkey when the text has from Turkey, with a comma after the country, or a demonym for Turkey followed immediately by a person-origin noun.

Code text for this tag

```

212:     "Turkey",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the *next* place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS}))\b[A-Za-z][a-zA-Z]+)?"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*," )
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:({_PREPOSITIONS})\s+({_PLACE}))\b")
1229: # "<Place>,<State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE}),\s+({_PLACE})\b")

1238: DEMONYM_TO_COUNTRY: dict[str, str] = {
1239:     "iranian": "Iran",
1240:     "mexican": "Mexico",
1241:     "venezuelan": "Venezuela",
1242:     "chinese": "China",
1243:     "russian": "Russia",
1244:     "cuban": "Cuba",
1245:     "colombian": "Colombia",
1246:     "salvadoran": "El Salvador",
1247:     "salvadorian": "El Salvador",
1248:     "honduran": "Honduras",
1249:     "guatemalan": "Guatemala",
1250:     "haitian": "Haiti",
1251:     "somali": "Somalia",
1252:     "somalian": "Somalia",
1253:     "afghan": "Afghanistan",
1254:     "syrian": "Syria",
1255:     "nigerian": "Nigeria",
1256:     "pakistani": "Pakistan",
1257:     "ukrainian": "Ukraine",
1258:     "israeli": "Israel",
1259:     "iraqi": "Iraq",
1260:     "egyptian": "Egypt",
1261:     "yemeni": "Yemen",
1262:     "brazilian": "Brazil",
1263:     "nicaraguan": "Nicaragua",
1264:     "ecuadorian": "Ecuador",
1265:     "peruvian": "Peru",
1266:     "jamaican": "Jamaica",
1267:     "filipino": "Philippines",
1268:     "vietnamese": "Vietnam",
1269:     "lebanese": "Lebanon",
1270:     "libyan": "Libya",
1271:     "sudanese": "Sudan",
1272:     "ethiopian": "Ethiopia",
1273:     "kenyan": "Kenya",
1274:     "moroccan": "Morocco",
1275:     "algerian": "Algeria",
1276:     "bangladeshi": "Bangladesh",
1277:     "cambodian": "Cambodia",
1278:     "indonesian": "Indonesia",
1279:     "dominican": "Dominican Republic",
1280:     "panamanian": "Panama",
1281:     "bolivian": "Bolivia",
1282:     "chilean": "Chile",
1283:     "argentine": "Argentina",
1284:     "argentinian": "Argentina",
1285:     "saudi": "Saudi Arabia",
1286:     "turkish": "Turkey",
1287: }
1288: FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
1289:     "tehran": "Iran",
1290:     "caracas": "Venezuela",
1291:     "beijing": "China",
1292:     "moscow": "Russia",
1293:     "havana": "Cuba",
1294:     "kabul": "Afghanistan",
1295:     "damascus": "Syria",
1296:     "baghdad": "Iraq",
1297:     "tripoli": "Libya",
1298:     "mogadishu": "Somalia",
1299:     "bogota": "Colombia",
1300:     "managua": "Nicaragua",
1301:     "tegucigalpa": "Honduras",
1302:     "kyiv": "Ukraine",
1303: }
1304:
1305:
1306: def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:
1307:     alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
1308:     return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)
1309:
1310:

```

```

1311: PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))
1312: PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
1313: # A demonym that immediately precedes one of these nouns describes a person's
1314: # nationality, so it also earns origin:<country>, not just a contextual country:.
1315: PATTERN_DEMONYM_PERSON = _compile(
1316:     r"^\W*(?:national|nationals|citizen|citizens|immigrant|immigrants|migrant|migrants|"

1677:     # origin:<COUNTRY> – validated against the sovereign-state vocab.
1678:     for m in PATTERN_ORIGIN_CANDIDATE.finditer(text):
1679:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1680:         if resolved:
1681:             add(f"origin:{_normalize_country(resolved)}", span=m.span(1))
1682:
1683:     # country:<NAME> – broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:
1687:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:     # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:     # country, which the bare-name validators above can't see. A demonym
1691:     # followed by a person noun also earns origin:.
1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

origin:Argentina (1 (0.0%))

Precise plain-English rule

Emitted as origin:Argentina when the text has from Argentina, with a comma after the country, or a demonym for Argentina followed immediately by a person-origin noun.

Code text for this tag

```

55:     "Argentina",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the 'next' place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]+)?"
1225: # "from <Country>," – anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>, <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE})\s+({_PLACE})\b")

1238: DEMONYM_TO_COUNTRY: dict[str, str] = {
1239:     "iranian": "Iran",
1240:     "mexican": "Mexico",
1241:     "venezuelan": "Venezuela",
1242:     "chinese": "China",
1243:     "russian": "Russia",
1244:     "cuban": "Cuba",
1245:     "colombian": "Colombia",
1246:     "salvadoran": "El Salvador",
1247:     "salvadorian": "El Salvador",
1248:     "honduran": "Honduras",
1249:     "guatemalan": "Guatemala",
1250:     "haitian": "Haiti",
1251:     "somali": "Somalia",
1252:     "somalian": "Somalia",
1253:     "afghan": "Afghanistan",
1254:     "syrian": "Syria",
1255:     "nigerian": "Nigeria",
1256:     "pakistani": "Pakistan",
1257:     "ukrainian": "Ukraine",
1258:     "israeli": "Israel",
1259:     "iraqi": "Iraq",
1260:     "egyptian": "Egypt",
1261:     "yemeni": "Yemen",
1262:     "brazilian": "Brazil",
1263:     "nicaraguan": "Nicaragua",
1264:     "ecuadorian": "Ecuador",
1265:     "peruvian": "Peru",
1266:     "jamaican": "Jamaica",
1267:     "filipino": "Philippines",

```

```

1268: "vietnamese": "Vietnam",
1269: "lebanese": "Lebanon",
1270: "libyan": "Libya",
1271: "sudanese": "Sudan",
1272: "ethiopian": "Ethiopia",
1273: "kenyan": "Kenya",
1274: "moroccan": "Morocco",
1275: "algerian": "Algeria",
1276: "bangladeshi": "Bangladesh",
1277: "cambodian": "Cambodia",
1278: "indonesian": "Indonesia",
1279: "dominican": "Dominican Republic",
1280: "panamanian": "Panama",
1281: "bolivian": "Bolivia",
1282: "chilean": "Chile",
1283: "argentine": "Argentina",
1284: "argentinian": "Argentina",
1285: "saudi": "Saudi Arabia",
1286: "turkish": "Turkey",
1287: }
1288: FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
1289:     "tehran": "Iran",
1290:     "caracas": "Venezuela",
1291:     "beijing": "China",
1292:     "moscow": "Russia",
1293:     "havana": "Cuba",
1294:     "kabul": "Afghanistan",
1295:     "damascus": "Syria",
1296:     "baghdad": "Iraq",
1297:     "tripoli": "Libya",
1298:     "mogadishu": "Somalia",
1299:     "bogota": "Colombia",
1300:     "managua": "Nicaragua",
1301:     "tegucigalpa": "Honduras",
1302:     "kyiv": "Ukraine",
1303: }
1304:
1305:
1306: def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:
1307:     alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
1308:     return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)
1309:
1310:
1311: PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))
1312: PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
1313: # A demonym that immediately precedes one of these nouns describes a person's
1314: # nationality, so it also earns origin:<country>, not just a contextual country:.
1315: PATTERN_DEMONYM_PERSON = _compile(
1316:     r"^\w*(?:national|nationals|citizen|citizens|immigrant|immigrants|migrant|migrants|"
1317:
1318:
1319:
1320:
1321:
1322:
1323:
1324:
1325:
1326:
1327:
1328:
1329:
1330:
1331:
1332:
1333:
1334:
1335:
1336:
1337:
1338:
1339:
1340:
1341:
1342:
1343:
1344:
1345:
1346:
1347:
1348:
1349:
1350:
1351:
1352:
1353:
1354:
1355:
1356:
1357:
1358:
1359:
1360:
1361:
1362:
1363:
1364:
1365:
1366:
1367:
1368:
1369:
1370:
1371:
1372:
1373:
1374:
1375:
1376:
1377:
1378:
1379:
1380:
1381:
1382:
1383:
1384:
1385:
1386:
1387:
1388:
1389:
1390:
1391:
1392:
1393:
1394:
1395:
1396:
1397:
1398:
1399:
1400:
1401:
1402:
1403:
1404:
1405:
1406:
1407:
1408:
1409:
1410:
1411:
1412:
1413:
1414:
1415:
1416:
1417:
1418:
1419:
1420:
1421:
1422:
1423:
1424:
1425:
1426:
1427:
1428:
1429:
1430:
1431:
1432:
1433:
1434:
1435:
1436:
1437:
1438:
1439:
1440:
1441:
1442:
1443:
1444:
1445:
1446:
1447:
1448:
1449:
1450:
1451:
1452:
1453:
1454:
1455:
1456:
1457:
1458:
1459:
1460:
1461:
1462:
1463:
1464:
1465:
1466:
1467:
1468:
1469:
1470:
1471:
1472:
1473:
1474:
1475:
1476:
1477:
1478:
1479:
1480:
1481:
1482:
1483:
1484:
1485:
1486:
1487:
1488:
1489:
1490:
1491:
1492:
1493:
1494:
1495:
1496:
1497:
1498:
1499:
1500:
1501:
1502:
1503:
1504:
1505:
1506:
1507:
1508:
1509:
1510:
1511:
1512:
1513:
1514:
1515:
1516:
1517:
1518:
1519:
1520:
1521:
1522:
1523:
1524:
1525:
1526:
1527:
1528:
1529:
1530:
1531:
1532:
1533:
1534:
1535:
1536:
1537:
1538:
1539:
1540:
1541:
1542:
1543:
1544:
1545:
1546:
1547:
1548:
1549:
1550:
1551:
1552:
1553:
1554:
1555:
1556:
1557:
1558:
1559:
1560:
1561:
1562:
1563:
1564:
1565:
1566:
1567:
1568:
1569:
1570:
1571:
1572:
1573:
1574:
1575:
1576:
1577:
1578:
1579:
1580:
1581:
1582:
1583:
1584:
1585:
1586:
1587:
1588:
1589:
1590:
1591:
1592:
1593:
1594:
1595:
1596:
1597:
1598:
1599:
1600:
1601:
1602:
1603:
1604:
1605:
1606:
1607:
1608:
1609:
1610:
1611:
1612:
1613:
1614:
1615:
1616:
1617:
1618:
1619:
1620:
1621:
1622:
1623:
1624:
1625:
1626:
1627:
1628:
1629:
1630:
1631:
1632:
1633:
1634:
1635:
1636:
1637:
1638:
1639:
1640:
1641:
1642:
1643:
1644:
1645:
1646:
1647:
1648:
1649:
1650:
1651:
1652:
1653:
1654:
1655:
1656:
1657:
1658:
1659:
1660:
1661:
1662:
1663:
1664:
1665:
1666:
1667:
1668:
1669:
1670:
1671:
1672:
1673:
1674:
1675:
1676:
1677:
1678:
1679:
1680:
1681:
1682:
1683:
1684:
1685:
1686:
1687:
1688:
1689:
1690:
1691:
1692:
1693:
1694:
1695:
1696:
1697:
1698:
1699:
1700:
1701:
1702:
1703:
1704:
1705:
1706:
1707:
1708:
1709:
1710:
1711:
1712:
1713:
1714:
1715:
1716:
1717:
1718:
1719:
1720:
1721:
1722:
1723:
1724:
1725:
1726:
1727:
1728:
1729:
1730:
1731:
1732:
1733:
1734:
1735:
1736:
1737:
1738:
1739:
1740:
1741:
1742:
1743:
1744:
1745:
1746:
1747:
1748:
1749:
1750:
1751:
1752:
1753:
1754:
1755:
1756:
1757:
1758:
1759:
1760:
1761:
1762:
1763:
1764:
1765:
1766:
1767:
1768:
1769:
1770:
1771:
1772:
1773:
1774:
1775:
1776:
1777:
1778:
1779:
1780:
1781:
1782:
1783:
1784:
1785:
1786:
1787:
1788:
1789:
1790:
1791:
1792:
1793:
1794:
1795:
1796:
1797:
1798:
1799:
1800:
1801:
1802:
1803:
1804:
1805:
1806:
1807:
1808:
1809:
1810:
1811:
1812:
1813:
1814:
1815:
1816:
1817:
1818:
1819:
1820:
1821:
1822:
1823:
1824:
1825:
1826:
1827:
1828:
1829:
1830:
1831:
1832:
1833:
1834:
1835:
1836:
1837:
1838:
1839:
1840:
1841:
1842:
1843:
1844:
1845:
1846:
1847:
1848:
1849:
1850:
1851:
1852:
1853:
1854:
1855:
1856:
1857:
1858:
1859:
1860:
1861:
1862:
1863:
1864:
1865:
1866:
1867:
1868:
1869:
1870:
1871:
1872:
1873:
1874:
1875:
1876:
1877:
1878:
1879:
1880:
1881:
1882:
1883:
1884:
1885:
1886:
1887:
1888:
1889:
1890:
1891:
1892:
1893:
1894:
1895:
1896:
1897:
1898:
1899:
1900:
1901:
1902:
1903:
1904:
1905:
1906:
1907:
1908:
1909:
1910:
1911:
1912:
1913:
1914:
1915:
1916:
1917:
1918:
1919:
1920:
1921:
1922:
1923:
1924:
1925:
1926:
1927:
1928:
1929:
1930:
1931:
1932:
1933:
1934:
1935:
1936:
1937:
1938:
1939:
1940:
1941:
1942:
1943:
1944:
1945:
1946:
1947:
1948:
1949:
1950:
1951:
1952:
1953:
1954:
1955:
1956:
1957:
1958:
1959:
1960:
1961:
1962:
1963:
1964:
1965:
1966:
1967:
1968:
1969:
1970:
1971:
1972:
1973:
1974:
1975:
1976:
1977:
1978:
1979:
1980:
1981:
1982:
1983:
1984:
1985:
1986:
1987:
1988:
1989:
1990:
1991:
1992:
1993:
1994:
1995:
1996:
1997:
1998:
1999:
2000:
2001:
2002:
2003:
2004:
2005:
2006:
2007:
2008:
2009:
2010:
2011:
2012:
2013:
2014:
2015:
2016:
2017:
2018:
2019:
2020:
2021:
2022:
2023:
2024:
2025:
2026:
2027:
2028:
2029:
2030:
2031:
2032:
2033:
2034:
2035:
2036:
2037:
2038:
2039:
2040:
2041:
2042:
2043:
2044:
2045:
2046:
2047:
2048:
2049:
2050:
2051:
2052:
2053:
2054:
2055:
2056:
2057:
2058:
2059:
2060:
2061:
2062:
2063:
2064:
2065:
2066:

```

origin:Azerbaijan (1 (0.0%))

Precise plain-English rule

Emitted as origin:Azerbaijan when the text has from Azerbaijan, with a comma after the country, or a demonym for Azerbaijan followed immediately by a person-origin noun.

Code text for this tag

```

59:     "Azerbaijan",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the *next* place

```

```

1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). `_resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?:{:_PREPOSITIONS_})\b)[A-Za-z][a-zA-Z]*(?)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{:_PREPOSITIONS_})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE})\s+({_PLACE})\b")

1238: DEMONYM_TO_COUNTRY: dict[str, str] = {
1239:     "iranian": "Iran",
1240:     "mexican": "Mexico",
1241:     "venezuelan": "Venezuela",
1242:     "chinese": "China",
1243:     "russian": "Russia",
1244:     "cuban": "Cuba",
1245:     "colombian": "Colombia",
1246:     "salvadoran": "El Salvador",
1247:     "salvadorian": "El Salvador",
1248:     "honduran": "Honduras",
1249:     "guatemalan": "Guatemala",
1250:     "haitian": "Haiti",
1251:     "somali": "Somalia",
1252:     "somalian": "Somalia",
1253:     "afghan": "Afghanistan",
1254:     "syrian": "Syria",
1255:     "nigerian": "Nigeria",
1256:     "pakistani": "Pakistan",
1257:     "ukrainian": "Ukraine",
1258:     "israeli": "Israel",
1259:     "iraqi": "Iraq",
1260:     "egyptian": "Egypt",
1261:     "yemeni": "Yemen",
1262:     "brazilian": "Brazil",
1263:     "nicaraguan": "Nicaragua",
1264:     "ecuadorian": "Ecuador",
1265:     "peruvian": "Peru",
1266:     "jamaican": "Jamaica",
1267:     "filipino": "Philippines",
1268:     "vietnamese": "Vietnam",
1269:     "lebanese": "Lebanon",
1270:     "libyan": "Libya",
1271:     "sudanese": "Sudan",
1272:     "ethiopian": "Ethiopia",
1273:     "kenyan": "Kenya",
1274:     "moroccan": "Morocco",
1275:     "algerian": "Algeria",
1276:     "bangladeshi": "Bangladesh",
1277:     "cambodian": "Cambodia",
1278:     "indonesian": "Indonesia",
1279:     "dominican": "Dominican Republic",
1280:     "panamanian": "Panama",
1281:     "bolivian": "Bolivia",
1282:     "chilean": "Chile",
1283:     "argentine": "Argentina",
1284:     "argentinian": "Argentina",
1285:     "saudi": "Saudi Arabia",
1286:     "turkish": "Turkey",
1287: }
1288: FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
1289:     "tehran": "Iran",
1290:     "caracas": "Venezuela",
1291:     "beijing": "China",
1292:     "moscow": "Russia",
1293:     "havana": "Cuba",
1294:     "kabul": "Afghanistan",
1295:     "damascus": "Syria",
1296:     "baghdad": "Iraq",
1297:     "tripoli": "Libya",
1298:     "mogadishu": "Somalia",
1299:     "bogota": "Colombia",
1300:     "managua": "Nicaragua",
1301:     "tegucigalpa": "Honduras",
1302:     "kyiv": "Ukraine",
1303: }
1304:
1305: def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:
1306:     alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
1307:     return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)
1310:
1311: PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))
1312: PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
1313: # A demonym that immediately precedes one of these nouns describes a person's
1314: # nationality, so it also earns origin:<country>, not just a contextual country:.
1315: PATTERN_DEMONYM_PERSON = re.compile(
1316:     r"^\w*(?:national|nationals|citizen|citizens|immigrant|immigrants|migrant|migrants|"
1677:     # origin:<COUNTRY> - validated against the sovereign-state vocab.
1678:     for m in PATTERN_ORIGIN_CANDIDATE.finditer(text):
1679:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1680:         if resolved:
1681:             add(f"origin:{_normalize_country(resolved)}", span=m.span(1))
1682:
1683:     # country:<NAME> - broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:
1687:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:     # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:     # country, which the bare-name validators above can't see. A demonym
1691:     # followed by a person noun also earns origin:.
1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())

```

```

1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):

2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
2052: if low == "burma":
2053:     return "Myanmar"
2054: return s.replace(" ", ".")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", ".")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

origin:Bangladesh (1 (0.0%))

Precise plain-English rule

Emitted as origin:Bangladesh when the text has from Bangladesh, with a comma after the country, or a demonym for Bangladesh followed immediately by a person-origin noun.

Code text for this tag

```

62:     "Bangladesh",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the *next* place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+?(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]+)?"
1225: # "from <Country>" - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s+", "")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>, <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE}),\s+({_PLACE})\b")

1238: DEMONYM_TO_COUNTRY: dict[str, str] = {
1239:     "iranian": "Iran",
1240:     "mexican": "Mexico",
1241:     "venezuelan": "Venezuela",
1242:     "chinese": "China",
1243:     "russian": "Russia",
1244:     "cuban": "Cuba",
1245:     "colombian": "Colombia",
1246:     "salvadoran": "El Salvador",
1247:     "salvadorian": "El Salvador",
1248:     "honduran": "Honduras",
1249:     "guatemalan": "Guatemala",
1250:     "haitian": "Haiti",
1251:     "somali": "Somalia",
1252:     "somalian": "Somalia",
1253:     "afghan": "Afghanistan",
1254:     "syrian": "Syria",
1255:     "nigerian": "Nigeria",
1256:     "pakistani": "Pakistan",
1257:     "ukrainian": "Ukraine",
1258:     "israeli": "Israel",
1259:     "iraqi": "Iraq",
1260:     "egyptian": "Egypt",
1261:     "yemeni": "Yemen",
1262:     "brazilian": "Brazil",
1263:     "nicaraguan": "Nicaragua",
1264:     "ecuadorian": "Ecuador",
1265:     "peruvian": "Peru",
1266:     "jamaican": "Jamaica",
1267:     "filipino": "Philippines",
1268:     "vietnamese": "Vietnam",
1269:     "lebanese": "Lebanon",
1270:     "libyan": "Libya",
1271:     "sudanese": "Sudan",
1272:     "ethiopian": "Ethiopia",
1273:     "kenyan": "Kenya",
1274:     "moroccan": "Morocco",
1275:     "algerian": "Algeria",
1276:     "bangladeshi": "Bangladesh",
1277:     "cambodian": "Cambodia",
1278:     "indonesian": "Indonesia",
1279:     "dominican": "Dominican Republic",
1280:     "panamanian": "Panama",
1281:     "bolivian": "Bolivia",
1282:     "chilean": "Chile",
1283:     "argentine": "Argentina",
1284:     "argentinian": "Argentina",
1285:     "saudi": "Saudi Arabia",
1286:     "turkish": "Turkey",
1287: }
1288: FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
1289:     "tehran": "Iran",
1290:     "caracas": "Venezuela",
1291:     "beijing": "China",

```

```

1292:     "moscow": "Russia",
1293:     "havana": "Cuba",
1294:     "kabul": "Afghanistan",
1295:     "damascus": "Syria",
1296:     "baghdad": "Iraq",
1297:     "tripoli": "Libya",
1298:     "mogadishu": "Somalia",
1299:     "bogota": "Colombia",
1300:     "managua": "Nicaragua",
1301:     "tegucigalpa": "Honduras",
1302:     "kyiv": "Ukraine",
1303: }
1304:
1305:
1306: def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:
1307:     alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
1308:     return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)
1309:
1310:
1311: PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))
1312: PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
1313: # A demonym that immediately precedes one of these nouns describes a person's
1314: # nationality, so it also earns origin:<country>, not just a contextual country:.
1315: PATTERN_DEMONYM_PERSON = _compile(
1316:     r"^\w*(?:national|nationals|citizen|citizens|immigrant|immigrants|migrant|migrants|"
1317:
1318:
1319:     # origin:<COUNTRY> – validated against the sovereign-state vocab.
1320:     for m in PATTERN_ORIGIN_CANDIDATE.finditer(text):
1321:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1322:         if resolved:
1323:             add(f"origin:{_normalize_country(resolved)}", span=m.span(1))
1324:
1325:     # country:<NAME> – broader: any contextual country mention.
1326:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1327:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1328:         if resolved:
1329:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1330:
1331:     # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1332:     # country, which the bare-name validators above can't see. A demonym
1333:     # followed by a person noun also earns origin:.
1334:     for m in PATTERN_DEMONYM.finditer(text):
1335:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1336:         if country and country.lower() in COUNTRY_LOWER:
1337:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1338:             if PATTERN_DEMONYM_PERSON.search(text[m.end(): m.end() + 24]):
1339:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1340:     for m in PATTERN_FOREIGN_CITY.finditer(text):
1341:
1342:
1343:
1344:
1345:
1346: if low in ("usa", "united states"):
1347:     return "United-States"
1348: if low == "uk":
1349:     return "United-Kingdom"
1350: if low == "uae":
1351:     return "United-Arab-Emirates"
1352: if low == "burma":
1353:     return "Myanmar"
1354: return s.replace(" ", "-")
1355:
1356:
1357: def _normalize_state(s: str) -> str:
1358:     return s.replace(" ", "-")
1359:
1360:
1361: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
1362:     """Resolve a candidate (any case, possibly two words) to its canonical
1363:     vocab spelling.
1364:
1365:     Tries the full phrase first, then its leading word, so a greedy
1366:     case-insensitive capture that swallowed a trailing lowercase token

```

origin:Burkina-Faso (1 (0.0%))

Precise plain-English rule

Emitted as origin:Burkina-Faso when the text has from Burkina Faso, with a comma after the country, or a demonym for Burkina Faso followed immediately by a person-origin noun.

Code text for this tag

```

75:     "Burkina Faso",
76:
77:
78:
79:
80:
81:
82:
83:
84:
85:
86:
87:
88:
89:
90:
91:
92:
93:
94:
95:
96:
97:
98:
99:
100:
101:
102:
103:
104:
105:
106:
107:
108:
109:
110:
111:
112:
113:
114:
115:
116:
117: # second word excludes those same prepositions via lookahead, so a greedy
118: # capture can't swallow the preposition that introduces the "next" place
119: # ("From USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab
120: # then validates against the vocab and falls back to the leading word for any
121: # other trailing token. Kept conservative because raw country names
122: # false-positive easily ("Chad", "Georgia").
123: _PREPOSITIONS = "from|in|to|of|with|by"
124: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!{PREPOSITIONS})\b)[A-Za-z][a-zA-Z]?"
125: # "from <Country>," – anchors the ORIGIN validator.
126: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:{PREPOSITIONS})\s+({_PLACE})\s*")
127: # Any sovereign-state mention after a preposition.
128: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{PREPOSITIONS})\s+({_PLACE})\b")
129: # "<Place>," <State>" – anchors the STATE validator.
130: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE}\s+({_PLACE})\b")
131:
132:
133: DEMONYM_TO_COUNTRY: dict[str, str] = {
134:     "iranian": "Iran",
135:     "mexican": "Mexico",
136:     "venezuelan": "Venezuela",
137:     "chinese": "China",
138:     "russian": "Russia",
139:     "cuban": "Cuba",
140:     "colombian": "Colombia",
141:     "salvadoran": "El Salvador",
142:     "salvadorian": "El Salvador",
143:     "honduran": "Honduras",

```

```

1249: "guatemalan": "Guatemala",
1250: "haitian": "Haiti",
1251: "somali": "Somalia",
1252: "somalian": "Somalia",
1253: "afghan": "Afghanistan",
1254: "syrian": "Syria",
1255: "nigerian": "Nigeria",
1256: "pakistani": "Pakistan",
1257: "ukrainian": "Ukraine",
1258: "israeli": "Israel",
1259: "iraqi": "Iraq",
1260: "egyptian": "Egypt",
1261: "yemeni": "Yemen",
1262: "brazilian": "Brazil",
1263: "nicaraguan": "Nicaragua",
1264: "ecuadorian": "Ecuador",
1265: "peruvian": "Peru",
1266: "jamaican": "Jamaica",
1267: "filipino": "Philippines",
1268: "vietnamese": "Vietnam",
1269: "lebanese": "Lebanon",
1270: "libyan": "Libya",
1271: "sudanese": "Sudan",
1272: "ethiopian": "Ethiopia",
1273: "kenyan": "Kenya",
1274: "moroccan": "Morocco",
1275: "algerian": "Algeria",
1276: "bangladeshi": "Bangladesh",
1277: "cambodian": "Cambodia",
1278: "indonesian": "Indonesia",
1279: "dominican": "Dominican Republic",
1280: "panamanian": "Panama",
1281: "bolivian": "Bolivia",
1282: "chilean": "Chile",
1283: "argentine": "Argentina",
1284: "argentinian": "Argentina",
1285: "saudi": "Saudi Arabia",
1286: "turkish": "Turkey",
1287: }
1288: FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
1289:     "tehran": "Iran",
1290:     "caracas": "Venezuela",
1291:     "beijing": "China",
1292:     "moscow": "Russia",
1293:     "havana": "Cuba",
1294:     "kabul": "Afghanistan",
1295:     "damascus": "Syria",
1296:     "baghdad": "Iraq",
1297:     "tripoli": "Libya",
1298:     "mogadishu": "Somalia",
1299:     "bogota": "Colombia",
1300:     "managua": "Nicaragua",
1301:     "tegucigalpa": "Honduras",
1302:     "kyiv": "Ukraine",
1303: }
1304:
1305:
1306: def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:
1307:     alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
1308:     return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)
1309:
1310:
1311: PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))
1312: PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
1313: # A demonym that immediately precedes one of these nouns describes a person's
1314: # nationality, so it also earns origin:<country>, not just a contextual country:.
1315: PATTERN_DEMONYM_PERSON = _compile(
1316:     r"^\W*(?:national|nationals|citizen|citizens|immigrant|immigrants|migrant|migrants)"
1317: )
1318:
1319:
1320: # origin:<COUNTRY> – validated against the sovereign-state vocab.
1321: for m in PATTERN_ORIGIN_CANDIDATE.finditer(text):
1322:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1323:     if resolved:
1324:         add(f"origin:{_normalize_country(resolved)}", span=m.span(1))
1325:
1326: # country:<NAME> – broader: any contextual country mention.
1327: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1328:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1329:     if resolved:
1330:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1331:
1332: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1333: # country, which the bare-name validators above can't see. A demonym
1334: # followed by a person noun also earns origin:.
1335: for m in PATTERN_DEMONYM.finditer(text):
1336:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1337:     if country and country.lower() in COUNTRY_LOWER:
1338:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1339:         if PATTERN_DEMONYM_PERSON.search(text[m.end(): m.end() + 24]):
1340:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1341:
1342: for m in PATTERN_FOREIGN_CITY.finditer(text):
1343:
1344:
1345: if low in ("usa", "united states"):
1346:     return "United-States"
1347:
1348: if low == "uk":
1349:     return "United-Kingdom"
1350:
1351: if low == "uae":
1352:     return "United-Arab-Emirates"
1353:
1354: if low == "burma":
1355:     return "Myanmar"
1356:
1357: return s.replace(" ", ".")
1358:
1359:
1360: def _normalize_state(s: str) -> str:
1361:     return s.replace(" ", "-")
1362:
1363:
1364: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
1365:     """Resolve a candidate (any case, possibly two words) to its canonical
1366:     vocab spelling.

```

```
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token
```

origin:Cameroon (1 (0.0%))

Precise plain-English rule

Emitted as origin:Cameroon when the text has from Cameroon, with a comma after the country, or a demonym for Cameroon followed immediately by a person-origin noun.

Code text for this tag

```
78:     "Cameroon",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). '_resolve_vocab'
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z](?:\s+(?!{_{PREPOSITIONS}})\b)[A-Za-z][a-zA-Z]?"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{_{PREPOSITIONS}})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE}\s+({_PLACE})\b")

1238: DEMONYM_TO_COUNTRY: dict[str, str] = {
1239:     "iranian": "Iran",
1240:     "mexican": "Mexico",
1241:     "venezuelan": "Venezuela",
1242:     "chinese": "China",
1243:     "russian": "Russia",
1244:     "cuban": "Cuba",
1245:     "colombian": "Colombia",
1246:     "salvadoran": "El Salvador",
1247:     "salvadorian": "El Salvador",
1248:     "honduran": "Honduras",
1249:     "guatemalan": "Guatemala",
1250:     "haitian": "Haiti",
1251:     "somali": "Somalia",
1252:     "somalian": "Somalia",
1253:     "afghan": "Afghanistan",
1254:     "syrian": "Syria",
1255:     "nigerian": "Nigeria",
1256:     "pakistani": "Pakistan",
1257:     "ukrainian": "Ukraine",
1258:     "israeli": "Israel",
1259:     "iraqi": "Iraq",
1260:     "egyptian": "Egypt",
1261:     "yemeni": "Yemen",
1262:     "brazilian": "Brazil",
1263:     "nicaraguan": "Nicaragua",
1264:     "ecuadorian": "Ecuador",
1265:     "peruvian": "Peru",
1266:     "jamaican": "Jamaica",
1267:     "filipino": "Philippines",
1268:     "vietnamese": "Vietnam",
1269:     "lebanese": "Lebanon",
1270:     "libyan": "Libya",
1271:     "sudanese": "Sudan",
1272:     "ethiopian": "Ethiopia",
1273:     "kenyan": "Kenya",
1274:     "moroccan": "Morocco",
1275:     "algerian": "Algeria",
1276:     "bangladeshi": "Bangladesh",
1277:     "cambodian": "Cambodia",
1278:     "indonesian": "Indonesia",
1279:     "dominican": "Dominican Republic",
1280:     "panamanian": "Panama",
1281:     "bolivian": "Bolivia",
1282:     "chilean": "Chile",
1283:     "argentine": "Argentina",
1284:     "argentinian": "Argentina",
1285:     "saudi": "Saudi Arabia",
1286:     "turkish": "Turkey",
1287: }
1288: FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
1289:     "tehran": "Iran",
1290:     "caracas": "Venezuela",
1291:     "beijing": "China",
1292:     "moscow": "Russia",
1293:     "havana": "Cuba",
1294:     "kabul": "Afghanistan",
1295:     "damascus": "Syria",
1296:     "baghdad": "Iraq",
1297:     "tripoli": "Libya",
1298:     "mogadishu": "Somalia",
1299:     "bogota": "Colombia",
1300:     "managua": "Nicaragua",
1301:     "tegucigalpa": "Honduras",
1302:     "kyiv": "Ukraine",
1303: }
1304:
1305:
1306: def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:
1307:     alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
1308:     return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)
1309:
1310:
1311: PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))
1312: PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
1313: # A demonym that immediately precedes one of these nouns describes a person's
1314: # nationality, so it also earns origin:<country>, not just a contextual country:.
1315: PATTERN_DEMONYM_PERSON = _compile(
```

```

1316: r"^\\W*(?:national|nationals|citizen|citizens|immigrant|immigrants|migrant|migrants|"
1677: # origin:<COUNTRY> – validated against the sovereign-state vocab.
1678: for m in PATTERN_ORIGIN_CANDIDATE.finditer(text):
1679:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1680:     if resolved:
1681:         add(f"origin:{_normalize_country(resolved)}", span=m.span(1))
1682:
1683: # country:<NAME> – broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
2052: if low == "burma":
2053:     return "Myanmar"
2054: return s.replace(" ", ".")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", ".")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

origin:Costa-Rica (1 (0.0%))

Precise plain-English rule

Emitted as origin:Costa-Rica when the text has from Costa Rica, with a comma after the country, or a demonym for Costa Rica followed immediately by a person-origin noun.

Code text for this tag

```

86:     "Costa Rica",
1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!(?:{_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <country>," – anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE})\s+({_PLACE})\b")
1238: DEMONYM_TO_COUNTRY: dict[str, str] = {
1239:     "iranian": "Iran",
1240:     "mexican": "Mexico",
1241:     "venezuelan": "Venezuela",
1242:     "chinese": "China",
1243:     "russian": "Russia",
1244:     "cuban": "Cuba",
1245:     "colombian": "Colombia",
1246:     "salvadoran": "El Salvador",
1247:     "salvadorian": "El Salvador",
1248:     "honduran": "Honduras",
1249:     "guatemalan": "Guatemala",
1250:     "haitian": "Haiti",
1251:     "somali": "Somalia",
1252:     "somalian": "Somalia",
1253:     "afghan": "Afghanistan",
1254:     "syrian": "Syria",
1255:     "nigerian": "Nigeria",
1256:     "pakistani": "Pakistan",
1257:     "ukrainian": "Ukraine",
1258:     "israeli": "Israel",
1259:     "iraqi": "Iraq",
1260:     "egyptian": "Egypt",
1261:     "yemeni": "Yemen",
1262:     "brazilian": "Brazil",
1263:     "nicaraguan": "Nicaragua",
1264:     "ecuadorian": "Ecuador",
1265:     "peruvian": "Peru",
1266:     "jamaican": "Jamaica",
1267:     "filipino": "Philippines",
1268:     "vietnamese": "Vietnam",
1269:     "lebanese": "Lebanon",
1270:     "libyan": "Libya",
1271:     "sudanese": "Sudan",
1272:     "ethiopian": "Ethiopia",

```

```

1273: "kenyan": "Kenya",
1274: "moroccan": "Morocco",
1275: "algerian": "Algeria",
1276: "bangladeshi": "Bangladesh",
1277: "cambodian": "Cambodia",
1278: "indonesian": "Indonesia",
1279: "dominican": "Dominican Republic",
1280: "panamanian": "Panama",
1281: "bolivian": "Bolivia",
1282: "chilean": "Chile",
1283: "argentine": "Argentina",
1284: "argentinian": "Argentina",
1285: "saudi": "Saudi Arabia",
1286: "turkish": "Turkey",
1287: }
1288: FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
1289:     "tehran": "Iran",
1290:     "caracas": "Venezuela",
1291:     "beijing": "China",
1292:     "moscow": "Russia",
1293:     "havana": "Cuba",
1294:     "kabul": "Afghanistan",
1295:     "damascus": "Syria",
1296:     "baghdad": "Iraq",
1297:     "tripoli": "Libya",
1298:     "mogadishu": "Somalia",
1299:     "bogota": "Colombia",
1300:     "managua": "Nicaragua",
1301:     "tegucigalpa": "Honduras",
1302:     "kyiv": "Ukraine",
1303: }
1304:
1305:
1306: def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:
1307:     alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
1308:     return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)
1309:
1310:
1311: PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))
1312: PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
1313: # A demonym that immediately precedes one of these nouns describes a person's
1314: # nationality, so it also earns origin:<country>, not just a contextual country:.
1315: PATTERN_DEMONYM_PERSON = _compile(
1316:     r"^\w*(?:national|nationals|citizen|citizens|immigrant|immigrants|migrant|migrants|"
1677:     # origin:<COUNTRY> – validated against the sovereign-state vocab.
1678:     for m in PATTERN_ORIGIN_CANDIDATE.finditer(text):
1679:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1680:         if resolved:
1681:             add(f"origin:{_normalize_country(resolved)}", span=m.span(1))
1682:
1683:     # country:<NAME> – broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:
1687:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:     # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:     # country, which the bare-name validators above can't see. A demonym
1691:     # followed by a person noun also earns origin:.
1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

origin:Germany (1 (0.0%))

Precise plain-English rule

Emitted as origin:Germany when the text has from Germany, with a comma after the country, or a demonym for Germany followed immediately by a person-origin noun.

Code text for this tag

```

109:     "Germany",
1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("From USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"

```

```

1224: _PLACE = rf"[A-Za-z][a-zA-Z](?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]?"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>,<State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1238: DEMONYM_TO_COUNTRY: dict[str, str] = {
1239:     "iranian": "Iran",
1240:     "mexican": "Mexico",
1241:     "venezuelan": "Venezuela",
1242:     "chinese": "China",
1243:     "russian": "Russia",
1244:     "cuban": "Cuba",
1245:     "colombian": "Colombia",
1246:     "salvadoran": "El Salvador",
1247:     "salvadorian": "El Salvador",
1248:     "honduran": "Honduras",
1249:     "guatemalan": "Guatemala",
1250:     "haitian": "Haiti",
1251:     "somali": "Somalia",
1252:     "somalian": "Somalia",
1253:     "afghan": "Afghanistan",
1254:     "syrian": "Syria",
1255:     "nigerian": "Nigeria",
1256:     "pakistani": "Pakistan",
1257:     "ukrainian": "Ukraine",
1258:     "israeli": "Israel",
1259:     "iraqi": "Iraq",
1260:     "egyptian": "Egypt",
1261:     "yemeni": "Yemen",
1262:     "brazilian": "Brazil",
1263:     "nicaraguan": "Nicaragua",
1264:     "ecuadorian": "Ecuador",
1265:     "peruvian": "Peru",
1266:     "jamaican": "Jamaica",
1267:     "filipino": "Philippines",
1268:     "vietnamese": "Vietnam",
1269:     "lebanese": "Lebanon",
1270:     "libyan": "Libya",
1271:     "sudanese": "Sudan",
1272:     "ethiopian": "Ethiopia",
1273:     "kenyan": "Kenya",
1274:     "moroccan": "Morocco",
1275:     "algerian": "Algeria",
1276:     "bangladeshi": "Bangladesh",
1277:     "cambodian": "Cambodia",
1278:     "indonesian": "Indonesia",
1279:     "dominican": "Dominican Republic",
1280:     "panamanian": "Panama",
1281:     "bolivian": "Bolivia",
1282:     "chilean": "Chile",
1283:     "argentine": "Argentina",
1284:     "argentinian": "Argentina",
1285:     "saudi": "Saudi Arabia",
1286:     "turkish": "Turkey",
1287: }
1288: FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
1289:     "tehran": "Iran",
1290:     "caracas": "Venezuela",
1291:     "beijing": "China",
1292:     "moscow": "Russia",
1293:     "havana": "Cuba",
1294:     "kabul": "Afghanistan",
1295:     "damascus": "Syria",
1296:     "baghdad": "Iraq",
1297:     "tripoli": "Libya",
1298:     "mogadishu": "Somalia",
1299:     "bogota": "Colombia",
1300:     "managua": "Nicaragua",
1301:     "tegucigalpa": "Honduras",
1302:     "kyiv": "Ukraine",
1303: }
1304:
1305: def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:
1306:     alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
1307:     return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)
1308:
1309:
1310:
1311: PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))
1312: PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
1313: # A demonym that immediately precedes one of these nouns describes a person's
1314: # nationality, so it also earns origin:<country>, not just a contextual country:.
1315: PATTERN_DEMONYM_PERSON = _compile(
1316:     r"^\W(?:national|nationals|citizen|citizens|immigrant|immigrants|migrant|migrants|"
1317:
1318:
1319:
1320:
1321:
1322:
1323:
1324:
1325:
1326:
1327:
1328:
1329:
1330:
1331:
1332:
1333:
1334:
1335:
1336:
1337:
1338:
1339:
1340:
1341:
1342:
1343:
1344:
1345:
1346:
1347:
1348:
1349:
1350:
1351:
1352:
1353:
1354:
1355:
1356:
1357:
1358:
1359:
1360:
1361:
1362:
1363:
1364:
1365:
1366:
1367:
1368:
1369:
1370:
1371:
1372:
1373:
1374:
1375:
1376:
1377:
1378:
1379:
1380:
1381:
1382:
1383:
1384:
1385:
1386:
1387:
1388:
1389:
1390:
1391:
1392:
1393:
1394:
1395:
1396:
1397:
1398:
1399:
1400:
1401:
1402:
1403:
1404:
1405:
1406:
1407:
1408:
1409:
1410:
1411:
1412:
1413:
1414:
1415:
1416:
1417:
1418:
1419:
1420:
1421:
1422:
1423:
1424:
1425:
1426:
1427:
1428:
1429:
1430:
1431:
1432:
1433:
1434:
1435:
1436:
1437:
1438:
1439:
1440:
1441:
1442:
1443:
1444:
1445:
1446:
1447:
1448:
1449:
1450:
1451:
1452:
1453:
1454:
1455:
1456:
1457:
1458:
1459:
1460:
1461:
1462:
1463:
1464:
1465:
1466:
1467:
1468:
1469:
1470:
1471:
1472:
1473:
1474:
1475:
1476:
1477:
1478:
1479:
1480:
1481:
1482:
1483:
1484:
1485:
1486:
1487:
1488:
1489:
1490:
1491:
1492:
1493:
1494:
1495:
1496:
1497:
1498:
1499:
1500:
1501:
1502:
1503:
1504:
1505:
1506:
1507:
1508:
1509:
1510:
1511:
1512:
1513:
1514:
1515:
1516:
1517:
1518:
1519:
1520:
1521:
1522:
1523:
1524:
1525:
1526:
1527:
1528:
1529:
1530:
1531:
1532:
1533:
1534:
1535:
1536:
1537:
1538:
1539:
1540:
1541:
1542:
1543:
1544:
1545:
1546:
1547:
1548:
1549:
1550:
1551:
1552:
1553:
1554:
1555:
1556:
1557:
1558:
1559:
1560:
1561:
1562:
1563:
1564:
1565:
1566:
1567:
1568:
1569:
1570:
1571:
1572:
1573:
1574:
1575:
1576:
1577:
1578:
1579:
1580:
1581:
1582:
1583:
1584:
1585:
1586:
1587:
1588:
1589:
1590:
1591:
1592:
1593:
1594:
1595:
1596:
1597:
1598:
1599:
1600:
1601:
1602:
1603:
1604:
1605:
1606:
1607:
1608:
1609:
1610:
1611:
1612:
1613:
1614:
1615:
1616:
1617:
1618:
1619:
1620:
1621:
1622:
1623:
1624:
1625:
1626:
1627:
1628:
1629:
1630:
1631:
1632:
1633:
1634:
1635:
1636:
1637:
1638:
1639:
1640:
1641:
1642:
1643:
1644:
1645:
1646:
1647:
1648:
1649:
1650:
1651:
1652:
1653:
1654:
1655:
1656:
1657:
1658:
1659:
1660:
1661:
1662:
1663:
1664:
1665:
1666:
1667:
1668:
1669:
1670:
1671:
1672:
1673:
1674:
1675:
1676:
1677:
1678:
1679:
1680:
1681:
1682:
1683:
1684:
1685:
1686:
1687:
1688:
1689:
1690:
1691:
1692:
1693:
1694:
1695:
1696:
1697:
1698:
1699:
1700:
1701:
1702:
1703:
1704:
1705:
1706:
1707:
1708:
1709:
1710:
1711:
1712:
1713:
1714:
1715:
1716:
1717:
1718:
1719:
1720:
1721:
1722:
1723:
1724:
1725:
1726:
1727:
1728:
1729:
1730:
1731:
1732:
1733:
1734:
1735:
1736:
1737:
1738:
1739:
1740:
1741:
1742:
1743:
1744:
1745:
1746:
1747:
1748:
1749:
1750:
1751:
1752:
1753:
1754:
1755:
1756:
1757:
1758:
1759:
1760:
1761:
1762:
1763:
1764:
1765:
1766:
1767:
1768:
1769:
1770:
1771:
1772:
1773:
1774:
1775:
1776:
1777:
1778:
1779:
1780:
1781:
1782:
1783:
1784:
1785:
1786:
1787:
1788:
1789:
1790:
1791:
1792:
1793:
1794:
1795:
1796:
1797:
1798:
1799:
1800:
1801:
1802:
1803:
1804:
1805:
1806:
1807:
1808:
1809:
1810:
1811:
1812:
1813:
1814:
1815:
1816:
1817:
1818:
1819:
1820:
1821:
1822:
1823:
1824:
1825:
1826:
1827:
1828:
1829:
1830:
1831:
1832:
1833:
1834:
1835:
1836:
1837:
1838:
1839:
1840:
1841:
1842:
1843:
1844:
1845:
1846:
1847:
1848:
1849:
1850:
1851:
1852:
1853:
1854:
1855:
1856:
1857:
1858:
1859:
1860:
1861:
1862:
1863:
1864:
1865:
1866:
1867:
1868:
1869:
1870:
1871:
1872:
1873:
1874:
1875:
1876:
1877:
1878:
1879:
1880:
1881:
1882:
1883:
1884:
1885:
1886:
1887:
1888:
1889:
1890:
1891:
1892:
1893:
1894:
1895:
1896:
1897:
1898:
1899:
1900:
1901:
1902:
1903:
1904:
1905:
1906:
1907:
1908:
1909:
1910:
1911:
1912:
1913:
1914:
1915:
1916:
1917:
1918:
1919:
1920:
1921:
1922:
1923:
1924:
1925:
1926:
1927:
1928:
1929:
1930:
1931:
1932:
1933:
1934:
1935:
1936:
1937:
1938:
1939:
1940:
1941:
1942:
1943:
1944:
1945:
1946:
1947:
1948:
1949:
1950:
1951:
1952:
1953:
1954:
1955:
1956:
1957:
1958:
1959:
1960:
1961:
1962:
1963:
1964:
1965:
1966:
1967:
1968:
1969:
1970:
1971:
1972:
1973:
1974:
1975:
1976:
1977:
1978:
1979:
1980:
1981:
1982:
1983:
1984:
1985:
1986:
1987:
1988:
1989:
1990:
1991:
1992:
1993:
1994:
1995:
1996:
1997:
1998:
1999:
2000:
2001:
2002:
2003:
2004:
2005:
2006:
2007:
2008:
2009:
2010:
2011:
2012:
2013:
2014:
2015:
2016:
2017:
2018:
2019:
2020:
2021:
2022:
2023:
2024:
2025:
2026:
2027:
2028:
2029:
2030:
2031:
2032:
2033:
2034:
2035:
2036:
2037:
2038:
2039:
2040:
2041:
2042:
2043:
2044:
2045:
2046:
2047:
2048:
2049:
2050:
2051:
2052:
2053:
2054:
2055:
2056:
2057:
2058:
2059:
2060:
2061:
2062:
2063:
2064:
2065:
2066:
2067:
2068:
2069:
2070:
2071:
2072:
2073:
2074:
2075:
2076:
2077:
2078:
2079:
2080:
2081:
2082:
2083:
2084:
2085:
2086:
2087:
2088:
2089:
2090:
2091:
2092:
2093:
2094:
2095:
2096:
2097:
2098:
2099:
2100:
2101:
2102:
2103:
2104:
2105:
2106:
2107:
2108:
2109:
2110:
2111:
2112:
2113:
2114:
2115:
2116:
2117:
2118:
2119:
2120:
2121:
2122:
2123:
2124:
2125:
2126:
2127:
2128:
2129:
2130:
2131:
2132:
2133:
2134:
2135:
2136:
2137:
2138:
2139:
2140:
2141:
2142:
2143:
2144:
2145:
2146:
2147:
2148:
2149:
2150:
2151:
2152:
2153:
2154:
2155:
2156:
2157:
2158:
2159:
2160:
2161:
2162:
2163:
2164:
2165:
2166:
2167:
2168:
2169:
2170:
2171:
2172:
2173:
2174:
2175:
2176:
2177:
2178:
2179:
2180:
2181:
2182:
2183:
2184:
2185:
2186:
2187:
2188:
2189:
2190:
2191:
2192:
2193:
2194:
2195:
2196:
2197:
2198:
2199:
2200:
2201:
2202:
2203:
2204:
2205:
2206:
2207:
2208:
2209:
2210:
2211:
2212:
2213:
2214:
2215:
2216:
2217:
2218:
2219:
2220:
2221:
2222:
2223:
2224:
2225:
2226:
2227:
2228:
2229:
2230:
2231:
2232:
2233:
2234:
2235:
2236:
2237:
2238:
2239:
2240:
2241:
2242:
2243:
2244:
2245:
2246:
2247:
2248:
2249:
2250:
2251:
2252:
2253:
2254:
2255:
2256:
2257:
2258:
2259:
2260:
2261:
2262:
2263:
2264:
2265:
2266:
2267:
2268:
2269:
2270:
2271:
2272:
2273:
2274:
2275:
2276:
2277:
2278:
2279:
2280:
2281:
2282:
2283:
2284:
2285:
2286:
2287:
2288:
2289:
2290:
2291:
2292:
2293:
2294:
2295:
2296:
2297:
2298:
2299:
2300:
2301:
2302:
2303:
2304:
2305:
2306:
2307:
2308:
2309:
2310:
2311:
2312:
2313:
2314:
2315:
2316:
2317:
2318:
2319:
2320:
2321:
2322:
2323:
2324:
2325:
2326:
2327:
2328:
2329:
2330:
2331:
2332:
2333:
2334:
2335:
2336:
2337:
2338:
2339:
2340:
2341:
2342:
2343:
2344:
2345:
2346:
2347:
2348:
2349:
2350:
2351:
2352:
2353:
2354:
2355:
2356:
2357:
2358:
2359:
2360:
2361:
2362:
2363:
2364:
2365:
2366:
2367:
2368:
2369:
2370:
2371:
2372:
2373:
2374:
2375:
2376:
2377:
2378:
2379:
2380:
2381:
2382:
2383:
2384:
2385:
2386:
2387:
2388:
2389:
2390:
2391:
2392:
2393:
2394:
2395:
2396:
2397:
2398:
2399:
2400:
2401:
2402:
2403:
2404:
2405:
2406:
2407:
2408:
2409:
2410:
2411:
2412:
2413:
2414:
2415:
2416:
2417:
2418:
2419:
2420:
2421:
2422:
2423:
2424:
2425:
2426:
2427:
2428:
2429:
2430:
2431:
2432:
2433:
2434:
2435:
2436:
2437:
2438:
2439:
2440:
2441:
2442:
2443:
2444:
2445:
2446:
2447:
2448:
2449:
2450:
2451:
2452:
2453:
2454:
2455:
2456:
2457:
2458:
2459:
2460:
2461:
2462:
2463:
2464:
2465:
2466:
2467:
2468:
2469:
2470:
2471:
2472:
2473:
2474:
2475:
2476:
2477:
2478:
2479:
2480:
2481:
2482:
2483:
2484:
2485:
2486:
2487:
2488:
2489:
2490:
2491:
2492:
2493:
2494:
2495:
2496:
2497:
2498:
2499:
2500:
2501:
2502:
2503:
2504:
2505:
2506:
2507:
2508:
2509:
2510:
2511:
2512:
2513:
2514:
2515:
2516:
2517:
2518:
2519:
2520:
2521:
2522:
2523:
2524:
2525:
2526:
2527:
2528:
2529:
2530:
2531:
2532:
2533:
2534:
2535:
2536:
2537:
2538:
2539:
2540:
2541:
2542:
2543:
2544:
2545:
2546:
2547:
2548:
2549:
2550:
2551:
2552:
2553:
2554:
2555:
2556:
2557:
2558:
2559:
2560:
2561:
2562:
2563:
2564:
2565:
2566:
2567:
2568:
2569:
2570:
2571:
2572:
2573:
2574:
2575:
2576:
2577:
2578:
2579:
2580:
2581:
2582:
2583:
2584:
2585:
2586:
2587:
2588:
2589:
2590:
2591:
2592:
2593:
2594:
2595:
2596:
2597:
2598:
2599:
2600:
2601:
2602:
2603:
2604:
2605:
2606:
2607:
2608:
2609:
2610:
2611:
2612:
2613:
2614:
2615:
2616:
2617:
2618:
2619:
2620:
2621:
2622:
2623:
2624:
2625:
2626:
2627:
2628:
2629:
2630:
2631:
2632:
2633:
2634:
2635:
2636:
2637:
2638:
2639:
2640:
2641:
2642:
2643:
2644:
2645:
2646:
2647:
2648:
2649:
2650:
2651:
2652:
2653:
2654:
2655:
2656:
2657:
2658:
2659:
2660:
2661:
2662:
2663:
2664:
2665:
2666:
2667:
2668:
2669:
2670:
2671:
2672:
2673:
2674:
2675:
2676:
2677:
2678:
2679:
2680:
2681:
2682:
2683:
2684:
2685:
2686:
2687:
2688:
2689:
2690:
2691:
2692:
2693:
2694:
2695:
2696:
2697:
2698:
2699:
2700:
2701:
2702:
2703:
2704:
2705:
2706:
2707:
2708:
2709:
2710:
2711:
2712:
2713:
2714:
2715:
2716:
2717:
2718:
2719:
2720:
2721:
2722:
2723:
2724:
2725:
2726:
2727:
2728:
2729:
2730:
2731:
2732:
2733:
2734:
2735:
2736:
2737:
2738:
2739:
2740:
2741:
2742:
2743:
2744:
2745:
2746:
2747:
2748:
2749:
2750:
2751:
2752:
2753:
2754:
2755:
2756:
2757:
2758:
2759:
2760:
2761:
2762:
2763:
2764:
2765:
2766:
2767:
2768:
2769:
2770:
2771:
2772:
2773:
2774:
2775:
2776:
2777:
2778:
2779:
2780:
2781:
2782:
2783:
2784:
2785:
2786:
2787:
2788:
2789:
2790:
2791:
2792:
2793:
2794:
2795:
2796:
2797:
2798:
2799:
2800:
2801:
2802:
2803:
2804:
2805:
2806:
2807:
2808:
2809:
2810:
2811:
2812:
2813:
2814:
2815:
2816:
2817:
2818:
2819:
2820:
2821:
2822:
2823:
2824:
2825:
2826:
2827:
2828:
2829:
2830:
2831:
2832:
2833:
2834:
2835:
2836:
2837:
2838:
2839:
2840:
2841:
2842:
2843:
2844:
2845:
2846:
2847:
2848:
2849:
2850:
2851:
2852:
2853:
2854:
2855:
2856:
2857:
2858:
2859:
2860:
2861:
2862:
2863:
2864:
2865:
2866:
2867:
2868:
2869:
2870:
2871:
2872:
2873:
2874:
2875:
2876:
2877:
2878:
2879:
2880:
2881:
2882:
2883:
2884:
2885:
2886:
2887:
2888:
2889:
2890:
2891:
2892:
2893:
2894:
2895:
2896:
2897:
2898:
2899:
2900:
2901:
2902:
2903:
2904:
2905:
2906:
2907:
2908:
2909:
2910:
2911:
2912:
2913:
2914:
2915:
2916:
2917:
2918:
2919:
2920:
2921:
2922:
2923:
2924:
2925:
2926:
2927:
2928:
2929:
2930:
2931:
2932:
2933:
2934:
2935:
2936:
2937:
2938:
2939:
2940:
2941:
2942:
2943:
2944:
2945:
2946:
2947:
2948:
2949:
2950:
2951:
2952:
2953:
2954:
2955:
2956:
2957:
2958:
2959:
2960:
2961:
2962:
2963:
2964:
2965:
2966:
2967:
2968:
2969:
2970:
2971:
2972:
2973:
2974:
2975:
2976:
2977:
2978:
2979:
2980:
2981:
2982:
2983:
2984:
2985:
2986:
2987:
2988:
2989:
2990:
2991:
2992:
2993:
2994:
2995:
2996:
2997:
2998:
2999:
3000:
3001:
3002:
3003:
3004:
3005:
3006:
3007:
3008:
3009:
3010:
3011:
3012:
3013:
3014:
3015:
3016:
3017:
3018:
3019:
3020:
3021:
3022:
3023:
3024:
3025:
3026:
3027:
3028:
3029:
3030:
3031:
3032:
3033:
3034:
3035:
3036:
3037:
3038:
3039:
3040:
3041:
3042:
3043:
3044:
3045:
3046:
3047:
3048:
3049:
3050:
3051:
3052:
3053:
3054:
3055:
3056:
3057:
3058:
3059:
3060:
3061:
3062:
3063:
3064:
3065:
3066:
3067:
3068:
3069:
3070:
3071:
3072:
3073:
3074:
3075:
3076:
3077:
3078:
3079:
3080:
3081:
3082:
3083:
3084:
3085:
3086:
3087:
3088:
3089:
3090:
3091:
3092:
3093:
3094:
3095:
3096:
3097:
3098:
3099:
3100:
3101:
3102:
3103:
3104:
3105:
3106:
3107:
3108:
3109:
3110:
3111:
3112:
3113:
3114:
3115:
3116:
3117:
3118:
3119:
3120:
3121:
3122:
3123:
3124:
3125:
3126:
3127:
3128:
3129:
3
```

```

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

origin:Ireland (1 (0.0%))

Precise plain-English rule

Emitted as origin:Ireland when the text has from Ireland, with a comma after the country, or a demonym for Ireland followed immediately by a person-origin noun.

Code text for this tag

```

124:     "Ireland",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z](?:\s+(?!({_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]+)?"
1225: # "from <Country>," - anchors the ORIGIN validator
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE}\s+({_PLACE})\b")

1238: DEMONYM_TO_COUNTRY: dict[str, str] = {
1239:     "iranian": "Iran",
1240:     "mexican": "Mexico",
1241:     "venezuelan": "Venezuela",
1242:     "chinese": "China",
1243:     "russian": "Russia",
1244:     "cuban": "Cuba",
1245:     "colombian": "Colombia",
1246:     "salvadoran": "El Salvador",
1247:     "salvadorian": "El Salvador",
1248:     "honduran": "Honduras",
1249:     "guatemalan": "Guatemala",
1250:     "haitian": "Haiti",
1251:     "somali": "Somalia",
1252:     "somalian": "Somalia",
1253:     "afghan": "Afghanistan",
1254:     "syrian": "Syria",
1255:     "nigerian": "Nigeria",
1256:     "pakistani": "Pakistan",
1257:     "ukrainian": "Ukraine",
1258:     "israeli": "Israel",
1259:     "iraqi": "Iraq",
1260:     "egyptian": "Egypt",
1261:     "yemeni": "Yemen",
1262:     "brazilian": "Brazil",
1263:     "nicaraguan": "Nicaragua",
1264:     "ecuadorian": "Ecuador",
1265:     "peruvian": "Peru",
1266:     "jamaican": "Jamaica",
1267:     "filipino": "Philippines",
1268:     "vietnamese": "Vietnam",
1269:     "lebanese": "Lebanon",
1270:     "libyan": "Libya",
1271:     "sudanese": "Sudan",
1272:     "ethiopian": "Ethiopia",
1273:     "kenyan": "Kenya",
1274:     "moroccan": "Morocco",
1275:     "algerian": "Algeria",
1276:     "bangladeshi": "Bangladesh",
1277:     "cambodian": "Cambodia",
1278:     "indonesian": "Indonesia",
1279:     "dominican": "Dominican Republic",
1280:     "panamanian": "Panama",
1281:     "bolivian": "Bolivia",
1282:     "chilean": "Chile",
1283:     "argentine": "Argentina",
1284:     "argentinian": "Argentina",
1285:     "saudi": "Saudi Arabia",
1286:     "turkish": "Turkey",
1287: }
1288: FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
1289:     "tehran": "Iran",
1290:     "caracas": "Venezuela",
1291:     "beijing": "China",
1292:     "moscow": "Russia",
1293:     "havana": "Cuba",
1294:     "kabul": "Afghanistan",
1295:     "damascus": "Syria",
1296:     "baghdad": "Iraq",
1297:     "tripoli": "Libya",

```

```

1298:     "mogadishu": "Somalia",
1299:     "bogota": "Colombia",
1300:     "managua": "Nicaragua",
1301:     "tegucigalpa": "Honduras",
1302:     "kyiv": "Ukraine",
1303: }
1304:
1305:
1306: def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:
1307:     alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
1308:     return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)
1309:
1310:
1311: PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))
1312: PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
1313: # A demonym that immediately precedes one of these nouns describes a person's
1314: # nationality, so it also earns origin:<country>, not just a contextual country:.
1315: PATTERN_DEMONYM_PERSON = _compile(
1316:     r"^\W(?:national|nationals|citizen|citizens|immigrant|immigrants|migrant|migrants|"
1317:
1318:
1319: # origin:<COUNTRY> – validated against the sovereign-state vocab.
1320: for m in PATTERN_ORIGIN_CANDIDATE.finditer(text):
1321:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1322:     if resolved:
1323:         add(f"origin:{_normalize_country(resolved)}", span=m.span(1))
1324:
1325: # country:<NAME> – broader: any contextual country mention.
1326: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1327:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1328:     if resolved:
1329:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1330:
1331: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1332: # country, which the bare-name validators above can't see. A demonym
1333: # followed by a person noun also earns origin:.
1334: for m in PATTERN_DEMONYM.finditer(text):
1335:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1336:     if country and country.lower() in COUNTRY_LOWER:
1337:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1338:         if PATTERN_DEMONYM_PERSON.search(text[m.end(): m.end() + 24]):
1339:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1340: for m in PATTERN_FOREIGN_CITY.finditer(text):
1341:
1342:
1343: if low in ("usa", "united states"):
1344:     return "United-States"
1345: if low == "uk":
1346:     return "United-Kingdom"
1347: if low == "uae":
1348:     return "United-Arab-Emirates"
1349: if low == "burma":
1350:     return "Myanmar"
1351: return s.replace(" ", "-")
1352:
1353:
1354: def _normalize_state(s: str) -> str:
1355:     return s.replace(" ", "-")
1356:
1357:
1358: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
1359:     """Resolve a candidate (any case, possibly two words) to its canonical
1360:     vocab spelling.
1361:
1362:     Tries the full phrase first, then its leading word, so a greedy
1363:     case-insensitive capture that swallowed a trailing lowercase token

```

origin:Israel (1 (0.0%))

Precise plain-English rule

Emitted as origin:Israel when the text has from Israel, with a comma after the country, or a demonym for Israel followed immediately by a person-origin noun.

Code text for this tag

```

125:     "Israel",

126:
127: # second word excludes those same prepositions via lookahead, so a greedy
128: # capture can't swallow the preposition that introduces the "next" place
129: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab
130: # then validates against the vocab and falls back to the leading word for any
131: # other trailing token. Kept conservative because raw country names
132: # false-positive easily ("Chad", "Georgia").
133: _PREPOSITIONS = "from|in|to|of|with|by"
134: _PLACE = rf"[A-Za-z][a-zA-Z]+(?:\s+(?!(?:{_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]+)?"
135: # "from <Country>," – anchors the ORIGIN validator.
136: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
137: # Any sovereign-state mention after a preposition.
138: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{_PREPOSITIONS})\s+({_PLACE})\b")
139: # "<Place>," <State>" – anchors the STATE validator.
140: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE})\s+({_PLACE})\b")

141:
142: DEMONYM_TO_COUNTRY: dict[str, str] = {
143:     "iranian": "Iran",
144:     "mexican": "Mexico",
145:     "venezuelan": "Venezuela",
146:     "chinese": "China",
147:     "russian": "Russia",
148:     "cuban": "Cuba",
149:     "colombian": "Colombia",
150:     "salvadoran": "El Salvador",
151:     "salvadorian": "El Salvador",
152:     "honduran": "Honduras",
153:     "guatemalan": "Guatemala",
154:     "haitian": "Haiti",
155:     "somali": "Somalia",
156:     "somalian": "Somalia",
157:     "afghan": "Afghanistan",
158:     "syrian": "Syria",

```

```

1255: "nigerian": "Nigeria",
1256: "pakistani": "Pakistan",
1257: "ukrainian": "Ukraine",
1258: "israeli": "Israel",
1259: "iraqi": "Iraq",
1260: "egyptian": "Egypt",
1261: "yemeni": "Yemen",
1262: "brazilian": "Brazil",
1263: "nicaraguan": "Nicaragua",
1264: "ecuadorian": "Ecuador",
1265: "peruvian": "Peru",
1266: "jamaican": "Jamaica",
1267: "filipino": "Philippines",
1268: "vietnamese": "Vietnam",
1269: "lebanese": "Lebanon",
1270: "libyan": "Libya",
1271: "sudanese": "Sudan",
1272: "ethiopian": "Ethiopia",
1273: "kenyan": "Kenya",
1274: "moroccan": "Morocco",
1275: "algerian": "Algeria",
1276: "bangladeshi": "Bangladesh",
1277: "cambodian": "Cambodia",
1278: "indonesian": "Indonesia",
1279: "dominican": "Dominican Republic",
1280: "panamanian": "Panama",
1281: "bolivian": "Bolivia",
1282: "chilean": "Chile",
1283: "argentine": "Argentina",
1284: "argentinian": "Argentina",
1285: "saudi": "Saudi Arabia",
1286: "turkish": "Turkey",
1287: }
1288: FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
1289:     "tehran": "Iran",
1290:     "caracas": "Venezuela",
1291:     "beijing": "China",
1292:     "moscow": "Russia",
1293:     "havana": "Cuba",
1294:     "kabul": "Afghanistan",
1295:     "damascus": "Syria",
1296:     "baghdad": "Iraq",
1297:     "tripoli": "Libya",
1298:     "mogadishu": "Somalia",
1299:     "bogota": "Colombia",
1300:     "managua": "Nicaragua",
1301:     "tegucigalpa": "Honduras",
1302:     "kyiv": "Ukraine",
1303: }
1304:
1305:
1306: def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:
1307:     alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
1308:     return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)
1309:
1310:
1311: PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))
1312: PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
1313: # A demonym that immediately precedes one of these nouns describes a person's
1314: # nationality, so it also earns origin:<country>, not just a contextual country:.
1315: PATTERN_DEMONYM_PERSON = compile(
1316:     r"^\w*(?:national|national|citizen|citizens|immigrant|immigrants|migrant|migrants|"
1677: # origin:<COUNTRY> – validated against the sovereign-state vocab.
1678: for m in PATTERN_ORIGIN_CANDIDATE.finditer(text):
1679:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1680:     if resolved:
1681:         add(f"origin:{_normalize_country(resolved)}", span=m.span(1))
1682:
1683: # country:<NAME> – broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
2052: if low == "burma":
2053:     return "Myanmar"
2054: return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

origin:Italy (1 (0.0%))

Precise plain-English rule

Emitted as origin:Italy when the text has from Italy, with a comma after the country, or a demonym for Italy followed immediately by a person-origin noun.

Code text for this tag

```
126:     "Italy",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). `_resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{[_PREPOSITIONS]})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE})\s+({_PLACE})\b")

1238: DEMONYM_TO_COUNTRY: dict[str, str] = {
1239:     "iranian": "Iran",
1240:     "mexican": "Mexico",
1241:     "venezuelan": "Venezuela",
1242:     "chinese": "China",
1243:     "russian": "Russia",
1244:     "cuban": "Cuba",
1245:     "colombian": "Colombia",
1246:     "salvadoran": "El Salvador",
1247:     "salvadorian": "El Salvador",
1248:     "honduran": "Honduras",
1249:     "guatemalan": "Guatemala",
1250:     "haitian": "Haiti",
1251:     "somali": "Somalia",
1252:     "somalian": "Somalia",
1253:     "afghan": "Afghanistan",
1254:     "syrian": "Syria",
1255:     "nigerian": "Nigeria",
1256:     "pakistani": "Pakistan",
1257:     "ukrainian": "Ukraine",
1258:     "israeli": "Israel",
1259:     "iraqi": "Iraq",
1260:     "egyptian": "Egypt",
1261:     "yemeni": "Yemen",
1262:     "brazilian": "Brazil",
1263:     "nicaraguan": "Nicaragua",
1264:     "ecuadorian": "Ecuador",
1265:     "peruvian": "Peru",
1266:     "jamaican": "Jamaica",
1267:     "filipino": "Philippines",
1268:     "vietnamese": "Vietnam",
1269:     "lebanese": "Lebanon",
1270:     "libyan": "Libya",
1271:     "sudanese": "Sudan",
1272:     "ethiopian": "Ethiopia",
1273:     "kenyan": "Kenya",
1274:     "moroccan": "Morocco",
1275:     "algerian": "Algeria",
1276:     "bangladeshi": "Bangladesh",
1277:     "cambodian": "Cambodia",
1278:     "indonesian": "Indonesia",
1279:     "dominican": "Dominican Republic",
1280:     "panamanian": "Panama",
1281:     "bolivian": "Bolivia",
1282:     "chilean": "Chile",
1283:     "argentine": "Argentina",
1284:     "argentinian": "Argentina",
1285:     "saudi": "Saudi Arabia",
1286:     "turkish": "Turkey",
1287: }
1288: FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
1289:     "tehran": "Iran",
1290:     "caracas": "Venezuela",
1291:     "beijing": "China",
1292:     "moscow": "Russia",
1293:     "havana": "Cuba",
1294:     "kabul": "Afghanistan",
1295:     "damascus": "Syria",
1296:     "baghdad": "Iraq",
1297:     "tripoli": "Libya",
1298:     "mogadishu": "Somalia",
1299:     "bogota": "Colombia",
1300:     "managua": "Nicaragua",
1301:     "tegucigalpa": "Honduras",
1302:     "kyiv": "Ukraine",
1303: }
1304:
1305:
1306: def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:
1307:     alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
1308:     return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)
1309:
1310:
1311: PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))
1312: PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
1313: # A demonym that immediately precedes one of these nouns describes a person's
1314: # nationality, so it also earns origin:<country>, not just a contextual country:.
1315: PATTERN_DEMONYM_PERSON = _compile(
1316:     r"^\W(?:national|nationals|citizen|citizens|immigrant|immigrants|migrant|migrants|"
1317:
1318:
1319:
1320:
1321:
1322:
1323:
1324:
1325:
1326:
1327:
1328:
1329:
1330:
1331:
1332:
1333:
1334:
1335:
1336:
1337:
1338:
1339:
1340:
1341:
1342:
1343:
1344:
1345:
1346:
1347:
1348:
1349:
1350:
1351:
1352:
1353:
1354:
1355:
1356:
1357:
1358:
1359:
1360:
1361:
1362:
1363:
1364:
1365:
1366:
1367:
1368:
1369:
1370:
1371:
1372:
1373:
1374:
1375:
1376:
1377:
1378:
1379:
1380:
1381:
1382:
1383:
1384:
1385:
1386:
1387:
1388:
1389:
1390:
1391:
1392:
1393:
1394:
1395:
1396:
1397:
1398:
1399:
1400:
1401:
1402:
1403:
1404:
1405:
1406:
1407:
1408:
1409:
1410:
1411:
1412:
1413:
1414:
1415:
1416:
1417:
1418:
1419:
1420:
1421:
1422:
1423:
1424:
1425:
1426:
1427:
1428:
1429:
1430:
1431:
1432:
1433:
1434:
1435:
1436:
1437:
1438:
1439:
1440:
1441:
1442:
1443:
1444:
1445:
1446:
1447:
1448:
1449:
1450:
1451:
1452:
1453:
1454:
1455:
1456:
1457:
1458:
1459:
1460:
1461:
1462:
1463:
1464:
1465:
1466:
1467:
1468:
1469:
1470:
1471:
1472:
1473:
1474:
1475:
1476:
1477:
1478:
1479:
1480:
1481:
1482:
1483:
1484:
1485:
1486:
1487:
1488:
1489:
1490:
1491:
1492:
1493:
1494:
1495:
1496:
1497:
1498:
1499:
1500:
1501:
1502:
1503:
1504:
1505:
1506:
1507:
1508:
1509:
1510:
1511:
1512:
1513:
1514:
1515:
1516:
1517:
1518:
1519:
1520:
1521:
1522:
1523:
1524:
1525:
1526:
1527:
1528:
1529:
1530:
1531:
1532:
1533:
1534:
1535:
1536:
1537:
1538:
1539:
1540:
1541:
1542:
1543:
1544:
1545:
1546:
1547:
1548:
1549:
1550:
1551:
1552:
1553:
1554:
1555:
1556:
1557:
1558:
1559:
1560:
1561:
1562:
1563:
1564:
1565:
1566:
1567:
1568:
1569:
1570:
1571:
1572:
1573:
1574:
1575:
1576:
1577:
1578:
1579:
1580:
1581:
1582:
1583:
1584:
1585:
1586:
1587:
1588:
1589:
1590:
1591:
1592:
1593:
1594:
1595:
1596:
1597:
1598:
1599:
1600:
1601:
1602:
1603:
1604:
1605:
1606:
1607:
1608:
1609:
1610:
1611:
1612:
1613:
1614:
1615:
1616:
1617:
1618:
1619:
1620:
1621:
1622:
1623:
1624:
1625:
1626:
1627:
1628:
1629:
1630:
1631:
1632:
1633:
1634:
1635:
1636:
1637:
1638:
1639:
1640:
1641:
1642:
1643:
1644:
1645:
1646:
1647:
1648:
1649:
1650:
1651:
1652:
1653:
1654:
1655:
1656:
1657:
1658:
1659:
1660:
1661:
1662:
1663:
1664:
1665:
1666:
1667:
1668:
1669:
1670:
1671:
1672:
1673:
1674:
1675:
1676:
1677:
1678:
1679:
1680:
1681:
1682:
1683:
1684:
1685:
1686:
1687:
1688:
1689:
1690:
1691:
1692:
1693:
1694:
1695:
1696:
1697:
1698:
1699:
1700:
1701:
1702:
1703:
1704:
1705:
1706:
1707:
1708:
1709:
1710:
1711:
1712:
1713:
1714:
1715:
1716:
1717:
1718:
1719:
1720:
1721:
1722:
1723:
1724:
1725:
1726:
1727:
1728:
1729:
1730:
1731:
1732:
1733:
1734:
1735:
1736:
1737:
1738:
1739:
1740:
1741:
1742:
1743:
1744:
1745:
1746:
1747:
1748:
1749:
1750:
1751:
1752:
1753:
1754:
1755:
1756:
1757:
1758:
1759:
1760:
1761:
1762:
1763:
1764:
1765:
1766:
1767:
1768:
1769:
1770:
1771:
1772:
1773:
1774:
1775:
1776:
1777:
1778:
1779:
1780:
1781:
1782:
1783:
1784:
1785:
1786:
1787:
1788:
1789:
1790:
1791:
1792:
1793:
1794:
1795:
1796:
1797:
1798:
1799:
1800:
1801:
1802:
1803:
1804:
1805:
1806:
1807:
1808:
1809:
1810:
1811:
1812:
1813:
1814:
1815:
1816:
1817:
1818:
1819:
1820:
1821:
1822:
1823:
1824:
1825:
1826:
1827:
1828:
1829:
1830:
1831:
1832:
1833:
1834:
1835:
1836:
1837:
1838:
1839:
1840:
1841:
1842:
1843:
1844:
1845:
1846:
1847:
1848:
1849:
1850:
1851:
1852:
1853:
1854:
1855:
1856:
1857:
1858:
1859:
1860:
1861:
1862:
1863:
1864:
1865:
1866:
1867:
1868:
1869:
1870:
1871:
1872:
1873:
1874:
1875:
1876:
1877:
1878:
1879:
1880:
1881:
1882:
1883:
1884:
1885:
1886:
1887:
1888:
1889:
1890:
1891:
1892:
1893:
1894:
1895:
1896:
1897:
1898:
1899:
1900:
1901:
1902:
1903:
1904:
1905:
1906:
1907:
1908:
1909:
1910:
1911:
1912:
1913:
1914:
1915:
1916:
1917:
1918:
1919:
1920:
1921:
1922:
1923:
1924:
1925:
1926:
1927:
1928:
1929:
1930:
1931:
1932:
1933:
1934:
1935:
1936:
1937:
1938:
1939:
1940:
1941:
1942:
1943:
1944:
1945:
1946:
1947:
1948:
1949:
1950:
1951:
1952:
1953:
1954:
1955:
1956:
1957:
1958:
1959:
1960:
1961:
1962:
1963:
1964:
1965:
1966:
1967:
1968:
1969:
1970:
1971:
1972:
1973:
1974:
1975:
1976:
1977:
1978:
1979:
1980:
1981:
1982:
1983:
1984:
1985:
1986:
1987:
1988:
1989:
1990:
1991:
1992:
1993:
1994:
1995:
1996:
1997:
1998:
1999:
2000:
2001:
2002:
2003:
2004:
2005:
2006:
2007:
2008:
2009:
2010:
2011:
2012:
2013:
2014:
2015:
2016:
2017:
2018:
2019:
2020:
2021:
2022:
2023:
2024:
2025:
2026:
2027:
2028:
2029:
2030:
2031:
2032:
2033:
2034:
2035:
2036:
2037:
2038:
2039:
2040:
2041:
2042:
2043:
2044:
2045:
2046:
2047:
2048:
2049:
2050:
2051:
2052:
2053:
2054:
2055:
2056:
2057:
2058:
2059:
2060:
2061:
2062:
2063:
2064:
2065:
2066:
2067:
2068:
2069:
2070:
2071:
2072:
2073:
2074:
2075:
2076:
2077:
2078:
2079:
2080:
2081:
2082:
2083:
2084:
2085:
2086:
2087:
2088:
2089:
2090:
2091:
2092:
2093:
2094:
2095:
2096:
2097:
2098:
2099:
2100:
2101:
2102:
2103:
2104:
2105:
2106:
2107:
2108:
2109:
2110:
2111:
2112:
2113:
2114:
2115:
2116:
2117:
2118:
2119:
2120:
2121:
2122:
2123:
2124:
2125:
2126:
2127:
2128:
2129:
2130:
2131:
2132:
2133:
2134:
2135:
2136:
2137:
2138:
2139:
2140:
2141:
2142:
2143:
2144:
2145:
2146:
2147:
2148:
2149:
2150:
2151:
2152:
2153:
2154:
2155:
2156:
2157:
2158:
2159:
2160:
2161:
2162:
2163:
2164:
2165:
2166:
2167:
2168:
2169:
2170:
2171:
2172:
2173:
2174:
2175:
2176:
2177:
2178:
2179:
2180:
2181:
2182:
2183:
2184:
2185:
2186:
2187:
2188:
2189:
2190:
2191:
2192:
2193:
2194:
2195:
2196:
2197:
2198:
2199:
2200:
2201:
2202:
2203:
2204:
2205:
2206:
2207:
2208:
2209:
2210:
2211:
2212:
2213:
2214:
2215:
2216:
2217:
2218:
2219:
2220:
2221:
2222:
2223:
2224:
2225:
2226:
2227:
2228:
2229:
2230:
2231:
2232:
2233:
2234:
2235:
2236:
2237:
2238:
2239:
2240:
2241:
2242:
2243:
2244:
2245:
2246:
2247:
2248:
2249:
2250:
2251:
2252:
2253:
2254:
2255:
2256:
2257:
2258:
2259:
2260:
2261:
2262:
2263:
2264:
2265:
2266:
2267:
2268:
2269:
2270:
2271:
2272:
2273:
2274:
2275:
2276:
2277:
2278:
2279:
2280:
2281:
2282:
2283:
2284:
2285:
2286:
2287:
2288:
2289:
2290:
2291:
2292:
2293:
2294:
2295:
2296:
2297:
2298:
2299:
2300:
2301:
2302:
2303:
2304:
2305:
2306:
2307:
2308:
2309:
2310:
2311:
2312:
2313:
2314:
2315:
2316:
2317:
2318:
2319:
2320:
2321:
2322:
2323:
2324:
2325:
2326:
2327:
2328:
2329:
2330:
2331:
2332:
2333:
2334:
2335:
2336:
2337:
2338:
2339:
2340:
2341:
2342:
2343:
2344:
2345:
2346:
2347:
2348:
2349:
2350:
2351:
2352:
2353:
2354:
2355:
2356:
2357:
2358:
2359:
2360:
2361:
2362:
2363:
2364:
2365:
2366:
2367:
2368:
2369:
2370:
2371:
2372:
2373:
2374:
2375:
2376:
2377:
2378:
2379:
2380:
2381:
2382:
2383:
2384:
2385:
2386:
2387:
2388:
2389:
2390:
2391:
2392:
2393:
2394:
2395:
2396:
2397:
2398:
2399:
2400:
2401:
2402:
2403:
2404:
2405:
2406:
2407:
2408:
2409:
2410:
2411:
2412:
2413:
2414:
2415:
2416:
2417:
2418:
2419:
2420:
2421:
2422:
2423:
2424:
2425:
2426:
2427:
2428:
2429:
2430:
2431:
2432:
2433:
2434:
2435:
2436:
2437:
2438:
2439:
2440:
2441:
2442:
2443:
2444:
2445:
2446:
2447:
2448:
2449:
2450:
2451:
2452:
2453:
2454:
2455:
2456:
2457:
2458:
2459:
2460:
2461:
2462:
2463:
2464:
2465:
2466:
2467:
2468:
2469:
2470:
2471:
2472:
2473:
2474:
2475:
2476:
2477:
2478:
2479:
2480:
2481:
2482:
2483:
2484:
2485:
2486:
2487:
2488:
2489:
2490:
2491:
2492:
2493:
2494:
2495:
2496:
2497:
2498:
2499:
2500:
2501:
2502:
2503:
2504:
2505:
2506:
2507:
2508:
2509:
2510:
2511:
2512:
2513:
2514:
2515:
2516:
2517:
2518:
2519:
2520:
2521:
2522:
2523:
2524:
2525:
2526:
2527:
2528:
2529:
2530:
2531:
2532:
2533:
2534:
2535:
2536:
2537:
2538:
2539:
2540:
2541:
2542:
2543:
2544:
2545:
2546:
2547:
2548:
2549:
2550:
2551:
2552:
2553:
2554:
2555:
2556:
2557:
2558:
2559:
2560:
2561:
2562:
2563:
2564:
2565:
2566:
2567:
2568:
2569:
2570:
2571:
2572:
2573:
2574:
2575:
2576:
2577:
2578:
2579:
2580:
2581:
2582:
2583:
2584:
2585:
2586:
2587:
2588:
2589:
2590:
2591:
2592:
2593:
2594:
2595:
2596:
2597:
2598:
2599:
2600:
2601:
2602:
2603:
2604:
2605:
2606:
2607:
2608:
2609:
2610:
2611:
2612:
2613:
2614:
2615:
2616:
2617:
2618:
2619:
2620:
2621:
2622:
2623:
2624:
2625:
2626:
2627:
2628:
2629:
2630:
2631:
2632:
2633:
2634:
2635:
2636:
2637:
2638:
2639:
2640:
2641:
2642:
2643:
2644:
2645:
2646:
2647:
2648:
2649:
2650:
2651:
2652:
2653:
2654:
2655:
2656:
2657:
2658:
2659:
2660:
2661:
2662:
2663:
2664:
2665:
2666:
2667:
2668:
2669:
2670:
2671:
2672:
2673:
2674:
2675:
2676:
2677:
2678:
2679:
2680:
2681:
2682:
2683:
2684:
2685:
2686:
2687:
2688:
2689:
2690:
2691:
2692:
2693:
2694:
2695:
2696:
2697:
2698:
2699:
2700:
2701:
2702:
2703:
2704:
2705:
2706:
2707:
2708:
2709:
2710:
2711:
2712:
2713:
2714:
2715:
2716:
2717:
2718:
2719:
2720:
2721:
2722:
2723:
2724:
2725:
2726:
2727:
2728:
2729:
2730:
2731:
2732:
2733:
2734:
2735:
2736:
2737:
2738:
2739:
2740:
2741:
2742:
2743:
2744:
2745:
2746:
2747:
2748:
2749:
2750:
2751:
2752:
2753:
2754:
2755:
2756:
2757:
2758:
2759:
2760:
2761:
2762:
2763:
2764:
2765:
2766:
2767:
2768:
2769:
2770:
2771:
2772:
2773:
2774:
2775:
2776:
2777:
2778:
2779:
2780:
2781:
2782:
2783:
2784:
2785:
2786:
2787:
2788:
2789:
2790:
2791:
2792:
2793:
2794:
2795:
2796:
2797:
2798:
2799:
2800:
2801:
2802:
2803:
2804:
2805:
2806:
2807:
2808:
2809:
2810:
2811:
2812:
2813:
2814:
2815:
2816:
2817:
2818:
2819:
2820:
2821:
2822:
2823:
2824:
2825:
2826:
2827:
2828:
2829:
2830:
2831:
2832:
2833:
2834:
2835:
2836:
2837:
2838:
2839:
2840:
2841:
2842:
2843:
2844:
2845:
2846:
2847:
2848:
2849:
2850:
2851:
2852:
2853:
2854:
2855:
2856:
2857:
2858:
2859:
2860:
2861:
2862:
2863:
2864:
2865:
2866:
2867:
2868:
2869:
2870:
2871:
2872:
2873:
2874:
2875:
2876:
2877:
2878:
2879:
2880:
2881:
2882:
2883:
2884:
2885:
2886:
2887:
2888:
2889:
2890:
2891:
2892:
2893:
2894:
2895:
2896:
2897:
2898:
2899:
2900:
2901:
2902:
2903:
2904:
2905:
2906:
2907:
2908:
2909:
2910:
2911:
2912:
2913:
2914:
2915:
2916:
2917:
2918:
2919:
2920:
2921:
2922:
2923:
2924:
2925:
2926:
2927:
2928:
2929:
2930:
2931:
2932:
2933:
2934:
2935:
2936:
2937:
2938:
2939:
2940:
2941:
2942:
2943:
2944:
2945:
2946:
2947:
2948:
2949:
2950:
2951:
2952:
2953:
2954:
2955:
2956:
2957:
2958:
2959:
2960:
2961:
2962:
2963:
2964:
2965:
2966:
2967:
2968:
2969:
2970:
2971:
2972:
2973:
2974:
2975:
2976:
2977:
2978:
2979:
2980:
2981:
2982:
2983:
2984:
2985:
2986:
2987:
2988:
2989:
2990:
2991:
2992:
2993:
2994:
2995:
2996:
2997:
2998:
2999:
3000:
3001:
3002:
3003:
3004:
3005:
3006:
3007:
3008:
3009:
3010:
3011:
3012:
3013:
3014:
3015:
3016:
3017:
3018:
3019:
3020:
3021:
3022:
3023:
3024:
3025:
3026:
3027:
3028:
3029:
3030:
3031:
3032:
3033:
3034:
3035:
3036:
3037:
3038:
3039:
3040:
3041:
3042:
3043:
3044:
3045:
3046:
3047:
3048:
3049:
3050:
3051:
3052:
3053:
3054:
3055:
3056:
3057:
3058:
3059:
3060:
3061:
3062:
3063:
3064:
3065:
3066:
3067:
3068:
3069:
3070:
3071:
3072:
3073:
3074:
3075:
3076:
3077:
3078:
3079:

```

```

1682:
1683: # country:<NAME> – broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:
2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
2052: if low == "burma":
2053:     return "Myanmar"
2054: return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

origin:Kazakhstan (1 (0.0%))

Precise plain-English rule

Emitted as origin:Kazakhstan when the text has from Kazakhstan, with a comma after the country, or a demonym for Kazakhstan followed immediately by a person-origin noun.

Code text for this tag

```

130:     "Kazakhstan",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?:{:_PREPOSITIONS_})\b)[A-Za-z][a-zA-Z]*(?)"
1225: # "from <Country>," – anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:{:_PREPOSITIONS_})\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{:_PREPOSITIONS_})\s+({_PLACE})\b")
1229: # "<Place>," <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE_}\s+({_PLACE})\b")

1238: DEMONYM_TO_COUNTRY: dict[str, str] = {
1239:     "iranian": "Iran",
1240:     "mexican": "Mexico",
1241:     "venezuelan": "Venezuela",
1242:     "chinese": "China",
1243:     "russian": "Russia",
1244:     "cuban": "Cuba",
1245:     "colombian": "Colombia",
1246:     "salvadoran": "El Salvador",
1247:     "salvadorian": "El Salvador",
1248:     "honduran": "Honduras",
1249:     "guatemalan": "Guatemala",
1250:     "haitian": "Haiti",
1251:     "somali": "Somalia",
1252:     "somalian": "Somalia",
1253:     "afghan": "Afghanistan",
1254:     "syrian": "Syria",
1255:     "nigerian": "Nigeria",
1256:     "pakistani": "Pakistan",
1257:     "ukrainian": "Ukraine",
1258:     "israeli": "Israel",
1259:     "iraqi": "Iraq",
1260:     "egyptian": "Egypt",
1261:     "yemeni": "Yemen",
1262:     "brazilian": "Brazil",
1263:     "nicaraguan": "Nicaragua",
1264:     "ecuadorian": "Ecuador",
1265:     "peruvian": "Peru",
1266:     "jamaican": "Jamaica",
1267:     "filipino": "Philippines",
1268:     "vietnamese": "Vietnam",
1269:     "lebanese": "Lebanon",
1270:     "libyan": "Libya",
1271:     "sudanese": "Sudan",
1272:     "ethiopian": "Ethiopia",
1273:     "kenyan": "Kenya",
1274:     "moroccan": "Morocco",
1275:     "algerian": "Algeria",
1276:     "bangladeshi": "Bangladesh",
1277:     "cambodian": "Cambodia",
1278:     "indonesian": "Indonesia",
1279:     "dominican": "Dominican Republic",

```

```

1280: "panamanian": "Panama",
1281: "bolivian": "Bolivia",
1282: "chilean": "Chile",
1283: "argentine": "Argentina",
1284: "argentinian": "Argentina",
1285: "saudi": "Saudi Arabia",
1286: "turkish": "Turkey",
1287: }
1288: FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
1289:     "tehran": "Iran",
1290:     "caracas": "Venezuela",
1291:     "beijing": "China",
1292:     "moscow": "Russia",
1293:     "havana": "Cuba",
1294:     "kabul": "Afghanistan",
1295:     "damascus": "Syria",
1296:     "baghdad": "Iraq",
1297:     "tripoli": "Libya",
1298:     "mogadishu": "Somalia",
1299:     "bogota": "Colombia",
1300:     "managua": "Nicaragua",
1301:     "tegucigalpa": "Honduras",
1302:     "kyiv": "Ukraine",
1303: }
1304:
1305:
1306: def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:
1307:     alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
1308:     return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)
1309:
1310:
1311: PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))
1312: PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
1313: # A demonym that immediately precedes one of these nouns describes a person's
1314: # nationality, so it also earns origin:<country>, not just a contextual country:.
1315: PATTERN_DEMONYM_PERSON = _compile(
1316:     r"^\w*(?:national|nationals|citizen|citizens|immigrant|immigrants|migrant|migrants)"
1677:
1678: # origin:<COUNTRY> – validated against the sovereign-state vocab.
1679: for m in PATTERN_ORIGIN_CANDIDATE.finditer(text):
1680:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1681:     if resolved:
1682:         add(f"origin:{_normalize_country(resolved)}", span=m.span(1))
1683:
1684: # country:<NAME> – broader: any contextual country mention.
1685: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1686:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1687:     if resolved:
1688:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1689:
1690: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1691: # country, which the bare-name validators above can't see. A demonym
1692: # followed by a person noun also earns origin:.
1693: for m in PATTERN_DEMONYM.finditer(text):
1694:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1695:     if country and country.lower() in COUNTRY_LOWER:
1696:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1697:         if PATTERN_DEMONYM_PERSON.search(text[m.end(): m.end() + 24]):
1698:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1699:
1700: for m in PATTERN_FOREIGN_CITY.finditer(text):
2046:
2047: if low in ("usa", "united states"):
2048:     return "United-States"
2049:
2050: if low == "uk":
2051:     return "United-Kingdom"
2052:
2053: if low == "uae":
2054:     return "United-Arab-Emirates"
2055:
2056: if low == "burma":
2057:     return "Myanmar"
2058:
2059: return s.replace(" ", "-")
2060:
2061: def _normalize_state(s: str) -> str:
2062:     return s.replace(" ", "-")
2063:
2064:
2065: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2066:     """Resolve a candidate (any case, possibly two words) to its canonical
2067:     vocab spelling.
2068:
2069:     Tries the full phrase first, then its leading word, so a greedy
2070:     case-insensitive capture that swallowed a trailing lowercase token

```

origin:Latvia (1 (0.0%))

Precise plain-English rule

Emitted as origin:Latvia when the text has from Latvia, with a comma after the country, or a demonym for Latvia followed immediately by a person-origin noun.

Code text for this tag

```

137:     "Latvia",
1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the *next* place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-Z][a-zA-Z]*(?:\s+(?!{(?:{:_PREPOSITIONS}})\b)[A-Za-z][a-zA-Z]+)?"
1225: # "from <Country>," – anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:{:_PREPOSITIONS})\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{:_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>," <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE})\s+({_PLACE})\b")

```

```

1238: DEMONYM_TO_COUNTRY: dict[str, str] = {
1239:     "iranian": "Iran",
1240:     "mexican": "Mexico",
1241:     "venezuelan": "Venezuela",
1242:     "chinese": "China",
1243:     "russian": "Russia",
1244:     "cuban": "Cuba",
1245:     "colombian": "Colombia",
1246:     "salvadoran": "El Salvador",
1247:     "salvadorian": "El Salvador",
1248:     "honduran": "Honduras",
1249:     "guatemalan": "Guatemala",
1250:     "haitian": "Haiti",
1251:     "somali": "Somalia",
1252:     "somalian": "Somalia",
1253:     "afghan": "Afghanistan",
1254:     "syrian": "Syria",
1255:     "nigerian": "Nigeria",
1256:     "pakistani": "Pakistan",
1257:     "ukrainian": "Ukraine",
1258:     "israeli": "Israel",
1259:     "iraqi": "Iraq",
1260:     "egyptian": "Egypt",
1261:     "yemeni": "Yemen",
1262:     "brazilian": "Brazil",
1263:     "nicaraguan": "Nicaragua",
1264:     "ecuadorian": "Ecuador",
1265:     "peruvian": "Peru",
1266:     "jamaican": "Jamaica",
1267:     "filipino": "Philippines",
1268:     "vietnamese": "Vietnam",
1269:     "lebanese": "Lebanon",
1270:     "libyan": "Libya",
1271:     "sudanese": "Sudan",
1272:     "ethiopian": "Ethiopia",
1273:     "kenyan": "Kenya",
1274:     "moroccan": "Morocco",
1275:     "algerian": "Algeria",
1276:     "bangladeshi": "Bangladesh",
1277:     "cambodian": "Cambodia",
1278:     "indonesian": "Indonesia",
1279:     "dominican": "Dominican Republic",
1280:     "panamanian": "Panama",
1281:     "bolivian": "Bolivia",
1282:     "chilean": "Chile",
1283:     "argentine": "Argentina",
1284:     "argentinian": "Argentina",
1285:     "saudi": "Saudi Arabia",
1286:     "turkish": "Turkey",
1287: }
1288: FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
1289:     "tehran": "Iran",
1290:     "caracas": "Venezuela",
1291:     "beijing": "China",
1292:     "moscow": "Russia",
1293:     "havana": "Cuba",
1294:     "kabul": "Afghanistan",
1295:     "damascus": "Syria",
1296:     "baghdad": "Iraq",
1297:     "tripoli": "Libya",
1298:     "mogadishu": "Somalia",
1299:     "bogota": "Colombia",
1300:     "managua": "Nicaragua",
1301:     "tegucigalpa": "Honduras",
1302:     "kyiv": "Ukraine",
1303: }
1304:
1305:
1306: def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:
1307:     alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
1308:     return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)
1309:
1310:
1311: PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))
1312: PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
1313: # A demonym that immediately precedes one of these nouns describes a person's
1314: # nationality, so it also earns origin:country, not just a contextual country:.
1315: PATTERN_DEMONYM_PERSON = re.compile(
1316:     r"^\w*(?:national|nationals|citizen|citizens|immigrant|immigrants|migrant|migrants|"
1677:     # origin:<COUNTRY> – validated against the sovereign-state vocab.
1678:     for m in PATTERN_ORIGIN_CANDIDATE.finditer(text):
1679:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1680:         if resolved:
1681:             add(f"origin:{_normalize_country(resolved)}", span=m.span(1))
1682:
1683:     # country:<NAME> – broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:
1687:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:     # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:     # country, which the bare-name validators above can't see. A demonym
1691:     # followed by a person noun also earns origin:.
1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end(): m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
2046:
2047:     if low in ("usa", "united states"):
2048:         return "United-States"
2049:
2050:     if low == "uk":
2051:         return "United-Kingdom"
2052:
2053:     if low == "uae":
2054:         return "United-Arab-Emirates"
2055:
2056:     if low == "burma":

```

```

2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

origin:Liberia (1 (0.0%))

Precise plain-English rule

Emitted as origin:Liberia when the text has from Liberia, with a comma after the country, or a demonym for Liberia followed immediately by a person-origin noun.

Code text for this tag

```

140:     "Liberia",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-Z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS}))\b)[A-Za-z][a-zA-Z]*)"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*,")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{[_PREPOSITIONS]})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE}),\s+({_PLACE})\b")

1238: DEMONYM_TO_COUNTRY: dict[str, str] = {
1239:     "iranian": "Iran",
1240:     "mexican": "Mexico",
1241:     "venezuelan": "Venezuela",
1242:     "chinese": "China",
1243:     "russian": "Russia",
1244:     "cuban": "Cuba",
1245:     "colombian": "Colombia",
1246:     "salvadoran": "El Salvador",
1247:     "salvadorian": "El Salvador",
1248:     "honduran": "Honduras",
1249:     "guatemalan": "Guatemala",
1250:     "haitian": "Haiti",
1251:     "somali": "Somalia",
1252:     "somalian": "Somalia",
1253:     "afghan": "Afghanistan",
1254:     "syrian": "Syria",
1255:     "nigerian": "Nigeria",
1256:     "pakistani": "Pakistan",
1257:     "ukrainian": "Ukraine",
1258:     "israeli": "Israel",
1259:     "iraqi": "Iraq",
1260:     "egyptian": "Egypt",
1261:     "yemeni": "Yemen",
1262:     "brazilian": "Brazil",
1263:     "nicaraguan": "Nicaragua",
1264:     "ecuadorian": "Ecuador",
1265:     "peruvian": "Peru",
1266:     "jamaican": "Jamaica",
1267:     "filipino": "Philippines",
1268:     "vietnamese": "Vietnam",
1269:     "lebanese": "Lebanon",
1270:     "libyan": "Libya",
1271:     "sudanese": "Sudan",
1272:     "ethiopian": "Ethiopia",
1273:     "kenyan": "Kenya",
1274:     "moroccan": "Morocco",
1275:     "algerian": "Algeria",
1276:     "bangladeshi": "Bangladesh",
1277:     "cambodian": "Cambodia",
1278:     "indonesian": "Indonesia",
1279:     "dominican": "Dominican Republic",
1280:     "panamanian": "Panama",
1281:     "bolivian": "Bolivia",
1282:     "chilean": "Chile",
1283:     "argentine": "Argentina",
1284:     "argentinian": "Argentina",
1285:     "saudi": "Saudi Arabia",
1286:     "turkish": "Turkey",
1287: }
1288: FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
1289:     "tehran": "Iran",
1290:     "caracas": "Venezuela",
1291:     "beijing": "China",
1292:     "moscow": "Russia",
1293:     "havana": "Cuba",
1294:     "kabul": "Afghanistan",
1295:     "damascus": "Syria",
1296:     "baghdad": "Iraq",
1297:     "tripoli": "Libya",
1298:     "mogadishu": "Somalia",
1299:     "bogota": "Colombia",
1300:     "managua": "Nicaragua",
1301:     "tegucigalpa": "Honduras",
1302:     "kyiv": "Ukraine",
1303: }
1304:

```

```

1305:
1306: def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:
1307:     alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
1308:     return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)
1309:
1310:
1311: PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))
1312: PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
1313: # A demonym that immediately precedes one of these nouns describes a person's
1314: # nationality, so it also earns origin:<country>, not just a contextual country:.
1315: PATTERN_DEMONYM_PERSON = _compile(
1316:     r"^\w*(?:national|nationals|citizen|citizens|immigrant|immigrants|migrant|migrants|"
1677:     # origin:<COUNTRY> – validated against the sovereign-state vocab.
1678:     for m in PATTERN_ORIGIN_CANDIDATE.finditer(text):
1679:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1680:         if resolved:
1681:             add(f"origin:{_normalize_country(resolved)}", span=m.span(1))
1682:
1683:     # country:<NAME> – broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:
1687:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:     # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:     # country, which the bare-name validators above can't see. A demonym
1691:     # followed by a person noun also earns origin:.
1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056: def _normalize_state(s: str) -> str:
2057:     return s.replace(" ", "-")
2058:
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

origin:Mauritania (1 (0.0%))

Precise plain-English rule

Emitted as origin:Mauritania when the text has from Mauritania, with a comma after the country, or a demonym for Mauritania followed immediately by a person-origin noun.

Code text for this tag

```

151:     "Mauritania",
1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). '_resolve_vocab'
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?:i:{_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]*(?)"
1225: # "from <Country>" – anchors the ORIGIN validator
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s+",)
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:i:{_PREPOSITIONS})\s+({_PLACE})\b")
1229: # "<Place>, <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE})\s+({_PLACE})\b")
1238: DEMONYM_TO_COUNTRY: dict[str, str] = {
1239:     "iranian": "Iran",
1240:     "mexican": "Mexico",
1241:     "venezuelan": "Venezuela",
1242:     "chinese": "China",
1243:     "russian": "Russia",
1244:     "cuban": "Cuba",
1245:     "colombian": "Colombia",
1246:     "salvadoran": "El Salvador",
1247:     "salvadorian": "El Salvador",
1248:     "honduran": "Honduras",
1249:     "guatemalan": "Guatemala",
1250:     "haitian": "Haiti",
1251:     "somali": "Somalia",
1252:     "somalian": "Somalia",
1253:     "afghan": "Afghanistan",
1254:     "syrian": "Syria",
1255:     "nigerian": "Nigeria",
1256:     "pakistani": "Pakistan",
1257:     "ukrainian": "Ukraine",
1258:     "israeli": "Israel",
1259:     "iraqi": "Iraq",
1260:     "egyptian": "Egypt",
1261:     "yemeni": "Yemen",

```

```

1262:     "brazilian": "Brazil",
1263:     "nicaraguan": "Nicaragua",
1264:     "ecuadorian": "Ecuador",
1265:     "peruvian": "Peru",
1266:     "jamaican": "Jamaica",
1267:     "filipino": "Philippines",
1268:     "vietnamese": "Vietnam",
1269:     "lebanese": "Lebanon",
1270:     "libyan": "Libya",
1271:     "sudanese": "Sudan",
1272:     "ethiopian": "Ethiopia",
1273:     "kenyan": "Kenya",
1274:     "moroccan": "Morocco",
1275:     "algerian": "Algeria",
1276:     "bangladeshi": "Bangladesh",
1277:     "cambodian": "Cambodia",
1278:     "indonesian": "Indonesia",
1279:     "dominican": "Dominican Republic",
1280:     "panamanian": "Panama",
1281:     "bolivian": "Bolivia",
1282:     "chilean": "Chile",
1283:     "argentine": "Argentina",
1284:     "argentinian": "Argentina",
1285:     "saudi": "Saudi Arabia",
1286:     "turkish": "Turkey",
1287: }
1288: FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
1289:     "tehran": "Iran",
1290:     "caracas": "Venezuela",
1291:     "beijing": "China",
1292:     "moscow": "Russia",
1293:     "havana": "Cuba",
1294:     "kabul": "Afghanistan",
1295:     "damascus": "Syria",
1296:     "baghdad": "Iraq",
1297:     "tripoli": "Libya",
1298:     "mogadishu": "Somalia",
1299:     "bogota": "Colombia",
1300:     "managua": "Nicaragua",
1301:     "tegucigalpa": "Honduras",
1302:     "kyiv": "Ukraine",
1303: }
1304:
1305:
1306: def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:
1307:     alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
1308:     return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)
1309:
1310:
1311: PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))
1312: PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
1313: # A demonym that immediately precedes one of these nouns describes a person's
1314: # nationality, so it also earns origin:<country>, not just a contextual country:.
1315: PATTERN_DEMONYM_PERSON = _compile(
1316:     r"^\w*(?:national|nationals|citizen|citizens|immigrant|immigrants|migrant|migrants|"
1317:
1318:
1319:
1320:
1321:
1322:
1323:
1324:
1325:
1326:
1327:
1328:
1329:
1330:
1331:
1332:
1333:
1334:
1335:
1336:
1337:
1338:
1339:
1340:
1341:
1342:
1343:
1344:
1345:
1346:
1347:
1348:
1349:
1350:
1351:
1352:
1353:
1354:
1355:
1356:
1357:
1358:
1359:
1360:
1361:
1362:
1363:
1364:
1365:
1366:
1367:
1368:
1369:
1370:
1371:
1372:
1373:
1374:
1375:
1376:
1377:
1378:
1379:
1380:
1381:
1382:
1383:
1384:
1385:
1386:
1387:
1388:
1389:
1390:
1391:
1392:
1393:
1394:
1395:
1396:
1397:
1398:
1399:
1400:
1401:
1402:
1403:
1404:
1405:
1406:
1407:
1408:
1409:
1410:
1411:
1412:
1413:
1414:
1415:
1416:
1417:
1418:
1419:
1420:
1421:
1422:
1423:
1424:
1425:
1426:
1427:
1428:
1429:
1430:
1431:
1432:
1433:
1434:
1435:
1436:
1437:
1438:
1439:
1440:
1441:
1442:
1443:
1444:
1445:
1446:
1447:
1448:
1449:
1450:
1451:
1452:
1453:
1454:
1455:
1456:
1457:
1458:
1459:
1460:
1461:
1462:
1463:
1464:
1465:
1466:
1467:
1468:
1469:
1470:
1471:
1472:
1473:
1474:
1475:
1476:
1477:
1478:
1479:
1480:
1481:
1482:
1483:
1484:
1485:
1486:
1487:
1488:
1489:
1490:
1491:
1492:
1493:
1494:
1495:
1496:
1497:
1498:
1499:
1500:
1501:
1502:
1503:
1504:
1505:
1506:
1507:
1508:
1509:
1510:
1511:
1512:
1513:
1514:
1515:
1516:
1517:
1518:
1519:
1520:
1521:
1522:
1523:
1524:
1525:
1526:
1527:
1528:
1529:
1530:
1531:
1532:
1533:
1534:
1535:
1536:
1537:
1538:
1539:
1540:
1541:
1542:
1543:
1544:
1545:
1546:
1547:
1548:
1549:
1550:
1551:
1552:
1553:
1554:
1555:
1556:
1557:
1558:
1559:
1560:
1561:
1562:
1563:
1564:
1565:
1566:
1567:
1568:
1569:
1570:
1571:
1572:
1573:
1574:
1575:
1576:
1577:
1578:
1579:
1580:
1581:
1582:
1583:
1584:
1585:
1586:
1587:
1588:
1589:
1590:
1591:
1592:
1593:
1594:
1595:
1596:
1597:
1598:
1599:
1600:
1601:
1602:
1603:
1604:
1605:
1606:
1607:
1608:
1609:
1610:
1611:
1612:
1613:
1614:
1615:
1616:
1617:
1618:
1619:
1620:
1621:
1622:
1623:
1624:
1625:
1626:
1627:
1628:
1629:
1630:
1631:
1632:
1633:
1634:
1635:
1636:
1637:
1638:
1639:
1640:
1641:
1642:
1643:
1644:
1645:
1646:
1647:
1648:
1649:
1650:
1651:
1652:
1653:
1654:
1655:
1656:
1657:
1658:
1659:
1660:
1661:
1662:
1663:
1664:
1665:
1666:
1667:
1668:
1669:
1670:
1671:
1672:
1673:
1674:
1675:
1676:
1677:
1678:
1679:
1680:
1681:
1682:
1683:
1684:
1685:
1686:
1687:
1688:
1689:
1690:
1691:
1692:
1693:
1694:
1695:
1696:
1697:
1698:
1699:
1700:
1701:
1702:
1703:
1704:
1705:
1706:
1707:
1708:
1709:
1710:
1711:
1712:
1713:
1714:
1715:
1716:
1717:
1718:
1719:
1720:
1721:
1722:
1723:
1724:
1725:
1726:
1727:
1728:
1729:
1730:
1731:
1732:
1733:
1734:
1735:
1736:
1737:
1738:
1739:
1740:
1741:
1742:
1743:
1744:
1745:
1746:
1747:
1748:
1749:
1750:
1751:
1752:
1753:
1754:
1755:
1756:
1757:
1758:
1759:
1760:
1761:
1762:
1763:
1764:
1765:
1766:
1767:
1768:
1769:
1770:
1771:
1772:
1773:
1774:
1775:
1776:
1777:
1778:
1779:
1780:
1781:
1782:
1783:
1784:
1785:
1786:
1787:
1788:
1789:
1790:
1791:
1792:
1793:
1794:
1795:
1796:
1797:
1798:
1799:
1800:
1801:
1802:
1803:
1804:
1805:
1806:
1807:
1808:
1809:
1810:
1811:
1812:
1813:
1814:
1815:
1816:
1817:
1818:
1819:
1820:
1821:
1822:
1823:
1824:
1825:
1826:
1827:
1828:
1829:
1830:
1831:
1832:
1833:
1834:
1835:
1836:
1837:
1838:
1839:
1840:
1841:
1842:
1843:
1844:
1845:
1846:
1847:
1848:
1849:
1850:
1851:
1852:
1853:
1854:
1855:
1856:
1857:
1858:
1859:
1860:
1861:
1862:
1863:
1864:
1865:
1866:
1867:
1868:
1869:
1870:
1871:
1872:
1873:
1874:
1875:
1876:
1877:
1878:
1879:
1880:
1881:
1882:
1883:
1884:
1885:
1886:
1887:
1888:
1889:
1890:
1891:
1892:
1893:
1894:
1895:
1896:
1897:
1898:
1899:
1900:
1901:
1902:
1903:
1904:
1905:
1906:
1907:
1908:
1909:
1910:
1911:
1912:
1913:
1914:
1915:
1916:
1917:
1918:
1919:
1920:
1921:
1922:
1923:
1924:
1925:
1926:
1927:
1928:
1929:
1930:
1931:
1932:
1933:
1934:
1935:
1936:
1937:
1938:
1939:
1940:
1941:
1942:
1943:
1944:
1945:
1946:
1947:
1948:
1949:
1950:
1951:
1952:
1953:
1954:
1955:
1956:
1957:
1958:
1959:
1960:
1961:
1962:
1963:
1964:
1965:
1966:
1967:
1968:
1969:
1970:
1971:
1972:
1973:
1974:
1975:
1976:
1977:
1978:
1979:
1980:
1981:
1982:
1983:
1984:
1985:
1986:
1987:
1988:
1989:
1990:
1991:
1992:
1993:
1994:
1995:
1996:
1997:
1998:
1999:
2000:
2001:
2002:
2003:
2004:
2005:
2006:
2007:
2008:
2009:
2010:
2011:
2012:
2013:
2014:
2015:
2016:
2017:
2018:
2019:
2020:
2021:
2022:
2023:
2024:
2025:
2026:
2027:
2028:
2029:
2030:
2031:
2032:
2033:
2034:
2035:
2036:
2037:
2038:
2039:
2040:
2041:
2042:
2043:
2044:
2045:
2046:
2047:
2048:
2049:
2050:
2051:
2052:
2053:
2054:
2055:
2056:
2057:
2058:
2059:
2060:
2061:
2062:
2063:
2064:
2065:
2066:

```

origin:Myanmar (1 (0.0%))

Precise plain-English rule

Emitted as origin:Myanmar when the text has from Myanmar, with a comma after the country, or a demonym for Myanmar followed immediately by a person-origin noun.

Code text for this tag

```

160:     "Myanmar",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the 'next' place
1219: # ("From USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!{_{PREPOSITIONS}})\b)[A-Za-z][a-zA-Z]*(?)"
1225: # "from <Country>" - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:{_{PLACE}})\s*", "")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{_{PREPOSITIONS}})\s+({_{PLACE}})\b")
1229: # "<Place>, <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_{PLACE}},\s+({_{PLACE}})\b")

1238: DEMONYM_TO_COUNTRY: dict[str, str] = {
1239:     "iranian": "Iran",
1240:     "mexican": "Mexico",
1241:     "venezuelan": "Venezuela",
1242:     "chinese": "China",
1243:     "russian": "Russia",
1244:     "cuban": "Cuba",
1245:     "colombian": "Colombia",
1246:     "salvadoran": "El Salvador",
1247:     "salvadorian": "El Salvador",
1248:     "honduran": "Honduras",
1249:     "guatemalan": "Guatemala",
1250:     "haitian": "Haiti",
1251:     "somali": "Somalia",
1252:     "somalian": "Somalia",
1253:     "afghan": "Afghanistan",
1254:     "syrian": "Syria",
1255:     "nigerian": "Nigeria",
1256:     "pakistani": "Pakistan",
1257:     "ukrainian": "Ukraine",
1258:     "israeli": "Israel",
1259:     "iraqi": "Iraq",
1260:     "egyptian": "Egypt",
1261:     "yemeni": "Yemen",
1262:     "brazilian": "Brazil",
1263:     "nicaraguan": "Nicaragua",
1264:     "ecuadorian": "Ecuador",
1265:     "peruvian": "Peru",
1266:     "jamaican": "Jamaica",
1267:     "filipino": "Philippines",
1268:     "vietnamese": "Vietnam",
1269:     "lebanese": "Lebanon",
1270:     "libyan": "Libya",
1271:     "sudanese": "Sudan",
1272:     "ethiopian": "Ethiopia",
1273:     "kenyan": "Kenya",
1274:     "moroccan": "Morocco",
1275:     "algerian": "Algeria",
1276:     "bangladeshi": "Bangladesh",
1277:     "cambodian": "Cambodia",
1278:     "indonesian": "Indonesia",
1279:     "dominican": "Dominican Republic",
1280:     "panamanian": "Panama",
1281:     "bolivian": "Bolivia",
1282:     "chilean": "Chile",
1283:     "argentine": "Argentina",
1284:     "argentinian": "Argentina",
1285:     "saudi": "Saudi Arabia",
1286:     "turkish": "Turkey",
1287: }
1288: FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
1289:     "tehran": "Iran",
1290:     "caracas": "Venezuela",
1291:     "beijing": "China",
1292:     "moscow": "Russia",
1293:     "havana": "Cuba",
1294:     "kabul": "Afghanistan",
1295:     "damascus": "Syria",
1296:     "baghdad": "Iraq",
1297:     "tripoli": "Libya",
1298:     "mogadishu": "Somalia",
1299:     "bogota": "Colombia",
1300:     "managua": "Nicaragua",
1301:     "tegucigalpa": "Honduras",
1302:     "kyiv": "Ukraine",
1303: }
1304:
1305:
1306: def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:
1307:     alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
1308:     return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)
1309:
1310:
1311: PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))
1312: PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
1313: # A demonym that immediately precedes one of these nouns describes a person's
1314: # nationality, so it also earns origin:<country>, not just a contextual country:.
1315: PATTERN_DEMONYM_PERSON = _compile(
1316:     r"^\W*(?:national|nationals|citizen|citizens|immigrant|immigrants|migrant|migrants|"
1677: # origin:<COUNTRY> - validated against the sovereign-state vocab.
1678: for m in PATTERN_ORIGIN_CANDIDATE.finditer(text):
1679:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1680:     if resolved:
1681:         add(f"origin:{_normalize_country(resolved)}", span=m.span(1))
1682:
1683: # country:<NAME> - broader: any contextual country mention.
1684: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:     if resolved:
1687:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:

```

```

1689: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690: # country, which the bare-name validators above can't see. A demonym
1691: # followed by a person noun also earns origin:.
1692: for m in PATTERN_DEMONYM.finditer(text):
1693:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:     if country and country.lower() in COUNTRY_LOWER:
1695:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:         if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698: for m in PATTERN_FOREIGN_CITY.finditer(text):
1699:
2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
2052: if low == "burma":
2053:     return "Myanmar"
2054: return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

origin:Rwanda (1 (0.0%))

Precise plain-English rule

Emitted as origin:Rwanda when the text has from Rwanda, with a comma after the country, or a demonym for Rwanda followed immediately by a person-origin noun.

Code text for this tag

```

183:     "Rwanda",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the "next" place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]+)?"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*", "")
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{[_PREPOSITIONS]})\s+({_PLACE})\b")
1229: # "<Place>," <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE})\s+({_PLACE})\b")

1238: DEMONYM_TO_COUNTRY: dict[str, str] = {
1239:     "iranian": "Iran",
1240:     "mexican": "Mexico",
1241:     "venezuelan": "Venezuela",
1242:     "chinese": "China",
1243:     "russian": "Russia",
1244:     "cuban": "Cuba",
1245:     "colombian": "Colombia",
1246:     "salvadoran": "El Salvador",
1247:     "salvadorian": "El Salvador",
1248:     "honduran": "Honduras",
1249:     "guatemalan": "Guatemala",
1250:     "haitian": "Haiti",
1251:     "somali": "Somalia",
1252:     "somalian": "Somalia",
1253:     "afghan": "Afghanistan",
1254:     "syrian": "Syria",
1255:     "nigerian": "Nigeria",
1256:     "pakistanian": "Pakistan",
1257:     "ukrainian": "Ukraine",
1258:     "israeli": "Israel",
1259:     "iraqi": "Iraq",
1260:     "egyptian": "Egypt",
1261:     "yemeni": "Yemen",
1262:     "brazilian": "Brazil",
1263:     "nicaraguan": "Nicaragua",
1264:     "ecuadorian": "Ecuador",
1265:     "peruvian": "Peru",
1266:     "jamaican": "Jamaica",
1267:     "filipino": "Philippines",
1268:     "vietnamese": "Vietnam",
1269:     "lebanese": "Lebanon",
1270:     "libyan": "Libya",
1271:     "sudanese": "Sudan",
1272:     "ethiopian": "Ethiopia",
1273:     "kenyan": "Kenya",
1274:     "moroccan": "Morocco",
1275:     "algerian": "Algeria",
1276:     "bangladeshi": "Bangladesh",
1277:     "cambodian": "Cambodia",
1278:     "indonesian": "Indonesia",
1279:     "dominican": "Dominican Republic",
1280:     "panamanian": "Panama",
1281:     "bolivian": "Bolivia",
1282:     "chilean": "Chile",
1283:     "argentine": "Argentina",
1284:     "argentinian": "Argentina",
1285:     "saudi": "Saudi Arabia",
1286:     "turkish": "Turkey",

```

```

1287: }
1288: FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
1289:     "tehran": "Iran",
1290:     "caracas": "Venezuela",
1291:     "beijing": "China",
1292:     "moscow": "Russia",
1293:     "havana": "Cuba",
1294:     "kabul": "Afghanistan",
1295:     "damascus": "Syria",
1296:     "baghdad": "Iraq",
1297:     "tripoli": "Libya",
1298:     "mogadishu": "Somalia",
1299:     "bogota": "Colombia",
1300:     "managua": "Nicaragua",
1301:     "tegucigalpa": "Honduras",
1302:     "kyiv": "Ukraine",
1303: }
1304:
1305:
1306: def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:
1307:     alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
1308:     return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)
1309:
1310:
1311: PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))
1312: PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
1313: # A demonym that immediately precedes one of these nouns describes a person's
1314: # nationality, so it also earns origin:<country>, not just a contextual country:.
1315: PATTERN_DEMONYM_PERSON = _compile(
1316:     r"^\W*(?national|nationals|citizen|citizens|immigrant|immigrants|migrant|migrants)"
1317:
1318:
1319: # origin:<COUNTRY> – validated against the sovereign-state vocab.
1320: for m in PATTERN_ORIGIN_CANDIDATE.finditer(text):
1321:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1322:     if resolved:
1323:         add(f"origin:{_normalize_country(resolved)}", span=m.span(1))
1324:
1325:
1326: # country:<NAME> – broader: any contextual country mention.
1327: for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1328:     resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1329:     if resolved:
1330:         add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1331:
1332:
1333: # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1334: # country, which the bare-name validators above can't see. A demonym
1335: # followed by a person noun also earns origin:.
1336: for m in PATTERN_DEMONYM.finditer(text):
1337:     country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1338:     if country and country.lower() in COUNTRY_LOWER:
1339:         add(f"country:{_normalize_country(country)}", span=m.span(1))
1340:         if PATTERN_DEMONYM_PERSON.search(text[m.end(): m.end() + 24]):
1341:             add(f"origin:{_normalize_country(country)}", span=m.span(1))
1342:
1343: for m in PATTERN_FOREIGN_CITY.finditer(text):
1344:
1345:
1346: if low in ("usa", "united states"):
1347:     return "United-States"
1348: if low == "uk":
1349:     return "United-Kingdom"
1350: if low == "uae":
1351:     return "United-Arab-Emirates"
1352: if low == "burma":
1353:     return "Myanmar"
1354: return s.replace(" ", "-")
1355:
1356:
1357: def _normalize_state(s: str) -> str:
1358:     return s.replace(" ", "-")
1359:
1360:
1361: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
1362:     """Resolve a candidate (any case, possibly two words) to its canonical
1363:     vocab spelling.
1364:
1365:     Tries the full phrase first, then its leading word, so a greedy
1366:     case-insensitive capture that swallowed a trailing lowercase token

```

origin:Senegal (1 (0.0%))

Precise plain-English rule

Emitted as origin:Senegal when the text has from Senegal, with a comma after the country, or a demonym for Senegal followed immediately by a person-origin noun.

Code text for this tag

```

185:     "Senegal",
186:
187:
188: # second word excludes those same prepositions via lookahead, so a greedy
189: # capture can't swallow the preposition that introduces the "next" place
190: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). '_resolve_vocab'
191: # then validates against the vocab and falls back to the leading word for any
192: # other trailing token. Kept conservative because raw country names
193: # false-positive easily ("Chad", "Georgia").
194: _PREPOSITIONS = "from|in|to|of|with|by"
195: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!{:_PREPOSITIONS})\b)[A-Za-z][a-zA-Z]*(?)"
196: # "from <Country>," – anchors the ORIGIN validator.
197: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:{:_PREPOSITIONS})\s+({_PLACE})\s*,")
198: # Any sovereign-state mention after a preposition.
199: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:{:_PREPOSITIONS})\s+({_PLACE})\b")
200: # "<Place>," <State>" – anchors the STATE validator.
201: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE}),\s+({_PLACE})\b")
202:
203:
204: DEMONYM_TO_COUNTRY: dict[str, str] = {
205:     "iranian": "Iran",
206:     "mexican": "Mexico",
207:     "venezuelan": "Venezuela",
208:     "chinese": "China",
209:     "russian": "Russia",

```

```

1244: "cuban": "Cuba",
1245: "colombian": "Colombia",
1246: "salvadoran": "El Salvador",
1247: "salvadorian": "El Salvador",
1248: "honduran": "Honduras",
1249: "guatemalan": "Guatemala",
1250: "haitian": "Haiti",
1251: "somali": "Somalia",
1252: "somalian": "Somalia",
1253: "afghan": "Afghanistan",
1254: "syrian": "Syria",
1255: "nigerian": "Nigeria",
1256: "pakistani": "Pakistan",
1257: "ukrainian": "Ukraine",
1258: "israeli": "Israel",
1259: "iraqi": "Iraq",
1260: "egyptian": "Egypt",
1261: "yemeni": "Yemen",
1262: "brazilian": "Brazil",
1263: "nicaraguan": "Nicaragua",
1264: "ecuadorian": "Ecuador",
1265: "peruvian": "Peru",
1266: "jamaican": "Jamaica",
1267: "filipino": "Philippines",
1268: "vietnamese": "Vietnam",
1269: "lebanese": "Lebanon",
1270: "libyan": "Libya",
1271: "sudanese": "Sudan",
1272: "ethiopian": "Ethiopia",
1273: "kenyan": "Kenya",
1274: "moroccan": "Morocco",
1275: "algerian": "Algeria",
1276: "bangladeshi": "Bangladesh",
1277: "cambodian": "Cambodia",
1278: "indonesian": "Indonesia",
1279: "dominican": "Dominican Republic",
1280: "panamanian": "Panama",
1281: "bolivian": "Bolivia",
1282: "chilean": "Chile",
1283: "argentine": "Argentina",
1284: "argentinian": "Argentina",
1285: "saudi": "Saudi Arabia",
1286: "turkish": "Turkey",
1287: }
1288: FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
1289:     "tehran": "Iran",
1290:     "caracas": "Venezuela",
1291:     "beijing": "China",
1292:     "moscow": "Russia",
1293:     "havana": "Cuba",
1294:     "kabul": "Afghanistan",
1295:     "damascus": "Syria",
1296:     "baghdad": "Iraq",
1297:     "tripoli": "Libya",
1298:     "mogadishu": "Somalia",
1299:     "bogota": "Colombia",
1300:     "managua": "Nicaragua",
1301:     "tegucigalpa": "Honduras",
1302:     "kyiv": "Ukraine",
1303: }
1304:
1305:
1306: def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:
1307:     alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
1308:     return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)
1309:
1310:
1311: PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))
1312: PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
1313: # A demonym that immediately precedes one of these nouns describes a person's
1314: # nationality, so it also earns origin:<country>, not just a contextual country:.
1315: PATTERN_DEMONYM_PERSON = _compile(
1316:     r"^\w*(?:national|nationals|citizen|citizens|immigrant|immigrants|migrant|migrants|"
1677:     # origin:<COUNTRY> – validated against the sovereign-state vocab.
1678:     for m in PATTERN_ORIGIN_CANDIDATE.finditer(text):
1679:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1680:         if resolved:
1681:             add(f"origin:{_normalize_country(resolved)}", span=m.span(1))
1682:
1683:     # country:<NAME> – broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:
1687:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:     # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:     # country, which the bare-name validators above can't see. A demonym
1691:     # followed by a person noun also earns origin:.
1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):
2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
2052: if low == "burma":
2053:     return "Myanmar"
2054: return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")

```

```

2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

origin:United-Kingdom (1 (0.0%))

Precise plain-English rule

Emitted as origin:United-Kingdom when the text has from United Kingdom, with a comma after the country, or a demonym for United Kingdom followed immediately by a person-origin noun.

Code text for this tag

```

218:     "United Kingdom",

1217: # second word excludes those same prepositions via lookahead, so a greedy
1218: # capture can't swallow the preposition that introduces the *next* place
1219: # ("from USA to Mexico" -> "USA" + "Mexico", not "USA to"). _resolve_vocab`
1220: # then validates against the vocab and falls back to the leading word for any
1221: # other trailing token. Kept conservative because raw country names
1222: # false-positive easily ("Chad", "Georgia").
1223: _PREPOSITIONS = "from|in|to|of|with|by"
1224: _PLACE = rf"[A-Za-z][a-zA-Z]*(?:\s+(?!({_PREPOSITIONS}))\b[A-Za-z][a-zA-Z]+)?"
1225: # "from <Country>," - anchors the ORIGIN validator.
1226: PATTERN_ORIGIN_CANDIDATE = re.compile(rf"\b(?:from)\s+({_PLACE})\s*," )
1227: # Any sovereign-state mention after a preposition.
1228: PATTERN_COUNTRY_CANDIDATE = re.compile(rf"\b(?:({_PREPOSITIONS}))\s+({_PLACE})\b")
1229: # "<Place>,<State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b({_PLACE}),\s+({_PLACE})\b")

1238: DEMONYM_TO_COUNTRY: dict[str, str] = {
1239:     "iranian": "Iran",
1240:     "mexican": "Mexico",
1241:     "venezuelan": "Venezuela",
1242:     "chinese": "China",
1243:     "russian": "Russia",
1244:     "cuban": "Cuba",
1245:     "colombian": "Colombia",
1246:     "salvadoran": "El Salvador",
1247:     "salvadorian": "El Salvador",
1248:     "honduran": "Honduras",
1249:     "guatemalan": "Guatemala",
1250:     "haitian": "Haiti",
1251:     "somali": "Somalia",
1252:     "somalian": "Somalia",
1253:     "afghan": "Afghanistan",
1254:     "syrian": "Syria",
1255:     "nigerian": "Nigeria",
1256:     "pakistani": "Pakistan",
1257:     "ukrainian": "Ukraine",
1258:     "israeli": "Israel",
1259:     "iraqi": "Iraq",
1260:     "egyptian": "Egypt",
1261:     "yemeni": "Yemen",
1262:     "brazilian": "Brazil",
1263:     "nicaraguan": "Nicaragua",
1264:     "ecuadorian": "Ecuador",
1265:     "peruvian": "Peru",
1266:     "jamaican": "Jamaica",
1267:     "filipino": "Philippines",
1268:     "vietnamese": "Vietnam",
1269:     "lebanese": "Lebanon",
1270:     "libyan": "Libya",
1271:     "sudanese": "Sudan",
1272:     "ethiopian": "Ethiopia",
1273:     "kenyan": "Kenya",
1274:     "moroccan": "Morocco",
1275:     "algerian": "Algeria",
1276:     "bangladeshi": "Bangladesh",
1277:     "cambodian": "Cambodia",
1278:     "indonesian": "Indonesia",
1279:     "dominican": "Dominican Republic",
1280:     "panamanian": "Panama",
1281:     "bolivian": "Bolivia",
1282:     "chilean": "Chile",
1283:     "argentine": "Argentina",
1284:     "argentinian": "Argentina",
1285:     "saudi": "Saudi Arabia",
1286:     "turkish": "Turkey",
1287: }
1288: FOREIGN_CITY_TO_COUNTRY: dict[str, str] = {
1289:     "tehran": "Iran",
1290:     "caracas": "Venezuela",
1291:     "beijing": "China",
1292:     "moscow": "Russia",
1293:     "havana": "Cuba",
1294:     "kabul": "Afghanistan",
1295:     "damascus": "Syria",
1296:     "baghdad": "Iraq",
1297:     "tripoli": "Libya",
1298:     "mogadishu": "Somalia",
1299:     "bogota": "Colombia",
1300:     "managua": "Nicaragua",
1301:     "tegucigalpa": "Honduras",
1302:     "kyiv": "Ukraine",
1303: }
1304:
1305:
1306: def _alternation_pattern(keys: list[str]) -> re.Pattern[str]:
1307:     alts = sorted((re.escape(k) for k in keys), key=len, reverse=True)
1308:     return re.compile(r"\b(" + "|".join(alts) + r")\b", re.IGNORECASE)
1309:
1310:

```

```

1311: PATTERN_DEMONYM = _alternation_pattern(list(DEMONYM_TO_COUNTRY))
1312: PATTERN_FOREIGN_CITY = _alternation_pattern(list(FOREIGN_CITY_TO_COUNTRY))
1313: # A demonym that immediately precedes one of these nouns describes a person's
1314: # nationality, so it also earns origin:<country>, not just a contextual country:.
1315: PATTERN_DEMONYM_PERSON = _compile(
1316:     r"^\W*(?:(national|nationals|citizen|citizens|immigrant|immigrants|migrant|migrants)|"

1677:     # origin:<COUNTRY> – validated against the sovereign-state vocab.
1678:     for m in PATTERN_ORIGIN_CANDIDATE.finditer(text):
1679:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1680:         if resolved:
1681:             add(f"origin:{_normalize_country(resolved)}", span=m.span(1))
1682:
1683:     # country:<NAME> – broader: any contextual country mention.
1684:     for m in PATTERN_COUNTRY_CANDIDATE.finditer(text):
1685:         resolved = _resolve_vocab(m.group(1) or "", COUNTRY_BY_LOWER)
1686:         if resolved:
1687:             add(f"country:{_normalize_country(resolved)}", span=m.span(1))
1688:
1689:     # Demonyms ("Iranian regime") and major foreign cities ("in Tehran") ->
1690:     # country, which the bare-name validators above can't see. A demonym
1691:     # followed by a person noun also earns origin:.
1692:     for m in PATTERN_DEMONYM.finditer(text):
1693:         country = DEMONYM_TO_COUNTRY.get(m.group(1).lower())
1694:         if country and country.lower() in COUNTRY_LOWER:
1695:             add(f"country:{_normalize_country(country)}", span=m.span(1))
1696:             if PATTERN_DEMONYM_PERSON.search(text[m.end() : m.end() + 24]):
1697:                 add(f"origin:{_normalize_country(country)}", span=m.span(1))
1698:     for m in PATTERN_FOREIGN_CITY.finditer(text):

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
2052:     if low == "burma":
2053:         return "Myanmar"
2054:     return s.replace(" ", "-")
2055:
2056:
2057: def _normalize_state(s: str) -> str:
2058:     return s.replace(" ", "-")
2059:
2060:
2061: def _resolve_vocab(candidate: str, vocab: dict[str, str]) -> str | None:
2062:     """Resolve a candidate (any case, possibly two words) to its canonical
2063:     vocab spelling.
2064:
2065:     Tries the full phrase first, then its leading word, so a greedy
2066:     case-insensitive capture that swallowed a trailing lowercase token

```

parody

The parody namespace is produced by the franchise gazetteer. A franchise match adds both `genre:parody` and `parody:<franchise>`. Star Wars also has a two-cue rule: this is the way plus at least one other Star Wars cue.

parody:star-wars (2 (0.0%))

Precise plain-English rule

Matches May the 4th/fourth be with you, force-with-you language, Jedi/Sith/Death Star/lightsaber/Darth Vader/Skywalker, or galaxy/republic/empire/rebellion/Mandalorian within 200 characters of Jedi/Sith/Death Star/the way. Also matches this is the way plus another Star Wars cue.

Code text for this tag

```

850: PARODY_FRANCHISE_GAZETTEER: tuple[tuple[re.Pattern[str], str], ...] = (
851:     (
852:         _compile(
853:             r"\bmay\s+the\s+(?:4th|fourth)\s+be\s+with\s+you\b"
854:             r"|\bthe\s+force\s+(?:is|be|will\s+be)\s+with\b"
855:             r"|\b(?:jedi|sith|death\s+star|lightsaber|darth\s+vader|skywalker)\b"
856:             r"|\b(?:galaxy|republic|empire|rebellion|mandalorian)\b.{0,200}\b(?:jedi|sith|death\s+star|the\s+way)\b"
857:         ),
858:         "star-wars",
859:     ),
860:     (
861:         _compile(

1656:         if m := franchise_pat.search(text):
1657:             add("genre:parody", span=m.span())
1658:             add(f"parody:{franchise_slug}", span=m.span())
1659:             add("theme:pop-culture-reference")
1660:         # Also catch "this is the way" + one more Star Wars cue (e.g. "a galaxy").
1661:         if (
1662:             PATTERN_PARODY_STAR_WARS_MULTI.search(text)
1663:             and PATTERN_PARODY_STAR_WARS_EXTRA_CUE.search(text)
1664:             and not any(e["tag"] == "parody:star-wars" for e in entries)
1665:         ):
1666:             m_way = PATTERN_PARODY_STAR_WARS_MULTI.search(text)
1667:             if m_way:
1668:                 add("genre:parody", span=m_way.span())
1669:                 add("parody:star-wars", span=m_way.span())
1670:                 add("theme:pop-culture-reference")

```

parody:game-of-thrones (1 (0.0%))

Precise plain-English rule

Matches winter is coming, Game of Thrones, the iron throne, or you know nothing Jon Snow.

Code text for this tag

```
871:     ),
872:     "top-gun",
873:   ),
874:   (
875:     _compile(
876:       r"\bwinter\s+is\s+coming\b|\bgame\s+of\s+thrones\b|\bthe\s+iron\s+throne\b"
877:       r"|\byou\s+know\s+nothing,?\s+jon\s+snow\b"
878:     ),
879:     "game-of-thrones",
880:   ),
881:   (
882:     _compile(
1656:       if m := franchise_pat.search(text):
1657:         add("genre:parody", span=m.span())
1658:         add(f"parody:{franchise_slug}", span=m.span())
1659:         add("theme:pop-culture-reference")
1660:       # Also catch "this is the way" + one more Star Wars cue (e.g. "a galaxy").
1661:       if (
1662:         PATTERN_PARODY_STAR_WARS_MULTI.search(text)
1663:         and PATTERN_PARODY_STAR_WARS_EXTRA_CUE.search(text)
1664:         and not any(e["tag"] == "parody:star-wars" for e in entries)
1665:       ):
1666:         m_way = PATTERN_PARODY_STAR_WARS_MULTI.search(text)
1667:         if m_way:
1668:           add("genre:parody", span=m_way.span())
1669:           add("parody:star-wars", span=m_way.span())
1670:           add("theme:pop-culture-reference")
```

parody:ghostbusters (1 (0.0%))

Precise plain-English rule

Matches who you/ya gonna call or Ghostbusters.

Code text for this tag

```
906:     _compile(
907:       r"\bavengers\s+assemble\b|\bwakanda\s+forever\b|\binfinity\s+(?:gauntlet|stones?)\b"
908:       r"|\bi\s+am\s+iron\s+man\b|\bthanos\b"
909:     ),
910:     "marvel",
911:   ),
912:   (
913:     _compile(r"\bwho\s+(?:you|ya)\s+gonna\s+call\b|\bghostbusters\b"),
914:     "ghostbusters",
915:   ),
916:   (
917:     _compile(r"\boffer\s+(?:he|you|they)\s+can'?t\s+refuse\b|\bthe\s+godfather\b"),
1656:       if m := franchise_pat.search(text):
1657:         add("genre:parody", span=m.span())
1658:         add(f"parody:{franchise_slug}", span=m.span())
1659:         add("theme:pop-culture-reference")
1660:       # Also catch "this is the way" + one more Star Wars cue (e.g. "a galaxy").
1661:       if (
1662:         PATTERN_PARODY_STAR_WARS_MULTI.search(text)
1663:         and PATTERN_PARODY_STAR_WARS_EXTRA_CUE.search(text)
1664:         and not any(e["tag"] == "parody:star-wars" for e in entries)
1665:       ):
1666:         m_way = PATTERN_PARODY_STAR_WARS_MULTI.search(text)
1667:         if m_way:
1668:           add("genre:parody", span=m_way.span())
1669:           add("parody:star-wars", span=m_way.span())
1670:           add("theme:pop-culture-reference")
```

parody:pawn-stars (1 (0.0%))

Precise plain-English rule

Matches Pawn Stars.

Code text for this tag

```
914:     "ghostbusters",
915:   ),
916:   (
917:     _compile(r"\boffer\s+(?:he|you|they)\s+can'?t\s+refuse\b|\bthe\s+godfather\b"),
918:     "godfather",
919:   ),
920:   (
921:     _compile(r"\bpawn\s+stars\b"),
922:     "pawn-stars",
923:   ),
924: )
925: # Emit genre:parody ONLY when a franchise match fires; it never fires alone.
1656:       if m := franchise_pat.search(text):
1657:         add("genre:parody", span=m.span())
1658:         add(f"parody:{franchise_slug}", span=m.span())
1659:         add("theme:pop-culture-reference")
1660:       # Also catch "this is the way" + one more Star Wars cue (e.g. "a galaxy").
1661:       if (
1662:         PATTERN_PARODY_STAR_WARS_MULTI.search(text)
1663:         and PATTERN_PARODY_STAR_WARS_EXTRA_CUE.search(text)
1664:         and not any(e["tag"] == "parody:star-wars" for e in entries)
1665:       ):
1666:         m_way = PATTERN_PARODY_STAR_WARS_MULTI.search(text)
1667:         if m_way:
1668:           add("genre:parody", span=m_way.span())
1669:           add("parody:star-wars", span=m_way.span())
1670:           add("theme:pop-culture-reference")
```

phrase

The phrase namespace is produced by literal regexes for migrant(s) and immigrant(s). Each phrase also implies topic:immigration.

phrase:immigrant (234 (2.3%))

Precise plain-English rule

Matches immigrant/immigrants.

Code text for this tag

```
1177: PATTERN_PHRASE_IMMIGRANT = _compile(r"\bimmigrants?\b")

1586: (PATTERN_PHRASE_MIGRANT, "phrase:migrant"),
1587: (PATTERN_PHRASE_IMMIGRANT, "phrase:immigrant"),
1588: (PATTERN_THEME_STATISTICS, "format:statistics"),
```

phrase:migrant (142 (1.4%))

Precise plain-English rule

Matches migrant/migrants.

Code text for this tag

```
1176: PATTERN_PHRASE_MIGRANT = _compile(r"\bmigrants?\b")

1585: (PATTERN_SLOGAN_IMPORT_THIRD_WORLD, "slogan:import-third-world"),
1586: (PATTERN_PHRASE_MIGRANT, "phrase:migrant"),
1587: (PATTERN_PHRASE_IMMIGRANT, "phrase:immigrant"),
```

policy

The policy namespace is produced by regexes for named policy areas: border, sanctuary, worksite enforcement, and CBP Home/self-deportation.

policy:border (1,071 (10.7%))

Precise plain-English rule

Matches border, southwest border, crossing, or border wall.

Code text for this tag

```
652: PATTERN_THEME_BORDER = _compile(r"\b(border|southwest border|crossing|border wall)\b")

1551: (PATTERN_EVENT_PALESTINE, "event:palestine"),
1552: (PATTERN_THEME_BORDER, "policy:border"),
1553: (PATTERN_THEME_SANCTUARY, "policy:sanctuary-cities"),
```

policy:cbp-home (236 (2.4%))

Precise plain-English rule

Matches CBP Home, Project Homecoming, self-deport variants, free/complimentary ticket/flight home language, exit bonus, take control of departure, or go home within 120 characters of alien/migrant/CBP Home/self-deport/deport context.

Code text for this tag

```
786: PATTERN_THEME_CBP_HOME = _compile(
787:     r"\bCBP\s+Home\b"
788:     r"|\bProject\s+Homecoming\b"
789:     r"|\bself[- ]deport(?:ed|ing|ation)?\b"
790:     r"|\bfree\s+(?:flight|plane\s+ticket|ticket)\s+home\b"
791:     r"|\bcomplimentary\s+plane\s+ticket\s+home\b"
792:     r"|\bexit\s+bonus\b"
793:     r"|\btake\s+control\s+of\s+(?:your|their|his|her)\s+departure\b"
794:     r"|\b(?:illegal\s+aliens?|migrants?|CBP\s+Home|self[- ]deport|deport)\b"
795:     r"|\s\S]{0,120}\bgo\s+home\b"
796:     r"|\bgo\s+home\b[\s\S]{0,120}\b"
797:     r"|(?:illegal\s+aliens?|aliens?|migrants?|CBP\s+Home|self[- ]deport|deport)\b"
798: )

1559: (PATTERN_THEME_CIVIL_DISTURBANCE, "theme:civil-disturbance"),
1560: (PATTERN_THEME_CBP_HOME, "policy:cbp-home"),
1561: (PATTERN_THEME_POP_CULTURE_REFERENCE, "theme:pop-culture-reference"),
```

policy:sanctuary-cities (191 (1.9%))

Precise plain-English rule

Matches sanctuary city/cities, sanctuary jurisdiction, sanctuary policy/policies, or sanctuary state.

Code text for this tag

```
653: PATTERN_THEME_SANCTUARY = _compile(
654:     r"\b(sanctuary(?:cit(?:y|ies))|jurisdiction|polic(?:y|ies))|sanctuary state\b"
655: )
1552: (PATTERN_THEME_BORDER, "policy:border"),
```

```
1553:      (PATTERN_THEME_SANCTUARY, "policy:sanctuary-cities"),
1554:      (PATTERN_THEME_WORKSITE, "policy:worksite-enforcement"),
```

policy:worksite-enforcement (21 (0.2%))

Precise plain-English rule

Matches worksite, workplace, I-9, E-Verify, employer audit, or employer raid.

Code text for this tag

```
656: PATTERN_THEME_WORKSITE = _compile(r"\b(worksite|workplace|I-9|E-Verify|employer (?audit|raid))\b")
```

```
1553:      (PATTERN_THEME_SANCTUARY, "policy:sanctuary-cities"),
1554:      (PATTERN_THEME_WORKSITE, "policy:worksite-enforcement"),
1555:      (PATTERN_THEME_HOMELAND, "theme:homeland"),
```

religion

The religion namespace in this workbook contains Christianity. It is produced by a Christian-specific regex over the combined text string.

religion:christianity (85 (0.8%))

Precise plain-English rule

Matches Christian/Christianity/Christians, Judeo-Christian, Christian faith/values/church/heritage/nation, Jesus/Jesus Christ, Christ is king/our lord, Bible/biblical/scripture/scriptural, Bible book chapter:verse references, or in Jesus' name.

Code text for this tag

```
681: PATTERN_THEME_CHRISTIANITY = _compile(
682:     r"\bchristian(?:ity|s)?\b"
683:     r"|\bjudeo[- ]christian\b"
684:     r"|\bchristian\s+(?:faith|values|church|churches|heritage|nation)\b"
685:     r"|\bjesus(?:\s+christ)?\b"
686:     r"|\bchrist\s+(?:is\s+king|the\s+king|our\s+lord)\b"
687:     r"|\b(?:bible|biblical|scripture|scriptural)\b"
688:     r"|\b(?:[1-3]\s+)?(?:Genesis|Exodus|Leviticus|Numbers|Deuteronomy|Joshua|Judges|Ruth|"
689:     r"Samuel|Kings|Chronicles|Ezra|Nehemiah|Esther|Job|Psalms|Proverbs|Ecclesiastes|"
690:     r"Songs+of\s+Songs|Isaiah|Jeremiah|Lamentations|Ezekiel|Daniel|Hosea|Joel|Amos|"
691:     r"Obadiah|Jonah|Micah|Nahum|Habakkuk|Zephaniah|Haggai|Zechariah|Malachi|Matthew|"
692:     r"Mark|Luke|John|Acts|Romans|Corinthians|Galatians|Ephesians|Philippians|"
693:     r"Colossians|Thessalonians|Timothy|Titus|Philemon|Hebrews|James|Peter|Jude|"
694:     r"Revelation)\s+\d{1,3}:\d{1,3}(?:[-\u2013]\d{1,3})?\b"
695:     r"|\bin\s+jesus(?:'s|')?\s+name\b"
696: )
```

```
1556:      (PATTERN_THEME_NATIVISM, "theme:nativism"),
1557:      (PATTERN_THEME_CHRISTIANITY, "religion:christianity"),
1558:      (PATTERN_THEME_TRANSGENDER, "theme:transgender"),
```

slogan

The slogan namespace is produced by individual recurring-phrase regexes. Some are literal phrases; some require nearby context, such as go home within 120 characters of immigration/deportation language.

slogan:illegal-alien (2,287 (22.9%))

Precise plain-English rule

Matches illegal alien or illegal aliens.

Code text for this tag

```
1149: PATTERN_SLOGAN_ILLEGAL_ALIEN = _compile(r"\billegal\s+aliens?\b")
```

```
1567:      (PATTERN_SLOGAN_CRIMINAL_ILLEGAL_ALIEN, "slogan:criminal-illegal-alien"),
1568:      (PATTERN_SLOGAN_ILLEGAL_ALIEN, "slogan:illegal-alien"),
1569:      (PATTERN_SLOGAN_FREE_TICKET_HOME, "slogan:free-ticket-home"),
```

slogan:criminal-illegal-alien (1,402 (14.0%))

Precise plain-English rule

Matches criminal illegal alien or criminal illegal aliens.

Code text for this tag

```
1148: PATTERN_SLOGAN_CRIMINAL_ILLEGAL_ALIEN = _compile(r"\bcriminal\s+illegal\s+aliens?\b")
```

```
1566:      (PATTERN_SLOGAN_REPORTRECON, "slogan:reportrecon"),
1567:      (PATTERN_SLOGAN_CRIMINAL_ILLEGAL_ALIEN, "slogan:criminal-illegal-alien"),
1568:      (PATTERN_SLOGAN_ILLEGAL_ALIEN, "slogan:illegal-alien"),
```

slogan:worst (222 (2.2%))

Precise plain-English rule

Matches the literal phrase WORST OF THE WORST.

Code text for this tag

```
1146: PATTERN_SLOGAN_WORST = _compile(r"\bWORST OF THE WORST\b")

1564: (PATTERN_SLOGAN_NICE, "slogan:nice"),
1565: (PATTERN_SLOGAN_WORST, "slogan:worst"),
1566: (PATTERN_SLOGAN_REPORTRECON, "slogan:reportrecon"),
```

slogan:masa (140 (1.4%))

Precise plain-English rule

Matches Make America Safe Again.

Code text for this tag

```
1164: PATTERN_SLOGAN_MASA = _compile(r"\bMake\s+America\s+Safe\s+Again\b")

1573: (PATTERN_SLOGAN_MAHA, "slogan:maha"),
1574: (PATTERN_SLOGAN_MASA, "slogan:masa"),
1575: (PATTERN_SLOGAN_AMERICA_FIRST, "slogan:america-first"),
```

slogan:america-first (72 (0.7%))

Precise plain-English rule

Matches America First or AmericaFirst.

Code text for this tag

```
1165: PATTERN_SLOGAN_AMERICA_FIRST = _compile(r"\bAmerica\s+First\b|\bAmericaFirst\b")

1574: (PATTERN_SLOGAN_MASA, "slogan:masa"),
1575: (PATTERN_SLOGAN_AMERICA_FIRST, "slogan:america-first"),
1576: (PATTERN_SLOGAN_GOLDEN_AGE, "slogan:golden-age"),
```

slogan:maga (58 (0.6%))

Precise plain-English rule

Matches MAGA or Make America Great Again.

Code text for this tag

```
1162: PATTERN_SLOGAN_MAGA = _compile(r"\bMAGA\b|\bMake\s+America\s+Great\s+Again\b")

1571: (PATTERN_SLOGAN_PROJECT_HOMECOMING, "slogan:project-homecoming"),
1572: (PATTERN_SLOGAN_MAGA, "slogan:maga"),
1573: (PATTERN_SLOGAN_MAHA, "slogan:maha"),
```

slogan:law-and-order (51 (0.5%))

Precise plain-English rule

Matches Law and Order or Law & Order.

Code text for this tag

```
1168: PATTERN_SLOGAN_LAW_AND_ORDER = _compile(r"\bLaw\s+and\s+Order\b|\bLaw\s*&\s*Order\b")

1577: (PATTERN_SLOGAN_SAVE_AMERICA, "slogan:save-america"),
1578: (PATTERN_SLOGAN_LAW_AND_ORDER, "slogan:law-and-order"),
1579: (PATTERN_SLOGAN_PEACE_THROUGH_STRENGTH, "slogan:peace-through-strength"),
```

slogan:promises-kept (50 (0.5%))

Precise plain-English rule

Matches Promises Made, Promises Kept, with comma/semicolon/colon flexibility, or Promises Kept.

Code text for this tag

```
1170: PATTERN_SLOGAN_PROMISES_KEPT = _compile(
1171:     r"\bPromises?\s+Made[,:;]?\s+Promises?\s+Kept\b|\bPromises?\s+Kept\b"
1172: )

1579: (PATTERN_SLOGAN_PEACE_THROUGH_STRENGTH, "slogan:peace-through-strength"),
1580: (PATTERN_SLOGAN_PROMISES_KEPT, "slogan:promises-kept"),
1581: (PATTERN_SLOGAN_MASS_DEPORTATION, "slogan:mass-deportation"),
```

slogan:most-secure-border (45 (0.4%))

Precise plain-English rule

Matches most secure border or most secured border.

Code text for this tag

```
1174: PATTERN_SLOGAN_MOST_SECURE_BORDER = _compile(r"\bmost\s+secure(?:d)?\s+border\b")

1581: (PATTERN_SLOGAN_MASS_DEPORTATION, "slogan:mass-deportation"),
1582: (PATTERN_SLOGAN_MOST_SECURE_BORDER, "slogan:most-secure-border"),
1583: (PATTERN_SLOGAN_CATCH_RELEASE, "slogan:catch-release"),
```

slogan:golden-age (42 (0.4%))

Precise plain-English rule

Matches Golden Age or Welcome to the Golden Age.

Code text for this tag

```
1166: PATTERN_SLOGAN_GOLDEN_AGE = _compile(r"\bGolden\s+Age\b|\bWelcome\s+to\s+the\s+Golden\s+Age\b")

1575: (PATTERN_SLOGAN_AMERICA_FIRST, "slogan:america-first"),
1576: (PATTERN_SLOGAN_GOLDEN_AGE, "slogan:golden-age"),
1577: (PATTERN_SLOGAN_SAVE_AMERICA, "slogan:save-america"),
```

slogan:free-ticket-home (26 (0.3%))

Precise plain-English rule

Matches free ticket home, free flight home, free plane ticket home, complimentary plane ticket home, or free flight to your/their/his/her home country.

Code text for this tag

```
1150: PATTERN_SLOGAN_FREE_TICKET_HOME = _compile(
1151:     r"\bfree\s+(?:ticket|flight|plane\s+ticket)\s+home\b"
1152:     r"|\bcomplimentary\s+plane\s+ticket\s+home\b"
1153:     r"|\bfree\s+flight\s+to\s+(?:your|their|his|her)\s+home\s+country\b"
1154: )

1568: (PATTERN_SLOGAN_ILLEGAL_ALIEN, "slogan:illegal-alien"),
1569: (PATTERN_SLOGAN_FREE_TICKET_HOME, "slogan:free-ticket-home"),
1570: (PATTERN_SLOGAN_GO_HOME, "slogan:go-home"),
```

slogan:mass-deportation (26 (0.3%))

Precise plain-English rule

Matches Mass Deportation or Mass Deportations.

Code text for this tag

```
1173: PATTERN_SLOGAN_MASS_DEPORTATION = _compile(r"\bMass\s+Deportations?\b")

1580: (PATTERN_SLOGAN_PROMISES_KEPT, "slogan:promises-kept"),
1581: (PATTERN_SLOGAN_MASS_DEPORTATION, "slogan:mass-deportation"),
1582: (PATTERN_SLOGAN_MOST_SECURE_BORDER, "slogan:most-secure-border"),
```

slogan:save-america (24 (0.2%))

Precise plain-English rule

Matches Save America or SAVEAMERICA.

Code text for this tag

```
1167: PATTERN_SLOGAN_SAVE_AMERICA = _compile(r"\bSave\s+America\b|\bSAVEAMERICA\b")

1576: (PATTERN_SLOGAN_GOLDEN_AGE, "slogan:golden-age"),
1577: (PATTERN_SLOGAN_SAVE_AMERICA, "slogan:save-america"),
1578: (PATTERN_SLOGAN_LAW_AND_ORDER, "slogan:law-and-order"),
```

slogan:go-home (16 (0.2%))

Precise plain-English rule

Matches go home only when it appears within 120 characters before or after immigration/deportation context: illegal alien(s), alien(s), migrant(s), CBP Home, self-deport, or deport.

Code text for this tag

```
1155: PATTERN_SLOGAN_GO_HOME = _compile(
1156:     r"\b(?:illegal\s+aliens?|aliens?|migrants?|CBP\s+Home|self[- ]deport|deport)\b"
1157:     r"[\s\S]{0,120}\bgo\s+home\b"
1158:     r"|\bgo\s+home\b[\s\S]{0,120}\b"
1159:     r"(?:illegal\s+aliens?|aliens?|migrants?|CBP\s+Home|self[- ]deport|deport)\b"
1160: )

1569: (PATTERN_SLOGAN_FREE_TICKET_HOME, "slogan:free-ticket-home"),
1570: (PATTERN_SLOGAN_GO_HOME, "slogan:go-home"),
1571: (PATTERN_SLOGAN_PROJECT_HOMECOMING, "slogan:project-homcoming"),
```

slogan:peace-through-strength (12 (0.1%))

Precise plain-English rule

Matches Peace Through Strength.

Code text for this tag

```
1169: PATTERN_SLOGAN_PEACE_THROUGH_STRENGTH = _compile(r"\bPeace\s+Through\s+Strength\b")

1578: (PATTERN_SLOGAN_LAW_AND_ORDER, "slogan:law-and-order"),
1579: (PATTERN_SLOGAN_PEACE_THROUGH_STRENGTH, "slogan:peace-through-strength"),
1580: (PATTERN_SLOGAN_PROMISES_KEPT, "slogan:promises-kept"),
```

slogan:catch-release (11 (0.1%))

Precise plain-English rule

Matches catch-and-release or catch and release.

Code text for this tag

```
1175: PATTERN_SLOGAN_CATCH_RELEASE = _compile(r"\bcatch(?:-and-|s+and\s+)release\b")
```

```
1582: (PATTERN_SLOGAN_MOST_SECURE_BORDER, "slogan:most-secure-border"),
1583: (PATTERN_SLOGAN_CATCH_RELEASE, "slogan:catch-release"),
1584: (PATTERN_SLOGAN_FIND_AND_KILL, "slogan:find-and-kill"),
```

slogan:import-third-world (11 (0.1%))

Precise plain-English rule

Matches import the Third World or if you import the Third World.

Code text for this tag

```
838: PATTERN_SLOGAN_IMPORT_THIRD_WORLD = _compile(
839:     r"\bimport\s+the\s+third\s+world\b"
840:     r"|\bif\s+you\s+import\s+the\s+third\s+world\b"
841: )
```

```
1584: (PATTERN_SLOGAN_FIND_AND_KILL, "slogan:find-and-kill"),
1585: (PATTERN_SLOGAN_IMPORT_THIRD_WORLD, "slogan:import-third-world"),
1586: (PATTERN_PHRASE_MIGRANT, "phrase:migrant"),
```

slogan:maha (11 (0.1%))

Precise plain-English rule

Matches MAHA or Make America Healthy Again.

Code text for this tag

```
1163: PATTERN_SLOGAN_MAHA = _compile(r"\bMAHA\b|\bMake\s+America\s+Healthy\s+Again\b")
```

```
1572: (PATTERN_SLOGAN_MAGA, "slogan:maga"),
1573: (PATTERN_SLOGAN_MAHA, "slogan:maha"),
1574: (PATTERN_SLOGAN_MASA, "slogan:masa"),
```

slogan:nice (9 (0.1%))

Precise plain-English rule

Matches NICE day, NICE morning, ICE is NICE, or NICE city.

Code text for this tag

```
1145: PATTERN_SLOGAN_NICE = _compile(r"\b(NICE day|NICE morning|ICE is NICE|NICE city)\b")
```

```
1563: (PATTERN_THEME_NATIVISM_BIRTHRIGHT, "theme:nativism"),
1564: (PATTERN_SLOGAN_NICE, "slogan:nice"),
1565: (PATTERN_SLOGAN_WORST, "slogan:worst"),
```

slogan:project-homecoming (4 (0.0%))

Precise plain-English rule

Matches Project Homecoming.

Code text for this tag

```
1161: PATTERN_SLOGAN_PROJECT_HOMECOMING = _compile(r"\bProject\s+Homecoming\b")
```

```
1570: (PATTERN_SLOGAN_GO_HOME, "slogan:go-home"),
1571: (PATTERN_SLOGAN_PROJECT_HOMECOMING, "slogan:project-homecoming"),
1572: (PATTERN_SLOGAN_MAGA, "slogan:maga"),
```

slogan:find-and-kill (2 (0.0%))

Precise plain-English rule

Matches we will find you and/& kill you, with optional repeated we will before kill, or find you and/& kill you.

Code text for this tag

```
832: PATTERN_SLOGAN_FIND_AND_KILL = _compile(
833:     r"\bwe\s+will\s+find\s+you\s+(?:and|&)\s+(?:we\s+will\s+)?kill\s+you\b"
834:     r"|\bfind\s+you\s+(?:and|&)\s+kill\s+you\b"
835: )
```

```
1583: (PATTERN_SLOGAN_CATCH_RELEASE, "slogan:catch-release"),
1584: (PATTERN_SLOGAN_FIND_AND_KILL, "slogan:find-and-kill"),
1585: (PATTERN_SLOGAN_IMPORT_THIRD_WORLD, "slogan:import-third-world"),
```

slogan:reportrecon (2 (0.0%))

Precise plain-English rule

Matches Report. Recon. Raid. with flexible whitespace after the periods.

Code text for this tag

```
1147: PATTERN_SLOGAN_REPORTRECON = _compile(r"\bReport\.\s*Recon\.\s*Raid\.")
```

```
1565: (PATTERN_SLOGAN_WORST, "slogan:worst"),
1566: (PATTERN_SLOGAN_REPORTRECON, "slogan:reportrecon"),
1567: (PATTERN_SLOGAN_CRIMINAL_ILLEGAL_ALIEN, "slogan:criminal-illegal-alien"),
```

speaker

The speaker namespace is produced only when a known speaker alias appears near speech context. One pattern is alias followed within 100 characters by a speech/interview/announcement/remarks word. The other is a speech/interview/announcement/remarks word followed within 80 characters by the alias.

speaker:President Trump (392 (3.9%))

Precise plain-English rule

Matches aliases for President Trump only when the alias is within 100 characters before, or 80 characters after, speech/interview/remarks/announcement/quote/briefing context.

Code text for this tag

```
1122: (
1123:     "President Trump",
1124:     r"President\s+(?:Donald\s+J\.?\s+)?Trump|Donald\s+Trump|@?POTUS|@?realDonaldTrump",
1125: ),
1126: (
1127:     "Vice President Vance",
1128:
2202:     """Return speaker tags only when a named official is tied to speech."""
2203:     out: list[tuple[str, tuple[int, int]]] = []
2204:     for canonical, alias in SPEAKER_ALIASES:
2205:         patterns = (
2206:             rf"(?<!\w)({alias})(?!\\w).{{0,100}}\b(?:{SPEAKER_ACTION_CONTEXT}|{SPEAKER_NOUN_CONTEXT})\\b",
2207:             rf"\b(?:{SPEAKER_ACTION_CONTEXT}|{SPEAKER_NOUN_CONTEXT})\\b"
2208:             rf".{{0,80}}\b(?:by|from|with|of|featuring|:)?\s*({alias})(?!\\w)",
2209:         )
2210:         for pattern in patterns:
2211:             match = re.search(pattern, text, re.I)
2212:             if match:
2213:                 out.append((f"speaker:{canonical}", match.span(1)))
2214:                 break
2215:     return out
2216:
2217:
```

speaker:Vice President Vance (31 (0.3%))

Precise plain-English rule

Matches aliases for Vice President Vance only when the alias is within 100 characters before, or 80 characters after, speech/interview/remarks/announcement/quote/briefing context.

Code text for this tag

```
1126: (
1127:     "Vice President Vance",
1128:     r"Vice\s+President\s+(?:JD\s+|J\.D\.?\s+)?Vance|VP\s+Vance|J\.?\D\.?\s+Vance|@?VP",
1129: ),
1130: (
1131:     "Tom Homan",
1132:
2202:     """Return speaker tags only when a named official is tied to speech."""
2203:     out: list[tuple[str, tuple[int, int]]] = []
2204:     for canonical, alias in SPEAKER_ALIASES:
2205:         patterns = (
2206:             rf"(?<!\w)({alias})(?!\\w).{{0,100}}\b(?:{SPEAKER_ACTION_CONTEXT}|{SPEAKER_NOUN_CONTEXT})\\b",
2207:             rf"\b(?:{SPEAKER_ACTION_CONTEXT}|{SPEAKER_NOUN_CONTEXT})\\b"
2208:             rf".{{0,80}}\b(?:by|from|with|of|featuring|:)?\s*({alias})(?!\\w)",
2209:         )
2210:         for pattern in patterns:
2211:             match = re.search(pattern, text, re.I)
2212:             if match:
2213:                 out.append((f"speaker:{canonical}", match.span(1)))
2214:                 break
2215:     return out
2216:
2217:
```

speaker:Secretary Mullin (22 (0.2%))

Precise plain-English rule

Matches aliases for Secretary Mullin only when the alias is within 100 characters before, or 80 characters after, speech/interview/remarks/announcement/quote/briefing context.

Code text for this tag

```
1118: (
1119:     "Secretary Mullin",
11120:     r"Secretary\s+Mullin|Sec\.?\s+Mullin|@?SecMullinDHS",
11121: ),
11122: (
11123:     "President Trump",
11124:
2202:     """Return speaker tags only when a named official is tied to speech."""
2203:     out: list[tuple[str, tuple[int, int]]] = []
2204:     for canonical, alias in SPEAKER_ALIASES:
2205:         patterns = (
2206:             rf"(?<!\w)({alias})(?!\\w).{{0,100}}\b(?:{SPEAKER_ACTION_CONTEXT}|{SPEAKER_NOUN_CONTEXT})\\b",
2207:             rf"\b(?:{SPEAKER_ACTION_CONTEXT}|{SPEAKER_NOUN_CONTEXT})\\b"
2208:             rf".{{0,80}}\b(?:by|from|with|of|featuring|:)?\s*({alias})(?!\\w)",
2209:         )
2210:         for pattern in patterns:
```

speaker:Tom Homan (11 (0.1%))

speaker:First Lady Melania Trump (6 (0.1%))

speaker:Stephen Miller (3 (0.0%))

```

2214:             break
2215:         return out
2216:
2217:

```

state

The state namespace is produced by a validated '<place>, <state>' extraction. The captured state must resolve against the U.S. state vocabulary.

state:Texas (133 (1.3%))

Precise plain-English rule

Emitted as state:Texas when '<place>, Texas' is captured and the state resolves against the U.S. state vocabulary.

Code text for this tag

```

277:         "Texas",

1229: # "<Place>, <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE}\s+({_PLACE})\b")

1703:     # state:<NAME> - the "<City>, <State>" pattern, validated.
1704:     for m in PATTERN_STATE_CANDIDATE.finditer(text):
1705:         resolved = _resolve_vocab(m.group(1) or "", STATE_BY_LOWER)
1706:         if resolved:
1707:             add(f"state:{_normalize_state(resolved)}", span=m.span(1))
1708:

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"

```

state:California (89 (0.9%))

Precise plain-English rule

Emitted as state:California when '<place>, California' is captured and the state resolves against the U.S. state vocabulary.

Code text for this tag

```

239:         "California",

1229: # "<Place>, <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE}\s+({_PLACE})\b")

1703:     # state:<NAME> - the "<City>, <State>" pattern, validated.
1704:     for m in PATTERN_STATE_CANDIDATE.finditer(text):
1705:         resolved = _resolve_vocab(m.group(1) or "", STATE_BY_LOWER)
1706:         if resolved:
1707:             add(f"state:{_normalize_state(resolved)}", span=m.span(1))
1708:

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"

```

state:New-York (57 (0.6%))

Precise plain-English rule

Emitted as state:New-York when '<place>, New York' is captured and the state resolves against the U.S. state vocabulary.

Code text for this tag

```

266:         "New York",

1229: # "<Place>, <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE}\s+({_PLACE})\b")

1703:     # state:<NAME> - the "<City>, <State>" pattern, validated.
1704:     for m in PATTERN_STATE_CANDIDATE.finditer(text):
1705:         resolved = _resolve_vocab(m.group(1) or "", STATE_BY_LOWER)
1706:         if resolved:
1707:             add(f"state:{_normalize_state(resolved)}", span=m.span(1))
1708:

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"

```

state:Florida (56 (0.6%))

Precise plain-English rule

Emitted as state:Florida when '<place>, Florida' is captured and the state resolves against the U.S. state vocabulary.

Code text for this tag

```
243:     "Florida",

1229: # "<Place>, <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1703:     # state:<NAME> – the "<City>, <State>" pattern, validated.
1704:     for m in PATTERN_STATE_CANDIDATE.finditer(text):
1705:         resolved = _resolve_vocab(m.group(1) or "", STATE_BY_LOWER)
1706:         if resolved:
1707:             add(f"state:{_normalize_state(resolved)}", span=m.span(1))
1708:

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
```

state:Virginia (46 (0.5%))

Precise plain-English rule

Emitted as state:Virginia when '<place>, Virginia' is captured and the state resolves against the U.S. state vocabulary.

Code text for this tag

```
280:     "Virginia",

1229: # "<Place>, <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1703:     # state:<NAME> – the "<City>, <State>" pattern, validated.
1704:     for m in PATTERN_STATE_CANDIDATE.finditer(text):
1705:         resolved = _resolve_vocab(m.group(1) or "", STATE_BY_LOWER)
1706:         if resolved:
1707:             add(f"state:{_normalize_state(resolved)}", span=m.span(1))
1708:

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
```

state:North-Carolina (23 (0.2%))

Precise plain-English rule

Emitted as state:North-Carolina when '<place>, North Carolina' is captured and the state resolves against the U.S. state vocabulary.

Code text for this tag

```
267:     "North Carolina",

1229: # "<Place>, <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1703:     # state:<NAME> – the "<City>, <State>" pattern, validated.
1704:     for m in PATTERN_STATE_CANDIDATE.finditer(text):
1705:         resolved = _resolve_vocab(m.group(1) or "", STATE_BY_LOWER)
1706:         if resolved:
1707:             add(f"state:{_normalize_state(resolved)}", span=m.span(1))
1708:

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
```

state:Utah (22 (0.2%))

Precise plain-English rule

Emitted as state:Utah when '<place>, Utah' is captured and the state resolves against the U.S. state vocabulary.

Code text for this tag

```
278:     "Utah",

1229: # "<Place>, <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1703:     # state:<NAME> – the "<City>, <State>" pattern, validated.
1704:     for m in PATTERN_STATE_CANDIDATE.finditer(text):
1705:         resolved = _resolve_vocab(m.group(1) or "", STATE_BY_LOWER)
1706:         if resolved:
1707:             add(f"state:{_normalize_state(resolved)}", span=m.span(1))
1708:

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
```

state:Illinois (21 (0.2%))

Precise plain-English rule

Emitted as state:Illinois when '<place>, Illinois' is captured and the state resolves against the U.S. state vocabulary.

Code text for this tag

```
247: "Illinois",

1229: # "<Place>, <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1703: # state:<NAME> - the "<City>, <State>" pattern, validated.
1704: for m in PATTERN_STATE_CANDIDATE.finditer(text):
1705:     resolved = _resolve_vocab(m.group(1) or "", STATE_BY_LOWER)
1706:     if resolved:
1707:         add(f"state:{_normalize_state(resolved)}", span=m.span(1))
1708:

2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
```

state:Tennessee (16 (0.2%))

Precise plain-English rule

Emitted as state:Tennessee when '<place>, Tennessee' is captured and the state resolves against the U.S. state vocabulary.

Code text for this tag

```
276: "Tennessee",

1229: # "<Place>, <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1703: # state:<NAME> - the "<City>, <State>" pattern, validated.
1704: for m in PATTERN_STATE_CANDIDATE.finditer(text):
1705:     resolved = _resolve_vocab(m.group(1) or "", STATE_BY_LOWER)
1706:     if resolved:
1707:         add(f"state:{_normalize_state(resolved)}", span=m.span(1))
1708:

2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
```

state:Minnesota (15 (0.1%))

Precise plain-English rule

Emitted as state:Minnesota when '<place>, Minnesota' is captured and the state resolves against the U.S. state vocabulary.

Code text for this tag

```
257: "Minnesota",

1229: # "<Place>, <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1703: # state:<NAME> - the "<City>, <State>" pattern, validated.
1704: for m in PATTERN_STATE_CANDIDATE.finditer(text):
1705:     resolved = _resolve_vocab(m.group(1) or "", STATE_BY_LOWER)
1706:     if resolved:
1707:         add(f"state:{_normalize_state(resolved)}", span=m.span(1))
1708:

2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
2051:     return "United-Arab-Emirates"
```

state:Washington (15 (0.1%))

Precise plain-English rule

Emitted as state:Washington when '<place>, Washington' is captured and the state resolves against the U.S. state vocabulary.

Code text for this tag

```
281: "Washington",

1229: # "<Place>, <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1703: # state:<NAME> - the "<City>, <State>" pattern, validated.
1704: for m in PATTERN_STATE_CANDIDATE.finditer(text):
1705:     resolved = _resolve_vocab(m.group(1) or "", STATE_BY_LOWER)
1706:     if resolved:
1707:         add(f"state:{_normalize_state(resolved)}", span=m.span(1))
1708:

2046: if low in ("usa", "united states"):
2047:     return "United-States"
2048: if low == "uk":
2049:     return "United-Kingdom"
2050: if low == "uae":
```

```
2051:         return "United-Arab-Emirates"
```

state:New-Jersey (14 (0.1%))

Precise plain-English rule

Emitted as state:New-Jersey when '<place>, New Jersey' is captured and the state resolves against the U.S. state vocabulary.

Code text for this tag

```
264:         "New Jersey",

1229: # "<Place>, <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1703:     # state:<NAME> – the "<City>, <State>" pattern, validated.
1704:     for m in PATTERN_STATE_CANDIDATE.finditer(text):
1705:         resolved = _resolve_vocab(m.group(1) or "", STATE_BY_LOWER)
1706:         if resolved:
1707:             add(f"state:{_normalize_state(resolved)}", span=m.span(1))
1708:

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
```

state:Colorado (13 (0.1%))

Precise plain-English rule

Emitted as state:Colorado when '<place>, Colorado' is captured and the state resolves against the U.S. state vocabulary.

Code text for this tag

```
240:         "Colorado",

1229: # "<Place>, <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1703:     # state:<NAME> – the "<City>, <State>" pattern, validated.
1704:     for m in PATTERN_STATE_CANDIDATE.finditer(text):
1705:         resolved = _resolve_vocab(m.group(1) or "", STATE_BY_LOWER)
1706:         if resolved:
1707:             add(f"state:{_normalize_state(resolved)}", span=m.span(1))
1708:

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
```

state:Oregon (13 (0.1%))

Precise plain-English rule

Emitted as state:Oregon when '<place>, Oregon' is captured and the state resolves against the U.S. state vocabulary.

Code text for this tag

```
271:         "Oregon",

1229: # "<Place>, <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1703:     # state:<NAME> – the "<City>, <State>" pattern, validated.
1704:     for m in PATTERN_STATE_CANDIDATE.finditer(text):
1705:         resolved = _resolve_vocab(m.group(1) or "", STATE_BY_LOWER)
1706:         if resolved:
1707:             add(f"state:{_normalize_state(resolved)}", span=m.span(1))
1708:

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
```

state:Pennsylvania (13 (0.1%))

Precise plain-English rule

Emitted as state:Pennsylvania when '<place>, Pennsylvania' is captured and the state resolves against the U.S. state vocabulary.

Code text for this tag

```
272:         "Pennsylvania",

1229: # "<Place>, <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1703:     # state:<NAME> – the "<City>, <State>" pattern, validated.
1704:     for m in PATTERN_STATE_CANDIDATE.finditer(text):
1705:         resolved = _resolve_vocab(m.group(1) or "", STATE_BY_LOWER)
1706:         if resolved:
1707:             add(f"state:{_normalize_state(resolved)}", span=m.span(1))
1708:
```

```

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"

```

state:Arizona (12 (0.1%))

Precise plain-English rule

Emitted as state:Arizona when '<place>, Arizona' is captured and the state resolves against the U.S. state vocabulary.

Code text for this tag

```

237:     "Arizona",

1229: # "<Place>, <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1703:     # state:<NAME> – the "<City>, <State>" pattern, validated.
1704:     for m in PATTERN_STATE_CANDIDATE.finditer(text):
1705:         resolved = _resolve_vocab(m.group(1) or "", STATE_BY_LOWER)
1706:         if resolved:
1707:             add(f"state:{_normalize_state(resolved)}", span=m.span(1))
1708:

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"

```

state:Maryland (12 (0.1%))

Precise plain-English rule

Emitted as state:Maryland when '<place>, Maryland' is captured and the state resolves against the U.S. state vocabulary.

Code text for this tag

```

254:     "Maryland",

1229: # "<Place>, <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1703:     # state:<NAME> – the "<City>, <State>" pattern, validated.
1704:     for m in PATTERN_STATE_CANDIDATE.finditer(text):
1705:         resolved = _resolve_vocab(m.group(1) or "", STATE_BY_LOWER)
1706:         if resolved:
1707:             add(f"state:{_normalize_state(resolved)}", span=m.span(1))
1708:

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"

```

state:Massachusetts (12 (0.1%))

Precise plain-English rule

Emitted as state:Massachusetts when '<place>, Massachusetts' is captured and the state resolves against the U.S. state vocabulary.

Code text for this tag

```

255:     "Massachusetts",

1229: # "<Place>, <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1703:     # state:<NAME> – the "<City>, <State>" pattern, validated.
1704:     for m in PATTERN_STATE_CANDIDATE.finditer(text):
1705:         resolved = _resolve_vocab(m.group(1) or "", STATE_BY_LOWER)
1706:         if resolved:
1707:             add(f"state:{_normalize_state(resolved)}", span=m.span(1))
1708:

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"

```

state:Connecticut (9 (0.1%))

Precise plain-English rule

Emitted as state:Connecticut when '<place>, Connecticut' is captured and the state resolves against the U.S. state vocabulary.

Code text for this tag

```

241:     "Connecticut",

1229: # "<Place>, <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1703:     # state:<NAME> – the "<City>, <State>" pattern, validated.

```

```

1704:     for m in PATTERN_STATE_CANDIDATE.finditer(text):
1705:         resolved = _resolve_vocab(m.group(1) or "", STATE_BY_LOWER)
1706:         if resolved:
1707:             add(f"state:{_normalize_state(resolved)}", span=m.span(1))
1708:
2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"

```

state:Georgia (9 (0.1%))

Precise plain-English rule

Emitted as state:Georgia when '<place>, Georgia' is captured and the state resolves against the U.S. state vocabulary.

Code text for this tag

```

244:     "Georgia",

1229: # "<Place>, <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1703:     # state:<NAME> – the "<City>, <State>" pattern, validated.
1704:     for m in PATTERN_STATE_CANDIDATE.finditer(text):
1705:         resolved = _resolve_vocab(m.group(1) or "", STATE_BY_LOWER)
1706:         if resolved:
1707:             add(f"state:{_normalize_state(resolved)}", span=m.span(1))
1708:
2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"

```

state:Louisiana (7 (0.1%))

Precise plain-English rule

Emitted as state:Louisiana when '<place>, Louisiana' is captured and the state resolves against the U.S. state vocabulary.

Code text for this tag

```

252:     "Louisiana",

1229: # "<Place>, <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1703:     # state:<NAME> – the "<City>, <State>" pattern, validated.
1704:     for m in PATTERN_STATE_CANDIDATE.finditer(text):
1705:         resolved = _resolve_vocab(m.group(1) or "", STATE_BY_LOWER)
1706:         if resolved:
1707:             add(f"state:{_normalize_state(resolved)}", span=m.span(1))
1708:
2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"

```

state:Michigan (7 (0.1%))

Precise plain-English rule

Emitted as state:Michigan when '<place>, Michigan' is captured and the state resolves against the U.S. state vocabulary.

Code text for this tag

```

256:     "Michigan",

1229: # "<Place>, <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1703:     # state:<NAME> – the "<City>, <State>" pattern, validated.
1704:     for m in PATTERN_STATE_CANDIDATE.finditer(text):
1705:         resolved = _resolve_vocab(m.group(1) or "", STATE_BY_LOWER)
1706:         if resolved:
1707:             add(f"state:{_normalize_state(resolved)}", span=m.span(1))
1708:
2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"

```

state:South-Carolina (7 (0.1%))

Precise plain-English rule

Emitted as state:South-Carolina when '<place>, South Carolina' is captured and the state resolves against the U.S. state vocabulary.

Code text for this tag

```

274:     "South Carolina",

1229: # "<Place>, <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1703:     # state:<NAME> – the "<City>, <State>" pattern, validated.
1704:     for m in PATTERN_STATE_CANDIDATE.finditer(text):
1705:         resolved = _resolve_vocab(m.group(1) or "", STATE_BY_LOWER)
1706:         if resolved:
1707:             add(f"state:{_normalize_state(resolved)}", span=m.span(1))
1708:

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"

```

state:Missouri (6 (0.1%))

Precise plain-English rule

Emitted as state:Missouri when '<place>, Missouri' is captured and the state resolves against the U.S. state vocabulary.

Code text for this tag

```

259:     "Missouri",

1229: # "<Place>, <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1703:     # state:<NAME> – the "<City>, <State>" pattern, validated.
1704:     for m in PATTERN_STATE_CANDIDATE.finditer(text):
1705:         resolved = _resolve_vocab(m.group(1) or "", STATE_BY_LOWER)
1706:         if resolved:
1707:             add(f"state:{_normalize_state(resolved)}", span=m.span(1))
1708:

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"

```

state:Nevada (6 (0.1%))

Precise plain-English rule

Emitted as state:Nevada when '<place>, Nevada' is captured and the state resolves against the U.S. state vocabulary.

Code text for this tag

```

262:     "Nevada",

1229: # "<Place>, <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1703:     # state:<NAME> – the "<City>, <State>" pattern, validated.
1704:     for m in PATTERN_STATE_CANDIDATE.finditer(text):
1705:         resolved = _resolve_vocab(m.group(1) or "", STATE_BY_LOWER)
1706:         if resolved:
1707:             add(f"state:{_normalize_state(resolved)}", span=m.span(1))
1708:

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"

```

state:New-Hampshire (6 (0.1%))

Precise plain-English rule

Emitted as state:New-Hampshire when '<place>, New Hampshire' is captured and the state resolves against the U.S. state vocabulary.

Code text for this tag

```

263:     "New Hampshire",

1229: # "<Place>, <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1703:     # state:<NAME> – the "<City>, <State>" pattern, validated.
1704:     for m in PATTERN_STATE_CANDIDATE.finditer(text):
1705:         resolved = _resolve_vocab(m.group(1) or "", STATE_BY_LOWER)
1706:         if resolved:
1707:             add(f"state:{_normalize_state(resolved)}", span=m.span(1))
1708:

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"

```

state:Wisconsin (6 (0.1%))

Precise plain-English rule

Emitted as state:Wisconsin when '<place>, Wisconsin' is captured and the state resolves against the U.S. state vocabulary.

Code text for this tag

```
283:     "Wisconsin",

1229: # "<Place>, <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1703:     # state:<NAME> – the "<City>, <State>" pattern, validated.
1704:     for m in PATTERN_STATE_CANDIDATE.finditer(text):
1705:         resolved = _resolve_vocab(m.group(1) or "", STATE_BY_LOWER)
1706:         if resolved:
1707:             add(f"state:{_normalize_state(resolved)}", span=m.span(1))
1708:

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
```

state:Alabama (5 (0.0%))

Precise plain-English rule

Emitted as state:Alabama when '<place>, Alabama' is captured and the state resolves against the U.S. state vocabulary.

Code text for this tag

```
235:     "Alabama",

1229: # "<Place>, <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1703:     # state:<NAME> – the "<City>, <State>" pattern, validated.
1704:     for m in PATTERN_STATE_CANDIDATE.finditer(text):
1705:         resolved = _resolve_vocab(m.group(1) or "", STATE_BY_LOWER)
1706:         if resolved:
1707:             add(f"state:{_normalize_state(resolved)}", span=m.span(1))
1708:

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
```

state:Idaho (5 (0.0%))

Precise plain-English rule

Emitted as state:Idaho when '<place>, Idaho' is captured and the state resolves against the U.S. state vocabulary.

Code text for this tag

```
246:     "Idaho",

1229: # "<Place>, <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1703:     # state:<NAME> – the "<City>, <State>" pattern, validated.
1704:     for m in PATTERN_STATE_CANDIDATE.finditer(text):
1705:         resolved = _resolve_vocab(m.group(1) or "", STATE_BY_LOWER)
1706:         if resolved:
1707:             add(f"state:{_normalize_state(resolved)}", span=m.span(1))
1708:

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
```

state:Iowa (5 (0.0%))

Precise plain-English rule

Emitted as state:Iowa when '<place>, Iowa' is captured and the state resolves against the U.S. state vocabulary.

Code text for this tag

```
249:     "Iowa",

1229: # "<Place>, <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1703:     # state:<NAME> – the "<City>, <State>" pattern, validated.
1704:     for m in PATTERN_STATE_CANDIDATE.finditer(text):
1705:         resolved = _resolve_vocab(m.group(1) or "", STATE_BY_LOWER)
1706:         if resolved:
1707:             add(f"state:{_normalize_state(resolved)}", span=m.span(1))
1708:

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
```

```
2051:         return "United-Arab-Emirates"
```

state:Vermont (5 (0.0%))

Precise plain-English rule

Emitted as state:Vermont when '<place>, Vermont' is captured and the state resolves against the U.S. state vocabulary.

Code text for this tag

```
279:         "Vermont",

1229: # "<Place>, <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1703:     # state:<NAME> – the "<City>, <State>" pattern, validated.
1704:     for m in PATTERN_STATE_CANDIDATE.finditer(text):
1705:         resolved = _resolve_vocab(m.group(1) or "", STATE_BY_LOWER)
1706:         if resolved:
1707:             add(f"state:{_normalize_state(resolved)}", span=m.span(1))
1708:

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
```

state:Indiana (4 (0.0%))

Precise plain-English rule

Emitted as state:Indiana when '<place>, Indiana' is captured and the state resolves against the U.S. state vocabulary.

Code text for this tag

```
248:         "Indiana",

1229: # "<Place>, <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1703:     # state:<NAME> – the "<City>, <State>" pattern, validated.
1704:     for m in PATTERN_STATE_CANDIDATE.finditer(text):
1705:         resolved = _resolve_vocab(m.group(1) or "", STATE_BY_LOWER)
1706:         if resolved:
1707:             add(f"state:{_normalize_state(resolved)}", span=m.span(1))
1708:

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
```

state:Kansas (4 (0.0%))

Precise plain-English rule

Emitted as state:Kansas when '<place>, Kansas' is captured and the state resolves against the U.S. state vocabulary.

Code text for this tag

```
250:         "Kansas",

1229: # "<Place>, <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1703:     # state:<NAME> – the "<City>, <State>" pattern, validated.
1704:     for m in PATTERN_STATE_CANDIDATE.finditer(text):
1705:         resolved = _resolve_vocab(m.group(1) or "", STATE_BY_LOWER)
1706:         if resolved:
1707:             add(f"state:{_normalize_state(resolved)}", span=m.span(1))
1708:

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"
```

state:Mississippi (4 (0.0%))

Precise plain-English rule

Emitted as state:Mississippi when '<place>, Mississippi' is captured and the state resolves against the U.S. state vocabulary.

Code text for this tag

```
258:         "Mississippi",

1229: # "<Place>, <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1703:     # state:<NAME> – the "<City>, <State>" pattern, validated.
1704:     for m in PATTERN_STATE_CANDIDATE.finditer(text):
1705:         resolved = _resolve_vocab(m.group(1) or "", STATE_BY_LOWER)
1706:         if resolved:
1707:             add(f"state:{_normalize_state(resolved)}", span=m.span(1))
1708:
```

```

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"

```

state:Kentucky (3 (0.0%))

Precise plain-English rule

Emitted as state:Kentucky when '<place>, Kentucky' is captured and the state resolves against the U.S. state vocabulary.

Code text for this tag

```

251:     "Kentucky",

1229: # "<Place>, <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1703:     # state:<NAME> – the "<City>, <State>" pattern, validated.
1704:     for m in PATTERN_STATE_CANDIDATE.finditer(text):
1705:         resolved = _resolve_vocab(m.group(1) or "", STATE_BY_LOWER)
1706:         if resolved:
1707:             add(f"state:{_normalize_state(resolved)}", span=m.span(1))
1708:

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"

```

state:New-Mexico (3 (0.0%))

Precise plain-English rule

Emitted as state:New-Mexico when '<place>, New Mexico' is captured and the state resolves against the U.S. state vocabulary.

Code text for this tag

```

265:     "New Mexico",

1229: # "<Place>, <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1703:     # state:<NAME> – the "<City>, <State>" pattern, validated.
1704:     for m in PATTERN_STATE_CANDIDATE.finditer(text):
1705:         resolved = _resolve_vocab(m.group(1) or "", STATE_BY_LOWER)
1706:         if resolved:
1707:             add(f"state:{_normalize_state(resolved)}", span=m.span(1))
1708:

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"

```

state:Oklahoma (3 (0.0%))

Precise plain-English rule

Emitted as state:Oklahoma when '<place>, Oklahoma' is captured and the state resolves against the U.S. state vocabulary.

Code text for this tag

```

270:     "Oklahoma",

1229: # "<Place>, <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1703:     # state:<NAME> – the "<City>, <State>" pattern, validated.
1704:     for m in PATTERN_STATE_CANDIDATE.finditer(text):
1705:         resolved = _resolve_vocab(m.group(1) or "", STATE_BY_LOWER)
1706:         if resolved:
1707:             add(f"state:{_normalize_state(resolved)}", span=m.span(1))
1708:

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"

```

state:Hawaii (2 (0.0%))

Precise plain-English rule

Emitted as state:Hawaii when '<place>, Hawaii' is captured and the state resolves against the U.S. state vocabulary.

Code text for this tag

```

245:     "Hawaii",

1229: # "<Place>, <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1703:     # state:<NAME> – the "<City>, <State>" pattern, validated.

```

```

1704:     for m in PATTERN_STATE_CANDIDATE.finditer(text):
1705:         resolved = _resolve_vocab(m.group(1) or "", STATE_BY_LOWER)
1706:         if resolved:
1707:             add(f"state:{_normalize_state(resolved)}", span=m.span(1))
1708:
2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"

```

state:Ohio (2 (0.0%))

Precise plain-English rule

Emitted as state:Ohio when '<place>, Ohio' is captured and the state resolves against the U.S. state vocabulary.

Code text for this tag

```

269:     "Ohio",

1229: # "<Place>, <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1703:     # state:<NAME> – the "<City>, <State>" pattern, validated.
1704:     for m in PATTERN_STATE_CANDIDATE.finditer(text):
1705:         resolved = _resolve_vocab(m.group(1) or "", STATE_BY_LOWER)
1706:         if resolved:
1707:             add(f"state:{_normalize_state(resolved)}", span=m.span(1))
1708:
2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"

```

state:Alaska (1 (0.0%))

Precise plain-English rule

Emitted as state:Alaska when '<place>, Alaska' is captured and the state resolves against the U.S. state vocabulary.

Code text for this tag

```

236:     "Alaska",

1229: # "<Place>, <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1703:     # state:<NAME> – the "<City>, <State>" pattern, validated.
1704:     for m in PATTERN_STATE_CANDIDATE.finditer(text):
1705:         resolved = _resolve_vocab(m.group(1) or "", STATE_BY_LOWER)
1706:         if resolved:
1707:             add(f"state:{_normalize_state(resolved)}", span=m.span(1))
1708:
2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"

```

state:Arkansas (1 (0.0%))

Precise plain-English rule

Emitted as state:Arkansas when '<place>, Arkansas' is captured and the state resolves against the U.S. state vocabulary.

Code text for this tag

```

238:     "Arkansas",

1229: # "<Place>, <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1703:     # state:<NAME> – the "<City>, <State>" pattern, validated.
1704:     for m in PATTERN_STATE_CANDIDATE.finditer(text):
1705:         resolved = _resolve_vocab(m.group(1) or "", STATE_BY_LOWER)
1706:         if resolved:
1707:             add(f"state:{_normalize_state(resolved)}", span=m.span(1))
1708:
2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"

```

state:Delaware (1 (0.0%))

Precise plain-English rule

Emitted as state:Delaware when '<place>, Delaware' is captured and the state resolves against the U.S. state vocabulary.

Code text for this tag

```

242:     "Delaware",

1229: # "<Place>, <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1703:     # state:<NAME> – the "<City>, <State>" pattern, validated.
1704:     for m in PATTERN_STATE_CANDIDATE.finditer(text):
1705:         resolved = _resolve_vocab(m.group(1) or "", STATE_BY_LOWER)
1706:         if resolved:
1707:             add(f"state:{_normalize_state(resolved)}", span=m.span(1))
1708:

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"

```

state:Maine (1 (0.0%))

Precise plain-English rule

Emitted as state:Maine when '<place>, Maine' is captured and the state resolves against the U.S. state vocabulary.

Code text for this tag

```

253:     "Maine",

1229: # "<Place>, <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1703:     # state:<NAME> – the "<City>, <State>" pattern, validated.
1704:     for m in PATTERN_STATE_CANDIDATE.finditer(text):
1705:         resolved = _resolve_vocab(m.group(1) or "", STATE_BY_LOWER)
1706:         if resolved:
1707:             add(f"state:{_normalize_state(resolved)}", span=m.span(1))
1708:

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"

```

state:Nebraska (1 (0.0%))

Precise plain-English rule

Emitted as state:Nebraska when '<place>, Nebraska' is captured and the state resolves against the U.S. state vocabulary.

Code text for this tag

```

261:     "Nebraska",

1229: # "<Place>, <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1703:     # state:<NAME> – the "<City>, <State>" pattern, validated.
1704:     for m in PATTERN_STATE_CANDIDATE.finditer(text):
1705:         resolved = _resolve_vocab(m.group(1) or "", STATE_BY_LOWER)
1706:         if resolved:
1707:             add(f"state:{_normalize_state(resolved)}", span=m.span(1))
1708:

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"

```

state:North-Dakota (1 (0.0%))

Precise plain-English rule

Emitted as state:North-Dakota when '<place>, North Dakota' is captured and the state resolves against the U.S. state vocabulary.

Code text for this tag

```

268:     "North Dakota",

1229: # "<Place>, <State>" – anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{_PLACE},\s+({_PLACE})\b")

1703:     # state:<NAME> – the "<City>, <State>" pattern, validated.
1704:     for m in PATTERN_STATE_CANDIDATE.finditer(text):
1705:         resolved = _resolve_vocab(m.group(1) or "", STATE_BY_LOWER)
1706:         if resolved:
1707:             add(f"state:{_normalize_state(resolved)}", span=m.span(1))
1708:

2046:     if low in ("usa", "united states"):
2047:         return "United-States"
2048:     if low == "uk":
2049:         return "United-Kingdom"
2050:     if low == "uae":
2051:         return "United-Arab-Emirates"

```

state:South-Dakota (1 (0.0%))

Precise plain-English rule

Emitted as state:South-Dakota when '<place>, South Dakota' is captured and the state resolves against the U.S. state vocabulary.

Code text for this tag

```
275:      "South Dakota",

1229: # "<Place>, <State>" - anchors the STATE validator.
1230: PATTERN_STATE_CANDIDATE = re.compile(rf"\b{ _PLACE },\s+({ _PLACE })\b")

1703:      # state:<NAME> - the "<City>, <State>" pattern, validated.
1704:      for m in PATTERN_STATE_CANDIDATE.finditer(text):
1705:          resolved = _resolve_vocab(m.group(1) or "", STATE_BY_LOWER)
1706:          if resolved:
1707:              add(f"state:{_normalize_state(resolved)}", span=m.span(1))
1708:

2046:      if low in ("usa", "united states"):
2047:          return "United-States"
2048:      if low == "uk":
2049:          return "United-Kingdom"
2050:      if low == "uae":
2051:          return "United-Arab-Emirates"
```

subject

The subject namespace is produced by direct subject regexes plus deterministic composites. subject:enforcement-op is added only when action:detention and theme:criminal have already both fired.

subject:enforcement-op (1,351 (13.5%))

Precise plain-English rule

Composite rule: action:detention and theme:criminal have both already fired.

Code text for this tag

```
1717:      # subject:enforcement-op heuristic from the deterministic rules above:
1718:      # if both action:detention and theme:criminal fire, that's strong enough
1719:      # to set this without tentative.
1720:      if any(e["tag"] == "action:detention" for e in entries) and any(
1721:          e["tag"] == "theme:criminal" for e in entries
1722:      ):
1723:          add("subject:enforcement-op")
1724:
1725:      # video:<kind> + video:duration-bucket - only when a video is attached.
```

subject:cbp-home-app (189 (1.9%))

Precise plain-English rule

Matches CBP Home App, CBP Home, DHS.GOV/CBPHOME, or DHS/CBP URLs containing cbp-home/cbphome/projecthomecoming.

Code text for this tag

```
780: PATTERN_SUBJECT_CBP_HOME_APP = _compile(
781:     r"\b(?:CBP\s+Home\s+App\b"
782:     r"|\b(?:CBP\s+Home\b"
783:     r"|\bDHS\.?GOV/CBPHOME\b"
784:     r"|\b(?:dhs|cbp)\.gov/(?:cbp|[_]home|projecthomecoming)\b"
785: )

1543:     (PATTERN_ACTION_REPORT_TO_ICE, "action:report-immigrants"),
1544:     (PATTERN_SUBJECT_CBP_HOME_APP, "subject:cbp-home-app"),
1545:     (PATTERN_SUBJECT_CELIBRITY, "subject:celebrity"),
```

subject:angel-family (54 (0.5%))

Precise plain-English rule

Matches angel family/families/mom/dad/parent/mother/father/wife/husband/son/daughter/child.

Code text for this tag

```
1209: PATTERN_ANGEL_FAMILY = _compile(
1210:     r"\bangel(?:famil(?:y|ies)|mom|dad|parent|mother|father|wife|husband|son|daughter|child)\b"
1211: )

1589:     (PATTERN_THEME_DIRECTIVE, "theme:directive"),
1590:     (PATTERN_ANGEL_FAMILY, "subject:angel-family"),
1591:     (PATTERN_NATIVE_BORN_CITIZEN, "subject:native-born-citizen"),
```

subject:native-born-citizen (3 (0.0%))

Precise plain-English rule

Matches native-born citizens/Americans/U.S. citizens/people/workers/taxpayers.

Code text for this tag

```
1212: PATTERN_NATIVE_BORN_CITIZEN = _compile(
1213:     r"\bnative[- ]born\s+(?:citizens?|americans?|u\.?s\.?s+citizens?|people|workers|taxpayers)\b"
1214: )

1590:     (PATTERN_ANGEL_FAMILY, "subject:angel-family"),
1591:     (PATTERN_NATIVE_BORN_CITIZEN, "subject:native-born-citizen"),
1592: ):
```

subject:celebrity (2 (0.0%))

Precise plain-English rule

Matches Sydney Sweeney, a celebrity-title phrase followed by a capitalized full name, or a capitalized full name followed by actress/influencer/celebrity/pop star/movie star.

Code text for this tag

```
799: PATTERN_SUBJECT_CELIBRITY = _compile(
800:     r"\bSydney\S+Sweeney\b"
801:     r"|\b(?:actress|influencer|celebrity|pop\s+star|movie\s+star|Hollywood\s+star)"
802:     r"\s+[A-Z][a-z]+(?:\s+[A-Z][a-z]+)+\b"
803:     r"|\b[A-Z][a-z]+(?:\s+[A-Z][a-z]+)+,\s+(?:an?\s+)"
804:     r"?(?:actress|influencer|celebrity|pop\s+star|movie\s+star)\b"
805: )

1544: (PATTERN_SUBJECT_CBP_HOME_APP, "subject:cbp-home-app"),
1545: (PATTERN_SUBJECT_CELIBRITY, "subject:celebrity"),
1546: (PATTERN_LEGAL_CRIMINAL_PROSECUTION, "legal:criminal-prosecution"),
```

subject:detainee (1 (0.0%))

Precise plain-English rule

No deterministic lexical emitter in scripts/tag_lexical.py; this appears only if imported from a vetted sidecar/manual tag source and normalized.

Code text for this tag

```
1364: "media:short-video": ("video:short",),
1365: # Namespace-migration aliases (media:* status flags -> media-status:*)
1366: "media:archived": ("media-status:archived",),
1367: "media:described": ("media-status:described",),
1368: "media:has-alt-text": ("media-status:has-alt-text",),
1369: "media:needs-vision": ("media-status:needs-vision",),
1370: "media:needs-ocr": ("media-status:needs-ocr",),
1371: "media:graphic-content": ("media-status:graphic-content",),
1372: # Namespace-migration aliases (theme:*/frame: -> policy:/format:/theme:)
1373: "theme:border": ("policy:border",),
1374: "theme:sanctuary-cities": ("policy:sanctuary-cities",),
1375: "theme:worksite-enforcement": ("policy:worksite-enforcement",),
1376: "theme:cbp-home": ("policy:cbp-home",),
1377: "theme:statistics": ("format:statistics",),
1378: "format:directive": ("theme:directive",),
1379: "frame:criminal": ("theme:criminal",),
1380: }
1381:
1382:
1383: # -----
1384: # Per-tweet tagger.
1385: # -----
1386:
1387:
1388: def normalized_media_tags(media_tags: list[Any] | None) -> list[dict[str, Any]]:
1389:     """Return vetted tag entries imported from media-recognition sidecars."""
1390:     out: list[dict[str, Any]] = []
1391:     seen: set[str] = set()
1392:     if not isinstance(media_tags, list):
1393:         return out
1394:     for entry in media_tags:
1395:         if isinstance(entry, str):
1396:             tag = entry.strip()
1397:             source = "media-description"
1398:             tentative = False
1399:         elif isinstance(entry, dict):
1400:             tag = str(entry.get("tag") or "").strip()
1401:             source = str(entry.get("source") or "media-description")
1402:             tentative = bool(entry.get("tentative"))
1403:         else:
1404:             continue
1405:         if not tag or tag in seen:
1406:             continue
1407:         aliased_tags = LEGACY_MEDIA_TAG_ALIASES.get(tag, (tag,))
1408:         for aliased_tag in aliased_tags:
1409:             if aliased_tag in seen:
1410:                 continue
1411:             if not any(
1412:                 aliased_tag.startswith(prefix) for prefix in MEDIA_TAG_PREFIXES_ALLOWED_IN_LEXICAL
1413:             ):
1414:                 continue
1415:             seen.add(aliased_tag)
```

This tag has no hard-coded lexical emitter in scripts/tag_lexical.py; the code above imports vetted sidecar/manual media tags when present.

subject:official (1 (0.0%))

Precise plain-English rule

No deterministic lexical emitter in scripts/tag_lexical.py; this appears only if imported from a vetted sidecar/manual tag source and normalized.

Code text for this tag

```
1364: "media:short-video": ("video:short",),
1365: # Namespace-migration aliases (media:* status flags -> media-status:*)
1366: "media:archived": ("media-status:archived",),
1367: "media:described": ("media-status:described",),
1368: "media:has-alt-text": ("media-status:has-alt-text",),
1369: "media:needs-vision": ("media-status:needs-vision",),
1370: "media:needs-ocr": ("media-status:needs-ocr",),
1371: "media:graphic-content": ("media-status:graphic-content",),
1372: # Namespace-migration aliases (theme:*/frame: -> policy:/format:/theme:)
1373: "theme:border": ("policy:border",),
1374: "theme:sanctuary-cities": ("policy:sanctuary-cities",),
```

```

1375:     "theme:worksite-enforcement": ("policy:worksite-enforcement",),
1376:     "theme:cbp-home": ("policy:cbp-home",),
1377:     "theme:statistics": ("format:statistics",),
1378:     "format:directive": ("theme:directive",),
1379:     "frame:criminal": ("theme:criminal",),
1380: }
1381:
1382:
1383: # -----
1384: # Per-tweet tagger.
1385: # -----
1386:
1387:
1388: def normalized_media_tags(media_tags: list[Any] | None) -> list[dict[str, Any]]:
1389:     """Return vetted tag entries imported from media-recognition sidecars."""
1390:     out: list[dict[str, Any]] = []
1391:     seen: set[str] = set()
1392:     if not isinstance(media_tags, list):
1393:         return out
1394:     for entry in media_tags:
1395:         if isinstance(entry, str):
1396:             tag = entry.strip()
1397:             source = "media-description"
1398:             tentative = False
1399:         elif isinstance(entry, dict):
1400:             tag = str(entry.get("tag") or "").strip()
1401:             source = str(entry.get("source") or "media-description")
1402:             tentative = bool(entry.get("tentative"))
1403:         else:
1404:             continue
1405:         if not tag or tag in seen:
1406:             continue
1407:         aliased_tags = LEGACY_MEDIA_TAG_ALIASES.get(tag, (tag,))
1408:         for aliased_tag in aliased_tags:
1409:             if aliased_tag in seen:
1410:                 continue
1411:             if not any(
1412:                 aliased_tag.startswith(prefix) for prefix in MEDIA_TAG_PREFIXES_ALLOWED_IN_LEXICAL
1413:             ):
1414:                 continue
1415:             seen.add(aliased_tag)

```

This tag has no hard-coded lexical emitter in scripts/tag_lexical.py; the code above imports vetted sidecar/manual media tags when present.

theme

The theme namespace is produced by rhetorical-frame regexes and guard functions. Those helpers handle expletive suppression for religion, context windows for coded nativism, and context windows for martyrdom.

theme:criminal (2,782 (27.8%))

Precise plain-English rule

Matches criminal illegal alien(s), illegal alien(s), criminal alien(s), aggravated felon(s), or convicted for/of.

Code text for this tag

```

515: PATTERN_FRAME_CRIMINAL = _compile(
516:     r"\b(criminal illegal aliens?|illegal aliens?|criminal aliens?|aggravated felons?|convicted (?:(for|of))\b"
517: )

1538:     for pat, tag in (
1539:         (PATTERN_FRAME_CRIMINAL, "theme:criminal"),
1540:         (PATTERN_ACTION_DETENTION, "action:detention"),

```

theme:directive (420 (4.2%))

Precise plain-English rule

Matches sentence-start imperatives: Apply, Report, Call, Visit, Leave, Self-deport, Go to, Submit, Sign up, Tip, Click, Tap, See, or Read.

Code text for this tag

```

1206: PATTERN_THEME_DIRECTIVE = _compile(
1207:     r"(?:^[.!?]\s+)(Apply|Report|Call|Visit|Leave|Self[- ]deport|Go to|Submit|Sign up|Tip|Click|Tap|See|Read)\b"
1208: )

1588:     (PATTERN_THEME_STATISTICS, "format:statistics"),
1589:     (PATTERN_THEME_DIRECTIVE, "theme:directive"),
1590:     (PATTERN_ANGEL_FAMILY, "subject:angel-family"),

```

theme:religion (329 (3.3%))

Precise plain-English rule

Matches religion/religious, faith/faithful/faith-based, worship/church/synagogue/mosque/temple/chaplain, prayer/pray/praying/prayed, bless/blessed/blessing, God/Lord/Jesus/Christ/Christian, Bible/biblical/scripture. Suppresses common expletives within 18 characters.

Code text for this tag

```

697: PATTERN_THEME_RELIGION = _compile(
698:     r"\breligio(?:n|us)\b"
699:     r"|\bfaith(?:ful|s|[- ]based)?\b"
700:     r"|\b(?:worship|church(?:es)?|synagogue|mosque|temple|chaplain)\b"
701:     r"|\b(?:prayer|prayers|pray|praying|prayed)\b"
702:     r"|\b(?:bless|blessed|blessing|blessings)\b"
703:     r"|\b(?:god|lord|jesus|christ|christian(?:ity|s)?|judeo[- ]christian)\b"
704:     r"|\b(?:bible|biblical|scripture|scriptural)\b"
705: )

```

```

706: PATTERN_THEME_RELIGION_EXPLETIVE = _compile(
707:     r"\b(?:oh\s+my\s+god|omg|god\s+damn(?:ed|it)?|goddamn(?:ed)?|\"
708:     r\"for\s+god'\s+sake|good\s+lord|thank\s+god)\b\"
709: )

1777: for match in PATTERN_THEME_RELIGION.finditer(text):
1778:     start, end = match.span()
1779:     window = text[
1780:         max(0, start - RELIGION_EXPLETIVE_WINDOW_CHARS) : min(
1781:             len(text), end + RELIGION_EXPLETIVE_WINDOW_CHARS
1782:         )
1783:     ]
1784:     if PATTERN_THEME_RELIGION_EXPLETIVE.search(window):
1785:         continue
1786:     return match
1787: return None

```

theme:martyrdom (202 (2.0%))

Precise plain-English rule

Matches Angel Family/family-member language; memorial/victim/fallen/killed/murdered language within 180 characters of honor/remember/commemorate/mourn/never forget/say their names in either order; would still be alive/life was taken/preventable tragedy/line of duty/gave everything. This is our why also matches directly for tracked accounts.

Code text for this tag

```

770: PATTERN_MARTYRDOM_CONTEXT = _compile(
771:     r"\bangel\s+(?:famil(?:y|ies)|mom|dad|parent|mother|father|wife|husband|son|daughter|child)\b\"
772:     r"|\b(?:honou?:|remember|commemorate|mourn|never\s+forget|say\s+their\s+names?)\b\"
773:     r".{0,180}\b(?:victims?|fallen|killed|murdered|lives?|life|names?|memory|heroes|families)\b\"
774:     r"|\b(?:victims?|fallen|killed|murdered|lives?|life|names?|memory|heroes|families)\b\"
775:     r".{0,180}\b(?:honou?:|remember|commemorate|mourn|never\s+forget|say\s+their\s+names?)\b\"
776:     r"|\b(?:would\s+still\s+be\s+alive|life\s+was\s+taken|lives?\s+were\s+taken|\"
777:     r\"preventable\s+(?:murder|death|tragedy)|killed\s+in\s+the\s+line\s+of\s+duty|\"
778:     r\"gave\s+everything)\b\"
779: )

765: PATTERN_MARTYRDOM_WHY = _compile(
766:     r"\bthis\s+is\s+our\s+why\b\"
767:     r"|\bthey\s+are\s+our\s+why\b\"
768:     r"|\byou\s+are\s+why\s+we\s+fight\b\"
769: )

1831: if m := PATTERN_MARTYRDOM_CONTEXT.search(text):
1832:     return m
1833:
1834: why = PATTERN_MARTYRDOM_WHY.search(text)
1835: if not why:
1836:     return None
1837: existing = {str(e["tag"]) for e in entries}
1838: if account_category in {"core", "government", "officials"}:
1839:     return why
1840: if existing.intersection({"subject:angel-family", "subject:crime-victim", "crime:homicide"}):
1841:     return why
1842: start, end = why.span()
1843: window = text[max(0, start - 220) : min(len(text), end + 220)]
1844: if PATTERN_MARTYRDOM_CONTEXT.search(window):
1845:     return why

```

theme:civil-disturbance (85 (0.8%))

Precise plain-English rule

Matches riot/rioters/rioting, civil disturbance/unrest, mass unrest, violent protest(s), anti-ICE riot/protest, violent demonstrators, street violence, or mob violence.

Code text for this tag

```

725: PATTERN_THEME_CIVIL_DISTURBANCE = _compile(
726:     r"\b(?:riot(?:?:s|ers|ing)?|civil\s+disturbance|civil\s+unrest|mass\s+unrest|\"
727:     r\"violent\s+protests?|anti[- ]ICE\s+(?:riot(?:?:s|ers|ing)?|protests?)|\"
728:     r\"violent\s+demonstrators?|street\s+violence|mob\s+violence)\b\"
729: )

1558: (PATTERN_THEME_TRANSGENER, "theme:transgender"),
1559: (PATTERN_THEME_CIVIL_DISTURBANCE, "theme:civil-disturbance"),
1560: (PATTERN_THEME_CBP_HOME, "policy:cbp-home"),

```

theme:homeland (67 (0.7%))

Precise plain-English rule

Matches our/the/this/America's Homeland; protect/secure/defend/safeguard Homeland; or safe/secure Homeland.

Code text for this tag

```

657: PATTERN_THEME_HOMELAND = re.compile(
658:     r"\b(?:our|the|this|america's)\s+Homeland\b\"
659:     r"|\b(?:protect|protecting|secure|securing|defend|defending|safeguard|safeguarding)\"
660:     r"\s+(?:our|the|this|america's)\s+)?Homeland\b\"
661:     r"|\b(?:safe|secure)\s+Homeland\b\"
662: )

1554: (PATTERN_THEME_WORKSITE, "policy:worksite-enforcement"),
1555: (PATTERN_THEME_HOMELAND, "theme:homeland"),
1556: (PATTERN_THEME_NATIVISM, "theme:nativism"),

```

theme:nativism (30 (0.3%))

Precise plain-English rule

Matches native-born/American-born people, foreign-born/foreign workers with displacement/flood/cheap/replacement/betrayal/American-worker context within 140 characters, globalism has failed/Americanism will prevail, forefathers, birthright-as-stolen language, or inheritance language within 160 characters of nation/citizenship/Homeland/birthright/immigration context.

Code text for this tag

```
663: PATTERN_THEME_NATIVISM = _compile(
664:     r"\b(?:native[- ]born\s+(?:americans?|workers?|citizens?)\b"
665:     r"|\bamerican[- ]born\s+(?:americans?|workers?|citizens?)\b"
666:     r"|\bforeign[- ]born\s+workers?\b"
667:     r"|\bforeign\s+(?:workers?|labor)\b.{0,140}\b(?:flood|cheap|displac|replac|"
668:     r"betray|job_market|american\s+(?:workers?|jobs?)|americans?\s+first)\b"
669:     r"|\b(?:american\s+(?:workers?|jobs?)|americans?\s+first|job_market)\b.{0,140}"
670:     r"|\b(?:foreign\s+(?:workers?|labor)|foreign[- ]born\s+workers?)\b"
671:     r"|\bglobalism has failed\b|bamericanism will prevail\b"
672:     r"|\b(?:our\s+)?forefathers?\b"
673: )

821: PATTERN_THEME_NATIVISM_BIRTHRIGHT = _compile(
822:     r"\b(?:actual|real|true)\s+birthright\s+of\s+(?:every|all)?\s*americans?\b"
823:     r"|\bamerican(?:'s)?\s+birthright\b"
824:     r"|\b(?:our|the)\s+birthright\b.{0,160}\b(?:steal|steals?|stolen|dilut|eras|destroy|destroyed|replac|betray|undermin)\b"
825:     r"|\b(?:steal|steals?|stolen|dilut|eras|destroy|destroyed|replac|betray|undermin)\b.{0,160}\b(?:our|the)\s+birthright\b"
826:     r"|\b(?:destroy(?:ing)?|destroys?|erasing?|replac(?:e|ing)?|betray(?:ing)?|undermin(?:e|ing)?|dilut(?:e|ing)?)"
827:     r"\b.{0,120}\b(?:our|the|american(?:'s)?)\s+birthright\b"
828: )

1809:         "topic:immigration",
1810:     }
1811:     for tag in existing
1812: )
1813: for match in PATTERN_NATIVISM_INHERITANCE.finditer(text):
1814:     start, end = match.span()
1815:     window = text[max(0, start - 160) : min(len(text), end + 160)]
1816:     if strong_tag_context or PATTERN_NATIVISM_INHERITANCE_CONTEXT.search(window):
1817:         return match
1818: return None
1819:
```

theme:transgender (23 (0.2%))

Precise plain-English rule

Matches transgender, gender ideology/identity/affirming/transition/dysphoria, biological/transgender male/female/man/woman/boy/girl language, men/boys in women's/girls' sports, women/girls sports exclusion/protection language, or Title IX within 80 characters of transgender/gender/sports context.

Code text for this tag

```
711: PATTERN_THEME_TRANSGENER = _compile(
712:     r"\btransgender\b"
713:     r"|\bgender[- ](?:ideology|identity|affirming|transition|dysphoria)\b"
714:     r"|\b(?:biological|transgender)\s+"
715:     r"|(?:males?|females?|man|woman|men|women|boys?|girls?)\b"
716:     r"|\b(?:men|males|boys)\s+in\s+(?:women[\u2019]?s|girls[\u2019]?)\s+sports\b"
717:     r"|\b(?:men|males|boys)\s+(?:competing|playing|participating)\s+"
718:     r"|(?:in|against|with)\s+(?:women|girls|female\s+athletes?)\b"
719:     r"|\b(?:women[\u2019]?s|girls[\u2019]?)\s+sports\s+(?:are\s+)?"
720:     r"for\s+(?:women|girls|female\s+athletes?)\b"
721:     r"|\b(?:protect|save|defend)\s+(?:women[\u2019]?s|girls[\u2019]?)\s+sports\b"
722:     r"|\btitle\s+ix\b.{0,80}\b(?:transgender|gender|women[\u2019]?s\s+sports|girls[\u2019]?s\s+sports)\b"
723:     r"|\b(?:transgender|gender|women[\u2019]?s\s+sports|girls[\u2019]?s\s+sports)\b.{0,80}\bttitle\s+ix\b"
724: )

1557:         (PATTERN_THEME_CHRISTIANITY, "religion:christianity"),
1558:         (PATTERN_THEME_TRANSGENER, "theme:transgender"),
1559:         (PATTERN_THEME_CIVIL_DISTURBANCE, "theme:civil-disturbance"),
```

theme:pop-culture-reference (5 (0.0%))

Precise plain-English rule

Matches Sydney Sweeney; American Eagle within 100 characters of ICE/DHS/CBP/deport/illegal alien/border in either order; or good genes within 100 characters of jeans/ICE/DHS/CBP/deport/border/illegal alien. Also implied by parody matches.

Code text for this tag

```
806: PATTERN_THEME_POP_CULTURE_REFERENCE = _compile(
807:     r"\bSydney\s+Sweeney\b"
808:     r"|\bAmerican\s+Eagle\b.{0,100}\b(?:ICE|DHS|CBP|deport|illegal\s+alien|border)\b"
809:     r"|\b(?:ICE|DHS|CBP|deport|illegal\s+alien|border)\b.{0,100}\bAmerican\s+Eagle\b"
810:     r"|\bgood\s+genes\b.{0,100}\b(?:jeans|ICE|DHS|CBP|deport|border|illegal\s+alien)\b"
811: )

1560:         (PATTERN_THEME_CBP_HOME, "policy:cbp-home"),
1561:         (PATTERN_THEME_POP_CULTURE_REFERENCE, "theme:pop-culture-reference"),
1562:         (PATTERN_LEGAL_BIRTHRIGHT_CITIZENSHIP, "legal:birthright-citizenship"),
```

topic

The topic namespace is produced by direct topic regexes, parent-topic implications from narrower tags, and the immigration sticky default. For tracked core/government/official accounts, topic:immigration is added unless a non-immigration blocker fires; it is tentative when no explicit immigration signal exists.

topic:immigration (9,196 (91.9%))

Precise plain-English rule

Added when explicit immigration regex matches immigration/immigrant/migrant/asylum/illegal alien/undocumented alien/border patrol/CBP Home/deport/removal; or when narrower immigration tags imply it; or by default for core/government/official accounts unless a weather/disaster/sports/birthday/anniversary blocker or another non-immigration topic prevents it. Tentative when only the account default supports it.

Code text for this tag

```
1908: PATTERN_EXPLICIT_IMMIGRATION_TOPIC = _compile(
1909:     r"\b(immigration|immigrants?|migrants?|asylum|illegal\s+(?:alien|immigrant)s?)"
1910:     r"undocumented\s+(?:alien|immigrant)s?|border patrol|CBP\s+Home|"
1911:     r"deport(?:ed|ing|ation)?|removals?\b"
1912: )

1855: "action:deportation": ("topic:immigration",),
1856: "action:self-deportation": ("topic:immigration",),
1857: "action:report-immigrants": ("topic:immigration",),
1858: "agency:ICEgov": ("topic:immigration",),
1859: "agency:CBP": ("topic:immigration",),
1860: "agency:DHSgov": ("topic:immigration",),
1861: "agency:HSI_HQ": ("topic:immigration",),
1862: "agency:USBPChief": ("topic:immigration",),
1863: "agency:DeptofWar": ("topic:military",),
1864: "agency:CENTCOM": ("topic:military",),
1865: "agency:Southcom": ("topic:military",),
1866: "agency:USCG": ("topic:military",),
1867: "agency:USArmyNorth": ("topic:military",),
1868: "agency:USNationalGuard": ("topic:military",),
1869: "agency:USNorthernCmd": ("topic:military",),
1870: "theme:criminal": ("topic:immigration",),
1871: "genre:lineup": ("topic:immigration",),
1872: "subject:angel-family": ("theme:martyrdom", "topic:immigration"),
1873: "subject:crime-victim": ("theme:martyrdom", "topic:immigration"),
1874: "subject:cbp-home-app": ("topic:immigration",),
1875: "subject:enforcement-op": ("topic:immigration",),
1876: "policy:border": ("topic:immigration",),
1877: "policy:cbp-home": ("topic:immigration",),
1878: "policy:sanctuary-cities": ("topic:immigration",),
1879: "policy:worksite-enforcement": ("topic:economy", "topic:immigration"),
1880: "theme:nativism": ("topic:immigration",),
1881: "theme:pop-culture-reference": ("topic:immigration",),
1882: "religion:christianity": ("theme:religion",),
1883: "slogan:criminal-illegal-alien": ("topic:immigration",),
1884: "slogan:free-ticket-home": ("topic:immigration",),
1885: "slogan:go-home": ("topic:immigration",),
1886: "slogan:illegal-alien": ("topic:immigration",),
1887: "slogan:mass-deportation": ("topic:immigration",),
1888: "slogan:masa": ("topic:immigration",),
1889: "slogan:find-and-kill": ("topic:immigration",),
1890: "slogan:import-third-world": ("topic:immigration",),
1891: "legal:birthright-citizenship": ("topic:immigration",),
1892: "genre:parody": ("theme:pop-culture-reference",),
1893: "slogan:most-secure-border": ("topic:immigration", "policy:border"),
1894: "slogan:catch-release": ("topic:immigration",),
1895: "slogan:project-homecoming": ("topic:immigration",),
1896: "slogan:maga": ("topic:general",),
1897: "slogan:maha": ("topic:general",),
1898: "slogan:america-first": ("topic:general",),
1899: "slogan:golden-age": ("topic:general",),
1900: "slogan:save-america": ("topic:general",),
1901: "slogan:law-and-order": ("topic:general",),
1902: "slogan:peace-through-strength": ("topic:general",),
1903: "slogan:promises-kept": ("topic:general",),
1904: "phrase:immigrant": ("topic:immigration",),
1905: "phrase:migrant": ("topic:immigration",),
1906: }
1907: INTRINSIC_PARENT_TOPICS_PREFIXES: tuple[tuple[str, str], ...] = (("origin:", "topic:immigration"),)
1908: PATTERN_EXPLICIT_IMMIGRATION_TOPIC = _compile(
1909:     r"\b(immigration|immigrants?|migrants?|asylum|illegal\s+(?:alien|immigrant)s?)"

1936: if PATTERN_EXPLICIT_IMMIGRATION_TOPIC.search(text):
1937:     add("topic:immigration")
1938: if (
1939:     any(e["tag"] == "policy:worksite-enforcement" for e in entries)
1940:     and "topic:economy" not in existing_tags
1941: ):
1942:     add("topic:economy")
1943:
1944:
1945: # Tag names whose presence on a tweet promotes `topic:immigration` from
1946: # a tentative-by-account default to a confirmed-by-text classification.
1947: # Anything that explicitly references the immigration domain (origin
1948: # country pattern, deportation verb, border keyword, ICE/CBP/DHS handle,
1949: # the criminal-alien frame, etc.) clears the bar.
1950: IMMIGRATION_CONFIRMING_PREFIXES: tuple[str, ...] = (
1951:     "action:",
1952:     "origin:",
1953:     "country:",
1954:     "policy:border",
1955:     "policy:sanctuary",
1956:     "policy:worksite",
1957:     "policy:cbp-home",
1958:     "theme:nativism",
1959:     "theme:criminal",
1960:     "shape:",
1961:     "subject:cbp-home-app",
1962:     "subject:enforcement-op",
1963: )
1964: IMMIGRATION_CONFIRMING_EXACT: frozenset[str] = frozenset(
1965:     {
1966:         "agency:ICEgov",
1967:         "agency:CBP",
1968:         "agency:DHSgov",
1969:         "agency:HSI_HQ",
```

```

1970:         "agency:USBPChief",
1971:         "slogan:criminal-illegal-alien",
1972:         "slogan:free-ticket-home",
1973:         "slogan:go-home",
1974:         "slogan:illegal-alien",
1975:         "slogan:mass-deportation",
1976:         "slogan:masa",
1977:         "slogan:most-secure-border",
1978:         "slogan:catch-release",
1979:         "slogan:nice",
1980:         "slogan:project-homecoming",
1981:         "slogan:reportrecon",
1982:         "slogan:worst",
1983:         "phrase:immigrant",
1984:         "phrase:migrant",
1985:     }
1986: )
1987: # Last-ditch keyword check – picks up image-heavy / template-light
1988: # tweets that the namespace rules above don't flag but that still
1989: # clearly read as immigration content ("ICE arrested", "illegal alien",
1990: # "the border", standalone "immigration"). Kept narrow on purpose.
1991: PATTERN_IMMIGRATION_KEYWORD = _compile(
1992:     r"\b(immigration|immigrant|migrant|asylum|illegal alien|the border|"
1993:     r"border patrol|ICE\b|CBP\b)\b"
1994: )
1995:
1996:
1997: def _has_immigration_signal(entries: list[dict[str, Any]], text: str) -> bool:
1998:     """True if the tweet carries any explicit immigration-domain signal.
1999:     Drives the confirmed-vs-tentative split for 'topic:immigration'."""
2000:     for e in entries:
2001:         tag = e["tag"]
2002:         if tag in IMMIGRATION_CONFIRMING_EXACT:
2003:             return True
2004:         for pref in IMMIGRATION_CONFIRMING_PREFIXES:
2005:             if tag.startswith(pref):
2006:                 return True
2007:     return bool(PATTERN_IMMIGRATION_KEYWORD.search(text))
2008:
2009:
2010: def _has_non_immigration_topic(entries: list[dict[str, Any]]) -> bool:
2011:     return any(e["tag"].startswith("topic:") and e["tag"] != "topic:immigration" for e in entries)
2012:
2013:
2014: def _maybe_immigration_default(
2015:     entries: list[dict[str, Any]],
2016:     account_category: str,
2017:     text: str,
2018:     add: Callable[..., None],
2019: ) -> None:
2020:     """Apply 'topic:immigration' to every tweet from a tracked-tier
2021:     author unless an obvious non-immigration signal blocks it.
2022:     Confirmed when the tweet carries any explicit immigration-domain
2023:     marker; tentative otherwise (image-heavy, template-light tweets
2024:     where the author identity is the only signal we have).
2025:
2026:     See the 'topic:immigration' entry in 'config/tag_taxonomy.yaml' for
2027:     the rationale."""
2028:     if account_category not in IMMIGRATION_DEFAULT_CATEGORIES:
2029:         return
2030:     has_signal = _has_immigration_signal(entries, text)
2031:     if has_signal:
2032:         add("topic:immigration")
2033:         return
2034:     if any(e["tag"] == "event:palestine" for e in entries):
2035:         return
2036:     for pat in NON_IMMIGRATION_PATTERNS:
2037:         if pat.search(text):
2038:             return
2039:     if not _has_non_immigration_topic(entries):
2040:         add("topic:immigration", tentative=True)
2041:

```

topic:economy (1,083 (10.8%))

Precise plain-English rule

Matches economy/economic, jobs/job growth, workers/workforce/wages/labor market, employment/unemployment/hiring/manufacturing/business/apprenticeship/small business, taxes/tax cuts, inflation/prices/cost of living, tariffs/trade deficit/supply chain/GDP/stock market/markets. Also implied by policy:worksite-enforcement.

Code text for this tag

```

560: PATTERN_TOPIC_ECONOMY = _compile(
561:     r"\b(econom(?:y|ic)|jobs?|job growth|workers?|workforce|wages?|labor market|"
562:     r"employment|unemployment|hiring|manufactur(?:e|ing)|business(?:es)?|"
563:     r"apprenticeships?|small businesses?|tax cuts?|taxes|inflation|prices?|"
564:     r"cost of living|tariffs?|trade deficit|supply chains?|GDP|stock market|markets?)\b"
565: )
566:
567:
568:
569:
570:
571:
572:
573:
574:
575:
576:
577:
578:
579:
580:
581:
582:
583:
584:
585:
586:
587:
588:
589:
590:
591:
592:
593:
594:
595:
596:
597:
598:
599:
600:
601:
602:
603:
604:
605:
606:
607:
608:
609:
610:
611:
612:
613:
614:
615:
616:
617:
618:
619:
620:
621:
622:
623:
624:
625:
626:
627:
628:
629:
630:
631:
632:
633:
634:
635:
636:
637:
638:
639:
640:
641:
642:
643:
644:
645:
646:
647:
648:
649:
650:
651:
652:
653:
654:
655:
656:
657:
658:
659:
660:
661:
662:
663:
664:
665:
666:
667:
668:
669:
670:
671:
672:
673:
674:
675:
676:
677:
678:
679:
680:
681:
682:
683:
684:
685:
686:
687:
688:
689:
690:
691:
692:
693:
694:
695:
696:
697:
698:
699:
700:
701:
702:
703:
704:
705:
706:
707:
708:
709:
710:
711:
712:
713:
714:
715:
716:
717:
718:
719:
720:
721:
722:
723:
724:
725:
726:
727:
728:
729:
730:
731:
732:
733:
734:
735:
736:
737:
738:
739:
740:
741:
742:
743:
744:
745:
746:
747:
748:
749:
750:
751:
752:
753:
754:
755:
756:
757:
758:
759:
760:
761:
762:
763:
764:
765:
766:
767:
768:
769:
770:
771:
772:
773:
774:
775:
776:
777:
778:
779:
780:
781:
782:
783:
784:
785:
786:
787:
788:
789:
790:
791:
792:
793:
794:
795:
796:
797:
798:
799:
800:
801:
802:
803:
804:
805:
806:
807:
808:
809:
810:
811:
812:
813:
814:
815:
816:
817:
818:
819:
820:
821:
822:
823:
824:
825:
826:
827:
828:
829:
830:
831:
832:
833:
834:
835:
836:
837:
838:
839:
840:
841:
842:
843:
844:
845:
846:
847:
848:
849:
850:
851:
852:
853:
854:
855:
856:
857:
858:
859:
860:
861:
862:
863:
864:
865:
866:
867:
868:
869:
870:
871:
872:
873:
874:
875:
876:
877:
878:
879:
880:
881:
882:
883:
884:
885:
886:
887:
888:
889:
890:
891:
892:
893:
894:
895:
896:
897:
898:
899:
900:
901:
902:
903:
904:
905:
906:
907:
908:
909:
910:
911:
912:
913:
914:
915:
916:
917:
918:
919:
920:
921:
922:
923:
924:
925:
926:
927:
928:
929:
930:
931:
932:
933:
934:
935:
936:
937:
938:
939:
940:
941:
942:
943:
944:
945:
946:
947:
948:
949:
950:
951:
952:
953:
954:
955:
956:
957:
958:
959:
960:
961:
962:
963:
964:
965:
966:
967:
968:
969:
970:
971:
972:
973:
974:
975:
976:
977:
978:
979:
980:
981:
982:
983:
984:
985:
986:
987:
988:
989:
990:
991:
992:
993:
994:
995:
996:
997:
998:
999:

```

topic:general (971 (9.7%))

Precise plain-English rule

Added when at least three broad problem-domain regexes match among immigration, crime, economy, military, fraud, security, and costs; also implied by broad slogans such as MAGA/MAHA/America First/Golden Age/Save America/Law and Order/Peace Through Strength/Promises Kept.

Code text for this tag

```

643: GENERAL_TOPIC_PATTERNS: tuple[tuple[str, re.Pattern[str]], ...] = (
644:     ("immigration", _compile(r"\b(immigration|migrant|border|illegal alien|asylum\b)")),
645:     ("crime", _compile(r"\b(crime|criminal|drugs?|fentanyl|gang|violence|murder)\b")),
646:     ("economy", PATTERN_TOPIC_ECONOMY),
647:     ("military", PATTERN_TOPIC_MILITARY),
648:     ("fraud", _compile(r"\b(fraud|waste|abuse|corruption|scam)\b")),
649:     ("security", _compile(r"\b(security|terror|war|china|cartel|threat)\b")),
650:     ("costs", _compile(r"\b(inflation|prices?|taxes|cost of living)\b")),
651: )

1848:
1849: def _general_topic_score(text: str) -> int:
1850:     """Count broad problem domains in a multi-issue / grievance-style post."""
1851:     return sum(1 for _slug, pat in GENERAL_TOPIC_PATTERNS if pat.search(text))
1852:

1909:     r"\b(immigration|immigrants?|migrants?|asylum|illegal\s+(?:alien|immigrant)s?)|"
1910:     r"undocumented\s+(?:alien|immigrant)s?|border patrol|CBP\s+Home|"
1911:     r"deport(?:ed|ing|ation)?|removals?)\b"
1912: )
1913:
1914:
1915: def _ensure_intrinsic_parent_topics(
1916:     entries: list[dict[str, Any]],
1917:     text: str,
1918:     add: Callable[..., None],
1919: ) -> None:
1920:     """Add broad topic parents implied by narrower deterministic tags."""
1921:     existing_tags = {str(e["tag"]) for e in entries}
1922:     if (
1923:         any(tag.startswith("military:") for tag in existing_tags)
1924:         and "topic:military" not in existing_tags
1925:     ):
1926:         add("topic:military")
1927:         existing_tags.add("topic:military")
1928:     for tag in list(existing_tags):
1929:         for parent in INTRINSIC_PARENT_TOPICS_EXACT.get(tag, ()):
1930:             add(parent)
1931:             existing_tags.add(parent)
1932:         for prefix, parent in INTRINSIC_PARENT_TOPICS_PREFIXES:
1933:             if tag.startswith(prefix):
1934:                 add(parent)

```

topic:military (658 (6.6%))

Precise plain-English rule

Matches military/armed forces/service members/veterans/troops/soldiers/sailors/airmen/marines/guardsmen, service branches, Pentagon/DoD/VA, combat/battlefield/war zone/deployment, carrier/USS/USNS terms, service academies/cadets/midshipmen, commander-in-chief, or commandant. Also implied by any military:<branch> tag.

Code text for this tag

```

566: PATTERN_TOPIC_MILITARY = _compile(
567:     r"\b(military|armed forces|servicemembers?|service members?|veterans?|troops?)|"
568:     r"soldiers?|sailors?|airmen|marines|guardsmen|army|navy|air force|space force|"
569:     r"marine corps|coast guard|national guard|USAF|USSF|USMC|pentagon|"
570:     r"department of defense|DoD|DOD|veterans affairs|combat|battlefield|"
571:     r"war zone|deployed|deployment|carrier strike group|aircraft carrier|"
572:     r"carrier air wing|CVN\s*[A-Z][A-Za-z0-9-]+|USNS\s*[A-Z][A-Za-z0-9-]+|"
573:     r"service academ(?:y|ies)|military academy|naval academy|coast guard academy|"
574:     r"air force academy|west point|USMA|USNA|USCGA|cadets?|midshipmen|"
575:     r"commander[- ]in[- ]chief|commandant)\b"
576: )

1548:     (PATTERN_TOPIC_ECONOMY, "topic:economy"),
1549:     (PATTERN_TOPIC_MILITARY, "topic:military"),
1550:     (PATTERN_TOPIC_LAUDATORY, "topic:laudatory"),

```

topic:laudatory (99 (1.0%))

Precise plain-English rule

Matches accomplishment-list framing: Promises Made/Kept, Golden Age of America, two or more check-mark bullets, delivering on promises/for Americans, historic/record/biggest/greatest/unprecedented/largest accomplishment/achievement/win/progress/result/deal, list/litany/series of accomplishments, or administration/president/Trump accomplishment framing.

Code text for this tag

```

621: PATTERN_TOPIC_LAUDATORY = _compile(
622:     # Laudatory = a general touting of the administration's accomplishments
623:     # (a "laundry list" of wins), NOT any single booster word. Bare words like
624:     # "historic", "delivering", "winning", "momentum", "success" produced heavy
625:     # false positives ("delivering remarks", "Massie winning", "vaccine
626:     # delivered", "historic astronauts"), so require explicit accomplishment-
627:     # list framing: signature slogans, check-marked bullet lists, or an
628:     # accomplishment noun qualified as historic/record/biggest/etc.
629:     r"\bpromises?\s+made[;,:.]?\s+promises?\s+kept\b"
630:     r"|\bgolden\s+age\s+of\s+america\b"
631:     r"|\b(?:[■▼■][^▼■\n]{1,90}){2,}"
632:     r"|\b(?:deliver(?:ing|ed)\s+(?:on\s+(?:his|our|its|their|the)\s+promises\b|for\s+(?:the\s+)?american\b)"
633:     r"|\b(?:historic|record(?:[- ](?:breaking|setting))?)?biggest|greatest|unprecedented|largest)"
634:     r"|\s+(?:list\s+of\s+)?(?:accomplishments?|achievements?|wins?|victories|progress|results|deals?)\b"
635:     r"|\b(?:list|litany|series)\s+of\s+(?:accomplishments?|achievements?|wins?|victories)\b"
636:     r"|\b(?:the\s+administration|president|trump)(?:'s)?\s+(?:historic\s+|record\s+)"
637:     r"?(?:accomplishments?|achievements?|track\s+record)\b"
638: )

1549:     (PATTERN_TOPIC_MILITARY, "topic:military"),
1550:     (PATTERN_TOPIC_LAUDATORY, "topic:laudatory"),
1551:     (PATTERN_EVENT_PALESTINE, "event:palestine"),

```